MISP

INTELLIGENCE SHARING

An Introduction to Cybersecurity Information Sharing

MISP - Intelligence Sharing

CIRCL / Team MISP Project

MISP Project

https://www.misp-project.org/

CRI MISP 2024

Agenda

- Agenda and details available
  https://tinyurl.com/CRI-MISP

MISP and starting from a practical use-case

- During a malware analysis workgroup in 2012, we discovered that we worked on the analysis of the same malware.
- We wanted to share information in an easy and automated way **to avoid duplication of work**.
- Christophe Vandeplas (then working at the CERT for the Belgian MoD) showed us his work on a platform that later became MISP.
- A first version of the MISP Platform was used by the MALWG and **the increasing feedback of users** helped us to build an improved platform.
- MISP is now **a community-driven development**.

about CIRCL

The Computer Incident Response Center Luxembourg (CIRCL) is a government-driven initiative designed to provide a systematic response facility to computer security threats and incidents. CIRCL is the CERT for the private sector, communes and non-governmental entities in Luxembourg and is operated by LHC g.i.e.

MISP and CIRCL

- CIRCL is mandated by the Ministry of Economy and acting as the Luxembourg National CERT for private sector.
- CIRCL leads the development of the Open Source MISP threat intelligence platform which is used by many military or intelligence communities, private companies, financial sector, National CERTs and LEAs globally.
- **CIRCL runs multiple large MISP communities performing active daily threat-intelligence sharing**.

Co-financed by the European Union

Connecting Europe Facility

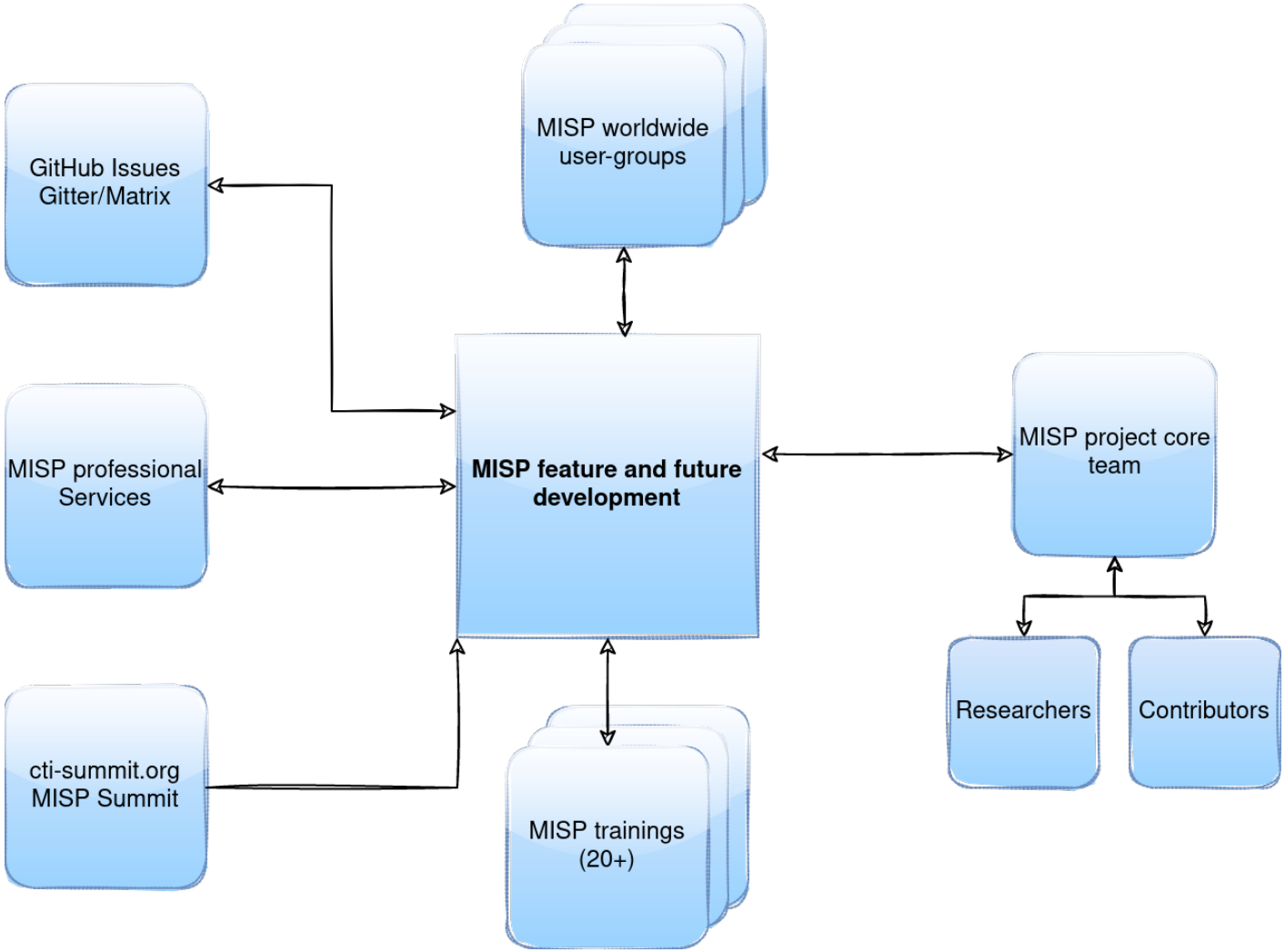
What is MISP?

- MISP is a **threat information sharing** platform that is free & open source software
- A tool that **collects** information from partners, your analysts, your tools, feeds
- Normalises, **correlates**, **enriches** the data
- Allows teams and communities to **collaborate**
- **Feeds** automated protective tools and analyst tools with the output

Development based on practical user feedback

- There are many different types of users of an information sharing platform like MISP:
  - **Malware reversers** willing to share indicators of analysis with respective colleagues.
  - **Security analysts** searching, validating and using indicators in operational security.
  - **Intelligence analysts** gathering information about specific adversary groups.
  - **Law-enforcement** relying on indicators to support or bootstrap their DFIR cases.
  - **Risk analysis teams** willing to know about the new threats, likelyhood and occurences.
  - **Fraud analysts** willing to share financial indicators to detect financial frauds.

MISP model of governance



Many objectives from different user-groups

- Sharing indicators for a **detection** matter.
  - 'Do I have infected systems in my infrastructure or the ones I operate?'
- Sharing indicators to **block**.
  - 'I use these attributes to block, sinkhole or divert traffic.'
- Sharing indicators to **perform intelligence**.
  - 'Gathering information about campaigns and attacks. Are they related? Who is targeting me? Who are the adversaries?'
- $\rightarrow$ These objectives can be conflicting (e.g. False-positives have different impacts)

Communities using MISP

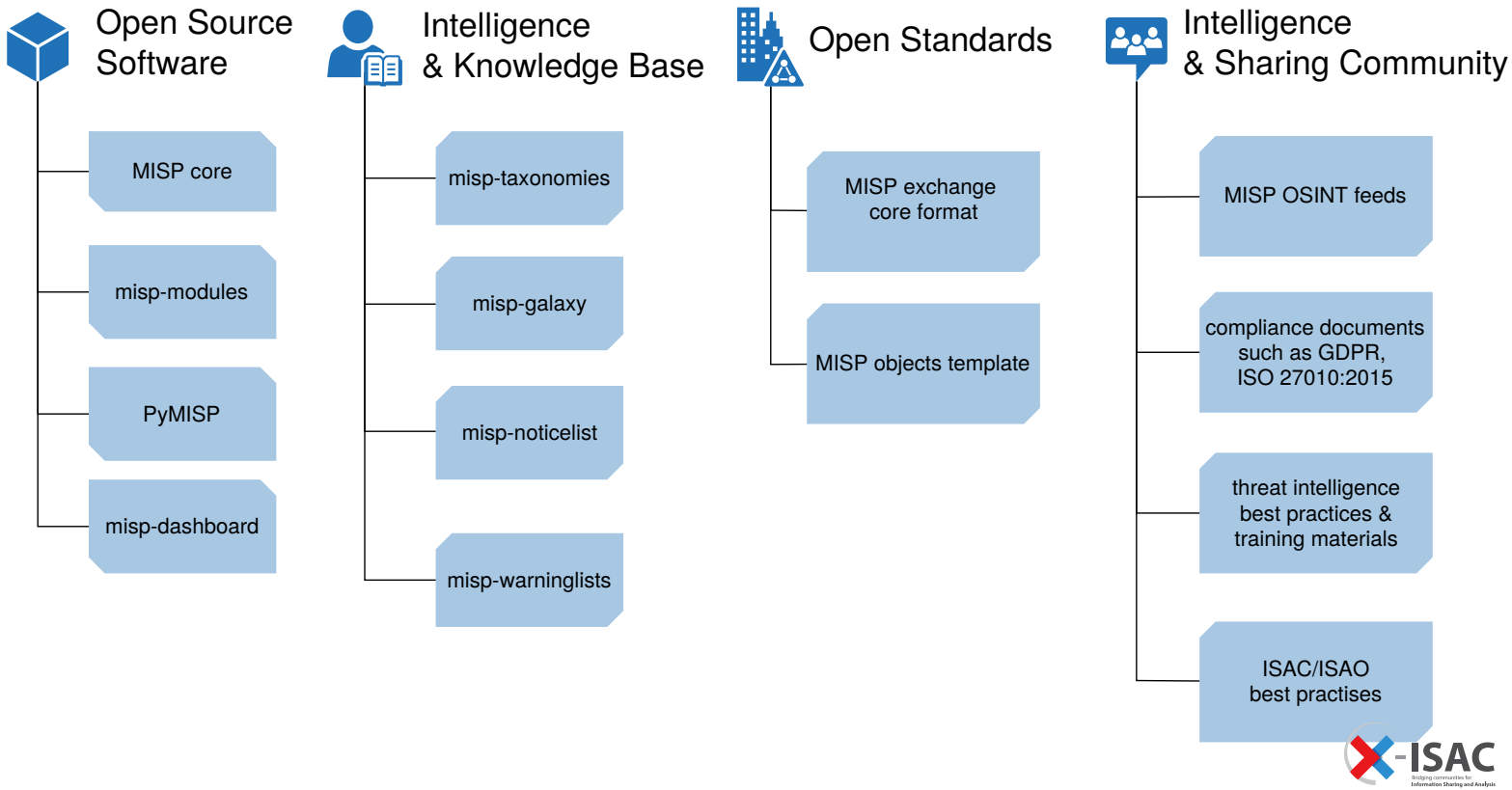
- Communities are groups of users sharing within a set of common objectives/values.
- CIRCL operates multiple MISP instances with a significant user base (more than 1200 organizations with more than 4000 users).
- **Trusted groups** running MISP communities in island mode (air gapped system) or partially connected mode.
- **Financial sector** (banks, ISACs, payment processing organizations) use MISP as a sharing mechanism.
- **Military and international organizations** (NATO, military CSIRTs, n/g CERTs,...).
- **Security vendors** running their own communities (e.g. Fidelis) or interfacing with MISP communities (e.g. OTX).
- **Topical communities** set up to tackle individual specific issues (COVID-19 MISP)

Sharing Difficulties

- Sharing difficulties are not really technical issues but often it's a matter of **social interactions** (e.g. **trust**).
- Legal restriction[1]
  - "Our legal framework doesn't allow us to share information."
  - "Risk of information-leak is too high and it's too risky for our organization or partners."
- Practical restriction
  - "We don't have information to share."
  - "We don't have time to process or contribute indicators."
  - "Our model of classification doesn't fit your model."
  - "Tools for sharing information are tied to a specific format, we use a different one."

[1] https://www.misp-project.org/compliance/

MISP Project Overview



Sharing in MISP

- Sharing via distribution lists - **Sharing groups**
- **Delegation** for pseudo-anonymised information sharing
- **Proposals** and **Extended events** for collaborated information sharing
- Synchronisation, Feed system, air-gapped sharing
- User defined **filtered sharing** for all the above mentioned methods
- Cross-instance information **caching** for quick lookups of large data-sets
- Support for multi-MISP internal enclaves

Information quality management

- Correlating data
- Feedback loop from detections via **Sightings**
- **False positive management** via the warninglist system
- **Enrichment system** via MISP-modules
- **workflow** system to review and control information publication
- **Integrations** with a plethora of tools and formats
- Flexible **API** and support **libraries** such as PyMISP to ease integration
- **Timelines** and giving information a temporal context
- Full chain for **indicator life-cycle management**

Conclusion

- **Information sharing practices come from usage** and by example (e.g. learning by imitation from the shared information).
- MISP is just a tool. What matters is your sharing practices. The tool should be as transparent as possible to support you.
- Enable users to customize MISP to meet their community's use-cases.
- MISP project combines open source software, open standards, best practices and communities to make information sharing a reality.

MISP

INTELLIGENCE SHARING

An Introduction to Cybersecurity Information Sharing

MISP - Intelligence Sharing

CIRCL / Team MISP Project

MISP Project

https://www.misp-project.org/

CRI MISP 2024

Agenda

- Agenda and details available
  https://tinyurl.com/CRI-MISP

MISP and starting from a practical use-case

- During a malware analysis workgroup in 2012, we discovered that we worked on the analysis of the same malware.
- We wanted to share information in an easy and automated way **to avoid duplication of work**.
- Christophe Vandeplas (then working at the CERT for the Belgian MoD) showed us his work on a platform that later became MISP.
- A first version of the MISP Platform was used by the MALWG and **the increasing feedback of users** helped us to build an improved platform.
- MISP is now **a community-driven development**.

about CIRCL

The Computer Incident Response Center Luxembourg (CIRCL) is a government-driven initiative designed to provide a systematic response facility to computer security threats and incidents. CIRCL is a CERT with national responsibilities for the private sector, communes and non-governmental entities in Luxembourg and is operated by LHC (Luxembourg House of Cybersecurity).

MISP and CIRCL

- CIRCL is mandated by the Ministry of Economy and acting as the Luxembourg National CERT for private sector.
- CIRCL leads the development of the Open Source MISP threat intelligence platform which is used by many military or intelligence communities, private companies, financial sector, National CERTs and LEAs globally.
- **CIRCL runs multiple large MISP communities performing active daily threat-intelligence sharing**.

Co-financed by the European Union

Connecting Europe Facility

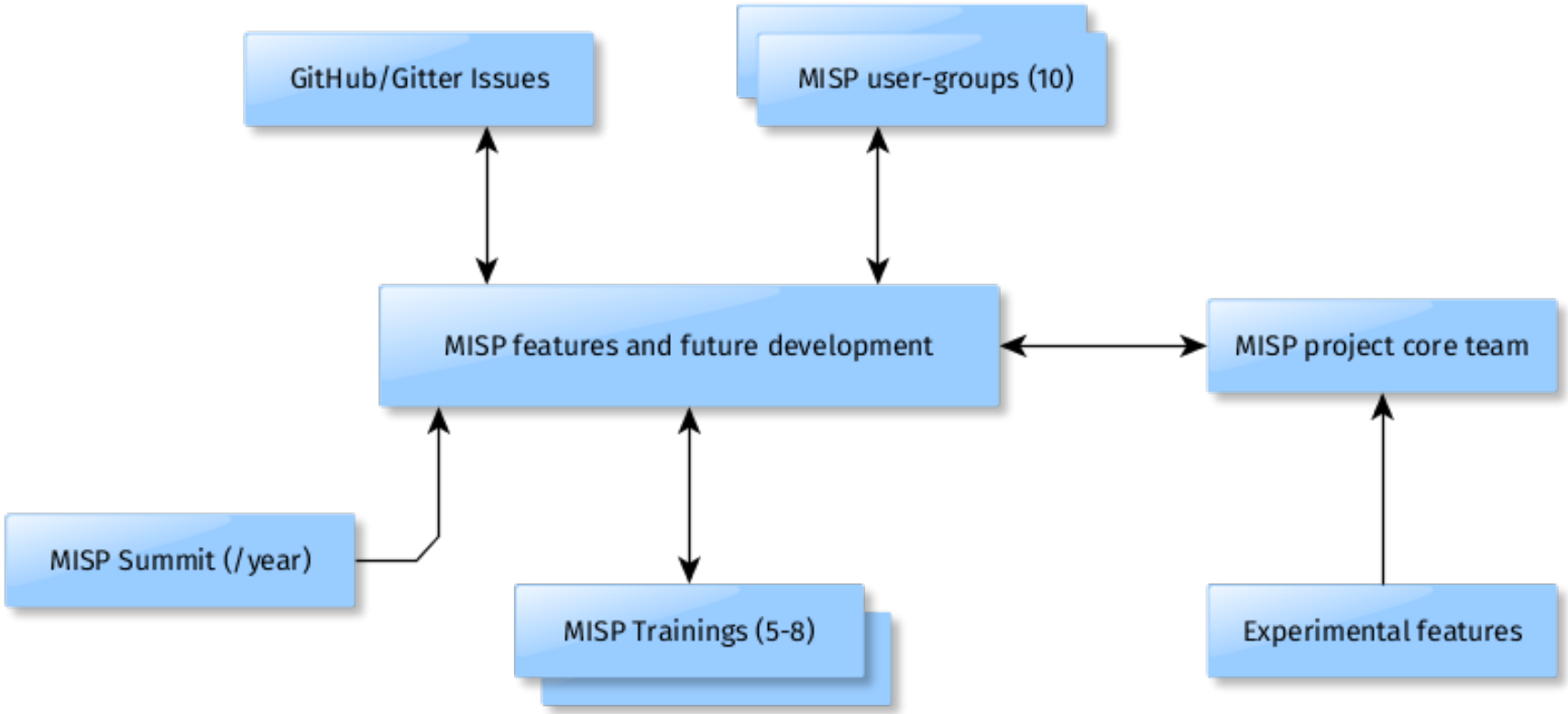
What is MISP?

- MISP is a **threat information sharing** platform that is free & open source software
- A tool that **collects** information from partners, your analysts, your tools, feeds
- Normalises, **correlates**, **enriches** the data
- Allows teams and communities to **collaborate**
- **Feeds** automated protective tools and analyst tools with the output

Development based on practical user feedback

- There are many different types of users of an information sharing platform like MISP:
  - **Malware reversers** willing to share indicators of analysis with respective colleagues.
  - **Security analysts** searching, validating and using indicators in operational security.
  - **Intelligence analysts** gathering information about specific adversary groups.
  - **Law-enforcement** relying on indicators to support or bootstrap their DFIR cases.
  - **Risk analysis teams** willing to know about the new threats, likelyhood and occurences.
  - **Fraud analysts** willing to share financial indicators to detect financial frauds.

MISP model of governance



Many objectives from different user-groups

- Sharing indicators for a **detection** matter.
  - 'Do I have infected systems in my infrastructure or the ones I operate?'
- Sharing indicators to **block**.
  - 'I use these attributes to block, sinkhole or divert traffic.'
- Sharing indicators to **perform intelligence**.
  - 'Gathering information about campaigns and attacks. Are they related? Who is targeting me? Who are the adversaries?'
- $\rightarrow$ These objectives can be conflicting (e.g. False-positives have different impacts)

Communities using MISP

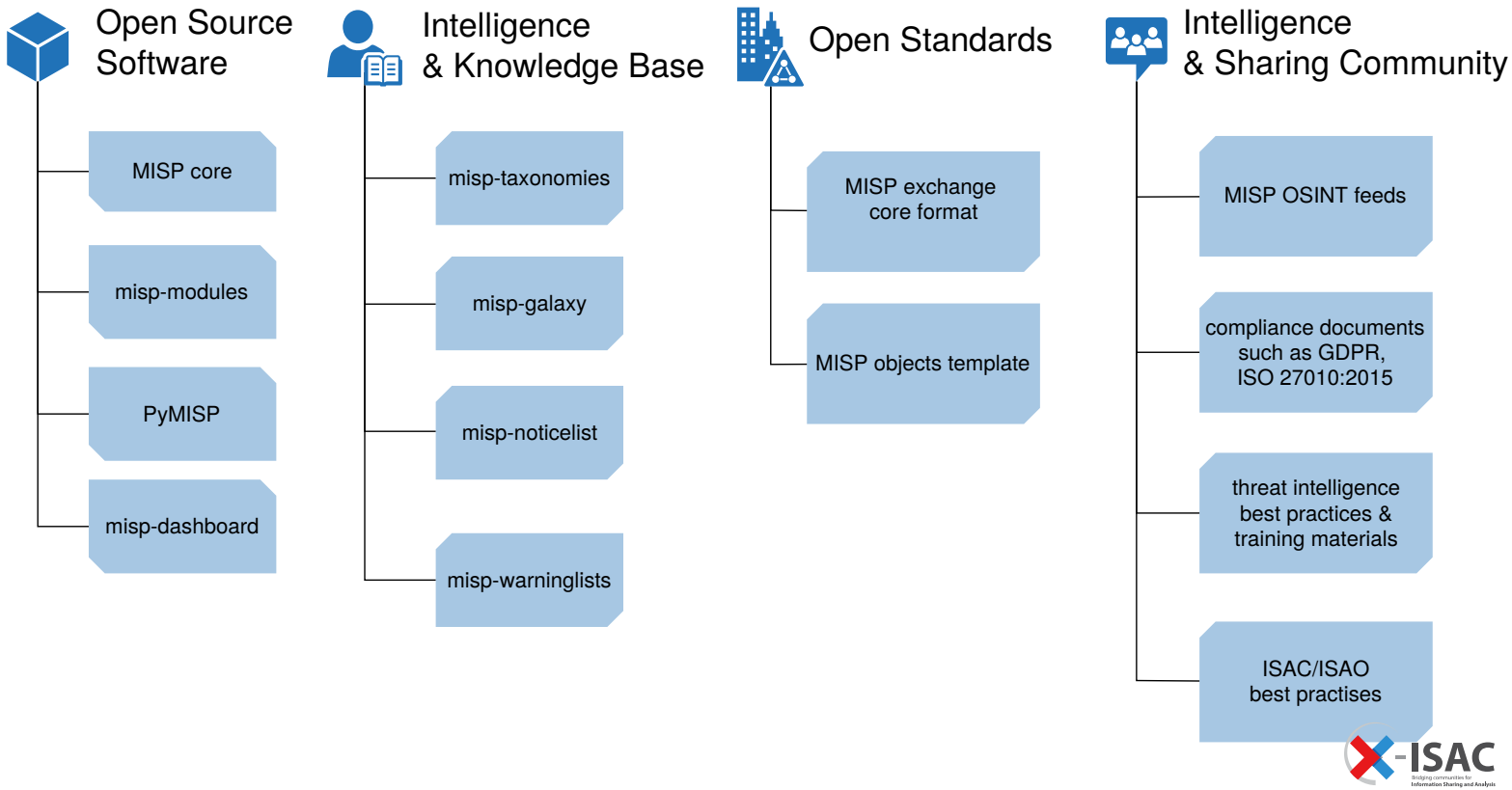
- Communities are groups of users sharing within a set of common objectives/values.
- CIRCL operates multiple MISP instances with a significant user base (more than 1200 organizations with more than 4000 users).
- **Trusted groups** running MISP communities in island mode (air gapped system) or partially connected mode.
- **Financial sector** (banks, ISACs, payment processing organizations) use MISP as a sharing mechanism.
- **Military and international organizations** (NATO, military CSIRTs, n/g CERTs,...).
- **Security vendors** running their own communities (e.g. Fidelis) or interfacing with MISP communities (e.g. OTX).
- **Topical communities** set up to tackle individual specific issues (COVID-19 MISP)

Sharing Difficulties

- Sharing difficulties are not really technical issues but often it's a matter of **social interactions** (e.g. **trust**).
- Legal restriction[1]
  - "Our legal framework doesn't allow us to share information."
  - "Risk of information-leak is too high and it's too risky for our organization or partners."
- Practical restriction
  - "We don't have information to share."
  - "We don't have time to process or contribute indicators."
  - "Our model of classification doesn't fit your model."
  - "Tools for sharing information are tied to a specific format, we use a different one."

[1] https://www.misp-project.org/compliance/

MISP Project Overview



Getting some naming conventions out of the way...

- Data layer
  - **Events** are encapsulations for contextually linked information
  - **Attributes** are individual data points, which can be indicators or supporting data
  - **Objects** are custom templated Attribute compositions
  - **Object references** are the relationships between other building blocks
  - **Sightings** are time-specific occurances of a given data-point detected
- Context layer
  - **Tags** are labels attached to events/attributes and can come from **Taxonomies**
  - **Galaxy-clusters** are knowledge base items used to label events/attributes and come from **Galaxies**
  - **Cluster relationships** denote pre-defined relationships between clusters

Terminology about Indicators

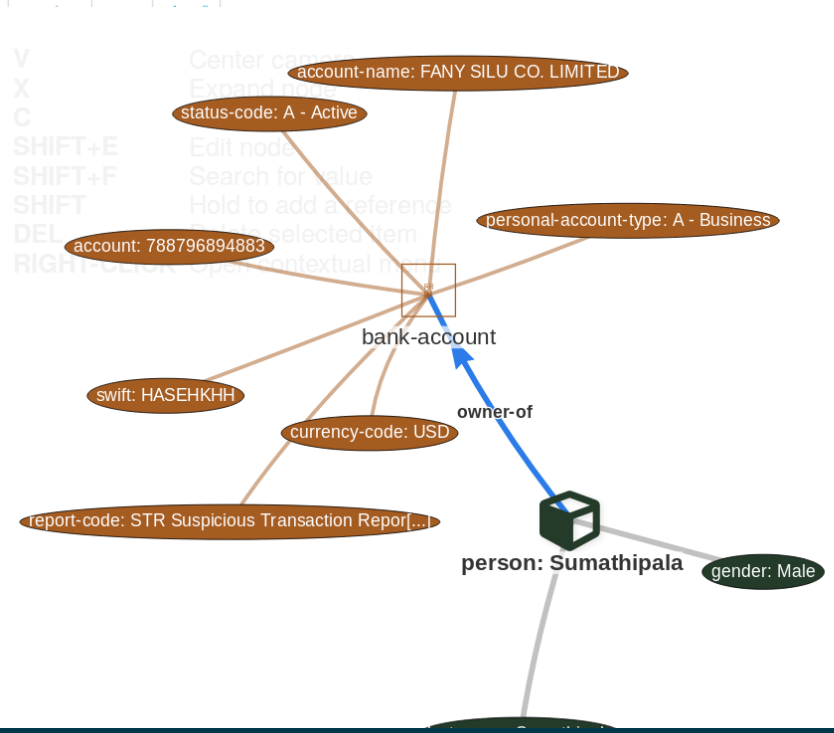
- Indicators[2]
  - Indicators contain a pattern that can be used to detect suspicious or malicious cyber activity.
- Attributes in MISP can be network indicators (e.g. IP address), system indicators (e.g. a string in memory) or even bank account details.
  - **A type (e.g. MD5, url) is how an attribute is described**.
  - An attribute is always in a category (e.g. Payload delivery) which puts it in a context.
    - **A category is what describes** an attribute.
  - An IDS flag on an attribute allows to determine if **an attribute can be automatically used for detection**.

[2] IoC (Indicator of Compromise) is a subset of indicators

A rich data-model: telling stories via relationships

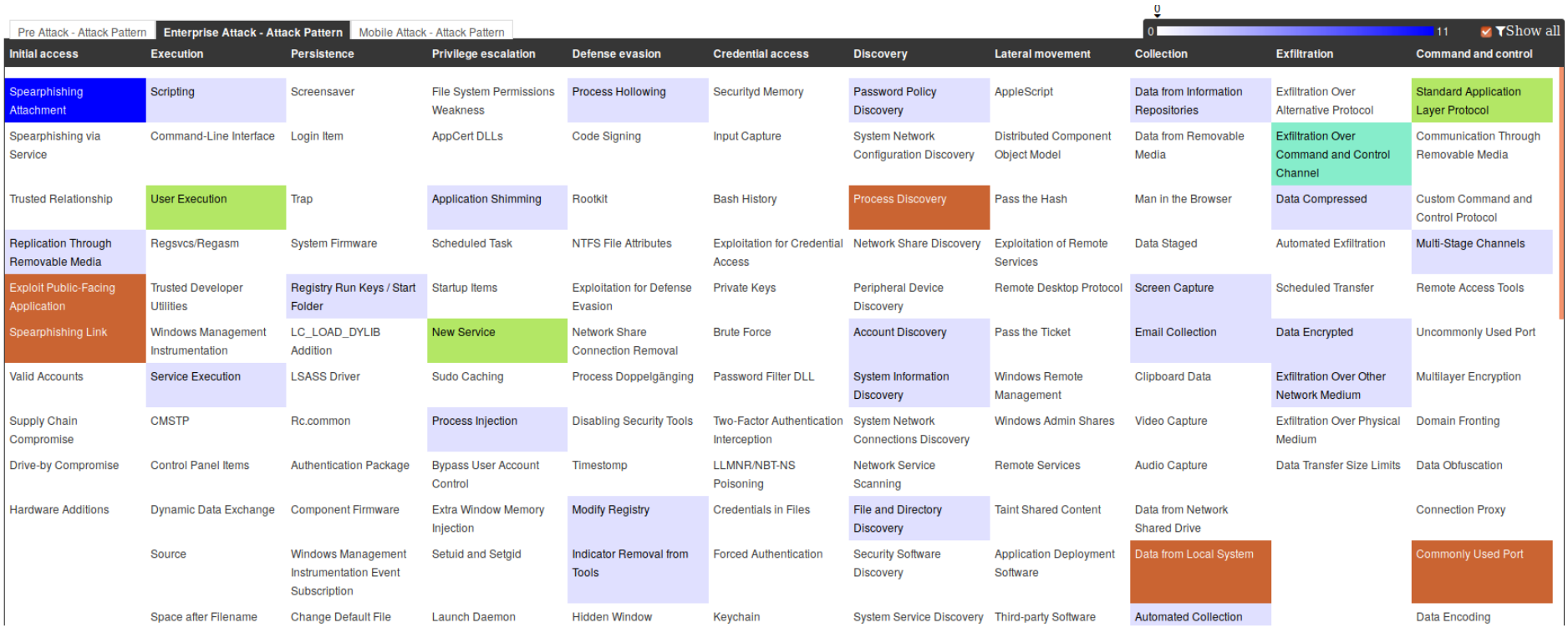
Filters: All | File | Network | Financial | Proposal | Correlation | Warnings | Include deleted attributes | Show context fields

Date	Org	Category	Type	Value	Tags	Galaxies	Comment	Correlate	Related Events
2018-09-28		Name: bank-account References: 0							
2018-09-28		Other	status-code: text	A - Active	+	Add		☐	
2018-09-28		Other	report-code: text	STR Suspicious Transaction Report	+	Add		☐	
2018-09-28		Other	personal-account-type: text	A - Business	+	Add		☐	
2018-09-28		Financial fraud	swift: bic	HASEHKHH	+	Add		☑	3849 11320 11584
2018-09-28		Financial fraud	account: bank-account-nr	788796894883	+	Add		☑	
2018-09-28		Other	account-name: text	FANY SILU CO. LIMITED	+	Add		☑	
2018-09-28		Other	currency-code: text	USD	+	Add		☐	



Contextualisation and aggregation

- MISP integrates at the event and the attribute levels MITRE's Adversarial Tactics, Techniques, and Common Knowledge (ATT&CK).



Pre Attack - Attack Pattern | **Enterprise Attack - Attack Pattern** | Mobile Attack - Attack Pattern

0 — 11 — Show all

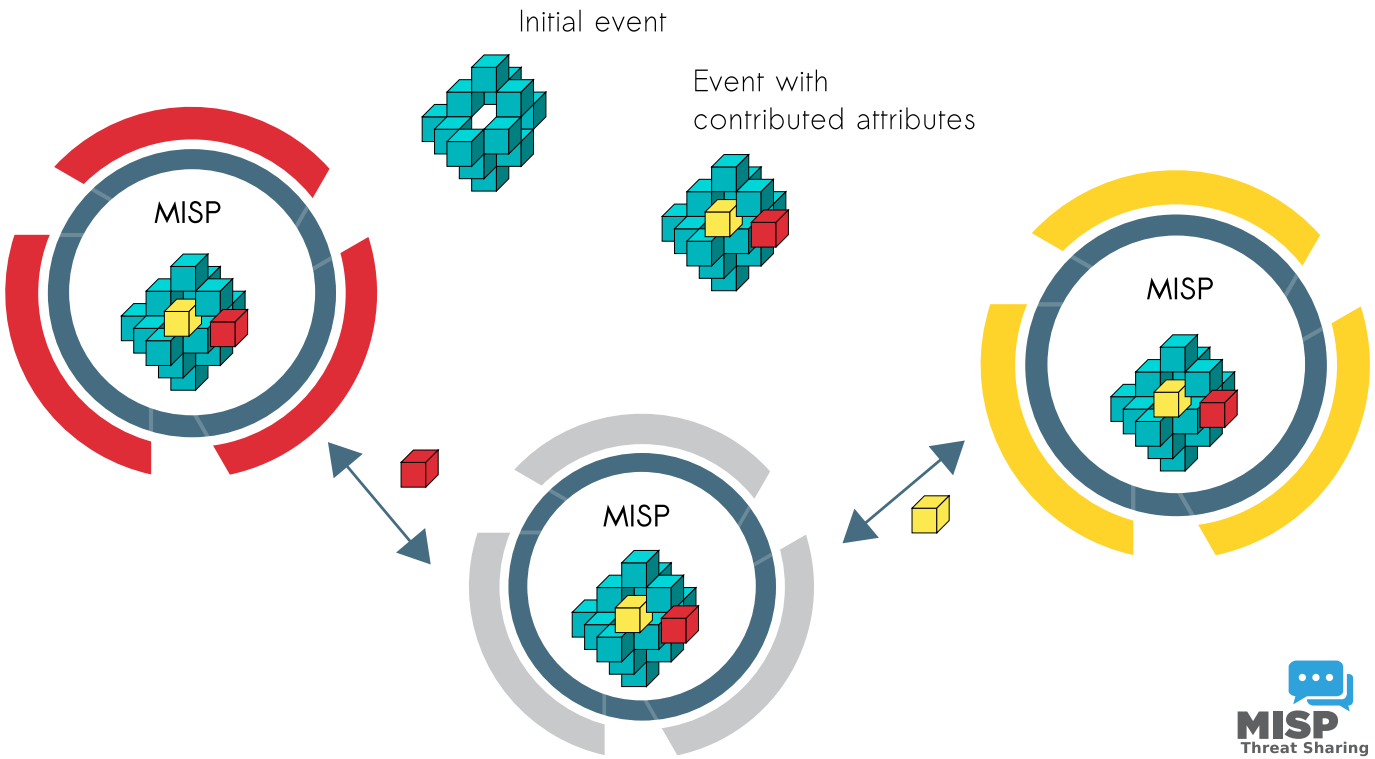
Initial access	Execution	Persistence	Privilege escalation	Defense evasion	Credential access	Discovery	Lateral movement	Collection	Exfiltration	Command and control
Spearphishing Attachment	Scripting	Screensaver	File System Permissions Weakness	Process Hollowing	Securityd Memory	Password Policy Discovery	AppleScript	Data from Information Repositories	Exfiltration Over Alternative Protocol	Standard Application Layer Protocol
Spearphishing via Service	Command-Line Interface	Login Item	AppCert DLLs	Code Signing	Input Capture	System Network Configuration Discovery	Distributed Component Object Model	Data from Removable Media	Exfiltration Over Command and Control Channel	Communication Through Removable Media
Trusted Relationship	User Execution	Trap	Application Shimming	Rootkit	Bash History	Process Discovery	Pass the Hash	Man in the Browser	Data Compressed	Custom Command and Control Protocol
Replication Through Removable Media	Regsvcs/Regasm	System Firmware	Scheduled Task	NTFS File Attributes	Exploitation for Credential Access	Network Share Discovery	Exploitation of Remote Services	Data Staged	Automated Exfiltration	Multi-Stage Channels
Exploit Public-Facing Application	Trusted Developer Utilities	Registry Run Keys / Start Folder	Startup Items	Exploitation for Defense Evasion	Private Keys	Peripheral Device Discovery	Remote Desktop Protocol	Screen Capture	Scheduled Transfer	Remote Access Tools
Spearphishing Link	Windows Management Instrumentation	LC_LOAD_DYLIB Addition	New Service	Network Share Connection Removal	Brute Force	Account Discovery	Pass the Ticket	Email Collection	Data Encrypted	Uncommonly Used Port
Valid Accounts	Service Execution	LSASS Driver	Sudo Caching	Process Doppelgänging	Password Filter DLL	System Information Discovery	Windows Remote Management	Clipboard Data	Exfiltration Over Other Network Medium	Multilayer Encryption
Supply Chain Compromise	CMSTP	Rc.common	Process Injection	Disabling Security Tools	Two-Factor Authentication Interception	System Network Connections Discovery	Windows Admin Shares	Video Capture	Exfiltration Over Physical Medium	Domain Fronting
Drive-by Compromise	Control Panel Items	Authentication Package	Bypass User Account Control	Timestomp	LLMNR/NBT-NS Poisoning	Network Service Scanning	Remote Services	Audio Capture	Data Transfer Size Limits	Data Obfuscation
Hardware Additions	Dynamic Data Exchange	Component Firmware	Extra Window Memory Injection	Modify Registry	Credentials in Files	File and Directory Discovery	Taint Shared Content	Data from Network Shared Drive		Connection Proxy
	Source	Windows Management Instrumentation Event Subscription	Setuid and Setgid	Indicator Removal from Tools	Forced Authentication	Security Software Discovery	Application Deployment Software	Data from Local System		Commonly Used Port
	Space after Filename	Change Default File	Launch Daemon	Hidden Window	Keychain	System Service Discovery	Third-party Software	Automated Collection		Data Encoding

Sharing in MISP

- Sharing via distribution lists - **Sharing groups**
- **Delegation** for pseudo-anonymised information sharing
- **Proposals** and **Extended events** for collaborated information sharing
- Synchronisation, Feed system, air-gapped sharing
- User defined **filtered sharing** for all the above mentioned methods
- Cross-instance information **caching** for quick lookups of large data-sets
- Support for multi-MISP internal enclaves

MISP core distributed sharing functionality

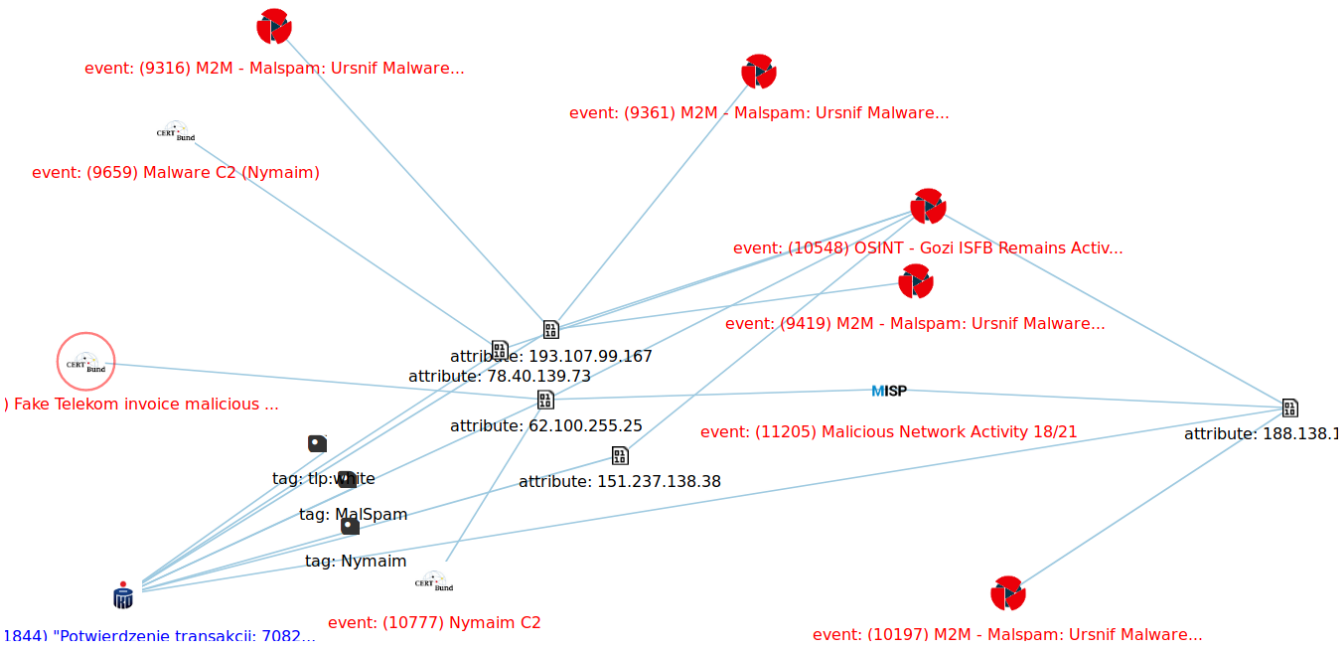
- MISPs' core functionality is sharing where everyone can be a consumer and/or a contributor/producer."
- Quick benefit without the obligation to contribute.
- Low barrier access to get acquainted to the system.



Information quality management

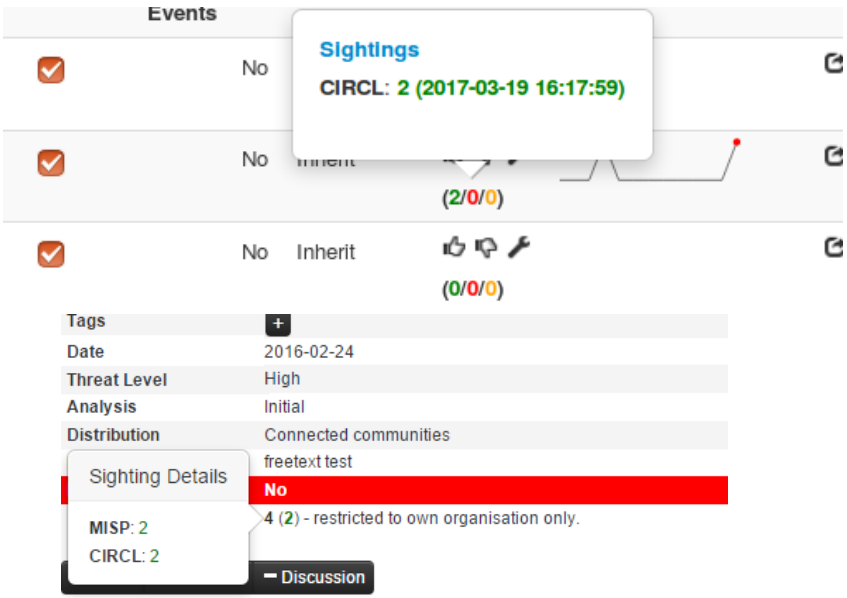
- Correlating data
- Feedback loop from detections via **Sightings**
- **False positive management** via the warninglist system
- **Enrichment system** via MISP-modules
- **Integrations** with a plethora of tools and formats
- Flexible **API** and support **libraries** such as PyMISP to ease integration
- **Timelines** and giving information a temporal context
- Full chain for **indicator life-cycle management**

Correlation features: a tool for analysts



- To **corroborate a finding** (e.g. is this the same campaign?), **reinforce an analysis** (e.g. do other analysts have the same hypothesis?), **confirm a specific aspect** (e.g. are the sinkhole IP addresses used for one campaign?) or just find if this **threat is new or unknown in your community**.

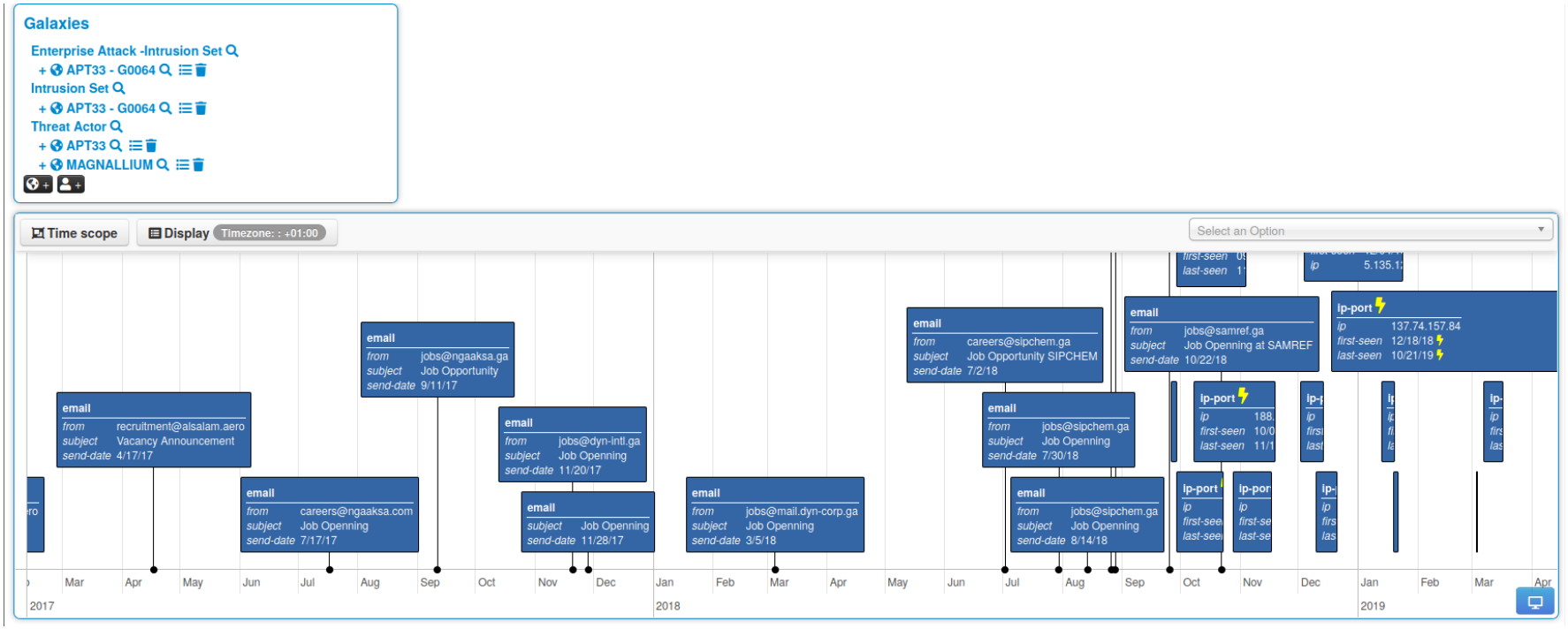
Sightings support



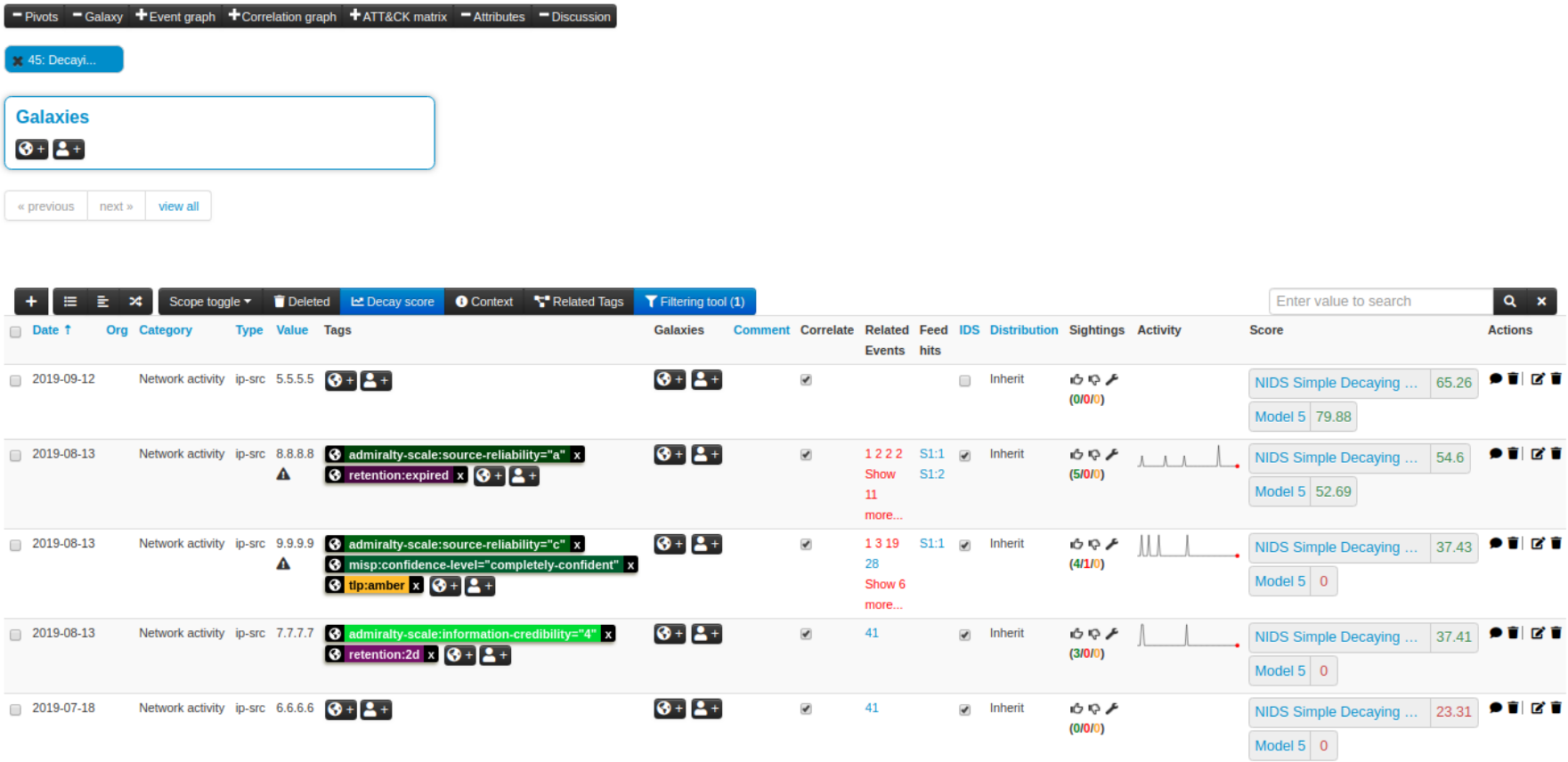
- Has a data-point been **sighted** by me or the community before?
- Additionally, the sighting system supports negative sigthings (FP) and expiration sightings.
- Sightings can be performed via the API or the UI.
- Many use-cases for **scoring indicators** based on users sighting.
- For large quantities of data, **SightingDB** by Devo

Timelines and giving information a temporal context

- Recently introduced **`first_seen`** and **`last_seen`** data points
- All data-points can be placed in time
- Enables the **visualisation** and **adjustment** of indicators timeframes

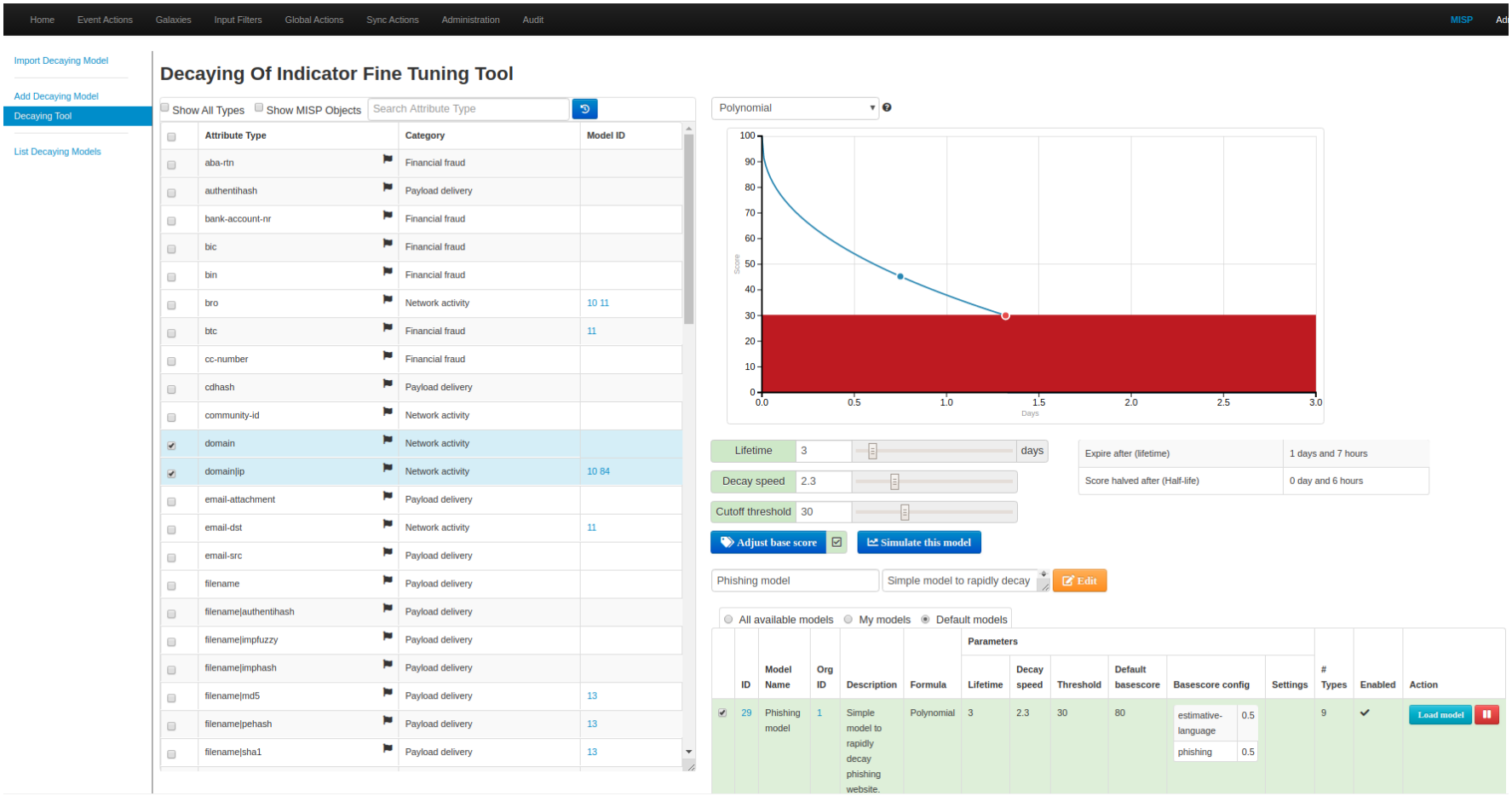


Life-cycle management via decaying of indicators



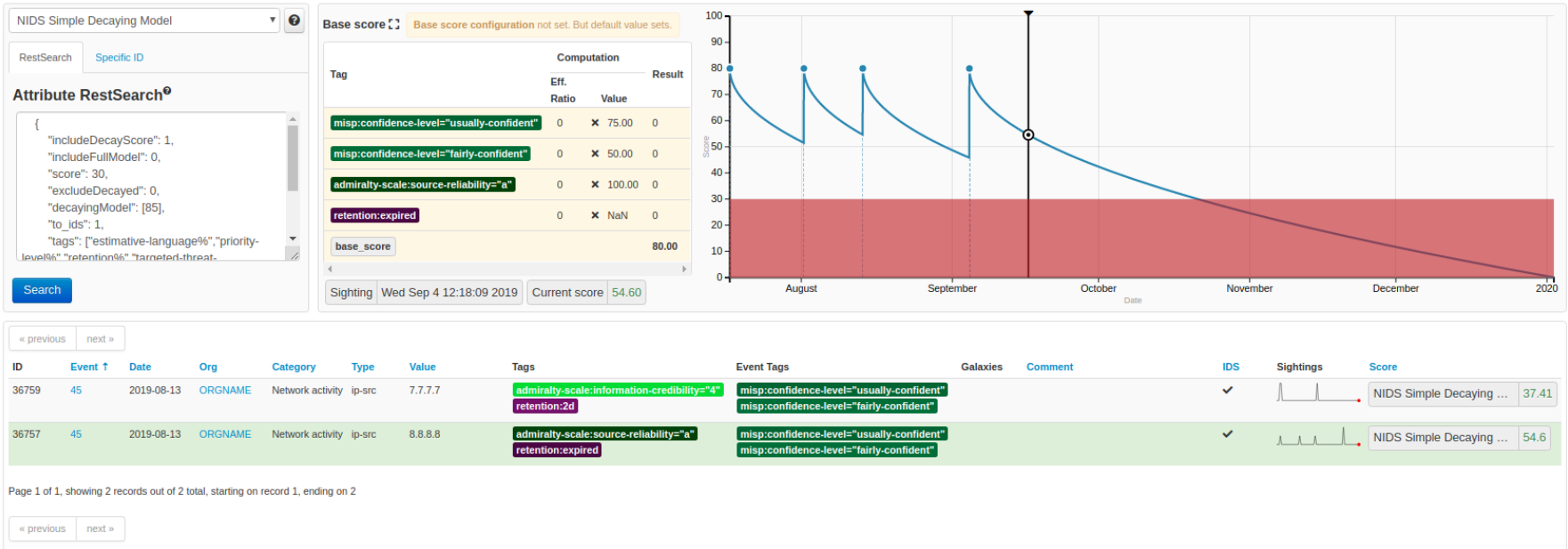
- Decay score toggle button
  - Shows Score for each *Models* associated to the *Attribute* type

Decaying of indicators: Fine tuning tool



Create, modify, visualise, perform mapping

Decaying of indicators: simulation tool



Simulate *Attributes* with different *Models*

Bootstrapping your MISP with data

- We maintain the default CIRCL OSINT feeds (TLP:WHITE selected from our communities) in MISP to allow users to ease their bootstrapping.
- The format of the OSINT feed is based on standard MISP JSON output pulled from a remote TLS/HTTP server.
- Additional content providers can provide their own MISP feeds. (https://botvrij.eu/)
- Allows users to **test their MISP installations and synchronisation with a real dataset**.
- Opening contribution to other threat intel feeds but also allowing the analysis of overlapping data[3].

[3] A recurring challenge in information sharing

Conclusion

- **Information sharing practices come from usage** and by example (e.g. learning by imitation from the shared information).
- MISP is just a tool. What matters is your sharing practices. The tool should be as transparent as possible to support you.
- Enable users to customize MISP to meet their community's use-cases.
- MISP project combines open source software, open standards, best practices and communities to make information sharing a reality.

MISP

INTELLIGENCE SHARING

MISP User Training - General usage of MISP

MISP - Intelligence Sharing

CIRCL / Team MISP Project

http://www.misp-project.org/

Twitter: *@MISPProject*

CRI MISP 2024

MISP - VM

- Credentials
  - MISP admin: admin@admin.test/admin
  - SSH: misp/Password1234
- Available at the following location (VirtualBox and VMWare):
  - https://www.circl.lu/misp-images/latest/

MISP - VM

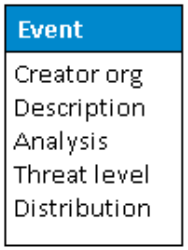
- It is a bit broken.
  - sudo -s
  - cd /var/www/MISP/
  - sudo pear install INSTALL/dependencies/Console_CommandLine/package.xml
  - sudo pear install INSTALL/dependencies/Crypt_GPG/package.xml
  - cd /usr/local/src/misp-modules
  - pip3 install -r REQUIREMENTS
  - pip3 install .
  - reboot

MISP - General Usage

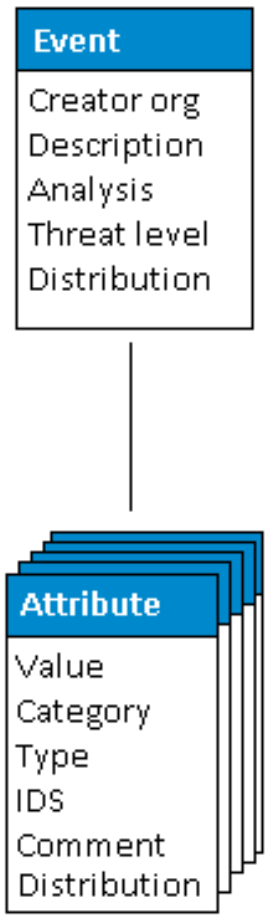
Plan for this part of the training

- Data model
- Viewing data
- Creating data
- Co-operation
- Distribution
- Exports

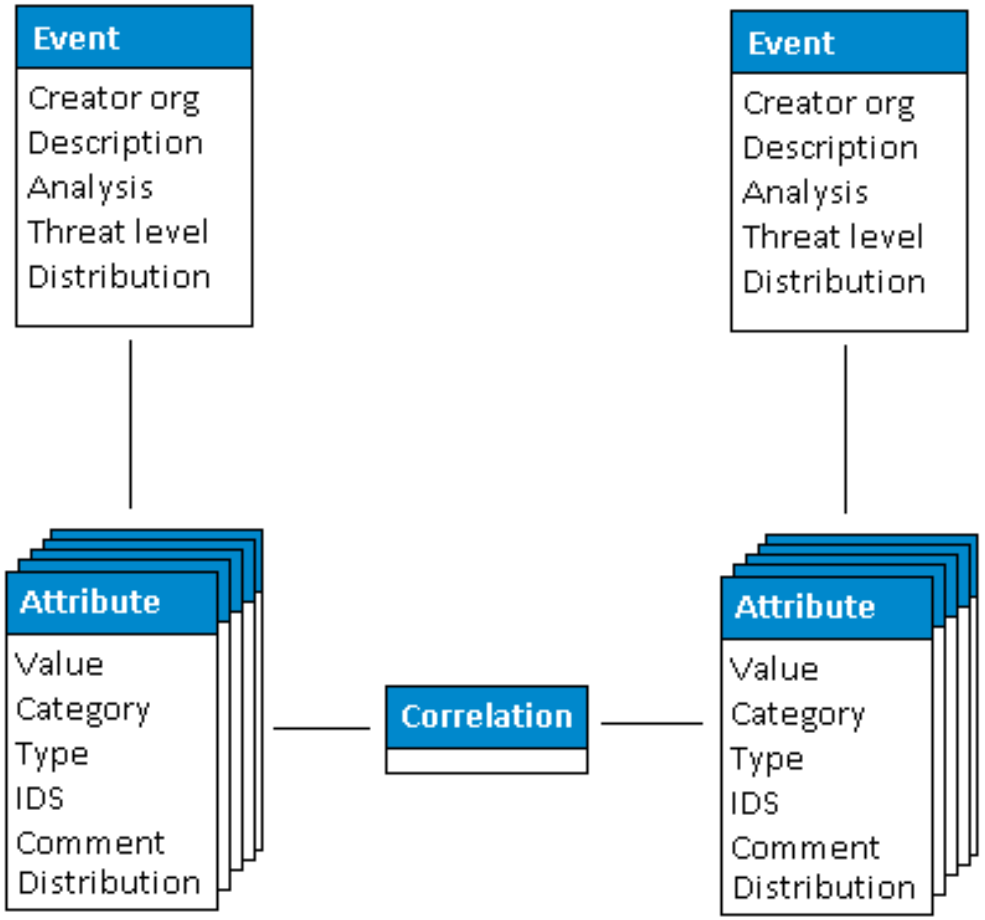
MISP – Event (MISP's basic building block)



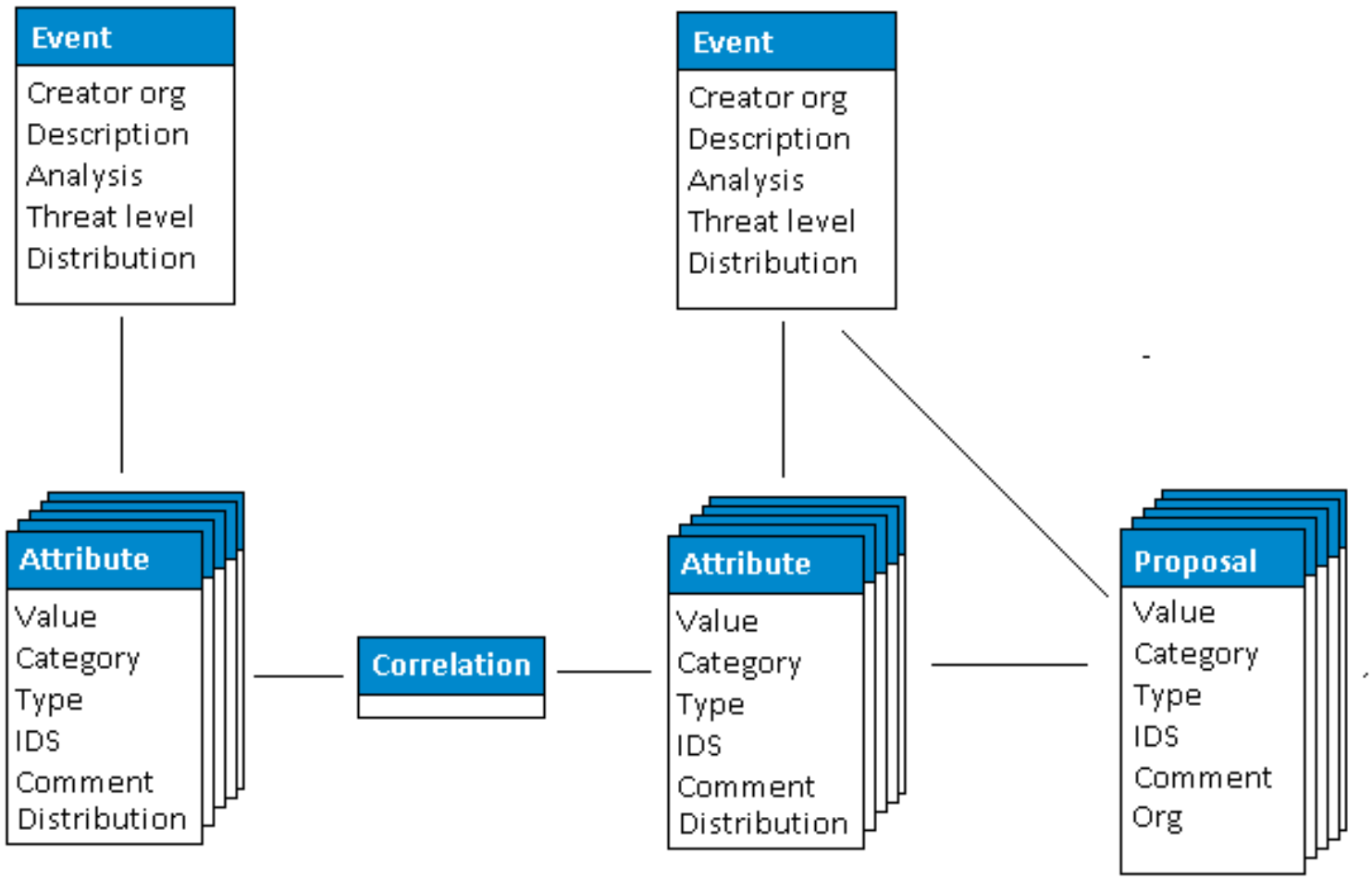
MISP - Event (Attributes, giving meaning to events)



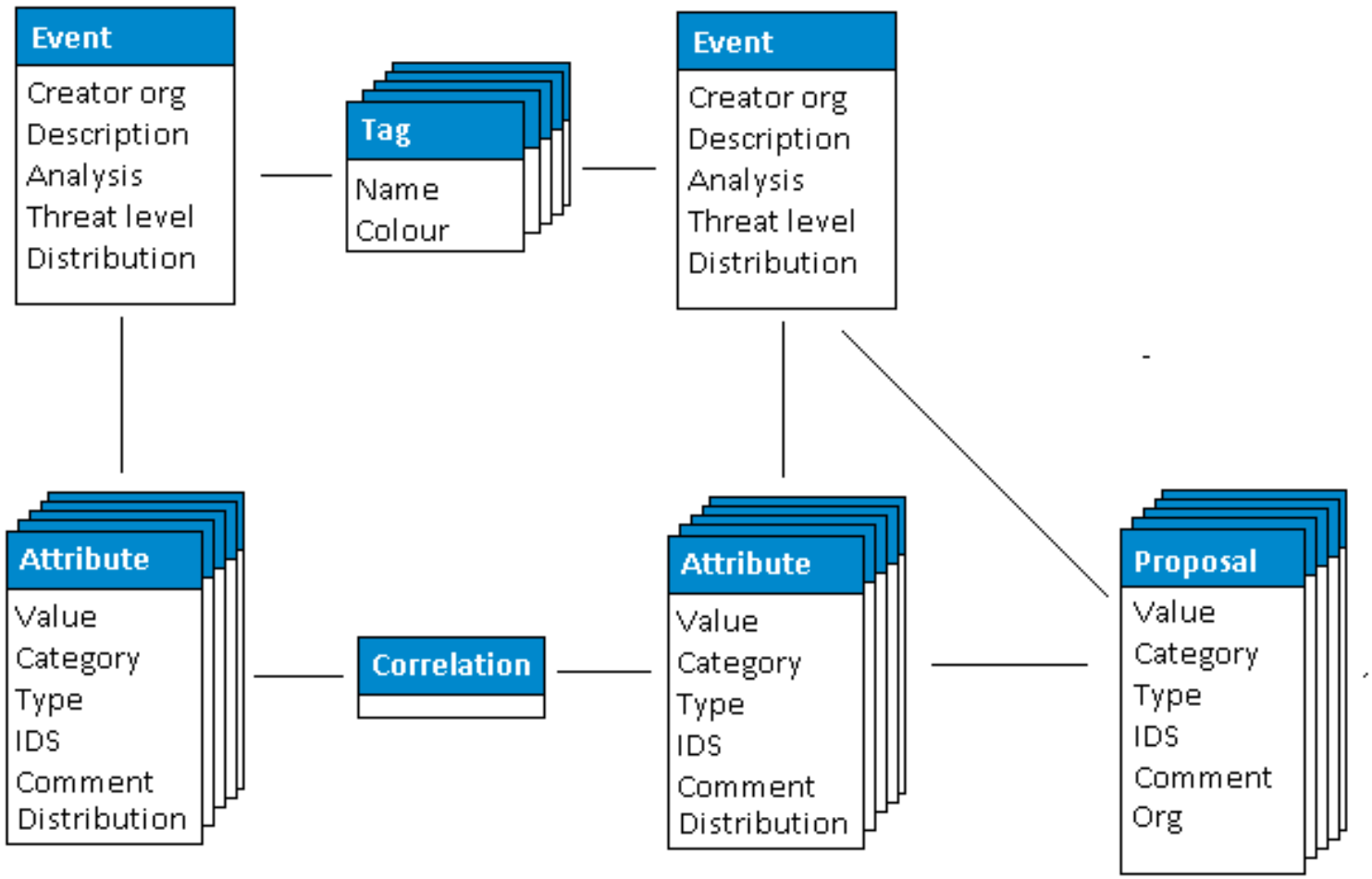
MISP - Event (Correlations on similar attributes)



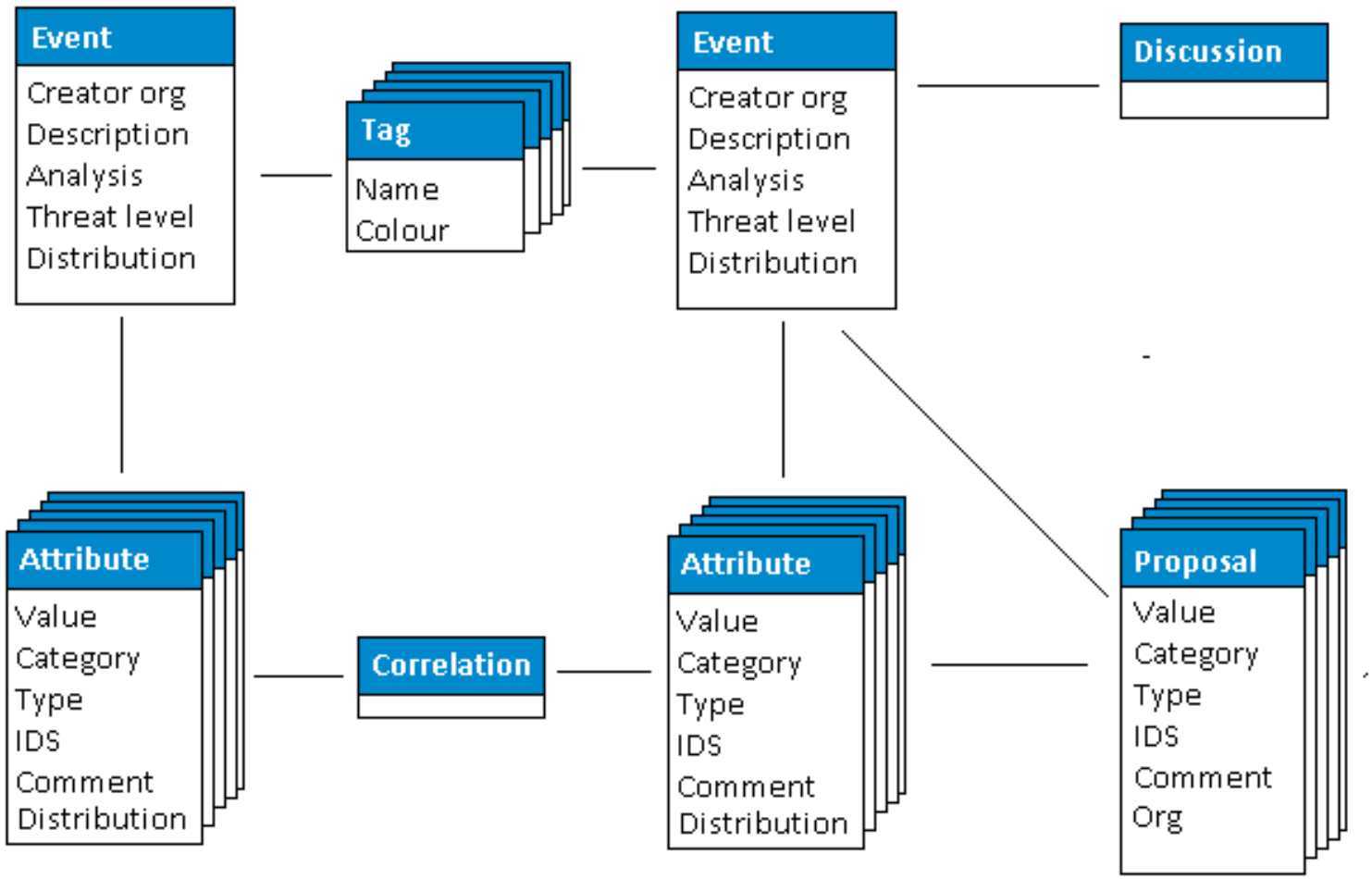
MISP - Event (Proposals)



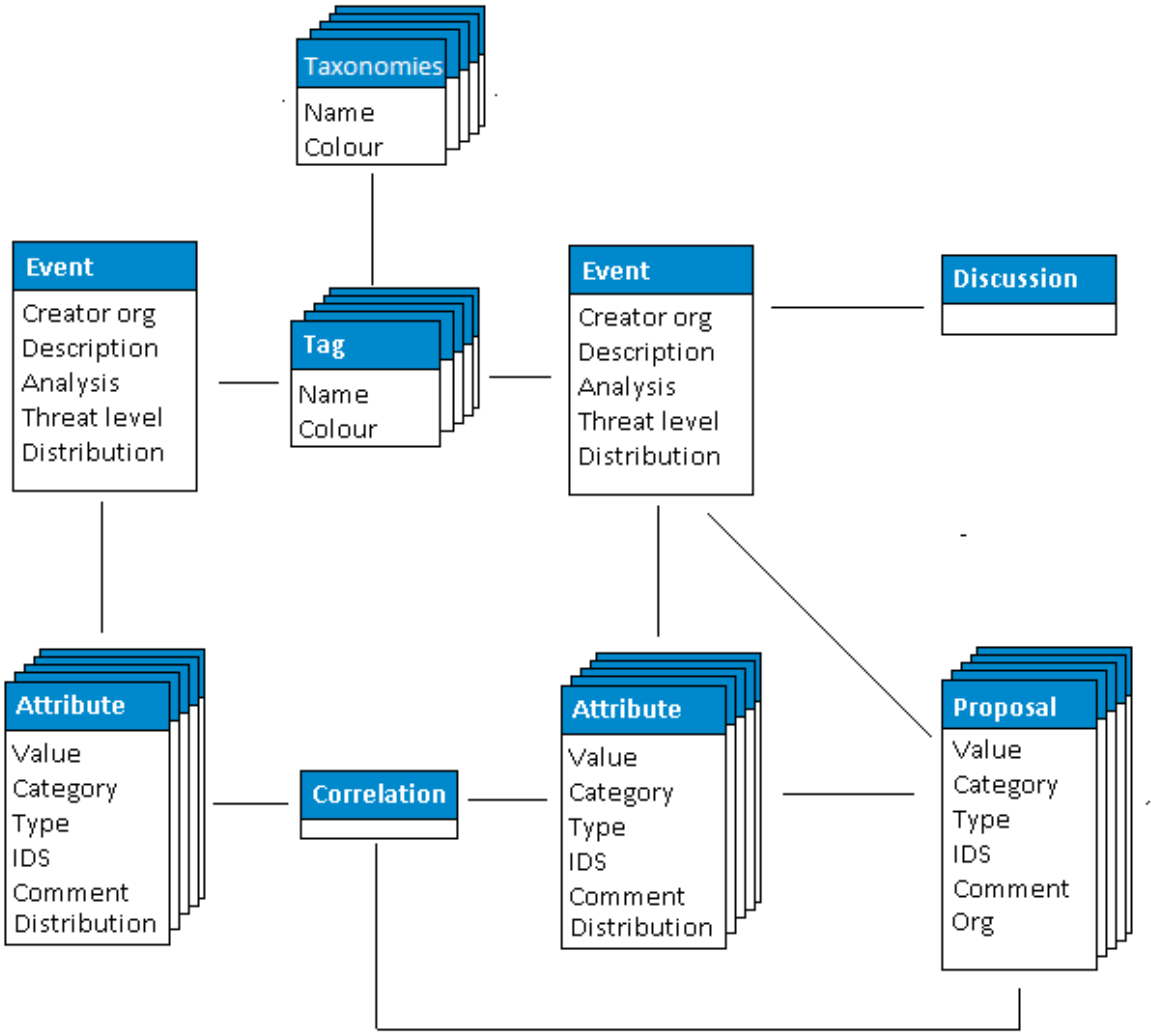
MISP - Event (Tags)



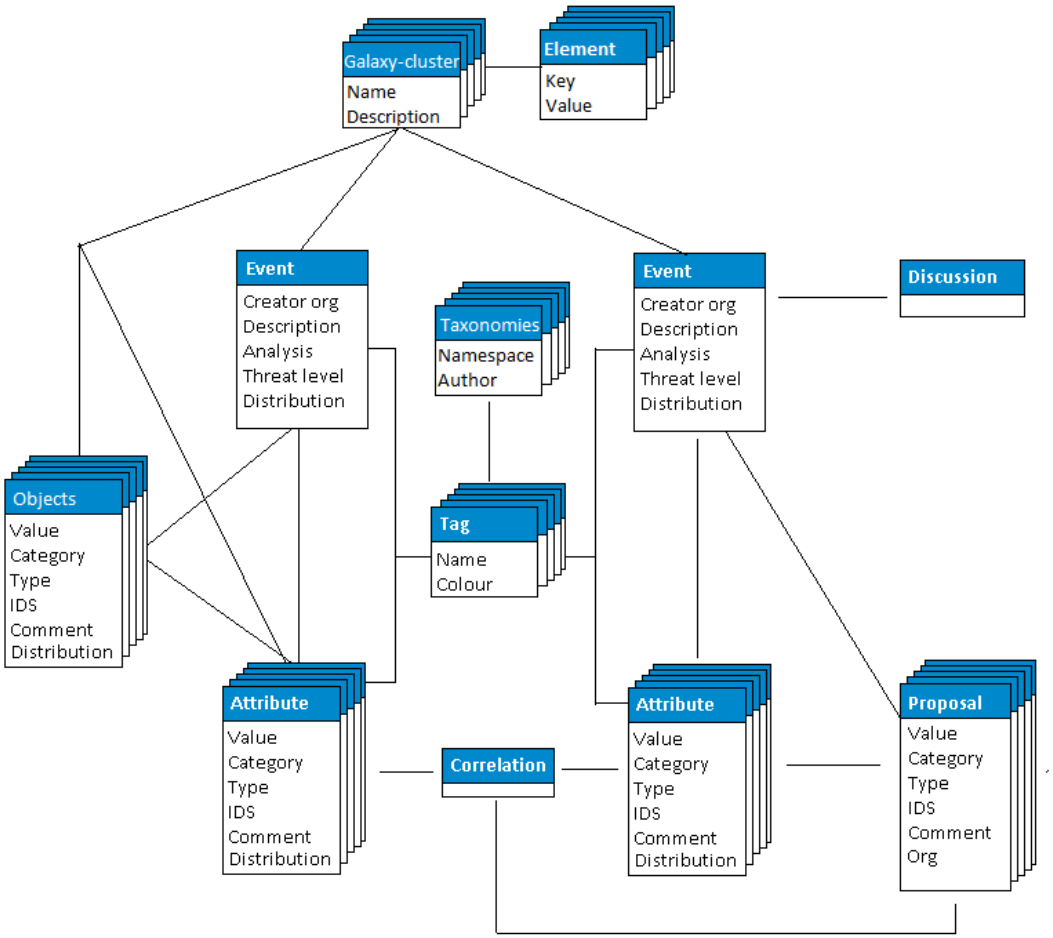
MISP - Event (Discussions)



MISP - Event (Taxonomies and proposal correlations)



MISP - Event (The state of the art MISP datamodel)



MISP - Viewing the Event Index

- Event Index
  - Event context
  - Tags
  - Distribution
  - Correlations
- Filters

MISP - Viewing an Event

- Event View
  - Event context
  - Attributes
    - Category/type, IDS, Correlations
  - Objects
  - Galaxies
  - Proposals
  - Discussions
- Tools to find what you are looking for
- Correlation graphs

MISP - Creating and populating events in various ways (demo)

- The main tools to populate an event
  - Adding attributes / batch add
  - Adding objects and how the object templates work
  - Freetext import
  - Import
  - Templates
  - Adding attachments / screenshots
  - API

MISP - Various features while adding data

- What happens automatically when adding data?
  - Automatic correlation
  - Input modification via validation and filters (regex)
  - Tagging / Galaxy Clusters
- Various ways to publish data
  - Publish with/without e-mail
  - Publishing via the API
  - Delegation

MISP - Using the data

- Correlation graphs
- Downloading the data in various formats
- API (explained later)
- Collaborating with users (proposals, discussions, emails)

MISP - Sync explained (if no admin training)

- Sync connections
- Pull/push model
- Previewing instances
- Filtering the sync
- Connection test tool
- Cherry pick mode

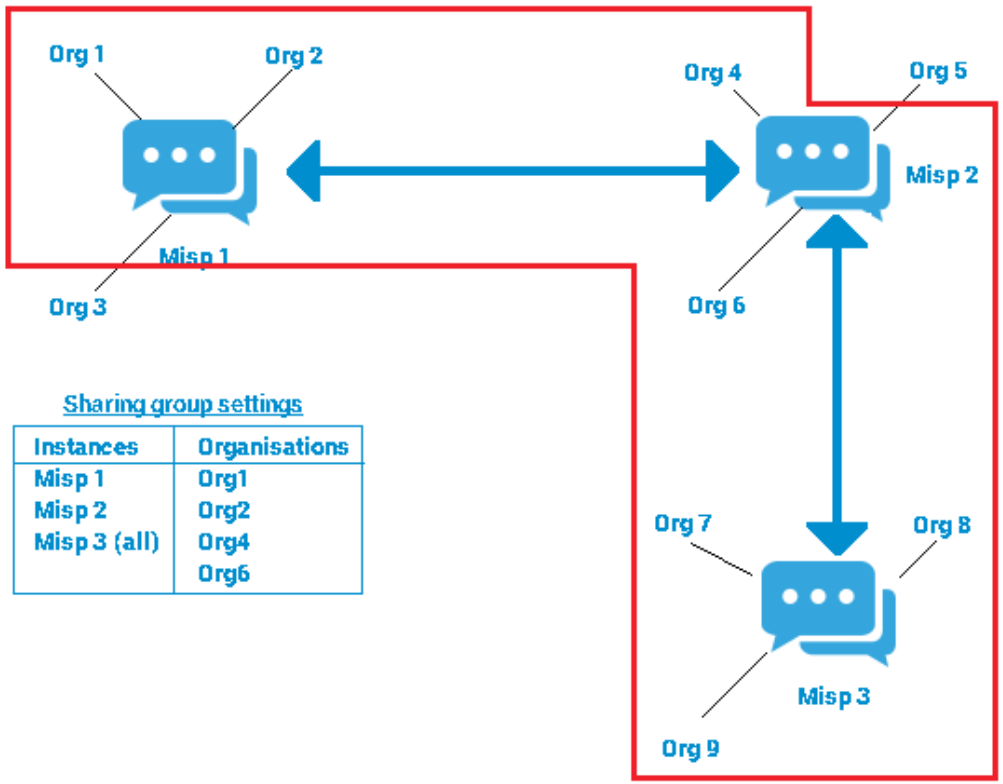
MISP - Feeds explained (if no admin training)

- Feed types (MISP, Freetext, CSV)
- Adding/editing feeds
- Previewing feeds
- Local vs Network feeds

MISP - Distributions explained

- Your Organisation Only
- This Community Only
- Connected Communities
- All Communities
- Sharing Group

MISP - Distribution and Topology



Sharing group settings

Instances	Organisations
Misp 1	Org1
Misp 2	Org2
Misp 3 (all)	Org4
	Org6

MISP - Exports and API

- Download an event
- Quick glance at the APIs
- Download search results
- ReST API and query builder

MISP - Shorthand admin (if no admin training)

- Settings
- Troubleshooting
- Workers
- Logs

MISP

INTELLIGENCE SHARING

MISP Training: MISP Deployment and Integration

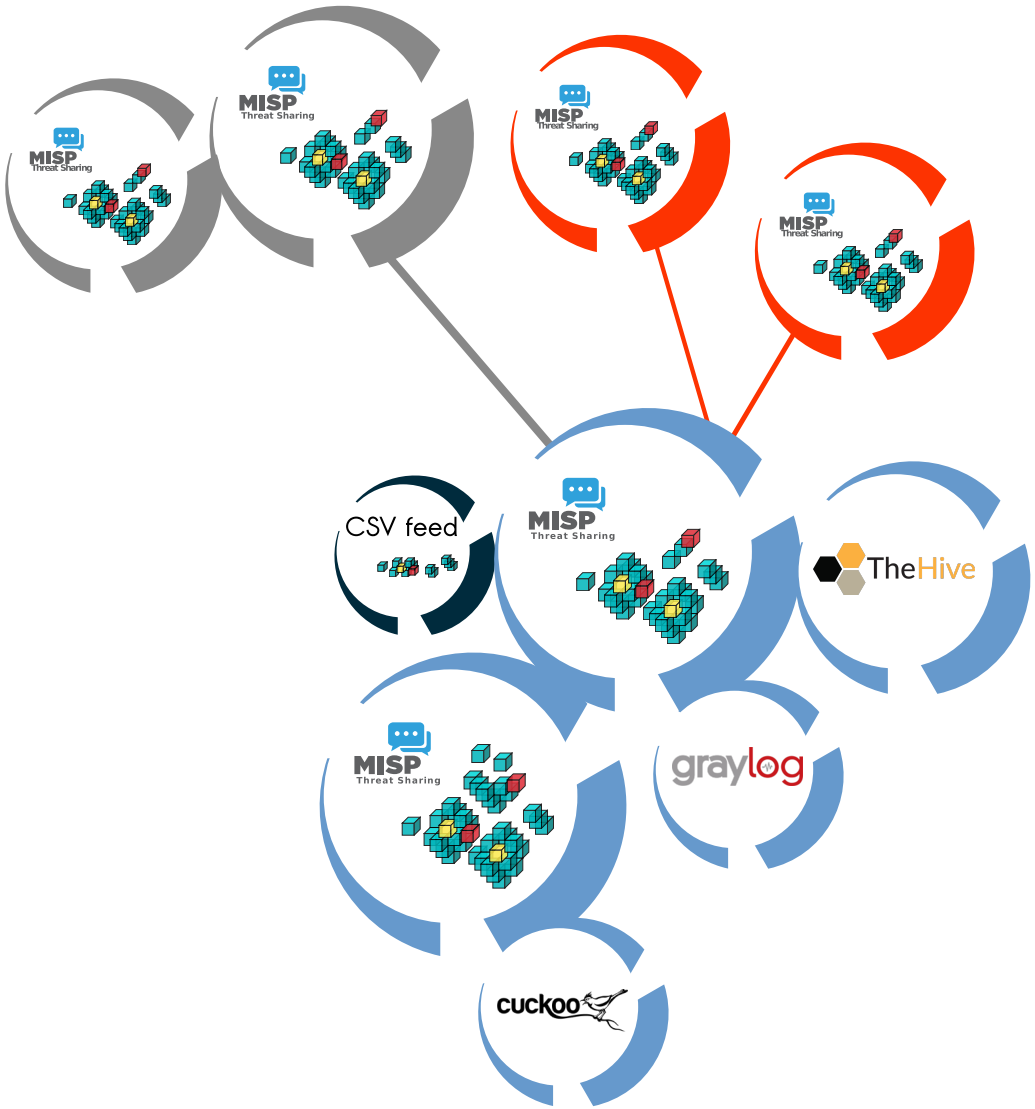
CIRCL / Team MISP Project

http://www.misp-project.org/

Twitter: *@MISPProject*

CRI MISP 2024

A Common Integration



Recommended MISP Setup

- Provisioning your MISP infrastructure depends heavily on the **number of attributes/events** (whether your dataset is below or above 50 million attributes).
- Number of MISP instances and the overall design depends on the following factors:
  - Is your community private? Are you gathering MISP events from other communities? Are you **publishing events to external** (trusted/untrusted) communities.
  - Do you plan to have **automatic tools** (e.g. sandbox analysis or low-value information needing correlation or an analyst workbench) feeding MISP?

Vendors and Formats

- There is **a jungle of formats** with some vendors having little to no interest in keeping their users autonomous.
- Attacks and threats require a **dynamic format** to be efficiently shared (e.g. from financial indicators to personal information).
- **Review your current list of formats/vendors** to ensure a limited loss of information, especially when exporting from MISP to other formats (e.g. STIX not supporting financial indicators or taxonomies/galaxies).

Use case: Normalizing OSINT and Private Feeds

- Normalizing external input and feed into MISP (e.g. feed importer).
- Comparing feeds before import (how many similarities? false-positives?).
- Evaluating quality of information before import (warning-list lookup at feed evaluation).

Connecting Devices and Tools to MISP

- One of the main goals of MISP is to feed protective or detection tools with data
  - IDSes / IPSes (e.g. Suricata, Bro, Snort format as included in Cisco products)
  - SIEMs (e.g. CEF, CSV or real-time ZMQ pub-sub or Sigma)
  - Host scanners (e.g. OpenIOC, STIX, yara rule-set, CSV)
  - Various analysis tools (e.g. Maltego)
  - DNS policies (e.g. RPZ)
- Various ways of exporting this data (downloads of the selected data, full exports, APIs)
- The idea was to leave the selection process of the subset of data to be pushed to these up to the user using APIs.

SIEM and MISP Integration

- SIEMs and MISP can be integrated with different techniques depending on the processes at your SOC or IR:
  - Pulling events (via the API) or indicator lists at **regular intervals** in a given time frame to perform lookups.
  - Subscribing to the MISP ZMQ **pub-sub channel** to directly get the published events and use these in a lookup process.
  - **Lookup expansion module** in MISP towards the SIEM to have a direct view of the attributes matched against the SIEM.
- The above options can be combined, depending on your organisation or requirements to increase coverage and detection.

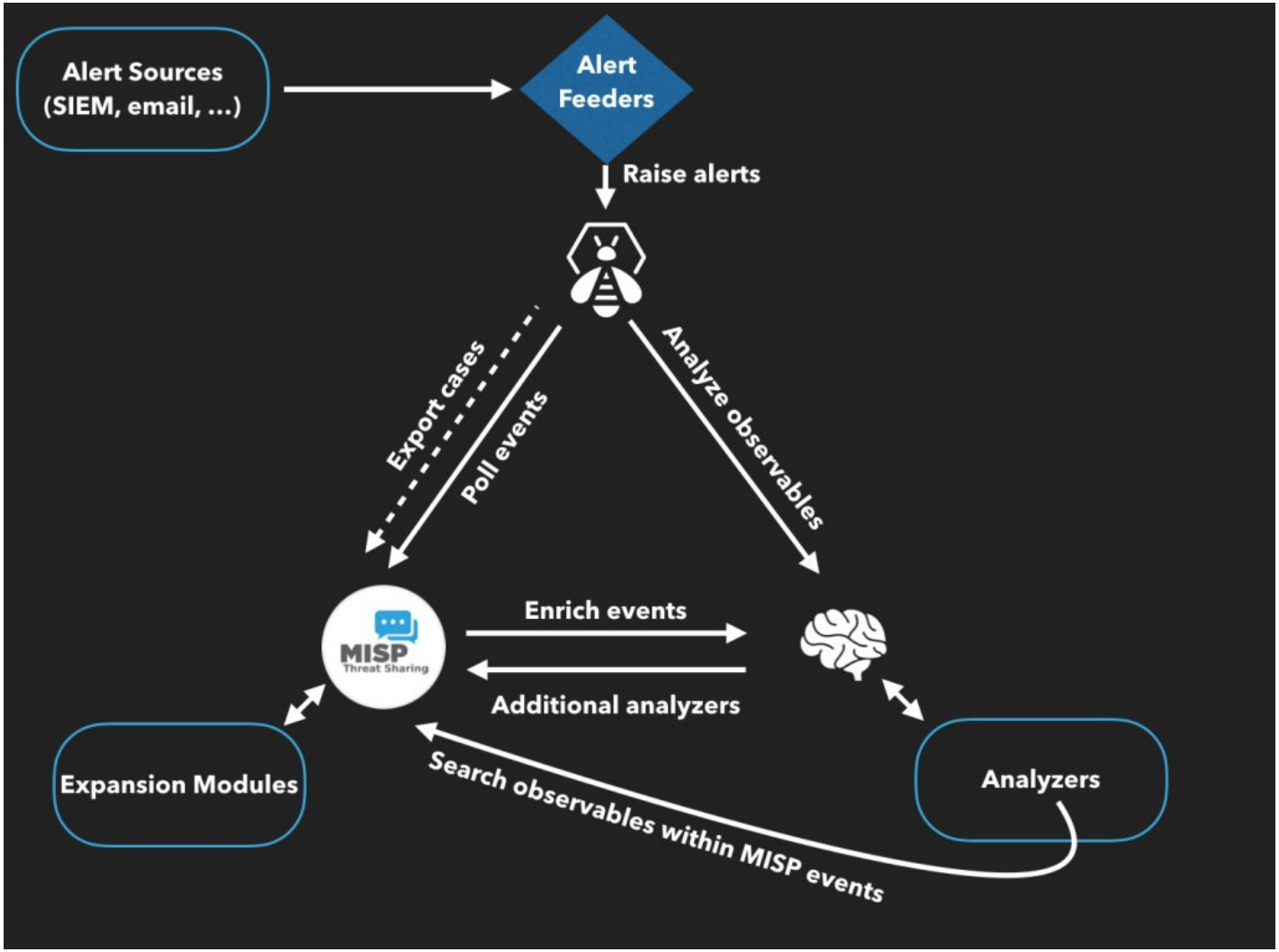
ZMQ integration: misp-dashboard

- A dashboard showing live data and statistics from the ZMQ pub-sub of one or more MISP instances.
- Building **low-latency software** by consuming pub-sub channel provides significant advantages over standard API use.
- Process information in **real-time** when it's updated, created, published or gathered in MISP.
- Demo!

New integrations: IR and threat hunting using MISP

- Close co-operation with **the Hive project** for IR
  - Interact with MISP directly from the Hive
  - Use both the MISP modules and the **Cortex** analysers in MISP or the Hive directly
- Using MISP to support your threat hunting via **McAfee OpenDXL**
- (https://securingtomorrow.mcafee.com/business/optimize-operations/expanding-automated-threat-hunting-response-open-

The Hive integration



Reporting Back from your Devices, Tools or Processes

As **Sightings** can be positive, negative or even based on expiration, different use cases are possible:

- **Sightings** allow users to notify a MISP instance about the activities related to an indicator.
- Activities can be from a SIEM (e.g. Splunk lookup validation or **false-positive feedback**), a NIDS or honeypot devices[1].
- Sighting can affect the API to limit the NIDS exports and improve the NIDS rule-set directly.

[1] https://www.github.com/MISP/misp-sighting-tools

Q&A

- info@circl.lu (if you want to join the CIRCL MISP sharing community)
- https://github.com/MISP/ - http://www.misp-project.org/
- We welcome any contributions to the project, be it pull requests, ideas, github issues,...

MISP

INTELLIGENCE SHARING

mail_to_misp

Connect your mail infrastructure to MISP to create events based on the information contained within mails

CIRCL / Team MISP Project

http://www.misp-project.org/

Twitter: *@MISPProject*

CRI MISP 2024

Context

- You receive emails with IoC's inside
- How to create an event out of it?
- Create event manually and copy paste
- $\rightarrow$ This works once or twice
- Forwarding the email would be nice
- $\rightarrow$ mail_to_misp

Features: Email handling

- Extraction of URLs and IP addresses and port numbers
- Extraction of hostnames from URLs
- Extraction of hashes (MD5, SHA1, SHA256)
- DNS expansion
- Subject filters
- Refanging of URLs ('hxxp://...')
- ... and more

Features: Support MISP features

- Add tags automatically
- Ignore 'whitelisted' domains
- Configurable list of attributes not to enable the IDS flag
- DNS expansion
- Automatically create 'external analysis' links based on filter list (e.g. VirusTotal, malwr.com)
- Automatically filter out attributes that are on a server side warning list
- Support for value sighting
- ... and more

Implementation

- Legacy
  - Email → Apple Mail → Mail rule → AppleScript → AppleScript → mail_to_misp → PyMISP → MISP
  - Email → Thunderbird → Mail rule → filterscript → thunderbird_wrapper → mail_to_misp → PyMISP → MISP
- Postfix and others
  - Email → mail_to_misp

Installation

- mail_to_misp
  1. `git clone git://github.com/MISP/mail_to_misp.git`
  2. Install dependencies - See Github site

- MTA (Postfix or alike)
  1. Setup a new email address in the aliases file (e.g. /etc/aliases)
     `misp_handler:  "|/path/to/mail_to_misp.py -"`
  2. Rebuild the DB
     `sudo newaliases`
  3. Configure mail_to_misp_config.py

```
misp_url = 'http://127.0.0.1/'
misp_key = 's5jPWCIud36Z8XHgsiCVI7SaL1XsMTyfEsN45tTe'
misp_verifycert = True
body_config_prefix = 'm2m'
...
...
```

Exercise: mail_2_misp.py

- Bonus: https://github.com/MISP/mail_to_misp_test

```
./mail_to_misp.py -r mail_to_misp_test/simple_forward.eml
```

- Bonus: Fake-SMTPD spamtrap

```
./fake_smtp.py

telnet 127.0.0.1 2526
    Trying 127.0.0.1...
    Connected to 127.0.0.1.
    Escape character is '^]'.
    220 misp Python SMTP 1.1
    helo misp
    250 misp
    mail from: mikel
    250 OK
    rcpt to: m2m
    250 OK
    data
    354 End data with <CR><LF>.<CR><LF>
```

MISP

INTELLIGENCE SHARING

MISP User Training - Administration of MISP 2.4

MISP - Intelligence Sharing

CIRCL / Team MISP Project

http://www.misp-project.org/

Twitter: *@MISPProject*

CRI MISP 2024

MISP - VM

- VM can be downloaded at https://www.circl.lu/misp-training/
- Credentials
  - MISP admin: admin@admin.test/admin
  - SSH: misp/Password1234
- 2 network interfaces
  - NAT
  - Host only adapter
- Start the enrichment system by typing:
  - cd /home/misp/misp-modules/bin
  - python3 misp-modules.py

MISP - Administration

- Plan for this part of the training
  - User and Organisaton administration
  - Sharing group creation
  - Templates
  - Tags and Taxonomy
  - Whitelisting and Regexp entries
  - Setting up the synchronisation
  - Scheduled tasks
  - Feeds
  - Settings and diagnostics
  - Logging
  - Troubleshooting and updating

MISP - Creating Users

- Add new user (andras.iklody@circl.lu)
- NIDS SID, Organisation, disable user
- Fetch the PGP key
- Roles
  - Re-using standard roles
  - Creating a new custom role
- Send out credentials

MISP - Creating Organisations

- Adding a new organisation
- UUID
- Local vs External organisation
- Making an organisation self sustaining with Org Admins
- Creating a sync user

MISP - Sharing groups

- The concept of a sharing group
- Creating a sharing group
- Adding extending rights to an organisation
- Include all organisations of an instance
- Not specifying an instance
- Making a sharing group active
- Reviewing the sharing group

MISP - Templates

- Why templating?
- Create a basic template
- Text fields
- Attribute fields
- Attachment fields
- Automatic tagging

MISP - Tags and Taxonomies

- git submodule init && git submodule update
- Loading taxonomies
- Enabling taxonomies and associated tags
- Tag management
- Exportable tags

MISP - Object Templates

- git submodule init && git submodule update
- Enabling objects (and what about versioning)

MISP - Whitelisting, Regexp entries, Warninglists

- Block from exports - whitelisting
- Block from imports - blacklisting via regexp
- Modify on import - modification via regexp
- Maintaining the warninglists

MISP - Setting up the synchronisation

- Requirements - versions
- Pull/Push
- One way vs Two way synchronisation
- Exchanging sync users
- Certificates
- Filtering
- Connection test tool
- Previewing an instance
- Cherry picking and keeping the list updated

MISP - Scheduled tasks

- How to schedule the next execution
- Frequency, next execution
- What happens if a job fails?

MISP - Setting up the synchronisation

- MISP Feeds and their generation
- PyMISP
- Default free feeds
- Enabling a feed
- Previewing a feed and cherry picking
- Feed filters
- Auto tagging

MISP - Settings and diagnostics

- Settings
  - Settings interface
  - The tabs explained at a glance
  - Issues and their severity
  - Setting guidance and how to best use it

MISP - Settings and diagnostics continued

- Basic instance setup
- Additional features released as hotfixes
- Customise the look and feel of your MISP
- Default behaviour (encryption, e-mailing, default distributions)
- Maintenance mode
- Disabling the e-mail alerts for an initial sync

MISP - Settings and diagnostics continued

- Plugins
  - Enrichment Modules
  - RPZ
  - ZeroMQ

MISP - Settings and diagnostics continued

- Diagnostics
  - Updating MISP
  - Writeable Directories
  - PHP settings
  - Dependency diagnostics

MISP - Settings and diagnostics continued

- Workers
  - What do the background workers do?
  - Queues
  - Restarting workers, adding workers, removing workers
  - Worker diagnostics (queue size, jobs page)
  - Clearing worker queues
  - Worker and background job debugging

MISP - Settings and diagnostics continued

- Seeking help
  - Dump your settings to a file!
  - Make sure to sanitise it
  - Send it to us together with your issue to make our lives easier
  - Ask Github (https://github.com/MISP/MISP)
  - Have a chat with us on gitter (https://gitter.im/MISP/MISP)
  - Ask the MISP mailing list
  - If this is security related, drop us a PGP encrypted email to mailto:info@circl.lu

MISP - Logging

- Audit logs in MISP
- Enable IP logging / API logging
- Search the logs, the fields explained
- External logs
  - /var/www/MISP/app/tmp/logs/error.log
  - /var/www/MISP/app/tmp/logs/resque-worker-error.log
  - /var/www/MISP/app/tmp/logs/resque-scheduler-error.log
  - /var/www/MISP/app/tmp/logs/resque-[date].log
  - /var/www/MISP/app/tmp/logs/error.log
  - apache access logs

MISP - Updating MISP

- git pull
- git submodule init && git submodule update
- reset the permissions if it goes wrong according to the INSTALL.txt
- when MISP complains about missing fields, make sure to clear the caches
  - in /var/www/MISP/app/tmp/cache/models remove myapp*
  - in /var/www/MISP/app/tmp/cache/persistent remove myapp*
- No additional action required on hotfix level
- Read the migration guide for major and minor version changes

MISP - Administrative tools

- Upgrade scripts for minor / major versions
- Maintenance scripts

MISP

INTELLIGENCE SHARING

Taxonomies and tagging in MISP 2.5

Classify consistently, filter precisely, share deliberately

CIRCL / MISP Project

https://www.misp-project.org/

Reference snapshot: 29 October 2026

Make four decisions with every classification

1. Which vocabulary expresses the assessment?
2. Does it describe the event or one attribute?
3. Is it shared context or a local assessment?
4. How will people and tools use it?

By the end

Apply canonical tags, interpret filters and review sharing intent.

Four concepts that answer different questions

Concept	Purpose	Example
Taxonomy	Shared classification vocabulary.	Reliability, confidence, handling.
Tag	A label attached to data.	`tlp:amber`
Galaxy cluster	A described entity or concept with relationships.	A technique or threat actor.
Warninglist	Match values needing extra scrutiny.	Special-use IP ranges.

Read the machine tag, then its explanation

Canonical machine value

```
admiralty-scale:source-reliability="b"
```

Part	Meaning

`admiralty-scale`
`source-reliability`
`"b"`
	Namespace: which vocabulary?
	Predicate: which question?
	Value: the selected answer. Expanded label: Usually reliable.

Some tags have no value: `tlp:amber`.

Separate source reliability from information credibility

What do we know about the source?

`admiralty-scale:source-reliability="b"`
Usually reliable.

What supports this particular information?

`admiralty-scale:information-credibility="2"`
Probably true.

Choose and prepare the vocabulary

Use the TLP 2.0 labels deliberately

Machine tag	Disclosure boundary, in brief
`tlp:clear`	No limit on disclosure, subject to applicable rules.
`tlp:green`	Within the community; no public channels.
`tlp:amber`	Need-to-know within the organisation and its clients.
`tlp:amber+strict`	Need-to-know within the organisation only.
`tlp:red`	Individual recipients only; no further disclosure.

FIRST TLP 2.0: `https://www.first.org/tlp/`

A handling label does not set access controls

Communicate intent

TLP tells recipients the permitted disclosure boundary.

Configure access

Distribution, sharing groups and user permissions govern access in MISP.

Review both before publishing or synchronising.

Prepare a small, useful set of taxonomies

1. Update the taxonomy files in the deployment.
2. Update the taxonomy library in MISP.
3. Enable the chosen taxonomy.
4. Enable all or selected tags from that vocabulary.

Check the result

Find the exact tag in the event or attribute tag picker.

Use governance controls with their actual scope

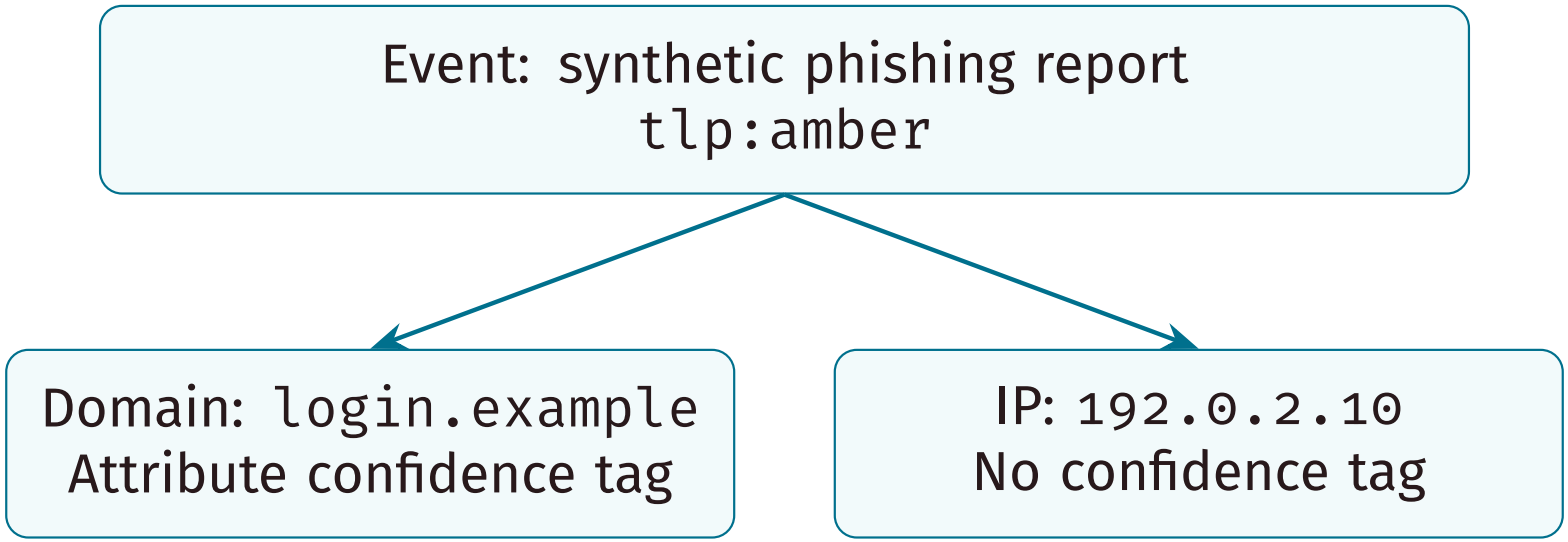
Control	What to expect
Exclusive	A taxonomy or predicate can disallow conflicting choices.
Required	An enabled taxonomy can be required in the UI publish check.
Highlighted	Surface selected taxonomy tags prominently in event views.

For automation

Validate classification explicitly; a UI check is not an API-wide gate.

Apply tags at the right scope

Event context and attribute context are separate



Event tags are not copied onto each stored attribute.

Choose shared context or a local assessment

Shared attachment

Context intended to travel with the data, subject to export and sync rules.

Local attachment

An instance’s assessment; excluded from ordinary external synchronisation.

Local does not mean private to one user.

Three tag settings that are easy to confuse

Setting	Meaning
`local_only`	The tag can only be attached as a local tag.
`exportable`	Whether the tag is eligible for export; other rules still apply.
Organisation / user restriction	Who may apply the tag, not who may read the event.

Inspect tag settings and the attachment’s local flag separately.

Use collections to apply a consistent starting set

- A tag collection groups labels for reuse.
- Select a collection from the tag picker.
- Review its meaning and scope before applying it.
- An administrator can configure a default event collection.

A starting point still needs review

A preset cannot know the source’s handling restrictions.

Keep the classification's meaning in context

- Event reports can also carry tags in the reviewed 2.5 source.
- A tag attachment can carry a relationship type.
- Use that annotation to explain how a label relates to its target.
- Keep the canonical taxonomy value unchanged.

An analyst’s tagging sequence

1. Read the evidence and the source’s handling instructions.
2. Choose the event or the specific attribute.
3. Select canonical tags and the intended attachment scope.
4. Review conflicts, distribution and the resulting export.

Worked example: classify a synthetic report

Event

```
tlp:amber
admiralty-scale:source-reliability="b"
```

Domain attribute: `login.example`

```
misp:confidence-level="fairly-confident"
```

IP `192.0.2.10`: no attribute confidence tag.
Event: organisation only; unpublished.

Find, process and review the data

Make filter logic explicit

Operator	Meaning within the selected endpoint and scope
`OR`	Match at least one of these labels.
`AND`	Match every required label.
`NOT`	Exclude data carrying a listed label.

Before interpreting a result

Check the endpoint, attachment scope, permissions and other filters.

Search for attributes with both classifications

`POST /attributes/restSearch`

```
{
  "returnFormat": "json",
  "eventid": "7fb18198-5f8f-4fc3-8ea7-d2d791c2c0be",
  "tags": {
    "AND": [
      "tlp:amber",
      "misp:confidence-level=\"fairly-confident\""
    ]
  },
  "includeEventTags": true
}
```

The domain matches event handling *and* attribute confidence.

A matching attribute can use event context

- Attribute tag searches can also match event tags.
- Request event tags explicitly when you need that context.
- Keep event and attribute provenance clear in consumers.
- Check returned values, not just the number of events.

In the example

`includeEventTags: true` adds context to the response; it does not change stored tags.

Tags help route work; configuration performs it

- Use labels such as `workflow:todo="review"` to coordinate work.
- Configure workflow conditions and actions for automation.
- Configure sync push/pull rules for the intended partner.
- Test a permitted and a rejected case before relying on the rule.

A warninglist hit is a reason to investigate

- Compare an observable against enabled reference lists.
- Consider shared infrastructure, reserved values or common mistakes.
- Read the matching list and why the value appears in it.
- Keep legitimate evidence even when it should not become a detection.

Avoid a false conclusion

A match does not prove that the observable is harmless.

Demonstrate a warning without using a live target

Training value

`192.0.2.10` falls in `192.0.2.0/24`.

- The reviewed RFC 5735 list contains this documentation range.
- Its matching types include `ip-dst`.
- Enable that list before expecting a hit in the exercise.

Choose warning visibility or result suppression

`POST /attributes/restSearch`

Option	Behavior in the reviewed attribute search
`includeWarninglistHits`	Set to `true`: return matches with warning details.
`enforceWarninglist`	Set to `true`: omit matches from enabled applicable lists.

Compare the two supplied request files and inspect the actual response.

Diagnose the cause before creating another tag

Symptom	Check first
Tag missing in picker	Taxonomy, enabled terms, hidden tags and assignment restrictions.
Duplicate-looking labels	Canonical spelling, case and expanded display text.
Label missing downstream	Local/exportable flags, sync route and export options.
Unexpected search results	Endpoint, event versus attribute context and Boolean groups.

Maintain and practise

A taxonomy definition starts with a clear question

```
{
  "namespace": "training-classification",
  "description": "Review state for a local exercise.",
  "version": 1,
  "predicates": [{
    "value": "review",
    "expanded": "Review state",
    "exclusive": true
  }]
}
```

This header is extended by values in the complete example file.

Values connect machine codes to readable labels

```
"values": [{
  "predicate": "review",
  "entry": [
    {"value": "draft", "expanded": "Draft"},
    {"value": "reviewed", "expanded": "Reviewed"}
  ]
}]
```

Resulting machine tag

```
training-classification:review="reviewed"
```

Contribute a vocabulary that others can maintain

1. Establish a gap and cite authoritative definitions.
2. Reuse stable codes and document the intended scope.
3. Validate the schema, predicate links and generated manifest.
4. Submit a focused PR with examples and compatibility notes.

https://github.com/MISP/misp-taxonomies

Use the vocabulary outside a MISP instance

- Read the JSON vocabulary and its canonical values directly.
- Use PyTaxonomies to load and search taxonomy libraries.
- Validate against the current schema and repository checks.
- Record the library revision used by your consumer.

Workshop: classify, search, explain

1. Import the synthetic event into a training instance.
2. Find its event tags and the domain's confidence tag.
3. Run the scoped tag and warninglist requests.
4. Explain scope, handling, detection intent and every warning.

Then review a local assessment

If permitted, attach a local review label and inspect its local flag.

Before you share or automate

1. Are labels canonical, justified and attached at the right scope?
2. Do distribution and recipients respect handling instructions?
3. Have local context and warninglist hits been reviewed?
4. Does the real consumer interpret returned data correctly?

References and the next practical steps

- Taxonomy reference and FIRST TLP 2.0.
- Taxonomy source and PyTaxonomies.
- Warninglists and MISP 2.5 source.
- This deck's `README.md`: exercise, build instructions, pinned evidence and validation limits.

MISP

INTELLIGENCE SHARING

MISP modules

Enrichment, data exchange and workflow actions

CIRCL / MISP Project

https://www.misp-project.org/

Reference snapshot: 29 October 2026

What you will be able to do

1. Select and discover modules for an analyst's task.
2. Connect MISP to a working module service.
3. Operate configuration, caching and query diagnostics.
4. Generate exports and distinguish them from workflow actions.
5. Use the standalone website and develop a local module.

Hands-on route

The accompanying lab uses synthetic data, local processing and an explicit dry run. No provider account or running MISP instance is needed.

A dated snapshot, not a promise about every install

- Reviewed against upstream `main`, commit `3a903305cf37`, on 29 October 2026.
- Package metadata declares version `3.1.1`. Features on this branch can postdate a published package.
- Source inventory: **270 candidate module files** across four families.
- Installed dependencies, credentials and service reachability determine which integrations can actually run.

Inspect the running service

Check `/version`, `/modules` and `/openapi.json`. A source file count is not the number of operational integrations.

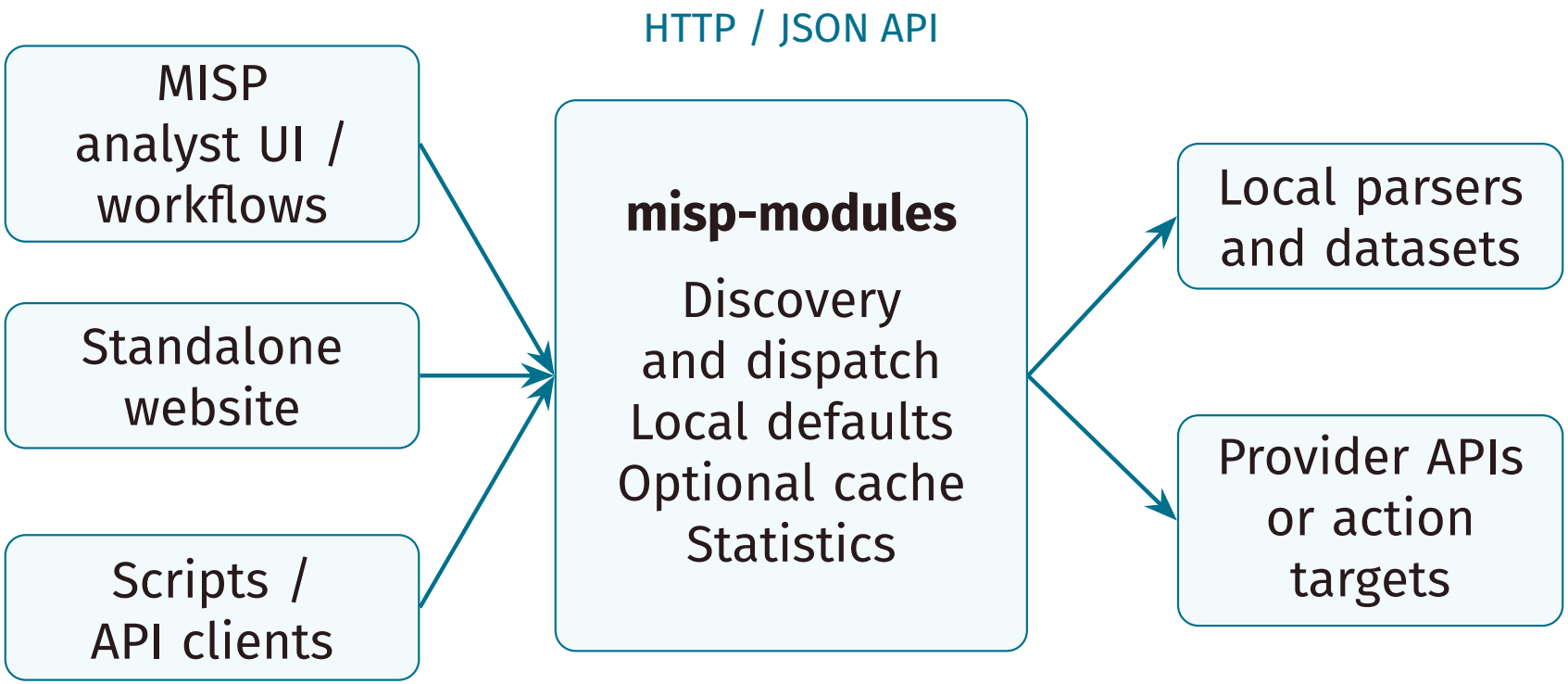
Pinned source links and the inventory method are in this deck's README.

Four families, different contracts

Family	Directory	Files	Purpose
Expansion	`expansion`	203	Enrich an observable; some modules also support hover.
Import	`import_mod`	22	Parse uploaded data or retrieve a report into MISP results.
Export	`export_mod`	32	Transform supplied MISP data into a downloadable format.
Action	`action_mod`	13	Run an operation from a workflow, such as a notification or publication.

Directory names are loader families. Metadata uses `expansion/hover`, `import`, `export` and `action`.

One service, several clients



The Python service can run without a MISP server. The website is a separate application.

Use and understand modules

Choose a module by the question you need to answer

Analyst question	Examples in the current source
What does this URL contain?	`extract_url_components`: local structured parsing.
What infrastructure is related?	`dns`, `reversedns`, `circl_passivedns`: different DNS evidence.
Is this observable known or noisy?	`hygiene`, `hashlookup`, `greynoise`.
What vulnerability context exists?	`cve`, `gcve_cpe`, `rapid7_attackerkb`.
What does analysis tooling know?	`mwdb_query`, `assemblyline_query`, sandbox query/submit integrations.

Read the module's input types, configuration, dependencies and side effects. A lookup and a submission are different operational choices.

Discover what is really loaded

```
curl --fail --silent --show-error \
  http://127.0.0.1:6666/modules |
  jq '.[] | {name, type, mispattributes, meta}'
```

- `name`: API identifier, usually the Python filename.
- `meta.name`: human-readable display name, when supplied.
- `mispattributes`: input/output capabilities and format.
- `meta.config`: expected settings, not configured secret values.
- `meta.module-type`: capabilities used by MISP.

Two expansion formats still coexist

Simplified / legacy

```
{"module": "training_uppercase",
 "text": "misp training"}

{"results": [
  {"types": ["text"],
   "values": ["LOCAL: MISP
    TRAINING"]}
]}
```

MISP standard

```
{"module": "...",
 "attribute": {
   "type": "url",
   "value": "...",
   "uuid": "..."
 }}

{"results": {
  "Attribute": [], "Object": []
}}
```

The module's introspection selects the contract. `format: misp_standard` enables structured results and references.

A complete local enrichment request

Save or use the included `examples/url-request.json`:

```
{
  "module": "extract_url_components",
  "attribute": {
    "type": "url",
    "value": "https://portal.example.org/a?item=1",
    "uuid": "13d8d18b-784a-42a0-9a58-a9cddbfda561"
  }
}
```

```
curl --fail --silent --show-error \
  http://127.0.0.1:6666/query \
  -H 'Content-Type: application/json' \
  --data @examples/url-request.json
```

Read the result as evidence

- URL parsing produces a `url` object, its component attributes, and a reference to the source attribute.
- Current DNS enrichment can express a `dns-record` object and related records, rather than flattening a lookup to one IP.
- Keep source identifiers and object references when pivoting.
- Review provenance, time, relevance and sharing constraints before adding or using results.

Enrichment is not a detection verdict

Context, reputation, warninglist matches and generated rules still need an analyst's interpretation.

Connect MISP: service discovery and user access

In **Administration** → **Server Settings & Maintenance** → **Plugin settings**:

1. Configure the relevant service family: `Enrichment`, `Import`, `Export` or `Action`.
2. Set its `services_enable`, `services_url` and `services_port` settings.
3. Reload settings, enable the desired module and set its credentials.
4. Check organisation restrictions and supported input types.

Example

`Plugin.Enrichment_services_enable`
Enable the enrichment family, then enable/configure its individual modules.

Verify connectivity from the MISP host/container; its localhost may not be the module server.

Deploy and operate the service

Choose the dependency footprint

Install	Python command	What to expect
Base	`pip install .`	Core service, workflow support and modules whose imports are satisfied.
Common extras	`pip install '.[minimal]'`	A broader set of Python dependencies, intended without extra system packages.
Broad extras	`pip install '.[all]'`	Wider coverage; some native tools/libraries are also required.

The snapshot requires **Python 3.10 or newer, below 4.0**. The lab uses Python 3.12 and the base installation.

Use a virtual environment and a supervised process

```
git clone https://github.com/MISP/misp-modules.git
cd misp-modules
python3 -m venv .venv
. .venv/bin/activate
python -m pip install .
misp-modules -l 127.0.0.1 -p 6666
```

- Pin a reviewed revision for a reproducible workshop.
- For development, use an editable installation or Poetry.
- In production, use a service manager with the actual venv executable and service account.

No global sudo pip installation or rc.local startup is needed. See the README for the pinned lab setup.

The service API is discoverable

Endpoint	What it tells you
`GET /healthcheck`	The HTTP service is responding.
`GET /version`	Installed package version.
`GET /modules`	Loaded module metadata and capabilities.
`GET /openapi.json`	Startup-generated API specification.
`GET /openapi`	Interactive Swagger UI using that specification.
`POST /query`	Dispatch a request to one selected module.
`GET /statistics`	Query counts, outcomes, latency and optional cache metrics.

A healthy service does not prove that a provider query will succeed.

Configure the right layer

- **MISP settings**: enablement, organisation restrictions and configuration sent in a query's `config`.
- **Module-server defaults**: optional `–module-config` JSON for credentials, URLs and other settings.
- **Service environment**: used by integrations that explicitly read environment variables, including some publication actions.
- **Website settings**: a separate application and stored module settings; not the MISP database.

Local defaults: explicit precedence

```
{
  "version": 1,
  "modules": {
    "training_uppercase": {
      "config": {"prefix": "LOCAL: "},
      "fallback_when_empty": ["prefix"]
    }
  }
}
```

Start with `--module-config examples/module-defaults.json`.

- Missing settings use defaults; explicit API values win.
- Empty strings/null use defaults only for named `fallback_when_empty` keys.
- Read once at startup; restart after edits or credential rotation.

Optional caching: choose read-only expansions

- Enable with `--cache-config`; disabled by default.
- Opt in each expansion module, with a TTL, entry limit and response-byte limit.
- Match the **whole effective request**, after local defaults: UUIDs, context, credentials and timeout matter.
- Identical concurrent requests can share one execution.
- Cache entries and counters are process-local and reset on restart.

Choose by behavior, not just family

Some expansion modules submit data or cause side effects. Caching one can suppress a repeated operation.

Observe queries, failures and cache savings

```
curl --fail --silent --show-error \
  http://127.0.0.1:6666/statistics | jq .
```

- Per-module and aggregate counts include loaded, unused modules.
- Distinguish module errors, exceptions and timeouts.
- Compare success/error rates, latency and in-flight requests.
- Optional cache metrics include hits, misses and coalesced queries.
- Query counts measure service requests, not provider HTTP calls.

Operate the service within its trust boundary

- Keep the API reachable only by intended clients; it has no built-in authentication layer in the reviewed dispatcher.
- Review which modules contact providers or submit samples.
- Keep local credentials outside the checkout and readable only by the service account.
- Use normal debug logs first. `--debug-extended` logs outbound `requests` exchanges, including credentials and full payloads.

Troubleshoot the failing layer

Symptom	First checks
Cannot reach service	Process, listen address, container network, host/port.
Module absent	Correct venv/package, optional dependencies, loader logs, restart.
Present, absent in MISP	Correct family settings, metadata type, reload settings.
Visible, unavailable to user	Module enablement, organisation restriction, supported input.
HTTP 200, failed query	Inspect `error`/`errors`; config, quotas, provider response.
Unexpected stale result	Effective request, TTL, correct process and cache configuration.

Import, export and act

Import: turn external material into MISP data

- Import sources vary: file upload, pasted text, or remote report retrieval.
- Check `inputSource`, `userConfig` and module settings.
- File-oriented API requests commonly carry base64 `data`.
- Review returned attributes, objects and references before ingestion.
- Examples include email, sandbox reports, PCAP and TAXII integrations.

Use the module's actual contract

A remote-report importer may need configuration instead of an uploaded file. File and report inputs are not interchangeable.

Export modules are an active part of the platform

Target	Examples / output
Detection and SIEM	`zeek_intel_export`, `elastic_ecs_export`, `splunk_export`, `wazuh_cdb_export`.
Host investigation	`osqueryexport`: query pack by default, optional raw SQL.
Network policy	PAN-OS, Junos, FortiOS, ASA, RouterOS, PF and nftables exporters.
Data interchange	CSV and other module-specific formats.

Native MISP exports also remain useful. Choose by target format and supported data, not an old assumption that module exports will disappear.

Lab: generate a Zeek Intel file

Use the included `examples/export-request.json`.

```
curl --fail --silent --show-error \
  http://127.0.0.1:6666/query \
  -H 'Content-Type: application/json' \
  --data @examples/export-request.json > /tmp/zeek-response.json

python3 -c 'import base64,json; r=json.load(open("/tmp/zeek-
    response.json")); print(base64.b64decode(r["data"]).decode("
    ascii"))'
```

Expected: `example.org` and `192.0.2.10`, with native Zeek types. The non-IDS attribute is excluded by default.

Exporting a file and publishing an action differ

	Export	Workflow action
Main result	A file or generated representation.	An operation and its status/result.
Examples	Zeek feed, osquery pack, firewall fragment.	Notification, local pack write, SIEM/EDR publication.
Credentials	Often unnecessary for local generation.	Often needed for the target; some support credential-free dry runs.
Completion	Output generated.	Check acknowledgement, partial failures and activation requirements.

Workflow actions connect intelligence to operations

- Notifications: Slack, Matrix, IRC, Signal and Nextcloud Talk.
- Target publication: CrowdStrike IOC, SentinelOne IOC, QRadar reference sets and Wazuh CDB lists.
- Local artifact publication: `osquery_pack_publish`.
- Inspect action metadata: parameters, blocking behavior, filters and expected MISP-core format.
- Supply the intended data selection and explicit target scope.

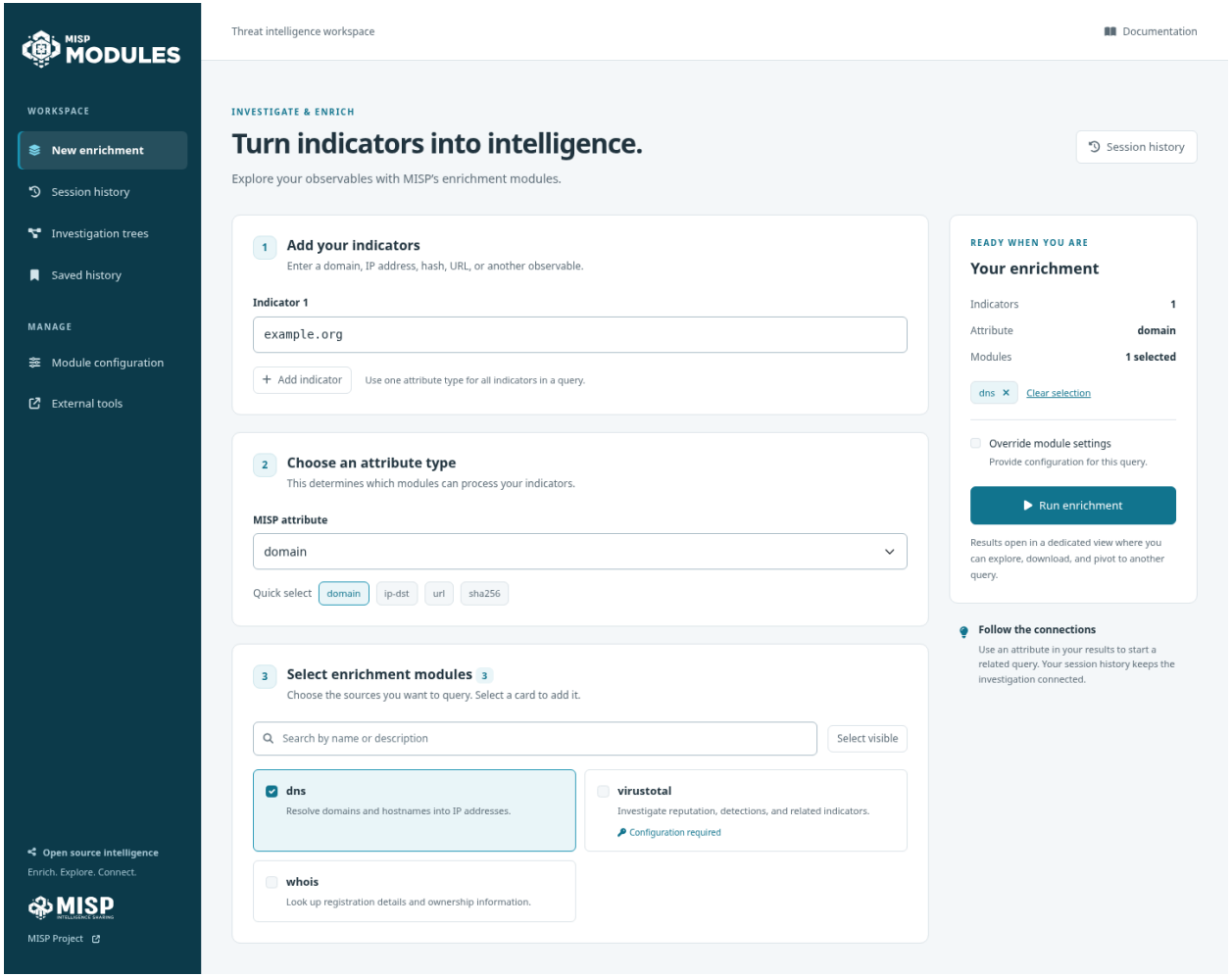
Lab: plan an osquery publication

```
{
  "module": "osquery_pack_publish",
  "params": {
    "pack_name": "training",
    "dry_run": "true",
    "overwrite": "false"
  },
  "data": [{"Event": {"Attribute": [
    {"type": "ip-dst", "value": "192.0.2.10", "to_ids": true}
  ]}}]
}
```

Expected: `data.status = planned`, a pack and provenance. This dry run writes no pack and executes no endpoint queries.

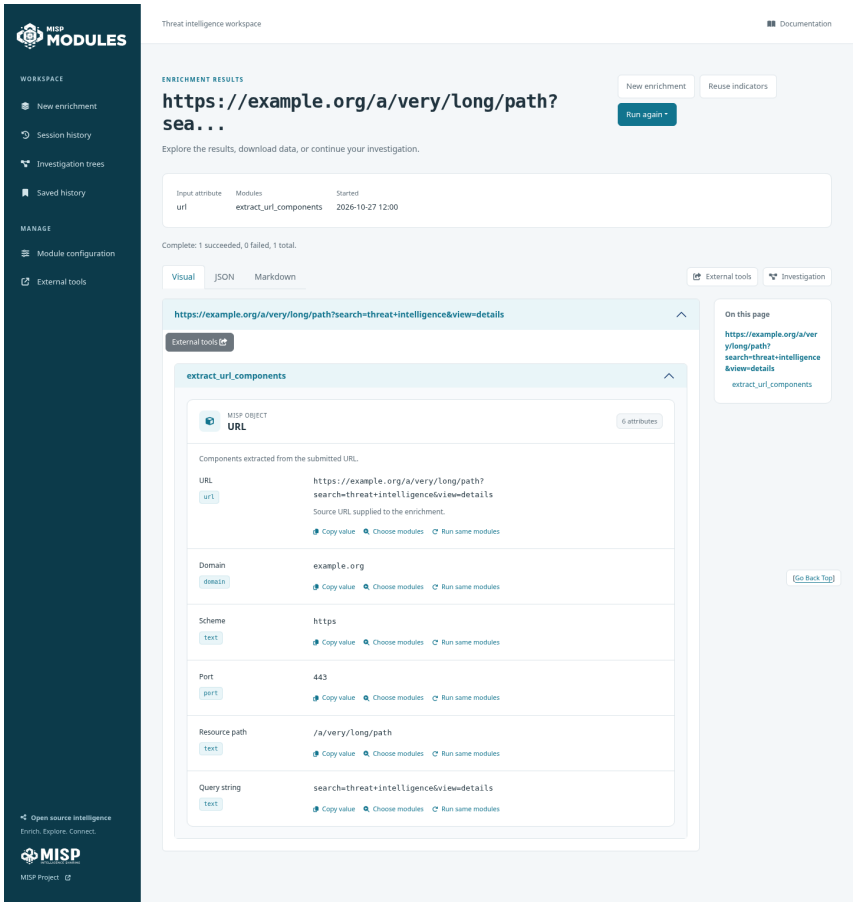
The standalone website

An enrichment workspace without a MISP instance



Current upstream documentation screenshot; sample catalogue from browser fixtures.

Structured results and investigation pivots



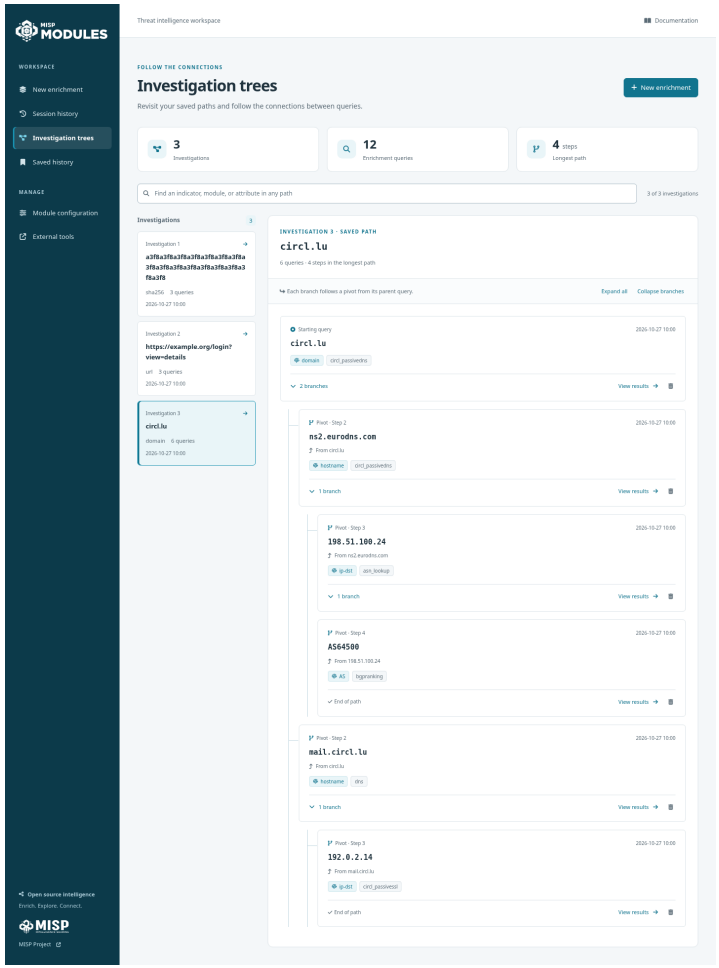
- Separate object cards and flat attributes.
- Visual, JSON and Markdown views.
- Copy complete values or start a related enrichment.
- Download results and use configured external tools.

Source: website/doc/interface.md and `object_result_cards.png` at the reviewed revision.

Import and export are website workflows too

- **Import data**: choose an active module and provide a supported upload, paste or remote-report configuration.
- **Export data**: provide MISP JSON, or choose a CIRCL OSINT feed event, then configure an active exporter.
- Results, text previews, decoded downloads and history use the shared interface.
- With `ADMIN_PASSWORD` configured, exports and their saved results require an administrator login.

Keep the investigation path



Session history, saved trees and related-query paths in the current upstream interface.

Develop and verify a module

A small interface, a real contract

- `introspection()`: supported input/output or import/export setup.
- `version()`: metadata, capabilities and configuration schema.
- `dict_handler(request=...)`: dictionary execution path supported by the current dispatcher.
- `handler(q=...)`: legacy JSON interface, still supported.
- Return useful `error` messages and valid result envelopes.

Choose the result format deliberately

The lab's simple text module uses legacy results. For structured enrichment, start from a current `misp_standard` module and retain source UUIDs and references.

Metadata is part of discovery

```
moduleinfo = {
    "version": "1.0",
    "author": "MISP Project training contributors",
    "name": "Training uppercase",
    "description": "Uppercase synthetic training text without
    network access.",
    "module-type": ["expansion", "hover"],
}
moduleconfig = ["prefix"]
mispattributes = {"input": ["text"], "output": ["text"]}
```

`training_uppercase.py` gives the API identifier. The display name is *Training uppercase*.

A complete handler, with validation

```
def dict_handler(request):
    if not isinstance(request, dict):
        return {"error": "Expected a request object."}
    value = request.get("text")
    if not isinstance(value, str) or not value.strip():
        return {"error": "A nonempty text value is required."}
    config = request.get("config", {})
    if not isinstance(config, dict):
        return {"error": "config must be an object."}
    prefix = config.get("prefix", "")
    if not isinstance(prefix, str):
        return {"error": "prefix must be a string."}
    return {"results": [{"types": ["text"], "values": [prefix +
    value.upper()]}]}
```

Load a local module without modifying the package

```
examples/custom/
  expansion/
    training_uppercase.py
```

```
misp-modules -l 127.0.0.1 \
  --custom "$PWD/examples/custom"
```

- Pass the custom *root*, not its `expansion` child.
- The loader discovers valid `.py` files; no `__all__` edit is needed.
- Restart, check logs and `/modules`, then enable/configure it in MISP.

Test behavior across the service boundary

- Compare dictionary and legacy handlers on valid and invalid input.
- Verify discovery and generated OpenAPI metadata.
- Check structured output and references using a local parser.
- Decode an export and verify indicator selection.
- Assert action status is `planned` in the dry run.
- Exercise defaults, API overrides, empty-value fallback, cache hits and error reporting.

The included `examples/verify_lab.py` performs 12 functional/contract tests.

Workshop: discover, transform, verify

1. **Discover**: find the lab modules and their formats in `/modules` and `/openapi.json`.
2. **Transform**: parse a URL, decode a Zeek file and inspect an osquery publication plan.
3. **Develop**: change the custom module's prefix and test a request override; inspect cache metrics and error responses.

```
python3 examples/verify_lab.py \
  --url http://127.0.0.1:6666 -v
```

Follow the README's full startup command before running the verifier.

What to take into the next integration

1. Discover the **running** module set and its contracts.
2. Separate service configuration, MISP enablement and provider readiness.
3. Preserve evidence and review enrichment before operational use.
4. Distinguish file generation, publication and activation.
5. Test through the dispatcher when relying on its configuration or cache.

Keep the material current

Recheck the pinned source audit, exercise results and module-specific documentation before teaching a newer revision.

References and next steps

- MISP/misp-modules: code and current catalogue
- Module documentation: install, contribute and module guides
- Standalone website and update instructions
- Standalone CLI client
- This deck's `README.md`: pinned sources, trainer notes, reproducible lab and validation scope.
- Continue with *Contributing MISP modules with an AI agent*; see the companion `a.14-ai-assisted-misp-modules` deck.

MISP

INTELLIGENCE SHARING

MISP Galaxy in 2.5

Explore the knowledge base, assess context, share carefully

CIRCL / MISP Project

https://www.misp-project.org/

Reference snapshot: 30 October 2026

Find context, assess evidence, make an assertion

- Explore the knowledge base beyond actor names.
- Choose the right cluster and attachment scope.
- Interpret relations and matrices carefully.
- Practise discovery, custom knowledge and contribution.

Explore the knowledge base

A large, reusable body of knowledge

- 136 Galaxy definitions and 136 cluster collections.
- 58,528 entries; 56,551 distinct cluster UUIDs.
- 79,372 listed directed relationship edges.

Start with the public catalogue

- Browse https://www.misp-galaxy.org/.
- Follow JSON and original-source references.
- Inspect collection scope, authors, version and status.

Choose a family that answers the question

- ATT&CK: observed techniques, software and activity groups.
- D3FEND and preventive measures: defensive context.
- DISARM: information-manipulation techniques and responses.
- Country, sector, detection and other domain knowledge.

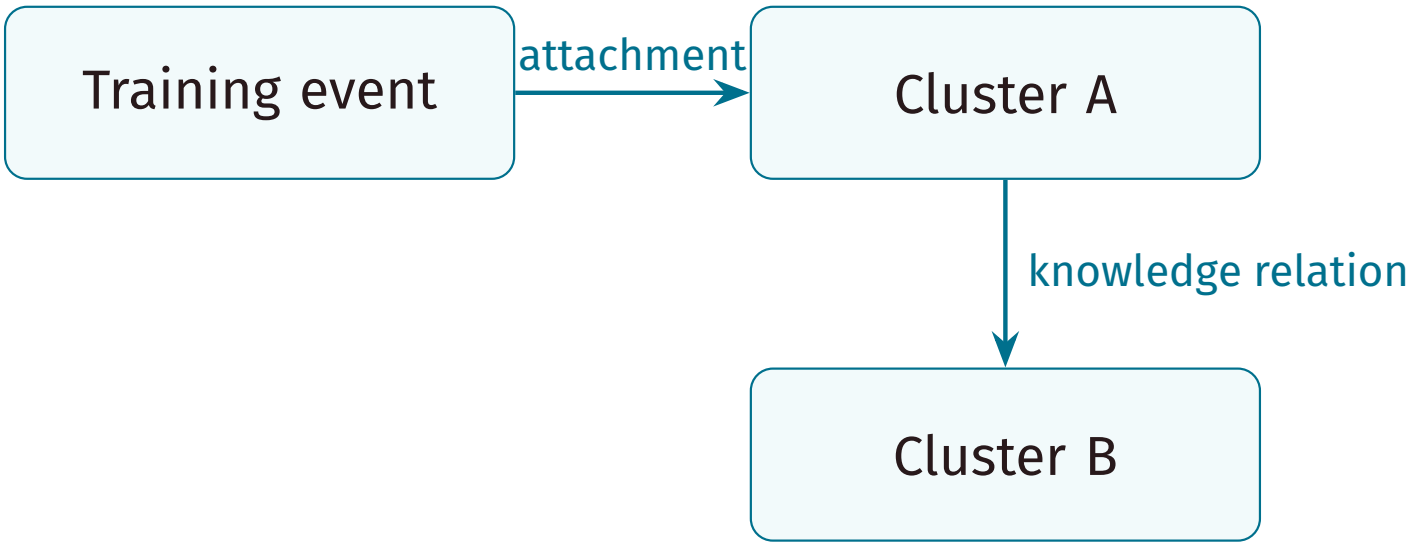
Galaxy, cluster and element

- Galaxy: the family and its presentation metadata.
- Cluster: one named concept with an individual UUID.
- Element: metadata such as synonyms, references or external IDs.

Context is different from incident data

- Taxonomy: classification vocabulary.
- MISP object: grouped typed attributes.
- Galaxy cluster: structured knowledge attached as context.

Three different links



An attachment does not create a cluster-to-cluster edge.

Search for identity, not just a familiar label

- Compare values, synonyms and external identifiers.
- Retain cluster UUID and collection provenance.
- Do not merge identities solely because names overlap.

Check whether the entry is current

- Read original references and collection version.
- Inspect revoked flags, deprecation metadata and historical descriptions.
- Check ATT&CK domain, platform and technique/sub-technique scope.

Apply context in MISP 2.5

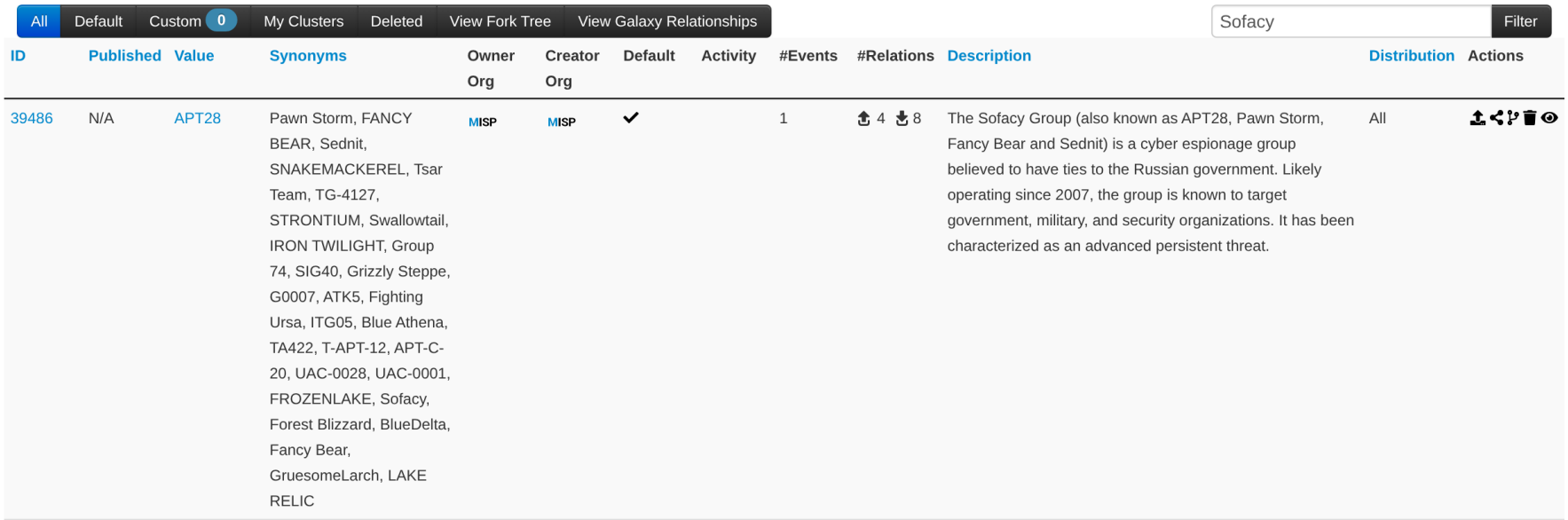
Choose the narrowest justified attachment

- Event scope describes the incident or report as a whole.
- Attribute scope can express a narrower observation.
- Verify the actual attachment after saving and exporting.

Use the installed cluster metadata

- Search and inspect the cluster before attaching it.
- MISP maintains a cluster-associated tag.
- Prefer UUID-backed identity over manually constructed labels.

Threat Actor: search for Sofacy



The APT28 result lists Sofacy as a synonym. Inspect the candidate.

MISP 2.5.50; synthetic lab, 1 November 2026.

Inspect the cluster identity and provenance

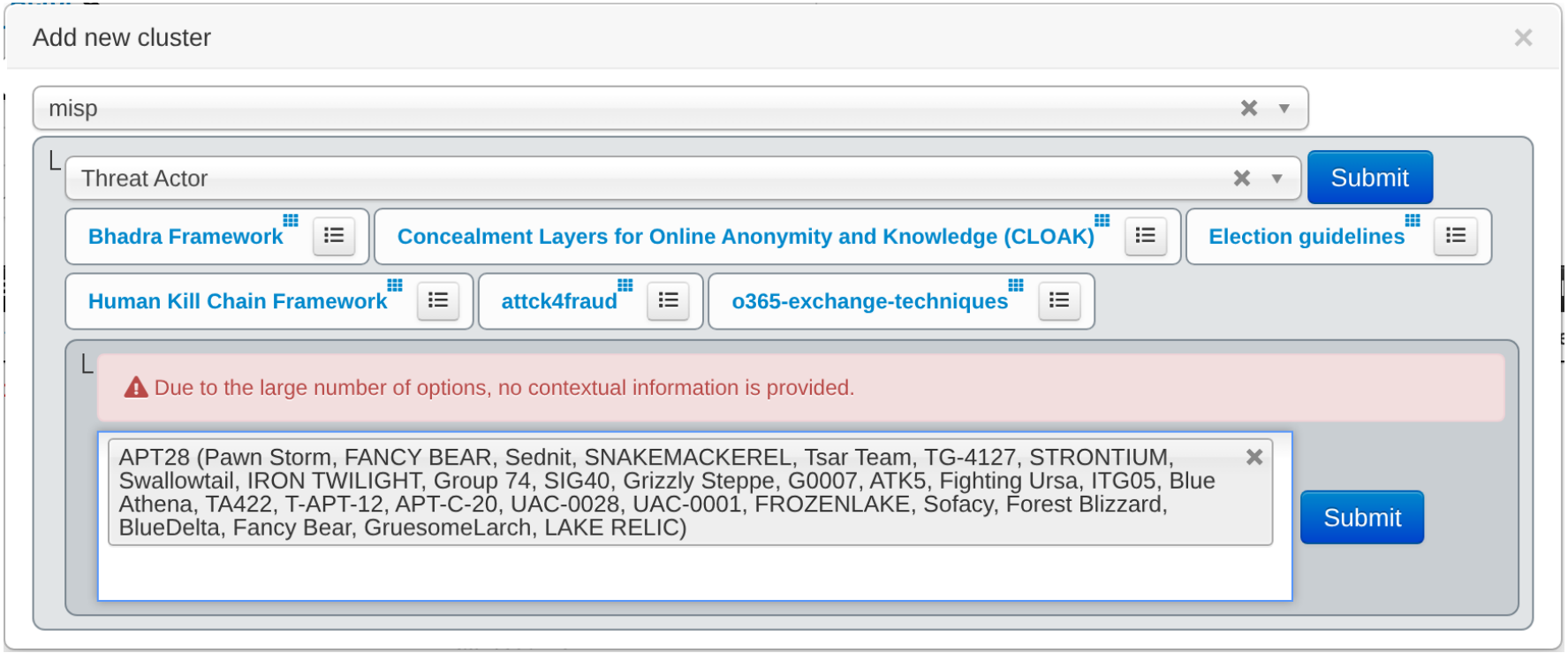
Threat Actor :: APT28

Cluster ID	39486
Name	APT28
Parent Galaxy	Threat Actor
Description	The Sofacy Group (also known as APT28, Pawn Storm, Fancy Bear and Sednit) is a cyber espionage group believed to have ties to the Russian government. Likely operating since 2007, the group is known to target government, military, and security organizations. It has been characterized as an advanced persistent threat.
Default	Yes
Version	357
UUID	5b4ee3ea-eee3-4c8e-8323-85ae32658754
Collection UUID	7cdff317-a673-4474-84ec-4f1754947823
Source	MISP Project
Authors	Alexandre Dulaunoy, Florian Roth, Thomas Schreck, Timo Steffens, Various
Distribution	All communities
Owner Organisation	MISP
Creator Organisation	MISP
Connector tag	misp-galaxy:threat-actor="APT28"
Events	1 event
Attributes	1 attribute

Check the cluster UUID, collection, version and source.

MISP 2.5.50; synthetic lab, 1 November 2026.

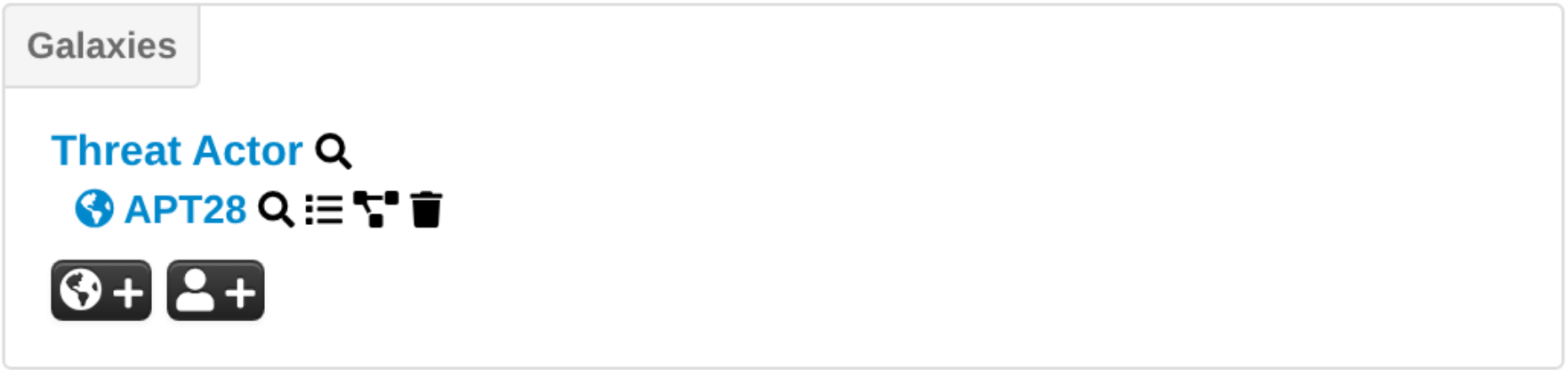
Attach the reviewed actor at event scope



Event Galaxies: Add new cluster → misp → Threat Actor.

MISP 2.5.50; synthetic lab, 1 November 2026.

Verify the saved event attachment



- APT28 appears under the event's Threat Actor Galaxy.
- Confirm UUID and scope in the saved event/export.
- An actor attachment records an assertion requiring evidence.

MISP 2.5.50; synthetic lab, 1 November 2026.

Keep visibility boundaries separate

- Event visibility controls incident access.
- Custom-cluster distribution controls knowledge access.
- Local attachment controls the association scope.
- Publication and sync settings control eligible sharing.

Matrix highlights are observations, not coverage

- Galaxy ordering defines matrix tabs and columns.
- Cluster kill-chain metadata places entries in that layout.
- A highlight does not establish detection or prevention efficacy.

Read graph edges as sourced assertions

- Inspect direction, relationship type and references.
- Review confidence or handling tags where supplied.
- Do not infer causality merely from a graph path.

Custom clusters and forks are supported

- Create permitted custom knowledge with explicit ownership.
- Fork a default cluster for local analysis.
- Record provenance, distribution and publication intent.

Updating files is different from updating the catalogue

- Select and update the intended Git revision.
- Import/update the MISP database through the operator workflow.
- Review changed knowledge and custom forks before sharing.

Explore, validate and contribute

Explore a pinned checkout offline

```
python3 examples/explore.py /tmp/misp-galaxy-lab --stats
python3 examples/explore.py /tmp/misp-galaxy-lab \
  --search T1059 \
  --collection mitre-enterprise-attack-attack-pattern \
  --limit 3
```

Repository structure: family versus collection

```
galaxies/training-knowledge.json
  type, namespace, name, uuid, version, description

clusters/training-knowledge.json
  type, name, uuid, version, source, authors, category
  values: [individual cluster entries]
```

An individual cluster can reference another

```
{
  "value": "Training observation A",
  "uuid": "<individual cluster UUID>",
  "meta": {"synonyms": ["Workshop A"]},
  "related": [{
    "dest-uuid": "<cluster B UUID>",
    "type": "related-to", "tags": []
  }]
}
```

Validate more than JSON syntax

- Check both repository schemas.
- Check identities, matching types and relationship targets.
- Exercise negative cases and inspect original sources.

Workshop: find, assess, attach

- Find a technique by external identifier and read its evidence.
- Compare a synonym search and a domain-specific collection.
- Attach to a disposable event; compare attribute and local scope.
- Inspect matrix/export results and remove training data.

Contribute at the correct source

- Manual collection: edit JSON with evidence and provenance.
- Generated collection: inspect the generator and upstream source.
- Preserve identities and update relevant versions.

Follow the current upstream validation workflow

```
./jq_all_the_things.sh
# Review and commit intended normalised changes.
./validate_all.sh
```

Make knowledge reusable and reviewable

- State source, identity, meaning and uncertainty.
- Keep attachment scope separate from knowledge distribution.
- Include reproducible examples and validation limits.
- Use the README for pinned references and workshop commands.

MISP

INTELLIGENCE SHARING

MISP objects and templates in 2.5

Model observations, validate instances, contribute templates

CIRCL / MISP Project

https://www.misp-project.org/

Reference snapshot: 29 October 2026

Model meaning, not just a bag of values

- Choose an existing model before inventing one.
- Separate template identity from instance identity.
- Validate definitions, instances and sharing policy.
- Practise an offline example and a contribution workflow.

Understand the model

Attribute, object and template

- An attribute is a typed observation with its own UUID.
- An object groups attributes describing one entity or observation.
- A template describes relation names, types and constraints.

Three identifiers with different jobs

- `uuid`: the object instance.
- `template_uuid`: the reusable model.
- `template_version`: the revision used to construct it.

Reuse the current repository

- Inspect the actual definition, not just its name.
- Look under `objects/<name>/definition.json`.
- Compare the model with your source data and downstream consumers.

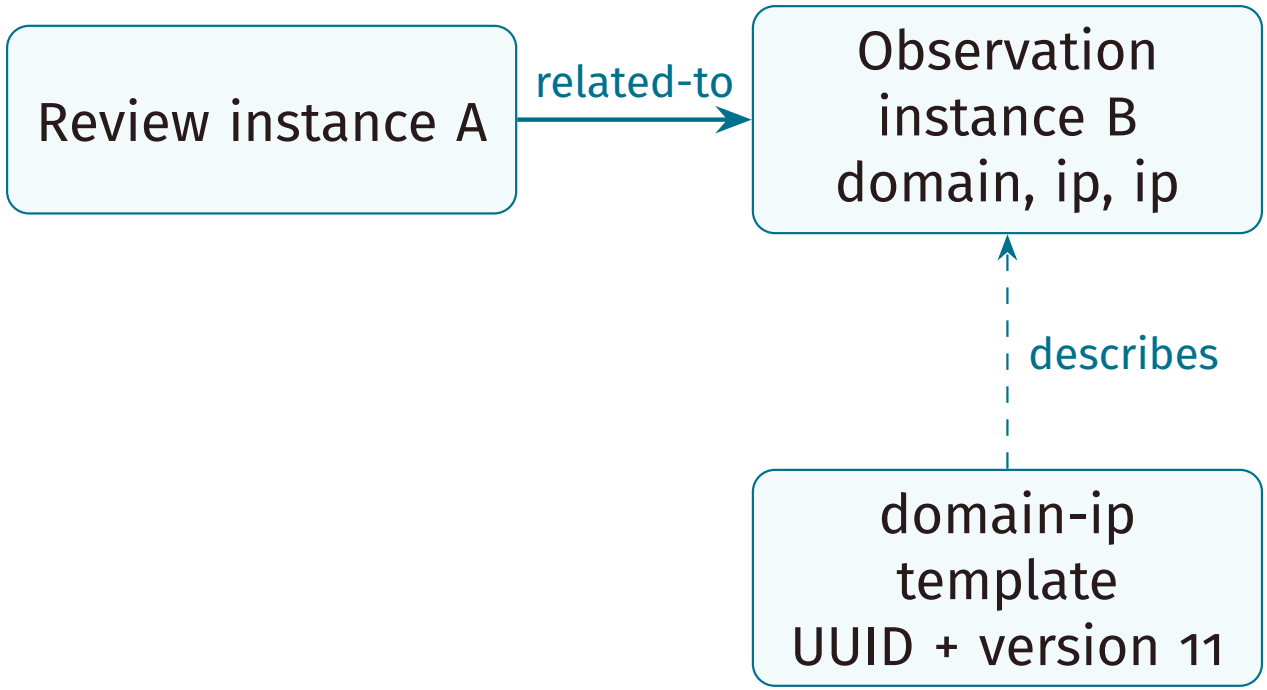
Current domain-ip: alternatives, not a mandatory pair

- `requiredOneOf`: ip, domain or hostname.
- Repeated domain and IP fields are supported.
- Use separate instances for different observation contexts.

Field relation is not a graph relationship

- `ip`: a field relation inside an object.
- `ip-dst`: the underlying MISP attribute type.
- `related-to`: a directed reference between instances.

Reference instances, not definitions



Build and validate

A definition has its own identity

```
{
  "name": "training-review",
  "uuid": "<fresh template UUID>",
  "version": 1,
  "meta-category": "misc",
  "description": "Teaching-only review record",
  "required": ["case-reference"],
  "requiredOneOf": ["domain", "ip"],
  "attributes": { "...": "relation definitions" }
}
```

A relation maps meaning to an existing type

```
"status": {
  "description": "Review status.",
  "misp-attribute": "text",
  "ui-priority": 1,
  "values_list": ["pending", "reviewed"],
  "to_ids": false,
  "disable_correlation": true
}
```

Required fields and repeated fields

- Every relation in required must be present.
- At least one relation in requiredOneOf must be present.
- Both rules can apply together; multiple permits repetition.

Vocabulary and suggestions are different

- `values_list` drives a vocabulary selector.
- `sane_default` offers suggestions plus manual input.
- Validate vocabulary membership explicitly when your workflow requires it.

UI priority is implemented in current MISP

- Higher-priority relations are ordered first in current object views.
- Low-priority fields can keep secondary details out of the main focus.
- Priority is presentation metadata, not a requirement or access rule.

Detection, correlation and visibility are separate

- Choose `to_ids` from the observation and deployment purpose.
- Choose correlation behaviour from the meaning of the value.
- Review event/object/attribute distribution and sharing context.

Four validation layers

- Definition schema: recognised fields and types.
- Definition semantics: identity, requirements and compatibility.
- Instance conformity: required and repeatable relations.
- Attribute validity and policy: values, flags and permissions.

Template enforcement depends on the path

- Template-backed object creation checks conformity.
- Event capture, editing and synchronisation have different paths.
- Strict client construction is useful evidence, not universal server enforcement.

Load the selected definition explicitly

```
definition = json.loads(definition_path.read_text())
obj = MISPObject(
    definition["name"], strict=True, standalone=False,
    misp_objects_template_custom=definition,
)
obj.add_attribute("domain", value="workshop.example",
                  to_ids=False)
obj.to_json(strict=True)
```

The offline workshop event

- One domain/IP object: one domain, two IPs.
- One teaching review: case reference, domain and review status.
- A related-to edge points from review to observation.

Validate, then inspect the output

```
python examples/verify_examples.py /tmp/misp-objects-lab
python examples/build_examples.py /tmp/misp-objects-lab \
  > training-event.json
```

Make failure cases part of the exercise

- Remove the required case reference.
- Remove all requiredOneOf alternatives.
- Repeat a unique relation or add an unknown relation.
- Misspell a schema field and observe the schema failure.

Deploy and contribute

Disk definitions and installed templates are separate

- Update the selected Git checkout deliberately.
- Import/update the template database through the operator workflow.
- Confirm the intended UUID and revision are available to users.

Unknown templates and revision upgrades

- Preserve the received attributes and template metadata.
- Normal UI editing needs a suitable definition.
- Review recovery/upgrade mappings and unmatched fields.

A controlled MISP lab

- Use an unpublished organisation-only event and an authorised writer.
- Add the official object, inspect fields and save.
- Link instances and export/read back the event.
- Test conformity through the actual UI/API path and clean up.

Evolve a model without losing its identity

- Existing model: retain UUID and increment version.
- New model: fresh UUID and initial version.
- Document changed semantics, requirements and consumer impact.

Contribute to the current repository

- Explain why existing templates do not cover the use case.
- Provide schema-valid JSON and synthetic positive/negative instances.
- Check formatting, UUID uniqueness and relationship consistency.
- Open a focused PR with compatibility and validation evidence.

Follow the upstream validation workflow

```
./jq_all_the_things.sh
# Review changes and commit normalised JSON.
./validate_all.sh
```

Make the result reviewable

- Show the model, an instance and the intended relationship direction.
- State the exact template and software revisions tested.
- Explain validation limits and migration choices.
- Keep real credentials and incident data out of examples.

MISP

INTELLIGENCE SHARING

The integrated dashboard in MISP 2.5

Build a useful view, understand its scope

CIRCL / MISP Project

https://www.misp-project.org/

Reference snapshot: 29 October 2026

Build a view that supports a decision

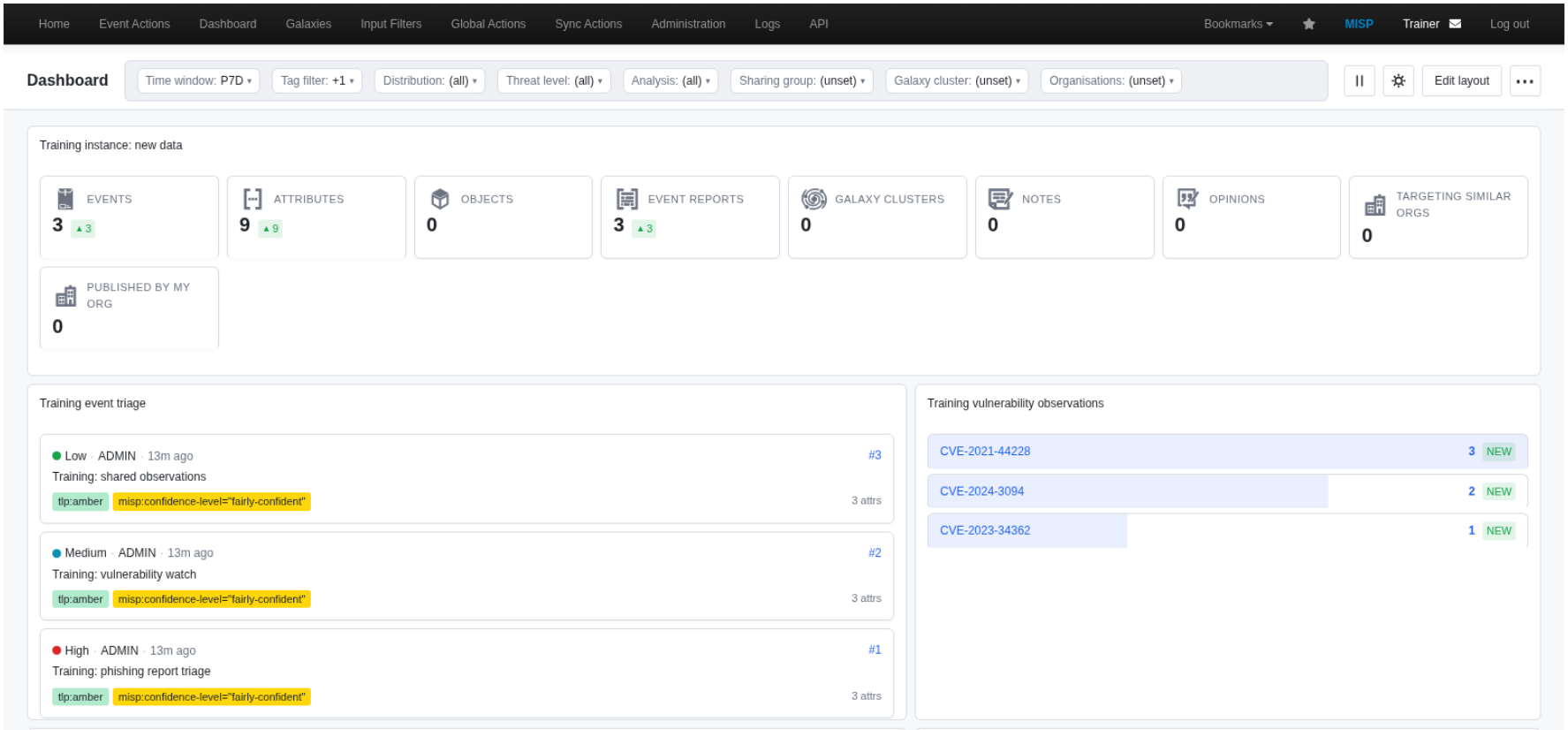
1. Choose widgets for the question you need to answer.
2. Set their filters and understand their scope.
3. Save a useful personal layout.
4. Follow interesting results back to the underlying data.

Two different dashboard applications

Surface	Integrated dashboard	Standalone misp-dashboard
Location	Inside MISP: `/dashboards`	Separate web application.
Model	Personal widgets, layouts and templates.	ZeroMQ activity stream and its own processing.
This training	Current standard course.	Historical course in the archive.

Build your personal view

A real MISP 2.5 dashboard



MISP 2.5.50, default theme, site-admin training account. Synthetic events; top of the board.

What determines your initial layout?

1. Your saved personal dashboard, if one exists.
2. Otherwise, an accessible default template.
3. Otherwise, an empty board.

An empty board can be intentional

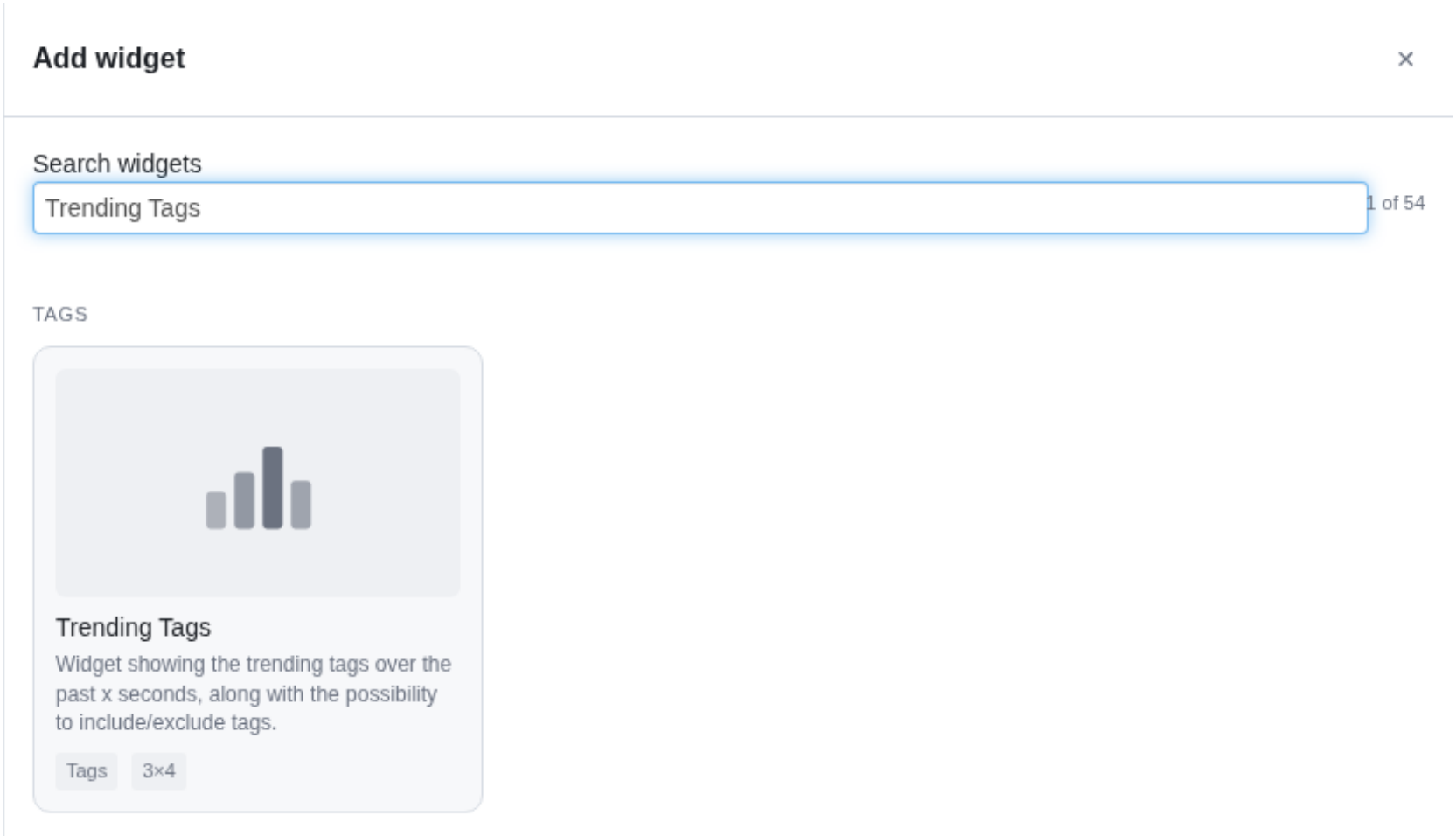
An explicitly saved empty layout is not replaced by a default on every visit.

Layout edits and widget settings save differently

Task	Save behavior
Add, move, resize or remove tiles	Enter *Edit layout*; use *Save changes* or *Discard*.
Configure a widget	Its panel's *Save* persists configuration separately.
Apply a shared filter	Updates supporting widgets' configuration.

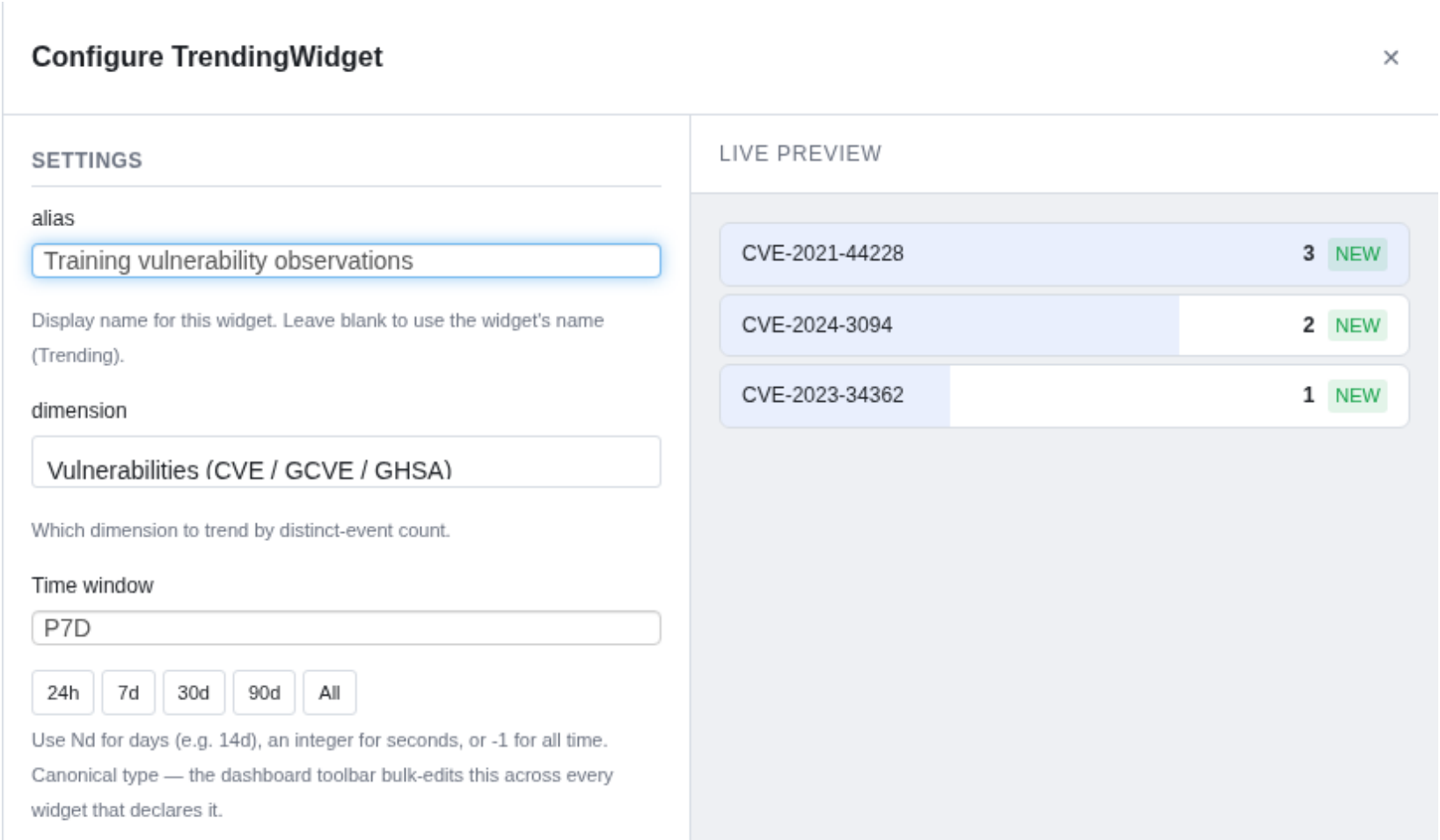
Discarding layout edits is not a general undo for saved widget settings.

Find a widget in the gallery



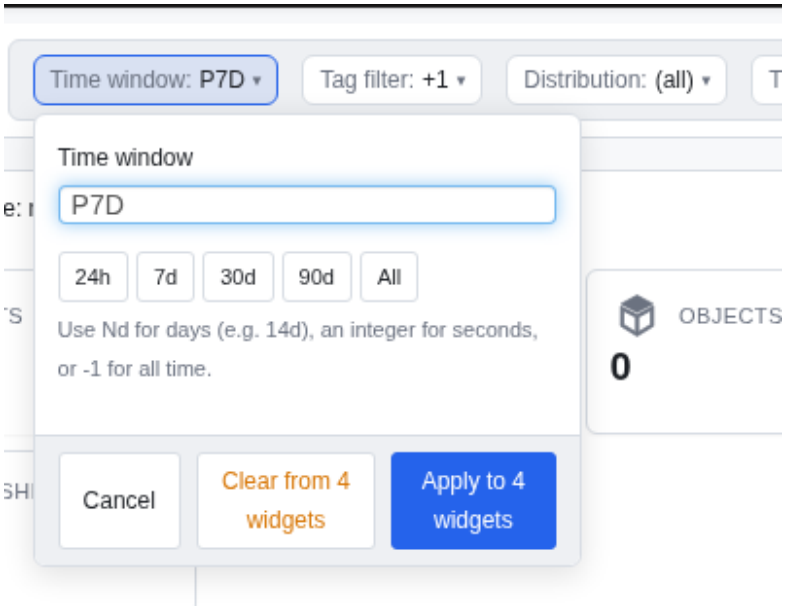
Actual gallery search for Trending Tags; the catalog is role-dependent.

Configure with typed controls and a preview



Alias, dimension and time window: detail of the actual configuration panel.

A shared filter targets supporting widgets



- Read the affected-widget count.
- A mixed value means settings differ.
- Apply changes only to widgets that declare that filter.

Make the scope visible to the next reader

- Give each widget a useful alias.
- Choose the time window and result limit deliberately.
- Check whether tags use exact labels or substring patterns.
- Use drill-down links to inspect the contributing data.

Interpret the numbers

Choose a widget for the question

Question	Useful starting point
What changed recently?	New data; event cards; recent reports or analyst data.
What recurs in the data?	Trending vulnerabilities, actors, techniques or tags.
Where is data associated?	Geolocation and country-map widgets.
Is the instance operating well?	Administrator health, resources and worker widgets.

Do not assume every metric has the same visibility

Detailed event views

Event Stream and Trending event counts use the viewer's event access.

Instance-scale aggregates

New data includes global counts and organisation-contextual metrics. Some geolocation summaries also aggregate at instance scope.

Understand what Trending counts

- Count distinct events per value, not repeated attribute rows.
- Compare with the immediately preceding equal-length window.
- Interpret minimum-count thresholds before trusting momentum.
- All-time mode has no previous window for a momentum comparison.

Training example

Public vulnerability identifiers appear in three synthetic events. A high count is not a severity score.

Refresh frequency is not data freshness

- Some widgets refresh automatically; others do not.
- Pause or resume automatic refresh from the header.
- Per-widget overrides are available where supported.
- A refresh may still return cached data.

Treat maps and coverage as evidence summaries

- Geolocation can be inferred from several kinds of data.
- A plotted origin is not proof of the attacker’s location.
- An ATT&CK tile reflects tagged information.
- Tagged coverage is not proof of detection coverage.

Separate an analyst view from an operator view

Analyst

Recent events, reports, trends and useful drill-downs.

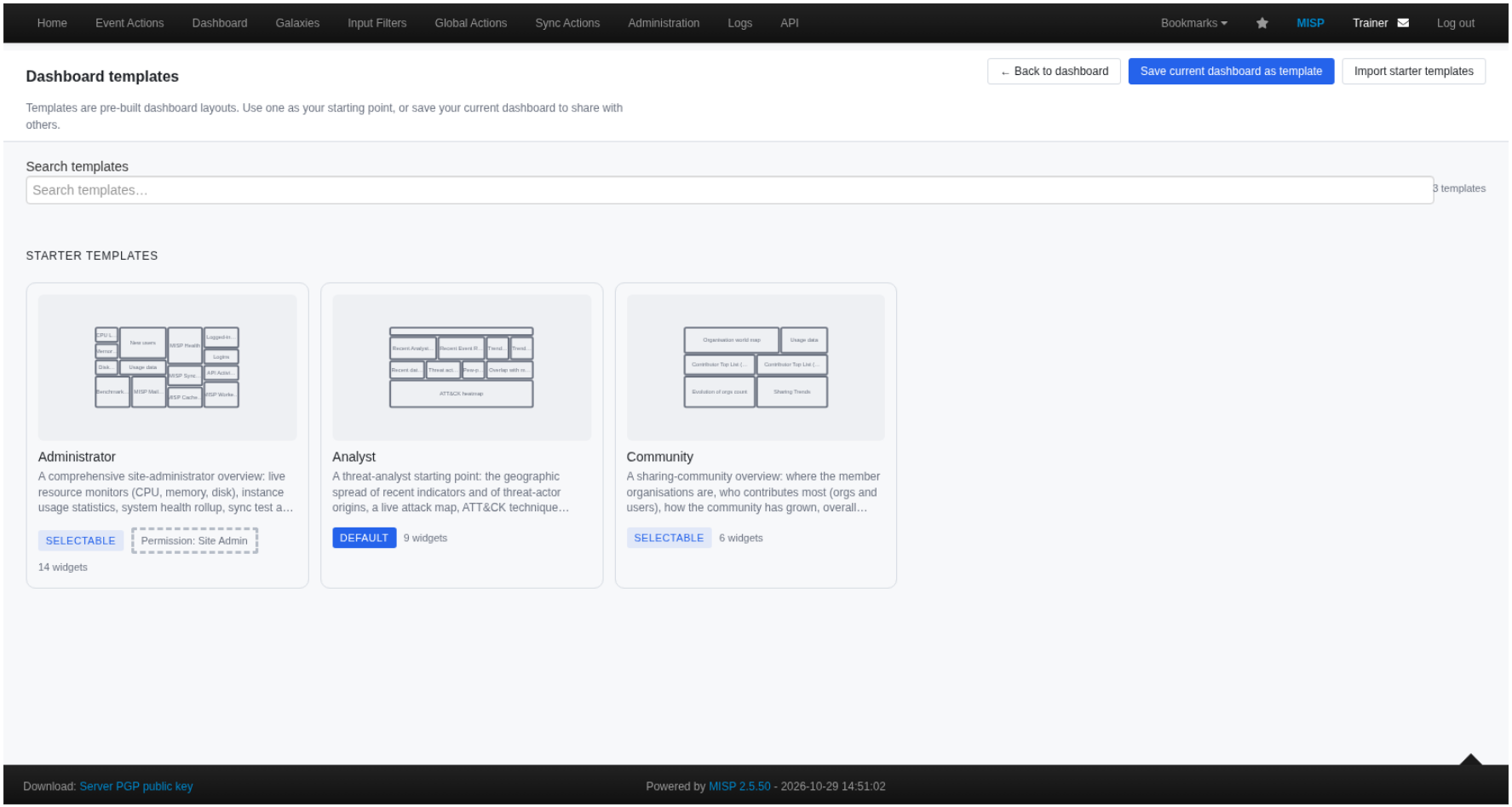
Administrator

Health, resources, queues and permission-gated actions.

A template arranges widgets; it does not grant privileges.

Reuse and verify

Choose a starting point from the template gallery



Actual gallery: imported Analyst, Administrator and Community templates.

Reuse settings without confusing them with data

- Export your configuration before replacing a layout.
- Import a configuration or use an accessible template.
- Save a useful personal view as a template when permitted.
- Review identifiers and configuration before sharing it.

Configuration export and widget-data export differ

Configuration

Widget classes, options and positions for reusing a view.

Widget data

The result produced by a widget, available as JSON or flattened CSV.

Inspect a widget through the API

```
POST /dashboards/renderWidget
```

```
{
  "widget": "TrendingWidget",
  "config": {
    "dimension": "vulnerability",
    "time_window": "P7D",
    "threshold": 5,
    "min_count": 1
  }
}
```

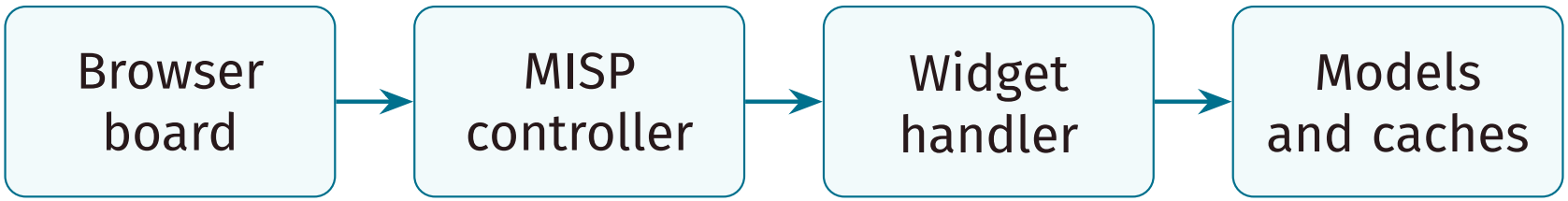
The JSON response identifies the widget, renderer, configuration and data.

What is stored in a dashboard layout?

Field	Purpose
`instance_id`	Stable identifier for one tile.
`widget`	Widget class name.
`config`	Widget-specific settings and supported filters.
`position`	Grid coordinates and dimensions: `x`, `y`, `w`, `h`.

The supplied `examples/dashboard-layout.json` is a portable widget array.

The integrated data path



Widget-specific prerequisites still apply.

Diagnose the scope before rebuilding the board

Symptom	Check first
Widget missing	Class availability, user role and permissions.
Empty or different counts	Widget definition, time window, filters and data access.
Results seem old	Refresh state, timestamps and the widget's cache.
Controls do not work	Browser console/network, JavaScript assets and server errors.

Workshop: build, inspect, restore

1. Use a disposable training account and export its current layout.
2. Import synthetic events and the five-widget sample.
3. Change a filter; inspect the affected widgets and underlying data.
4. Stage a layout removal, discard it, then verify restoration.

Finish with evidence

Export the configuration and explain the scope of each result.

What was verified for this course?

- Real MISP 2.5.50 browser screenshots with synthetic data.
- Layout import/export and widget rendering through the API.
- A staged removal and Discard in the actual browser.
- Slide and notes builds, examples and source references.

Sources and the next practical steps

- MISP 2.5 source and its shipped dashboard templates.
- This course's `README.md`: trainer guide, exercises and immutable source evidence.
- `images/misp25/README.md`: screenshot provenance.
- The standalone ZeroMQ course remains in the training archive.

MISP

INTELLIGENCE SHARING

Contributing to the MISP Project

Become part of the community to design, develop and improve information sharing

CIRCL / Team MISP Project

http://www.misp-project.org/

Twitter: *@MISPProject*

CRI MISP 2024

Code of Conduct

- The MISP project has a Contributor Covenant Code of Conduct[1].
- The goal of the code of conduct is to foster an **open, fun and welcoming environment**.
- Another important aspect of the MISP projects is to welcome different areas of expertise in information sharing and analysis. The **diversity of the MISP community** is important to make the project useful for everyone.

[1] https://github.com/MISP/MISP/code_of_conduct.md

Reporting a bug, an issue or suggesting features

- The most common way to contribute to the MISP project is to report a bug, issues or suggesting features.
- Each project (MISP core, misp-modules, misp-book, misp-taxonomies, misp-galaxy, misp-object or PyMISP) has their **own issue management**.
- Don't forget that you can **cross-reference issues** from other sub-projects.
- If you know an answer or could help on a specific issue, we welcome all contributions including **useful comments to reach a resolution**.

Reporting security vulnerabilities

- **If you find security vulnerabilities (even minor ones) in MISP project, send an encrypted email** (info@circl.lu) with the details and especially how to reproduce the issues. Avoid to share publicly the vulnerability before a fix is available in MISP. PGP key fingerprint: CA57 2205 C002 4E06 BA70 BE89 EAAD CFFC 22BD 4CD5.
- We usually fix reported and confirmed security vulnerabilities in less than 48 hours.
- **We will request a CVE number** if the reporters didn't ask for one (don't forget to mention how you want to be credited).

Automatic integration and testing

- The majority of the repositories within the MISP GitHub organisation includes automatic integration via Github Actions.
- If you contribute and make a pull-request, **verify if your changes affect the result of the tests**.
- Automatic integration is not perfect including Travis but it's a quick win to catch new bugs or major issues in contribution.
- When you do a pull-request, the CI suite is automatically called[2].
  - If this fails, no worries, **review the output at Github actions** (it's not always you).
- We are working on additional automatic tests including security testing for the MISP core software (contributors are welcome).

[2] https://github.com/MISP/MISP/actions

JSON validation for MISP libraries

- All JSON format (**galaxy, taxonomies, objects or warning-lists**) are described in a JSON Schema[3].
- The TravisCI tests are including JSON validation (via *jq*) and validated with the associated JSON schema.
- How to contribute a JSON library (objects, taxonomies, galaxy or warning-list):
  - If you update a JSON library, don't forget to run *jq_all_the_things.sh*. It's fast and easy. If it fails, review your JSON.
  - Commit your code and make a pull-request.
- Documentations (in PDF and HTML format) for the librairies are automatically generated from the JSON via asciidoctor[4].

[3] schema_name.json

[4] example https://github.com/MISP/misp-galaxy/blob/master/tools/adoc_galaxy.py

Documentation

- In addition to the automatic generation of documentations from JSON files, we maintain **misp-book**[5] which is a generic documentation for MISP including usage, API documentation, best practices and specific configuration settings.
- The book is generated in HTML, PDF, epub and mobi using GitBook[6] which is a framework to write documentation in MarkDown format.
- TravisCI is included in misp-book and **the book generation is tested at each commit**.
- The MISP book is regularly published on misp-project.org and circl.lu website.
- Contributors are welcome especially for new topics[7] and also fixing our broken english.

[5] https://github.com/MISP/misp-book
[6] https://github.com/GitbookIO
[7] Topics of interest are analysts best-practices,

Internet-Draft - IETF for MISP formats

- If you want to contribute to our IETF Internet-Draft for the MISP standard, misp-rfc[8] is the repository where to contribute.
- **Update only the markdown file**, the XML and ASCII for the IETF I-D are automatically generated.
- If a major release or updates happen in the format, we will publish the I-D to the IETF[9].
- The process is always MISP implementation $\rightarrow$ IETF I-D updates.

[8] https://github.com/MISP/misp-rfc

[9] https://datatracker.ietf.org/doc/search/?name=misp&activedrafts=on&rfcs=on

MISP

INTELLIGENCE SHARING

MISP core development crash course

How I learned to stop worrying and love the PHP

CIRCL / Team MISP Project

MISP Project

CRI MISP 2024

Some things to know in advance...

- MISP is based on PHP 7.3+
- Using the MVC framework CakePHP 2.x
- What we'll look at now will be a quick glance at the structuring / layout of the code

MVC frameworks in general

- separation of business logic and views, interconnected by controllers
- main advantage is clear separation of the various components
- lean controllers, fat models (kinda...)
- domain based code reuse
- No interaction between Model and Views, ever

Structure of MISP Core app directories

- Config: general configuration files
- Console: command line tools
- Controller: Code dealing with requests/responses, generating data for views based on interactions with the models
- Lib: Generic reusable code / libraries
- Model: Business logic, data gathering and modification
- Plugin: Alternative location for plugin specific codes, ordered into controller, model, view files
- View: UI views, populated by the controller

Controllers - scope

- Each public function in a controller is exposed as an API action
- request routing (admin routing)
- multi-use functions (POST/GET)
- request/response objects
- contains the action code, telling the application what data fetching/modifying calls to make, preparing the resulting data for the resulting view
- grouped into controller files based on model actions
- Accessed via UI, API, AJAX calls directly by users
- For code reuse: behaviours
- Each controller bound to a model

Controllers - functionalities of controllers

- pagination functionality
- logging functionality
- Controllers actions can access functionality / variables of Models
- Controllers cannot access code of other controller actions (kind of...)
- Access to the authenticated user's data
- beforeFilter(), afterFilter() methods
- Inherited code in AppController

Controllers - components

- Components = reusable code for Controllers
  - Authentication components
  - RestResponse component
  - ACL component
  - Cidr component
  - IOCImport component (should be moved)

Controllers - additional functionalities

- Handling API responses (RestResponseComponent)
- Handling API requests (IndexFilterComponent)
- auth/session management
- ACL management
- CRUD Component
- Security component
- important: quertString/PyMISP versions, MISP version handler
- future improvements to the export mechanisms

Models - scope

- Controls anything that has to do with:
  - finding subsets of data
  - altering existing data
  - inherited model: AppModel
  - reusable code for models: Behaviours
  - regex, trim

Models - hooking system

- Versatile hooking system
  - manipulate the data at certain stages of execution
  - code can be located in 3 places: Model hook, AppModel hook, behaviour

Model - hooking pipeline (add/edit)

- Hooks / model pipeline for data creation / edits
  - beforeValidate() (lowercase all hashes)
  - validate() (check hash format)
  - afterValidate() (we never use it
  - could be interesting if we ever validated without saving)
  - beforeSave() (purge existing correlations for an attribute)
  - afterSave() (create new correlations for an attribute / zmq)

Models - hooking pipeline (delete/read)

- Hooks for deletions
  - beforeDelete() (purge correlations for an attribute)
  - afterDelete() (zmq)
- Hooks for retrieving data
  - beforeFind() (modify the find parameters before execution, we don't use it)
  - afterFind() (json decode json fields)

Models - misc

- code to handle version upgrades contained in AppModel
- generic cleanup/data migration tools
- centralised redis/pubsub handlers
- (Show example of adding an attribute with trace)

Views - scope and structure

- templates for views
- layouts
- reusable template code: elements
  - attribute list, rows (if reused)
- reusable code: helpers
  - commandhelper (for discussion boards), highlighter for searches, tag colour helper
- views per controller

Views - Types of views and helpers

- ajax views vs normal views
- data views vs normal views vs serialisation in the controller
- sanitisation h()
- creating forms
  - sanitisation
  - CSRF

Views - Generators

- Mostly in genericElements
- Preparing the move to Cake4
- Important ones
  - Form - generate forms in a standardised way (/add, /edit, etc)
  - IndexTable - index lists using Field templates (/index, etc)
  - SingleViews - key-value lists with child elements (/view, etc)
  - Menues - to be refactored, see Cerebrate

General reusable libraries

- Located in app/Lib
- Code that is to be reused across several layers
- Important ones
  - Dashboard - Dashboard widget backend code
  - EventReport - Report generation
  - Export - MISP -> external format converter modules
  - Tools - List of generic helper libraries - examples:
    - Attachment, JSON conversion, random generation, emailing, sync request generation
    - Kafka, ZMQ, AWS S3, Elastic integration, PGP encryption, CIDR operations

Distribution

- algorithm for checking if a user has access to an attribute
- creator vs owner organisation
- distribution levels and inheritance (events -> objects -> attributes)
- shorthand inherit level
- sharing groups (org list, instance list)
- correlation distribution
- algorithms for safe data fetching (fetchEvents(), fetchAttributes(),...)

Testing your code

- funtional testing
- Github actions
- impact scope
  - view code changes: only impacts request type based views
  - controller code changes: Should only affect given action
  - model code changes: can have impact on entire application
  - lib changes: can have affect on the entire application
- Don’t forget: queryACL, change querystring

MISP

INTELLIGENCE SHARING

PyMISP with MISP 2.5

Build models, query context, automate deliberate changes

CIRCL / MISP Project

https://www.misp-project.org/

Reference snapshot: 2 November 2026

From a notebook to a deliberate integration

- Build and inspect models without contacting a server.
- Map Python operations to the MISP 2.5 REST API.
- Query the right scope and recognise incomplete results.
- Create, verify and remove a synthetic training event.

Client, models and API

Pin the client and identify the server

- Reviewed: MISP 2.5.50 and PyMISP 2.5.34.4.
- PyMISP requires Python 3.10 or later, below 4.0.
- PyPI and GitHub release checked on 2 November 2026.
- Server and client version numbers need not match.

Two complementary interfaces

- `PyMISP`: authenticated requests to the server.
- `MISPEvent`, `MISPAttribute`, `MISPObject`: local models.
- Model changes and JSON conversion do not save remotely.
- Use the public client methods to persist intended changes.

The Python methods still use REST routes

Python operation	REST route in this snapshot
`search_index(...)`	`POST events/index`
`search(...)` (attributes)	`POST attributes/restSearch`
`get_event(uuid)`	`GET events/view/{uuid}`
`add_event(event)`	`POST events/add`
`update_attribute(...)`	`POST attributes/edit/{uuid}`

Authentication and visibility remain server decisions

- The raw auth key goes in the Authorization header.
- An API-enabled role and visible data are still required.
- Advanced keys can restrict IPs, expiry and read-only use.
- Read-only keys can POST searches; they cannot create events.

Install the reviewed release in a virtual environment

```
python3 -m venv .venv
. .venv/bin/activate
python -m pip install -r examples/requirements.txt
python -m unittest examples.test_examples -v
python -m examples.models
```

Connect with trusted TLS and bounded request timeouts

```
from pymisp import PyMISP
from examples.common import Config

config = Config.from_env()
misp = PyMISP(config.url, config.key,
              ssl=config.verify, timeout=(5, 60),
              debug=False)
```

Check the response before using it

```
from pymisp import MISPEvent
from examples.common import expect_model

saved = expect_model(
    misp.get_event(event_uuid, pythonify=True),
    MISPEvent,
)
```

Read the right data

Choose the search scope first

- `search_index`: event summaries from the index.
- `search(controller="events")`: matching events.
- `search(controller="attributes")`: matching attribute rows.
- `get_event(uuid)`: one known event and its context.

One typed attribute page

```
rows = misp.search(
    controller="attributes", return_format="json",
    type_attribute="domain", value="%.example",
    include_event_uuid=True, include_event_tags=True,
    order="Attribute.id ASC", limit=10, page=1,
    pythonify=True,
)
```

Raw JSON, normalised JSON and typed models differ

Interface	Attribute-search result
Raw HTTP JSON	`response.Attribute` array
PyMISP JSON	Dictionary containing `Attribute`
PyMISP with `pythonify=True`	List of `MISPAttribute` objects

Do not reuse the raw-envelope parser on typed results.

Translate parameter names and states deliberately

REST JSON	PyMISP keyword
`type`	`type_attribute`
`returnFormat`	`return_format`
`includeEventUuid`	`include_event_uuid`
`includeEventTags`	`include_event_tags`
`enforceWarninglist`	`enforce_warninglist`

A matching event can contain nonmatching attributes

- Event search selects events using the supplied filters.
- A full event result can include broader incident context.
- Attribute search selects the rows required by a consumer.
- Retain event UUID and inherited-tag context when needed.

Traverse bounded pages without claiming false completion

- Start at page 1 and use a stable order.
- A role cap can make every nonempty page short.
- For this ordinary JSON query, continue to an empty page.
- Fail on repeated UUIDs or the maximum-page bound.

Keep polling clocks and checkpoints explicit

```
from time import time
upper = int(time())
lower = previous_checkpoint - overlap_seconds
# Use the same bounds for every page:
rows = misp.search(controller="attributes",
                   timestamp=[lower, upper],
                   limit=100, page=1, pythonify=True)
```

Export format is also a selection policy

- Ordinary JSON can include unpublished contextual attributes.
- Detection formats can default to published and to_ids records.
- Set publication, to_ids and warninglist policy explicitly.
- A text export is a string, not a list of typed attributes.

Build, save and verify

Build a private synthetic event offline

```
from pymisp import MISPEvent

event = MISPEvent()
event.info = "PyMISP 2.5 training / synthetic"
event.distribution = 0
event.threat_level_id = 4
event.analysis = 0
event.published = False
```

Add a contextual attribute to the local model

```
attribute = event.add_attribute(
    "domain", "observer.example", to_ids=False,
    comment="Synthetic observation only.",
)
attribute.comment = "Reviewed synthetic observation."
print(attribute.uuid, attribute.type)
```

Group related observations in a template-backed object

```
from pymisp import MISPObject
obj = MISPObject("domain-ip", strict=True,
                 standalone=False)
obj.add_attribute("domain", value="host.example",
                  to_ids=False)
obj.add_attribute("ip", value="192.0.2.10",
                  to_ids=False)
event.add_object(obj)
```

Inspect JSON and round-trip the local model

```
encoded = event.to_json(indent=2)
restored = MISPEvent()
restored.load(encoded)
assert restored.uuid == event.uuid
print(len(restored.attributes), len(restored.objects))
```

Persist once, then use the server response

```
from examples.common import expect_model

created = expect_model(
    misp.add_event(event, pythonify=True), MISPEvent)
verified = expect_model(
    misp.get_event(created.uuid, pythonify=True), MISPEvent
    )
```

Update a known attribute and verify its identity

```
from pymisp import MISPAttribute
attribute = verified.attributes[0]
attribute.comment = "Reviewed in the workshop."
updated = expect_model(
    misp.update_attribute(attribute, pythonify=True),
    MISPAttribute,
)
assert updated.uuid == attribute.uuid
```

Reference an attribute from an object

```
obj.add_reference(attribute.uuid, "related-to",
                  "Synthetic teaching relation.")
# Add this before saving the event in the workshop.
```

- Keep the referenced UUID and relationship meaning explicit.
- A local reference is not a correlation or Galaxy attachment.

Tag at the intended scope

```
from examples.common import expect_success

expect_success(misp.tag(created.uuid, "tlp:clear"))
expect_success(misp.tag(attribute.uuid, "tlp:clear",
                        local=True))
```

Sightings and proposals have distinct meanings

- `MISPSighting` plus `add_sighting` records an observation.
- Target the intended attribute UUID, not a broad shared value.
- Proposal methods create suggested changes for review.
- A returned proposal does not confirm an original was edited.

Run the disposable write workshop deliberately

```
python -m examples.workshop --run
```

- Create a private event, attributes, object and reference.
- Edit, tag, add a sighting and refetch the result.
- Query bounded pages restricted to the new event UUID.
- Delete only that fixture and verify it is absent.

Handle failures and diagnostics without losing meaning

- Separate transport failure, API error and valid empty result.
- Check saved state before retrying an uncertain write.
- Keep keys, private payloads and verbose logs out of notebooks.
- Record versions, query policy and validation limits.

Use the notebook as an experiment

- Default: create, inspect and round-trip local models.
- Optional live reads: compare raw and typed attribute UUIDs.
- Separate opt-in: execute the disposable write workshop.
- Clear outputs before sharing; retain source provenance.

Make the integration reproducible

- Pin tested versions and preserve UUID-based identity.
- Separate local construction from remote persistence.
- Keep scope, selection policy and completion explicit.
- Use the README, executable examples and REST API course.

MISP

INTELLIGENCE SHARING

MISP feeds - A simple and secure approach to generate, select and collect intelligence

Providing ready-to-use threat intelligence in MISP standard format

CIRCL / Team MISP Project

TLP:WHITE

http://www.misp-project.org/

Twitter: *@MISPProject*

CRI MISP 2024

MISP Feed - Basics

MISP Feeds provide a way to

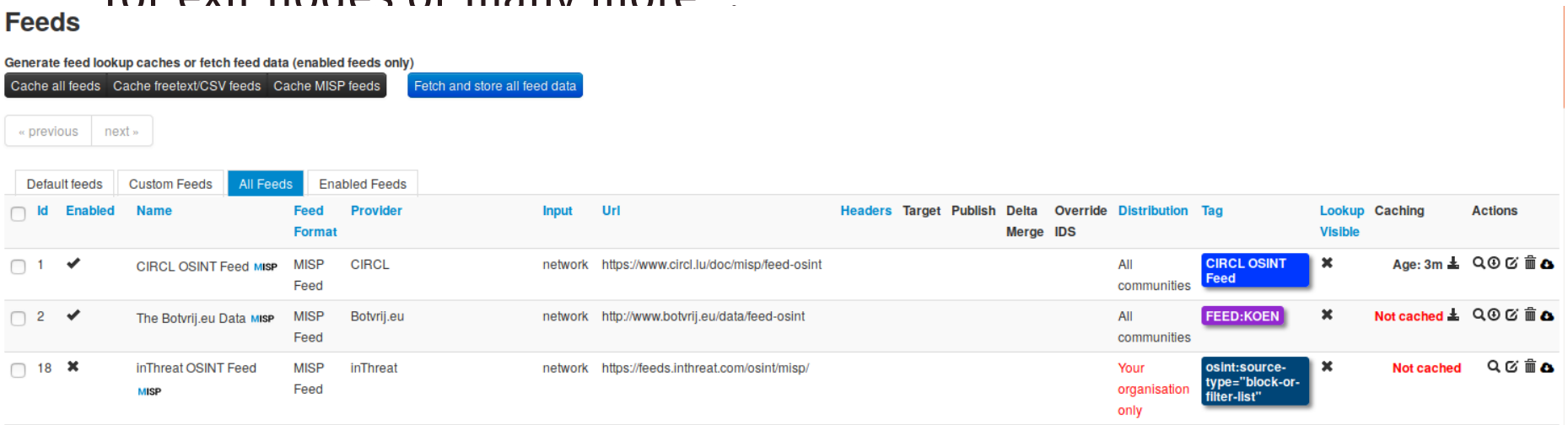
- **Exchange information via any transports** (e.g. HTTP, TLS, USB keys)
- Preview events along with their attributes, objects
- Select and import events
- **Correlate attributes using caching**

MISP Feeds have the following advantages

- Feeds work without the need of MISP synchronisation (reducing attack surface and complexity to a static directory with the events)
- **Feeds can be produced without a MISP instance** (e.g. security devices, honeypot sensors)

Feed - Overview

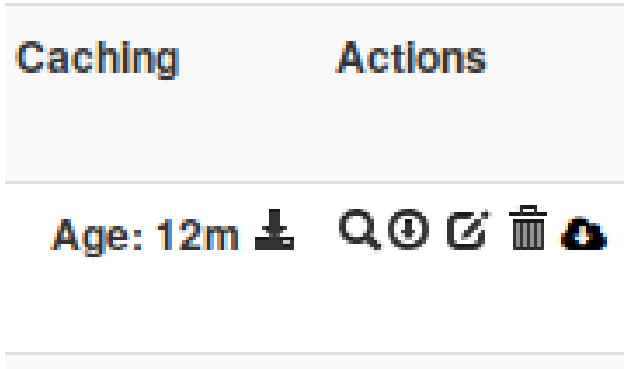
- By default, MISP is bundled with ∼50 default feeds (MISP feeds, CSV or freetext feeds) which are not enabled by default and described in a simple JSON file[1].
- The feeds include CIRCL OSINT feed but also feeds like abuse.ch, Tor exit nodes or many more [2].



[1] https://github.com/MISP/MISP/blob/2.4/app/files/feed-metadata/defaults.json

[2] http://www.misp-project.org/feeds/

Feed - Operations



- Cache feed attributes for correlation (not imported but visible in MISP)
- Disable feed
- Explore remote events
- Fetch all events (imported in MISP as event)
- Edit the feed configuration (e.g. authentication, URL,...)
- Remove feed
- Download feed metadata (to share feed details)

Feed - Creation using PyMISP `feed generator`

`feed generator` fetches events (matching some filtering) from a MISP instance and construct the manifest (defined in *MISP core format*) needed to export data.

Particularly,

- Used to generate the **CIRCL OSINT feed**
- Export events as json based on tags, organisation, events, ...
- Automatically update the dumps and the metadata file
- Comparable to a lighweight **TAXII interface**

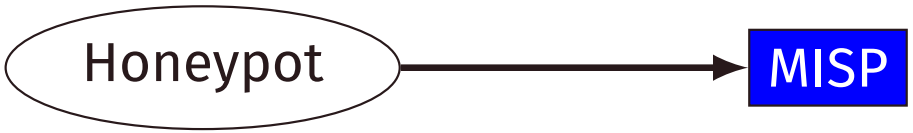
Feed generator - configuration file

```
1  url = 'your/misp/url'
2  key = 'YourAPIKey'
3  ssl = True
4  outputdir = 'output_directory'
5
6  filters = {
7      'tag':'tlp:white|feed-export|!privint',
8      'org':'CIRCL'
9  }
10 # the above would generate a feed for all events created by CIRCL,
       tagged tlp:white and/or feed-export but exclude anything
       tagged privint
11
12 valid_attribute_distribution_levels = ['0', '1', '2', '3', '4', '5
       ']
13 # 0: Your Organisation Only
14 # 4: Sharing Group
15 # 5: Inherit Event
16
```

Real-time **Feed generator - Purpose**

The PyMISP feed generator is great but may be inadequate or ineficient:

- Batch import of attributes/objects
- Data producer doesn't have a MISP instance at hand and only wants to **produce a directly consumable feed**:



ip-src
payload-delivery
url
malware
...

Real-time **Feed generator - Usage**

- `generator.py` exposes a class allowing to generate a MISP feed in real-time
- Each items can be appended on daily generated events

Example:

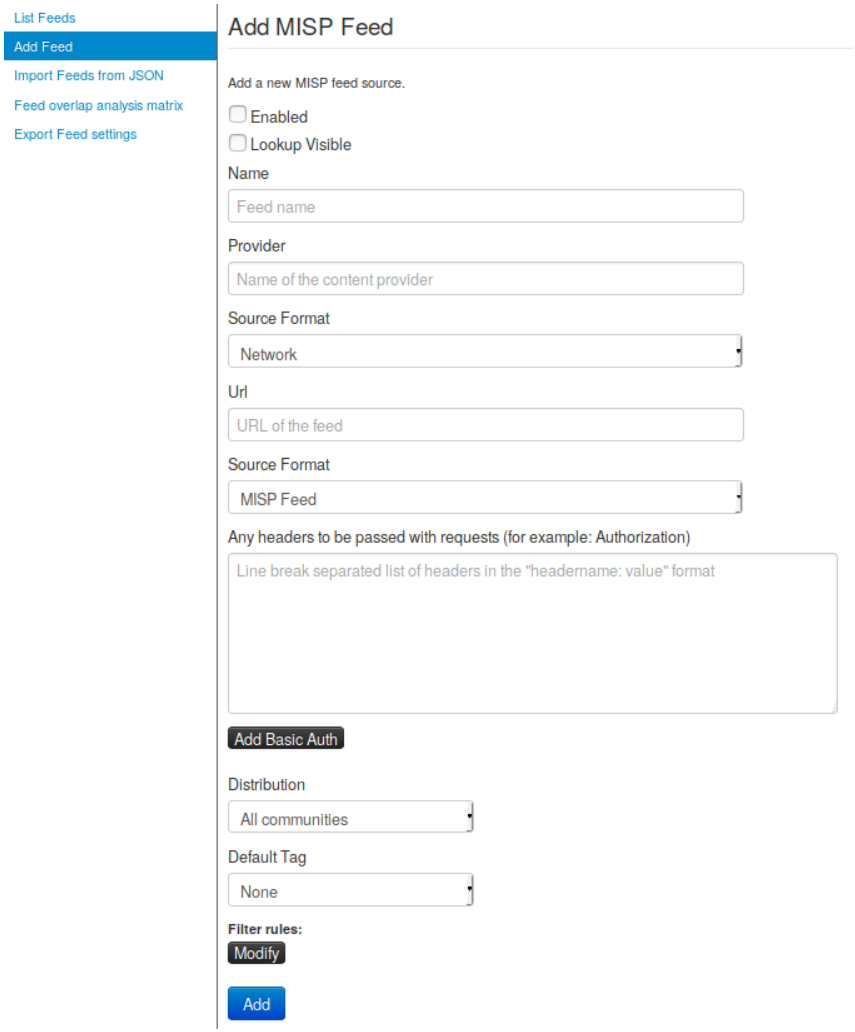
```
#  Init generator
generator = FeedGenerator()

#  Adding an attribute to the daily event
attr_type = "ip-src"
attr_value = "8.8.8.8"
additional_data = {}
generator.add_attribute_to_event(attr_type,
                                 attr_value,
                                 **additional_data)
```

Real-time **Feed generator - Usage (2)**

```
#  Adding a MISP object (cowrie) to the daily event
obj_name = "cowrie"
obj_data = {
    "session": "session_id",
    "username": "admin",
    "password": "admin",
    "protocol": "telnet"
    }
generator.add_object_to_event(obj_name, **obj_data)
```

Adding custom feed to MISP



- Enabled
- Lookup visible
- Name
- Provider
- Source Format
- Url
- Source Format
- Headers
- Distribution
- Default Tag
- Filter rules

Q&A

- https://github.com/MISP/PyMISP
- https://github.com/MISP/
- We welcome new functionalities and pull requests.

MISP

INTELLIGENCE SHARING

MISP workshop

Introduction into Information Sharing using MISP for CSIRTs

Team CIRCL

TLP:WHITE

CRI MISP 2024

Plan for this session

- Explanation of the CSIRT use case for information sharing and what CIRCL does
- Building an information sharing community and best practices[1]
- Quick demo of MISP capabilities

[1] We published the complete guidelines in https://www.x-isac.org/assets/images/guidelines_to_set-up_an_ISAC.pdf

Communities operated by CIRCL

- As a CSIRT, CIRCL operates a wide range of communities
- We use it as an **internal tool** to cover various day-to-day activities
- Whilst being the main driving force behind the development, we're also one of the largest consumers
- Different communities have different needs and restrictions

Communities operated by CIRCL

- Private sector community (fall-back community)
  - Our largest sharing community
  - Over **+1500 organisations**
  - **+4000 users**
  - Functions as a central hub for a lot of sharing communities
  - Private organisations, Researchers, Various SoCs, some CSIRTs, etc
- CSIRT community
  - Tighter community
  - National CSIRTs, connections to international organisations, etc

Communities operated by CIRCL

- Financial sector community
  - Banks, payment processors, etc.
  - Sharing of **mule accounts** and **non-cyber threat information**
- X-ISAC[2]
  - **Bridging the gap** between the various sectorial and geographical ISACs
  - Goal is to **bootstrap the cross-sectorial sharing** along with building the infrastructure to enable sharing when needed
  - Provide a basic set of threat intelligence for new ISACs

[2] https://www.x-isac.org/

Communities operated by CIRCL

- The ATT&CK EU community[3]
  - Work on attacker modelling
  - With the assistance of MITRE themselves
  - Unique opportunity to **standardise on TTPs**
  - Increasing the use of TTPs[4] especially in sharing community like MITRE ATT&CK
  - Major increase of MITRE ATT&CK context in sharing communities

[3] https://www.attack-community.org/

[4] Tactics, Techniques and Procedures

Communities supported by CIRCL

- ISAC / specialised community MISPs
  - Topical or community specific instances hosted or co-managed by CIRCL
  - Examples, GSMA, FIRST.org, CSIRTs network, etc
  - Often come with their **own taxonomies and domain specific object definitions**
- FIRST.org's MISP community
- Telecom and Mobile operators' such as GSMA T-ISAC community
- Various ad-hoc communities for cyber security exercises
  - The ENISA exercise (Cyber Europe)
  - NATO Locked Shields exercise

Sharing Scenarios in MISP

- Sharing can happen for **many different reasons**. Let's see what we believe are the typical CSIRT scenarios
- We can generally split these activities into 4 main groups when we're talking about traditional CSIRT tasks:
  - Core services
  - Proactive services
  - Advanced services
  - Sharing communities managed by CSIRTs for various tasks

CSIRT core services

- Incident response
  - **Internal storage** of incident response data
  - Sharing of indicators **derived from incident response**
  - **Correlating data** derived and using the built in analysis tools
  - **Enrichment** services
  - **Collaboration** with affected parties via MISP during IR
  - **Co-ordination** and collaboration
  - **Takedown** requests
- Alerting of information leaks (integration with **AIL**[5])

[5] https://www.ail-project.org/

CSIRT proactive services

- **Contextualising** both internal and external data
- **Collection** and **dissimination** of data from various sources (including OSINT)
- Storing, correlating and sharing own manual research (**reversing, behavioural analysis**)
- Aggregating automated collection (**sandboxing, honeypots, spamtraps, sensors**)
  - MISP allows for the creation of **internal MISP "clouds"**
  - Store **large specialised datasets** (for example honeypot data)
  - MISP has **interactions with** a large set of such **tools** (Cuckoo, Mail2MISP, etc)
- **Situational awareness** tools to monitor trends and adversary TTPs within my sector/geographical region (MISP-dashboard, built in statistics)

CSIRT advanced services

- Supporting **forensic analysts**
- Collaboration with **law enforcement**
- **Vulnerability** information sharing
  - **Notifications** to the constituency about relevant vulnerabilities
  - **Co-ordinating** with vendors for notifications (*)
  - Internal / closed community sharing of pentest results

CSIRTs' management of sharing communities for constituent actions:

- **Reporting** non-identifying information about incidents (such as outlined in NISD)
- **Seeking** and engaging in **collaboration** with CSIRT or other parties during an incident
- Pre-sharing information to **request for help** / additional information from the community
- **Pseudo-anonymised sharing** through 3rd parties to **avoid attribution** of a potential target
- Building processes for **other types of sharing** to get the community engaged and acquainted with the methodologies of sharing (mule account information, disinformation campaigns, border control, etc)

A quick note on legal compliance...

- Collaboration with legal advisors as part of a CEF project for creating compliance documents
  - Information sharing and cooperation **such as GDPR**
  - How MISP enables stakeholders identified by the **NISD** to perform key activities
  - **AIL** and MISP
- For more information: `https://github.com/CIRCL/compliance` about DORA, GDPR, ISO 27010 and MISP compliance

Bringing different sharing communities together

- We generally all **end up sharing with peers that face similar threats**
- Division is either **sectorial or geographical**
- So why even bother with trying to bridge these communities?

Advantages of cross sectorial sharing

- **Reuse of TTPs** across sectors
- Being hit by something that **another sector has faced before**
- **Hybrid threats** - how seemingly unrelated things may be interesting to correlate
- Prepare other communities for the capability and **culture of sharing** for when the need arises for them to reach out to CSIRT
- Generally our field is ahead of several other sectors when it comes to information sharing, might as well **spread the love**

Getting started with building your own sharing community

- Starting a sharing community is **both easy and difficult** at the same time
- Many moving parts and most importantly, you'll be dealing with a **diverse group of people**
- Understanding and working with your constituents to help them face their challenges is key

Getting started with building your own sharing community

- When you are starting out - you are in a unique position to drive the community and set best practices...

Running a sharing community using MISP - How to get going?

- Different models for constituents
  - **Connecting to** a MISP instance hosted by a CSIRT
  - **Hosting** their own instance and connecting to CSIRT's MISP
  - **Becoming member** of a sectorial MISP community that is connected to CSIRT's community
- Planning ahead for future growth
  - Estimating requirements
  - Deciding early on common vocabularies
  - Offering expansion,analysis and intelligence services through MISP

Rely on our instincts to immitate over expecting adherence to rules

- **Lead by example** - the power of immitation
- Encourage **improving by doing** instead of blocking sharing with unrealistic quality controls
  - What should the information look like?
  - How should it be contextualise
  - What do you consider as useful information?
  - What tools did you use to get your conclusions?
- Side effect is that you will end up **raising the capabilities of your constituents**

What counts as valuable data?

- Sharing comes in many shapes and sizes
  - Sharing **results** / reports is the classical example
  - Sharing **enhancements** to existing data/intelligence
  - Validating data / flagging false positives (**sighting**)
  - Asking for **support and collaboration** from the community
- **Embrace all of them**. Even the ones that don't make sense right now, you never know when they come handy...

How to deal with organisations that only "leech"?

- From our own communities, only about **30%** of the organisations **actively share data**
- We have come across some communities with sharing requirements
- In our experience, this sets you up for failure because:
  - Organisations losing access are the ones who would possibly benefit the most from it
  - Organisations that want to stay above the thresholds will start sharing junk / fake data
  - You lose organisations that might turn into valuable contributors in the future

So how does one convert the passive organisations into actively sharing ones?

- Rely on **organic growth** and it takes time (+2 years is common)
- **Help** them increase their capabilities
- As mentioned before, lead by example
- Rely on the inherent value to one's self when sharing information (validation, enrichments, correlations)
- **Give credit** where credit is due, never steal the contributions of your community (that is incredibly demotivating)

Dispelling the myths around blockers when it comes to information sharing

- Sharing difficulties are not really technical issues but often it's a matter of **social interactions** (e.g. **trust**).
  - You can play a role here: organise regular workshops, conferences, have face to face meetings
- Legal restrictions
  - "Our legal framework doesn't allow us to share information."
  - "Risk of information leak is too high and it's too risky for our organization or partners."
- Practical restrictions
  - "We don't have information to share."
  - "We don't have time to process or contribute indicators."
  - "Our model of classification doesn't fit your model."
  - "Tools for sharing information are tied to a specific format, we use a different one."

Contextualising the information

- Sharing **technical information** is a **great start**
- However, to truly create valueable information for your community, always consider the context:
  - Your IDS might not care why it should alert on a rule
  - But your analysts will be interested in the threat landscape and the "big picture"
- Classify data to make sure your partners understand why it is **important for you**, so they can see why it could be **useful to them**
- Massively important once an organisation has the maturity to filter the most critical **subsets of information for their own defense**

Choice of vocabularies

- MISP has a verify **versatile system** (taxonomies) for classifying and marking data
- However, this includes different vocabularies with obvious overlaps
- MISP allows you to **pick and choose vocabularies** to use and enforce in a community
- Good idea to start with this process early
- If you don’t find what you’re looking for:
  - Create your own (JSON format, no coding skills required)
  - If it makes sense, share it with us via a pull request for redistribution

Shared libraries of meta-information (Galaxies)

- The MISPProject in co-operation with partners provides a **curated list of galaxy information**
- Can include information packages of different types, for example:
  - Threat actor information (event different models or approaches)
  - Specialised information such as Ransomware, Exploit kits, etc
  - Methodology information such as preventative actions
  - Classification systems for methodologies used by adversaries - ATT&CK
- Consider improving the default libraries or contributing your own (simple JSON format)
- If there is something you cannot share, run your own galaxies and **share it out of bound** with partners
- Pull requests are always welcome

False-positive handling

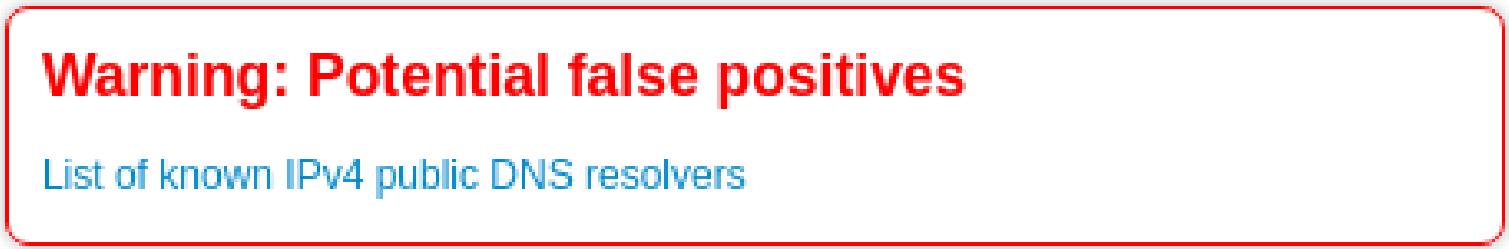
- You might often fall into the trap of discarding seemingly "junk" data
- Besides volume limitations (which are absolutely valid, fear of false-positives is the most common reason why people discard data) - Our recommendation:
  - Be lenient when considering what to keep
  - Be strict when you are feeding tools
- MISP allows you to **filter out the relevant data on demand** when feeding protective tools
- What may seem like **junk to you may** be absolutely **critical to other users**

Many objectives from different user-groups

- Sharing indicators for a **detection** matter.
  - 'Do I have infected systems in my infrastructure or the ones I operate?'
- Sharing indicators to **block**.
  - 'I use these attributes to block, sinkhole or divert traffic.'
- Sharing indicators to **perform intelligence**.
  - 'Gathering information about campaigns and attacks. Are they related? Who is targeting me? Who are the adversaries?'
- $\rightarrow$ These objectives can be conflicting (e.g. False-positives have different impacts)

False-positive handling

- **Analysts** will often be interested in the **modus operandi** of threat actors over **long periods of time**
- Even cleaned up infected hosts might become interesting again (embedded in code, recurring reuse)
- Use the tools provided to eliminate obvious false positives instead and limit your data-set to the most relevant sets



Managing sub-communities

- Often within a community **smaller bubbles of information sharing will form**
- For example: Within a national private sector sharing community, specific community for financial institutions
- Sharing groups serve this purpose mainly
- As a CSIRT running a national community, consider bootstraping these sharing communities
- Organisations can of course self-organise, but you are the ones with the know-how to get them started

Managing sub-communities

- Consider compartmentalisation - does it make sense to move a secret squirrel club to their own sharing hub to avoid accidental leaks?
- Use your **best judgement** to decide which communities should be separated from one another
- Create sharing hubs with **manual data transfer** if needed
- Some organisations will even have their data air-gapped - Feed system
- **Create guidance** on what should be shared outside of their bubbles - organisations often lack the insight / experience to decide how to get going. Take the initiative!

Get in touch if you need some help to get started

- Getting started with building a new community can be daunting. Feel free to get in touch with us if you have any questions!
- Contact: info@circl.lu
- https://www.circl.lu/
  https://www.misp-project.org/
- https://github.com/MISP
  https://gitter.im/MISP/MISP
  https://twitter.com/MISPProject

MISP
INTELLIGENCE SHARING

MISP and Decaying of Indicators

An indicator scoring method and ongoing implementation in MISP

Team CIRCL

info@circl.lu

November 2, 2026

Expiring IOCs: Why and How?

Indicators - Problem Statement

- **Sharing information** about threats **is crucial**
- Organisations are sharing more and more

Contribution by **unique organisation** (`Orgc.name`) on MISPPriv:

Date	Unique Org
2013	17
2014	43
2015	82
2016	105
2017	118
2018	125
2019-10	135

```
{
    "distribution": [1, 2, 3]
}
```

Indicators - Problem Statement

- Various users and organisations can share data via MISP, multiple parties can be involved
  - **Trust**, **data quality** and **time-to-live** issues
  - Each user/organisation has **different use-cases** and interests
    - Conflicting interests such as operational security, attribution,... (depends on the user)

  $\rightarrow$ Can be partially solved with *Taxonomies*

Indicators - Problem Statement

- Various users and organisations can share data via MISP, multiple parties can be involved
  - **Trust**, **data quality** and **time-to-live** issues
  - Each user/organisation has **different use-cases** and interests
    - Conflicting interests such as operational security, attribution,... (depends on the user)

  $\rightarrow$ Can be partially solved with *Taxonomies*

- Attributes can be shared in large quantities (more than 7.3 million on `MISPPRIV`)
  - Partial info about their **freshness** (*Sightings*)
  - Partial info about their **validity** (last update)

  $\rightarrow$ Can be partially solved with our *Decaying model*

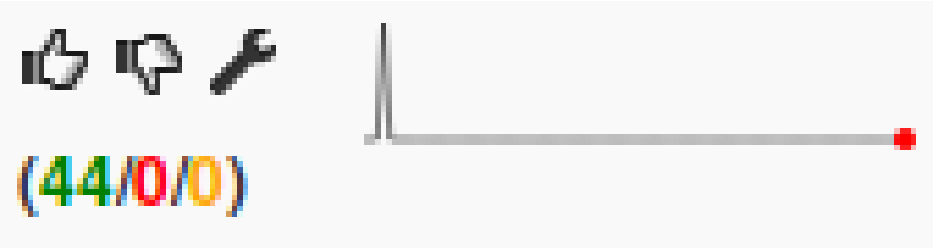
Requirements to enjoy the decaying feature in MISP

- Starting from **MISP 2.4.116**, the decaying feature is available
- Don't forget to update the decay models and enable the ones you want
- The decaying feature has no impact on the information in MISP, it's just an overlay to be used in the user-interface and API
- Decay strongly relies on *Taxonomies* and *Sightings*, don't forget to review their configuration

Sightings - Refresher

Sightings add temporal context to indicators. A user, script or an IDS can extend the information related to indicators by reporting back to MISP that an indicator has been `seen`, or that an indicator can be considered as a `false-positive`

- *Sightings* give more credibility/visibility to indicators
- This information can be used to **prioritise and decay indicators**



Organisations opt-in - setting a level of confidence

MISP is a peer-to-peer system, information passes through multiple instances.

- **Producers can add context** (such as tags from *Taxonomies*, *Galaxies*) about their asserted confidence or the reliability of the data
- Consumers can have **different levels of trust** in the producers and/or analysts themselves
- Users might have other contextual needs

$\rightarrow$ Achieved thanks to *Taxonomies*

Taxonomies - Refresher (1)

Taxonomies

« previous | 1 | 2 | next »

Id	Namespace	Description	Version	Enabled	Required	Active Tags	Actions
181	workflow	Workflow support language is a common language to support intelligence analysts to perform their analysis on data and information.	9	Yes	☐	**27** / 26 (enable all)	− 👁 🗑
180	vocabulaire-des-probabilites-estimatives	Ce vocabulaire attribue des valeurs en pourcentage à certains énoncés de probabilité	2	Yes	☐	**5** / 5	− 👁 🗑
179	threats-to-dns	An overview of some of the known attacks related to DNS as described by Torabi, S., Boukhtouta, A., Assi, C., & Debbabi, M. (2018) in Detecting Internet Abuse by Analyzing Passive DNS Traffic: A Survey of Implemented Systems. IEEE Communications Surveys & Tutorials, 1–1. doi:10.1109/comst.2018.2849614	1	No	☐	**0** / 18	+ 👁 🗑
178	targeted-threat-index	The Targeted Threat Index is a metric for assigning an overall threat ranking score to email messages that deliver malware to a victim's computer. The TTI metric was first introduced at SecTor 2013 by Seth Hardy as part of the talk "RATastrophe: Monitoring a Malware Menagerie" along with Katie Kleemola and Greg Wiseman.	2	Yes	☐	**11** / 11	− 👁 🗑

- Tagging is a simple way to attach a classification to an *Event* or an *Attribute*
- Classification must be globally used to be efficient

Taxonomies - Refresher (2)

ADMIRALTY-SCALE Taxonomy Library

Id	127
Namespace	admiralty-scale
Description	The Admiralty Scale or Ranking (also called the NATO System) is used to rank the reliability of a source and the credibility of an information. Reference based on FM 2-22.3 (FM 34-52) HUMAN INTELLIGENCE COLLECTOR OPERATIONS and NATO documents.
Version	4
Enabled	Yes (disable)

« previous | next »

Filter

Tag	Expanded	Numerical value	Events	Attributes	Tags	Action
admiralty-scale:information-credibility="1"	Information Credibility: Confirmed by other sources	100	6	0	admiralty-scale:information-credibility="1"	
admiralty-scale:information-credibility="2"	Information Credibility: Probably true	75	21	1	admiralty-scale:information-credibility="2"	
admiralty-scale:information-credibility="3"	Information Credibility: Possibly true	50	16	5	admiralty-scale:information-credibility="3"	
admiralty-scale:information-credibility="4"	Information Credibility: Doubtful	25	2	0	admiralty-scale:information-credibility="4"	
admiralty-scale:information-credibility="5"	Information Credibility: Improbable	0	1	0	admiralty-scale:information-credibility="5"	
admiralty-scale:information-credibility="6"	Information Credibility: Truth cannot be judged	50	9	2	admiralty-scale:information-credibility="6"	
admiralty-scale:source-reliability="a"	Source Reliability: Completely reliable	100	1	0	admiralty-scale:source-reliability="a"	
admiralty-scale:source-reliability="b"	Source Reliability: Usually reliable	75	21	76	admiralty-scale:source-reliability="b"	
admiralty-scale:source-reliability="c"	Source Reliability: Fairly reliable	50	9	8	admiralty-scale:source-reliability="c"	
admiralty-scale:source-reliability="d"	Source Reliability: Not usually reliable	25	2	0	admiralty-scale:source-reliability="d"	
admiralty-scale:source-reliability="e"	Source Reliability: Unreliable	0	0	0	admiralty-scale:source-reliability="e"	
admiralty-scale:source-reliability="f"	Source Reliability: Reliability cannot be judged	50	10	7	admiralty-scale:source-reliability="f"	
admiralty-scale:source-reliability="g"	Source Reliability: Deliberatly deceptive	0	N/A	N/A		+

→ Cherry-pick allowed *Tags*

Taxonomies - Refresher (3)

- Some taxonomies have `numerical_value`
  - → Can be used to prioritise *Attributes*

Description	Value
Completely reliable	100
Usually reliable	75
Fairly reliable	50
Not usually reliable	25
Unreliable	0
Reliability cannot be judged	50 ?
Deliberatly deceptive	0 ?

Description	Value
Confirmed by other sources	100
Probably true	75
Possibly true	50
Doubtful	25
Improbable	0
Truth cannot be judged	50 ?

Scoring Indicators: Our solution

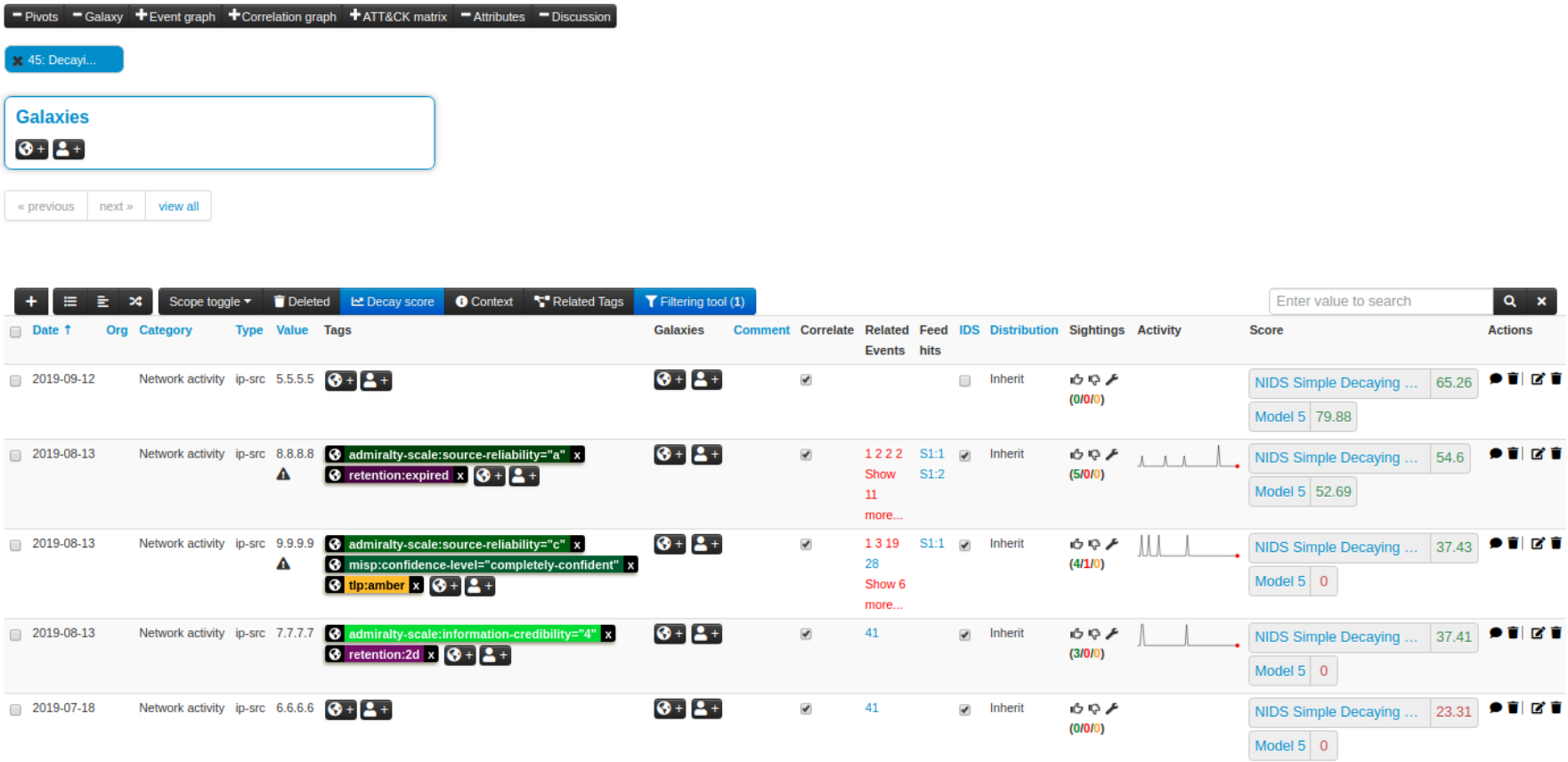
$$\texttt{score}(\texttt{Attribute}) = \texttt{base_score}(\texttt{Attribute, Model}) \bullet \texttt{decay}(\texttt{Model, time})$$

Where,

- $\texttt{score} \in [0, +\infty$
- $\texttt{base_score} \in [0, 100]$
- `decay` is a function defined by model's parameters controlling decay speed
- `Attribute` Contains *Attribute*'s values and metadata (*Taxonomies*, *Galaxies*, ...)
- `Model` Contains the *Model*'s configuration

Current implementation in MISP

Implementation in MISP: `Event/view`



- `Decay score` toggle button
  - Shows Score for each *Models* associated to the *Attribute* type

Implementation in MISP: API result

/attributes/restSearch

```
"Attribute": [
  {
    "category": "Network activity",
    "type": "ip-src",
    "to_ids": true,
    "timestamp": "1565703507",
    [...]
    "value": "8.8.8.8",
    "decay_score": [
      {
        "score": 54.475223849544456,
        "decayed": false,
        "DecayingModel": {
          "id": "85",
          "name": "NIDS Simple Decaying Model"
        }
      }
    ],
[...]
```

Implementation in MISP: Playing with Models

- **Automatic scoring** based on default values
- **User-friendly UI** to manually set *Model* configuration (lifetime, decay, etc.)
- **Simulation** tool
- Interaction through the **API**
- Opportunity to create your **own** formula or algorithm

Decaying Models in Depth

Scoring Indicators: base_score (1)

$$\texttt{score}(\texttt{Attribute}) = \texttt{base_score}(\texttt{Attribute, Model}) \bullet \texttt{decay}(\texttt{Model, time})$$

When scoring indicators[1], multiple parameters[2] can be taken into account. The **base score** is calculated with the following in mind:

- Data reliability, credibility, analyst skills, custom prioritisation tags (economical-impact), etc.
- Trust in the source

$$\texttt{base_score} = \omega_{tg} \cdot \textit{tags} + \omega_{sc} \cdot \textit{source_confidence}$$

Where,

$$\omega_{sc} + \omega_{tg} = 1$$

[1] Paper available: https://arxiv.org/pdf/1803.11052

[2] at a variable extent as required

Scoring Indicators: base_score (2)

Current implentation ignores source_confidence:

$$\rightarrow \texttt{base_score} = \textit{tags}$$

Tag	Computation			Result
	Eff. Ratio		numerical_value	
admiralty-scale:source-reliability="Completely reliable"	0.50	*	100.00	50.00
phishing:psychological-acceptability="high"	0.50	*	75.00	37.50
				87.50

→ The base_score can be use to prioritize attribute based on their attached context and source

Scoring Indicators: decay speed (1)

$$\texttt{score}(\texttt{Attribute}) = \texttt{base_score}(\texttt{Attribute, Model}) \bullet \texttt{decay}(\texttt{Model, time})$$

The `decay` is calculated using:

- The `lifetime` of the indicator
  - May vary depending on the indicator type
  - short for an IP, long for an hash
- The `decay rate`, or speed at which an attribute loses score over time
- The `time` elapsed since the latest update or sighting

Scoring Indicators: putting it all toghether

$\rightarrow$ `decay rate` is **re-initialized upon sighting** addition, or said differently, the `score` is reset to its base score as new *sightings* are applied.

$$score = base_score \cdot \left(1 - \left(\frac{t}{\tau}\right)^{\frac{1}{\delta}}\right)$$

- $\tau =$ `lifetime`
- $\delta =$ `decay speed`

Implementation in MISP: Models definition

$$\nearrow score = base_score \cdot \left(1 - \left(\frac{t}{\tau}\right)^{\frac{1}{\delta}}\right)$$

Models are an instanciation of the formula where elements can be defined:

- Parameters: `lifetime, decay_rate, threshold`
- `base_score`
- `default base_score`
- formula
- associate *Attribute* types
- creator organisation

Implementation in MISP: Models Types

Multiple model types are available

- **Default Models**: Models created and shared by the community. Available from `misp-decaying-models` repository[3].
  - $\rightarrow$ Not editable
- **Organisation Models**: Models created by a user belonging to an organisation
  - These models can be hidden or shared to other organisation
  - $\rightarrow$ Editable

[3] https://github.com/MISP/misp-decaying-models.git

Implementation in MISP: Index

Decaying Models

« previous | next »

All Models | My Models | Shared Models | Default Models

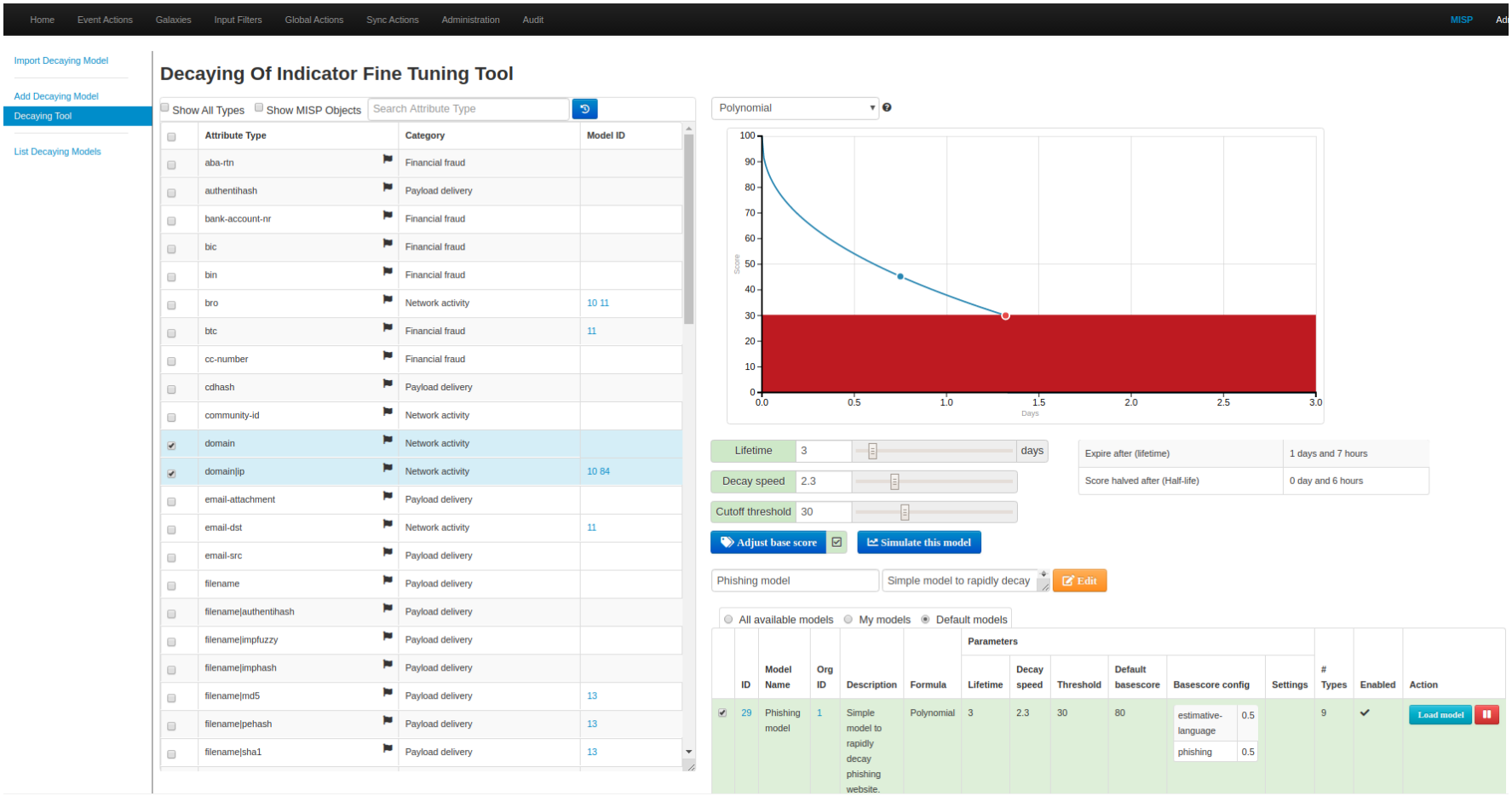
ID	Organization	Usable to everyone	Name	Description	Parameters { }	Formula	# Assigned Types	Version	Enabled	Actions
29	1	✔	Phishing model	Simple model to rapidly decay phishing website.	{ "lifetime": 3, "decay_speed": 2.3, "threshold": 30, "default_base_score": 80, "base_score_config": { "estimative-language": 0.5, "phishing": 0.5 } }	Polynomial	9	1	✔	
85	1	✘	NIDS Simple Decaying Model MISP	Simple decaying model for Network Intrusion Detection System (NIDS).	{ "lifetime": 120, "decay_speed": 2, "threshold": 30, "default_base_score": 80, "base_score_config": { "estimative-language": 0.25, "priority-level": 0.25, "retention": 0.25, "targeted-threat-index": 0.125, "false-positive": 0.125 } }	Polynomial	13	1	✔	

Page 1 of 1, showing 2 records out of 2 total, starting on record 1, ending on 2

« previous | next »

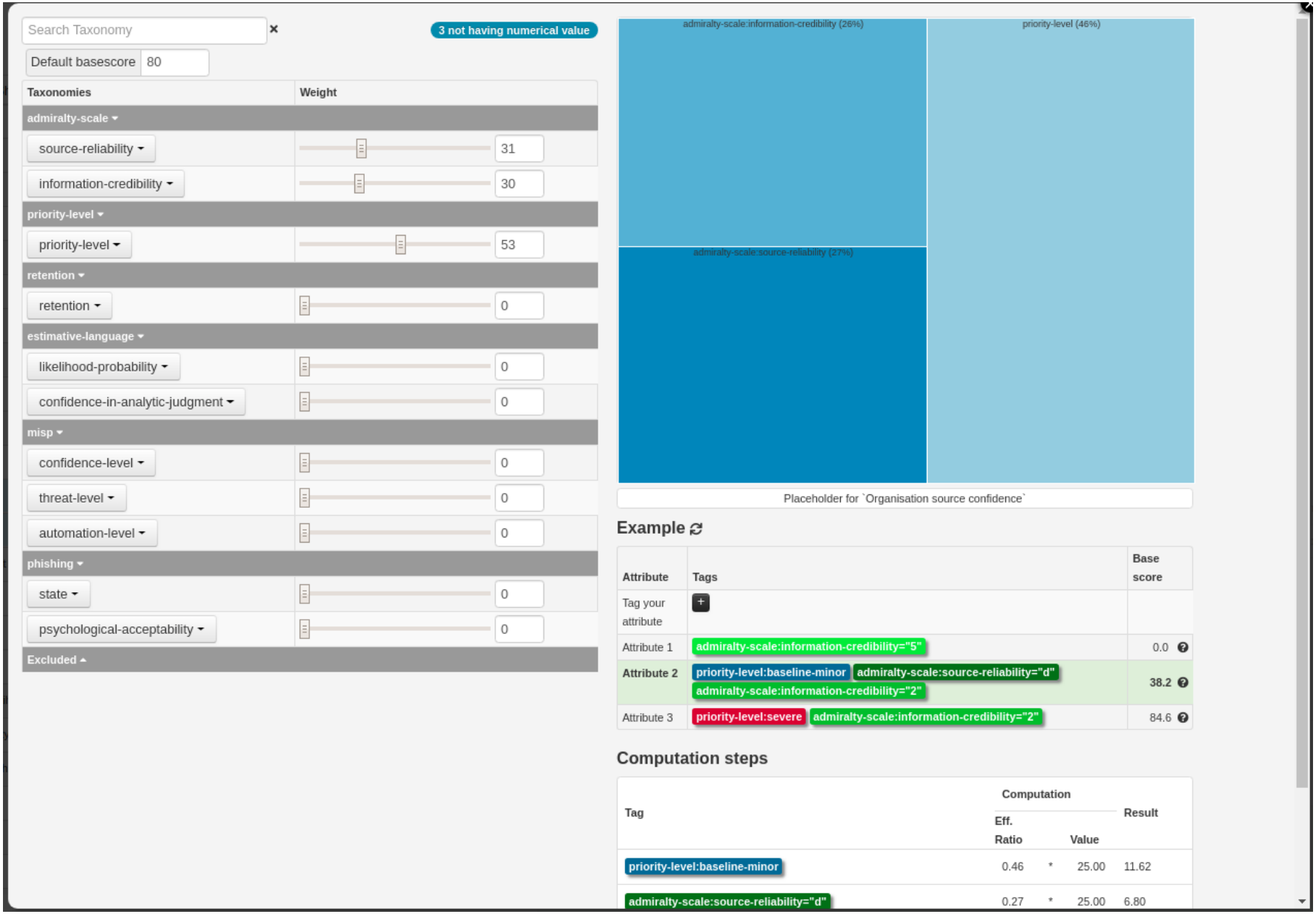
View, update, add, create, delete, enable, export, import

Implementation in MISP: Fine tuning tool



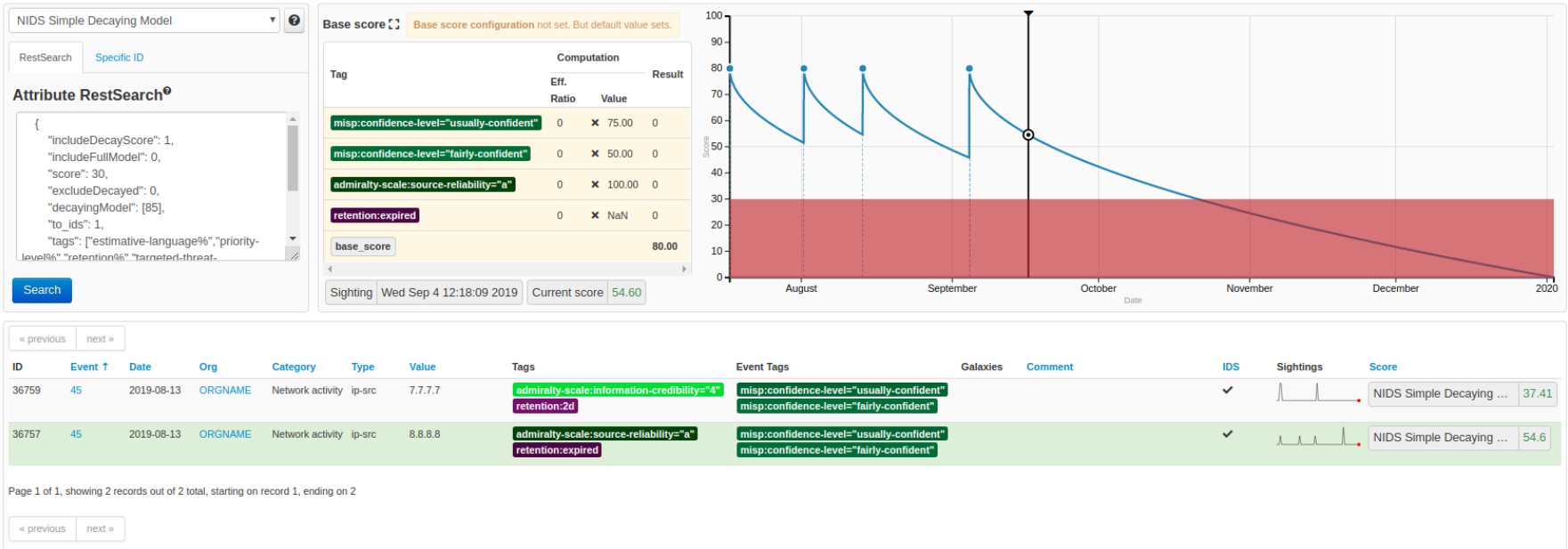
Create, modify, visualise, perform mapping

Implementation in MISP: base_score tool



Adjust Taxonomies relative weights

Implementation in MISP: simulation tool



Simulate *Attributes* with different *Models*

Implementation in MISP: API query body

/attributes/restSearch

```
{
    "includeDecayScore": 1,
    "includeFullModel": 0,
    "excludeDecayed": 0,
    "decayingModel": [85],
    "modelOverrides": {
        "threshold": 30
    }
    "score": 30,
}

```

Creating a new decay algorithm (1)

The current architecture allows users to create their **own** formulae.

1. Create a new file `$filename` in `app/Model/DecayingModelsFormulas/`
2. Extend the Base class as defined in `DecayingModelBase`
3. Implement the two mandatory functions `computeScore` and `isDecayed` using your own formula/algorithm
4. Create a Model and set the formula field to `$filename`

Use cases:

- Add support for **more feature** (expiration taxonomy)
- **Query external services** then influence the score
- Completely **different approach** (i.e streaming algorithm)
- ...

Creating a new decay algorithm (2)

```php
<?php
include_once 'Base.php';

class Polynomial extends DecayingModelBase
{
    public const DESCRIPTION = 'The description of your new decaying algorithm';

    public function computeScore($model, $attribute, $base_score, $elapsed_time)
    {
        // algorithm returning a numerical score
    }

    public function isDecayed($model, $attribute, $score)
    {
        // algorithm returning a boolean stating
        // if the attribute is expired or not
    }
}
?>
```

Decaying Models 2.0

- Improved support of *Sightings*
  - `False positive` *Sightings* should somehow reduce the score
  - `Expiration` *Sightings* should mark the attribute as decayed
- Potential *Model* improvements
  - Instead of resetting the score to `base_score` once a *Sighting* is set, the score should be increased additively (based on a defined coefficient); thus **prioritizing surges** rather than infrequent *Sightings*
  - Take into account related *Tags* or *Correlations* when computing score
- Increase *Taxonomy* coverage
  - Users should be able to manually override the `numerical_value` of *Tags*
- For specific type, take into account data from other services
  - Could fetch data from *BGP ranking*, *Virus Total*, *Passive X* for IP/domain/... and adapt the score

MISP

INTELLIGENCE SHARING

MISP and Decaying of Indicators

Primer for indicator scoring in MISP

Team CIRCL

info@circl.lu

November 2, 2026

Outline of the presentation

- Present the components used in MISP to expire IOCs
- Present the current state of Indicators life-cycle management in MISP

Expiring IOCs: Why and How?

Indicators lifecycle - Problem Statement

- **Sharing information** about threats **is crucial**
- Organisations are sharing more and more

Contribution by **unique organisation** (`Orgc.name`) on MISPPriv:

Date	Unique Org
2013	17
2014	43
2015	82
2016	105
2017	118
2018	125
2019-10	135

```
{
    "distribution": [1, 2, 3]
}
```

Indicators lifecycle - Problem Statement

- Various users and organisations can share data via MISP, multiple parties can be involved
  - **Trust**, **data quality** and **relevance** issues
  - Each user/organisation have **different use-cases** and interests
    - Conflicting interests: Operational security VS attribution

  $\rightarrow$ Can be partially solved with *Taxonomies*

Indicators lifecycle - Problem Statement

- Various users and organisations can share data via MISP, multiple parties can be involved
  - **Trust**, **data quality** and **relevance** issues
  - Each user/organisation have **different use-cases** and interests
    - Conflicting interests: Operational security VS attribution

  $\rightarrow$ Can be partially solved with *Taxonomies*

- Attributes can be shared in large quantities (more than 12M on `MISPPRIV` - Sept. 2020)
  - Partial info about their **freshness** (*Sightings*)
  - Partial info about their **validity** (*last_seen*)

  $\rightarrow$ Can be partially solved with our *Data model*

MISP's *Decaying model* combines the two

Requirements to enjoy the decaying feature in MISP

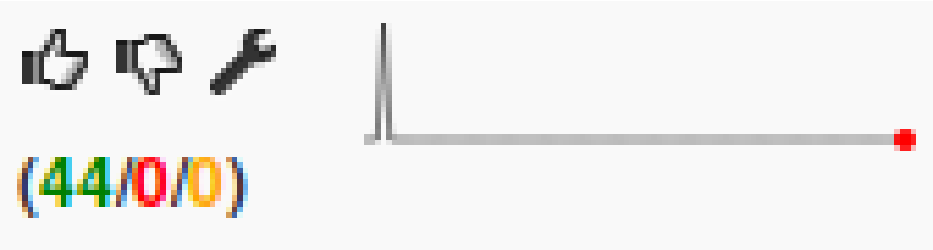
- Starting from **MISP 2.4.116**, the decaying feature is available
- **Update** decay models and **enable** some
- MISP Decaying strongly relies on *Taxonomies* and *Sightings*, don't forget to review their configuration

Note: The decaying feature has no impact on the information stored in MISP, it's just an **overlay** to be used in the user-interface and API

Sightings - Refresher (1)

Sightings add a **temporal context** to indicators.

- *Sightings* can be used to represent that you saw the IoC
- **Usecase:** Continuous feedback loop MISP $\leftrightarrow$ IDS

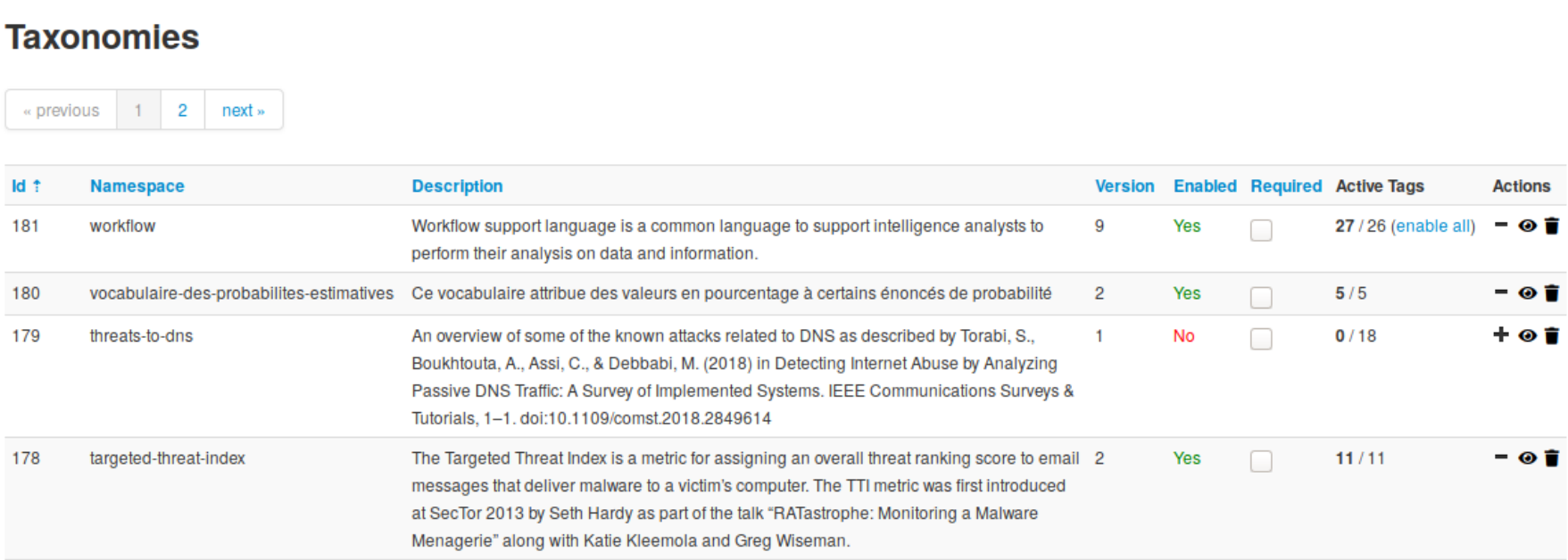


Sightings - Refresher (2)

Sightings add a **temporal context** to indicators.

- *Sightings* give more credibility/visibility to indicators
- This information can be used to **prioritise and decay indicators**

Taxonomies - Refresher (1)



Taxonomies

« previous | 1 | 2 | next »

Id	Namespace	Description	Version	Enabled	Required	Active Tags	Actions
181	workflow	Workflow support language is a common language to support intelligence analysts to perform their analysis on data and information.	9	Yes	☐	**27** / 26 (enable all)	− 👁 🗑
180	vocabulaire-des-probabilites-estimatives	Ce vocabulaire attribue des valeurs en pourcentage à certains énoncés de probabilité	2	Yes	☐	**5** / 5	− 👁 🗑
179	threats-to-dns	An overview of some of the known attacks related to DNS as described by Torabi, S., Boukhtouta, A., Assi, C., & Debbabi, M. (2018) in Detecting Internet Abuse by Analyzing Passive DNS Traffic: A Survey of Implemented Systems. IEEE Communications Surveys & Tutorials, 1–1. doi:10.1109/comst.2018.2849614	1	No	☐	**0** / 18	+ 👁 🗑
178	targeted-threat-index	The Targeted Threat Index is a metric for assigning an overall threat ranking score to email messages that deliver malware to a victim's computer. The TTI metric was first introduced at SecTor 2013 by Seth Hardy as part of the talk "RATastrophe: Monitoring a Malware Menagerie" along with Katie Kleemola and Greg Wiseman.	2	Yes	☐	**11** / 11	− 👁 🗑

- *Taxonomies* are a simple way to attach a classification to an *Event* or an *Attribute*
- Classification must be globally used to be efficient (or agreed on beforehand)

Taxonomies - Refresher (2)

ADMIRALTY-SCALE Taxonomy Library

Id	127
Namespace	admiralty-scale
Description	The Admiralty Scale or Ranking (also called the NATO System) is used to rank the reliability of a source and the credibility of an information. Reference based on FM 2-22.3 (FM 34-52) HUMAN INTELLIGENCE COLLECTOR OPERATIONS and NATO documents.
Version	4
Enabled	Yes (disable)

« previous | next »

Filter

Tag	Expanded	Numerical value	Events	Attributes	Tags	Action
admiralty-scale:information-credibility="1"	Information Credibility: Confirmed by other sources	100	6	0	admiralty-scale:information-credibility="1"	
admiralty-scale:information-credibility="2"	Information Credibility: Probably true	75	21	1	admiralty-scale:information-credibility="2"	
admiralty-scale:information-credibility="3"	Information Credibility: Possibly true	50	16	5	admiralty-scale:information-credibility="3"	
admiralty-scale:information-credibility="4"	Information Credibility: Doubtful	25	2	0	admiralty-scale:information-credibility="4"	
admiralty-scale:information-credibility="5"	Information Credibility: Improbable	0	1	0	admiralty-scale:information-credibility="5"	
admiralty-scale:information-credibility="6"	Information Credibility: Truth cannot be judged	50	9	2	admiralty-scale:information-credibility="6"	
admiralty-scale:source-reliability="a"	Source Reliability: Completely reliable	100	1	0	admiralty-scale:source-reliability="a"	
admiralty-scale:source-reliability="b"	Source Reliability: Usually reliable	75	21	76	admiralty-scale:source-reliability="b"	
admiralty-scale:source-reliability="c"	Source Reliability: Fairly reliable	50	9	8	admiralty-scale:source-reliability="c"	
admiralty-scale:source-reliability="d"	Source Reliability: Not usually reliable	25	2	0	admiralty-scale:source-reliability="d"	
admiralty-scale:source-reliability="e"	Source Reliability: Unreliable	0	0	0	admiralty-scale:source-reliability="e"	
admiralty-scale:source-reliability="f"	Source Reliability: Reliability cannot be judged	50	10	7	admiralty-scale:source-reliability="f"	
admiralty-scale:source-reliability="g"	Source Reliability: Deliberatly deceptive	0	N/A	N/A		+

→ Cherry-pick allowed *Tags*

Taxonomies - Refresher (3)

- Some taxonomies have a `numerical_value`
- Allows concepts to be used in an mathematical expression
  - → Can be used to prioritise IoCs

`admiralty-scale` taxonomy[1]

Description	Value
Completely reliable	100
Usually reliable	75
Fairly reliable	50
Not usually reliable	25
Unreliable	0
Reliability cannot be judged	50
Deliberatly deceptive	0

Description	Value
Confirmed by other sources	100
Probably true	75
Possibly true	50
Doubtful	25
Improbable	0
Truth cannot be judged	50

[1] https://github.com/MISP/misp-taxonomies/blob/master/admiralty-scale/machinetag.json

Taxonomies - Refresher (3)

`admiralty-scale` taxonomy[2]

Description	Value
Completely reliable	100
Usually reliable	75
Fairly reliable	50
Not usually reliable	25
Unreliable	0
Reliability cannot be judged	50 ?
Deliberatly deceptive	0 ?

Description	Value
Confirmed by other sources	100
Probably true	75
Possibly true	50
Doubtful	25
Improbable	0
Truth cannot be judged	50 ?

→ Users can override tag `numerical_value`

[2] https://github.com/MISP/misp-taxonomies/blob/master/admiralty-scale/machinetag.json

Scoring Indicators: Our solution

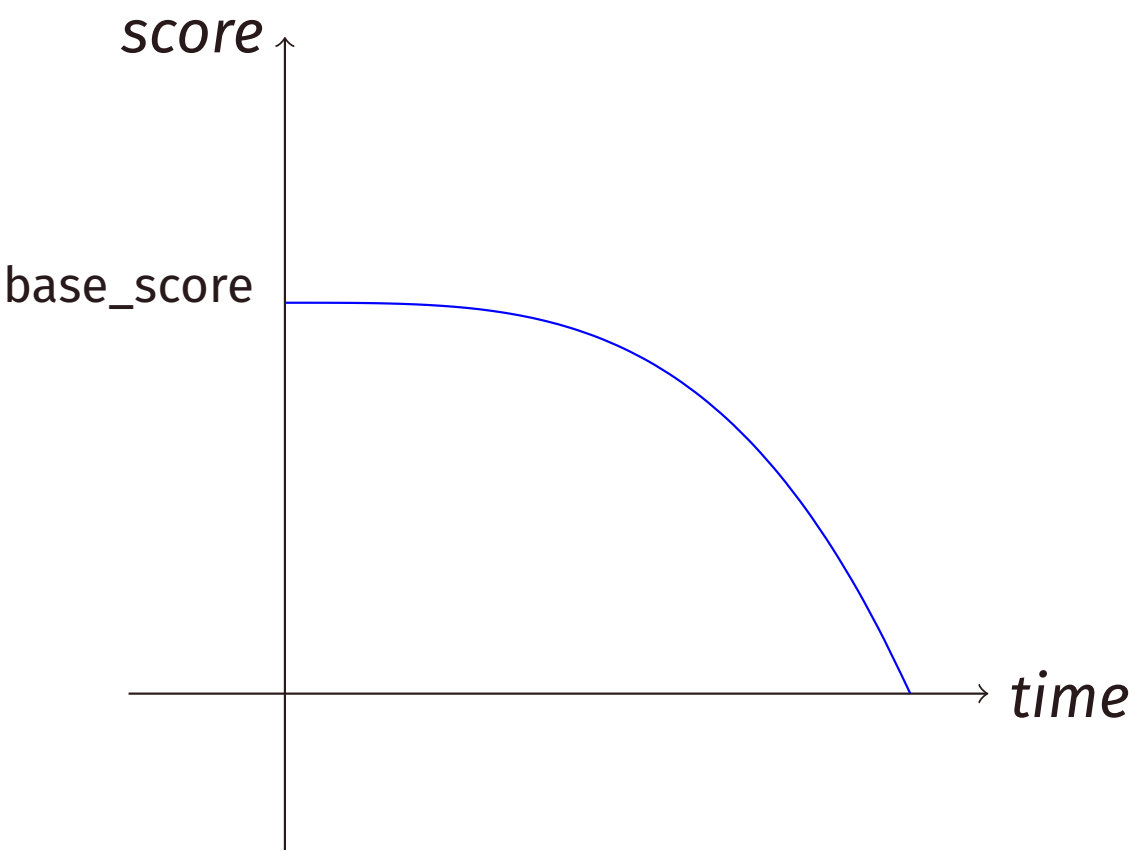
$$\texttt{score}(\texttt{Attribute}) = \texttt{base_score}(\texttt{Attribute, Model}) \bullet \texttt{decay}(\texttt{Model, time})$$

- base_score(Attribute, Model)
  - Initial score of the *Attribute* only considering the context (*Attribute's type*, *Tags*)

- decay(Model, time)
  - Function composed of the **lifetime** and **decay speed**
  - Decreases the base_score over time

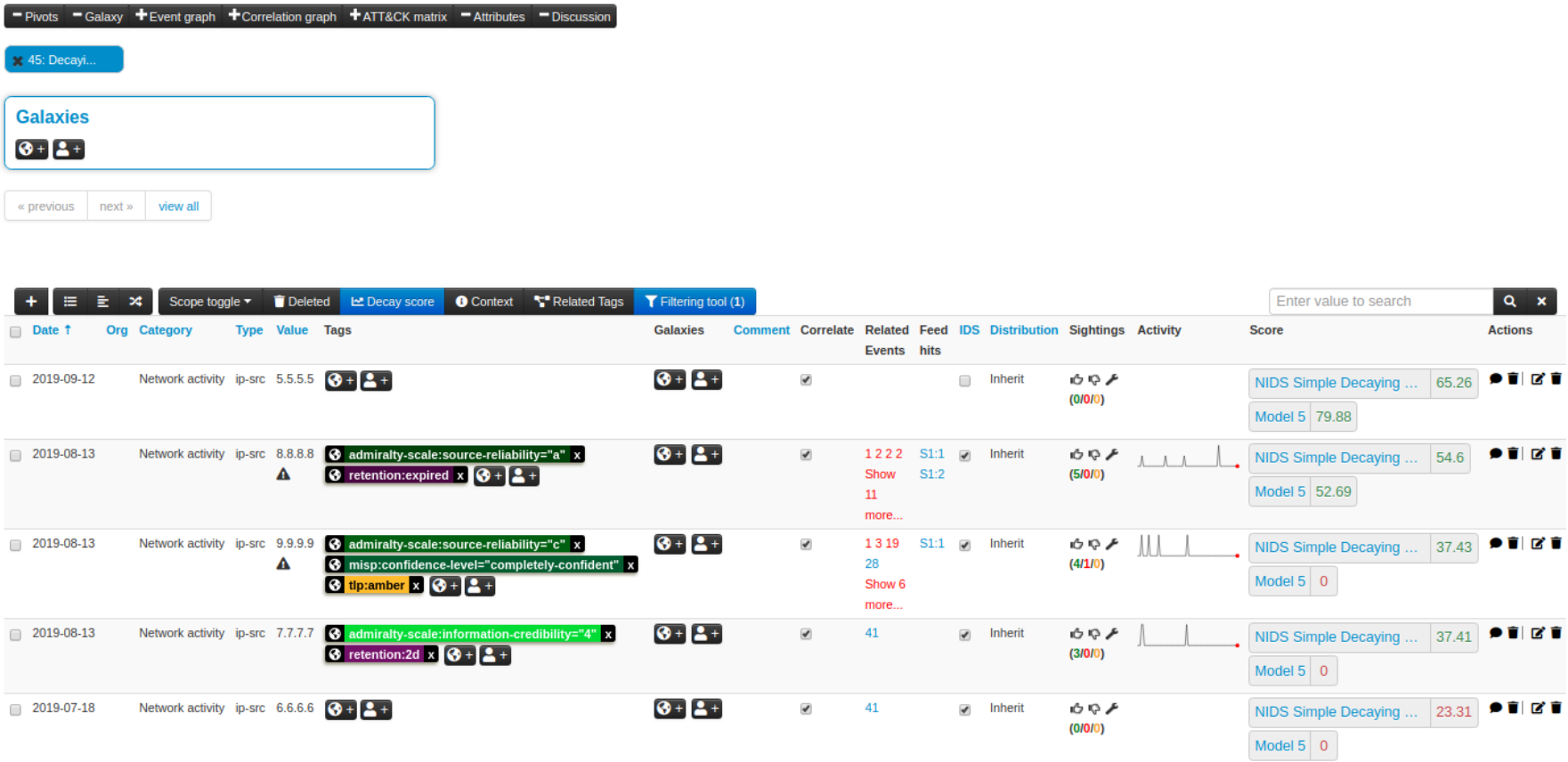
Scoring Indicators: Our solution

$$\texttt{score}(\texttt{Attribute}) = \texttt{base_score}(\texttt{Attribute, Model}) \bullet \texttt{decay}(\texttt{Model, time})$$



Current implementation in MISP

Implementation in MISP: `Event/view`



- `Decay score` toggle button
  - Shows Score for each *Models* associated to the *Attribute* type

Implementation in MISP: API result

/attributes/restSearch

```
"Attribute": [
  {
    "category": "Network activity",
    "type": "ip-src",
    "to_ids": true,
    "timestamp": "1565703507",
    [...]
    "value": "8.8.8.8",
    "decay_score": [
      {
        "score": 54.475223849544456,
        "decayed": false,
        "DecayingModel": {
          "id": "85",
          "name": "NIDS Simple Decaying Model"
        }
      }
    ],
[...]
```

Implementation in MISP: Objectives

- **Automatic scoring** based on default values
- **User-friendly UI** to manually set *Model* configuration (lifetime, decay, etc.)
- **Simulation** tool
- Interaction through the **API**
- Opportunity to create your **own** formula or algorithm

Implementation in MISP: Models definition

$$\overset{\curvearrowright}{\Gamma} \; score = base_score \cdot \left(1 - \left(\frac{t}{\tau}\right)^{\frac{1}{\delta}}\right)$$

Models are an instanciation of the formula with configurable parameters:

- Parameters: `lifetime, decay_rate, threshold`
- `base_score` computation
- `default base_score`
- associate *Attribute* types
- formula
- creator organisation

Implementation in MISP: Models Types

Two types of model are available

- **Default Models**: Created and shared by the community. Coming from `misp-decaying-models` repository[3].
  - → Not editable

- **Organisation Models**: Created by a user on MISP
  - Can be hidden or shared to other organisation
  - → Editable

[3] https://github.com/MISP/misp-decaying-models.git

Implementation in MISP: Index

Decaying Models

« previous | next »

All Models | My Models | Shared Models | Default Models

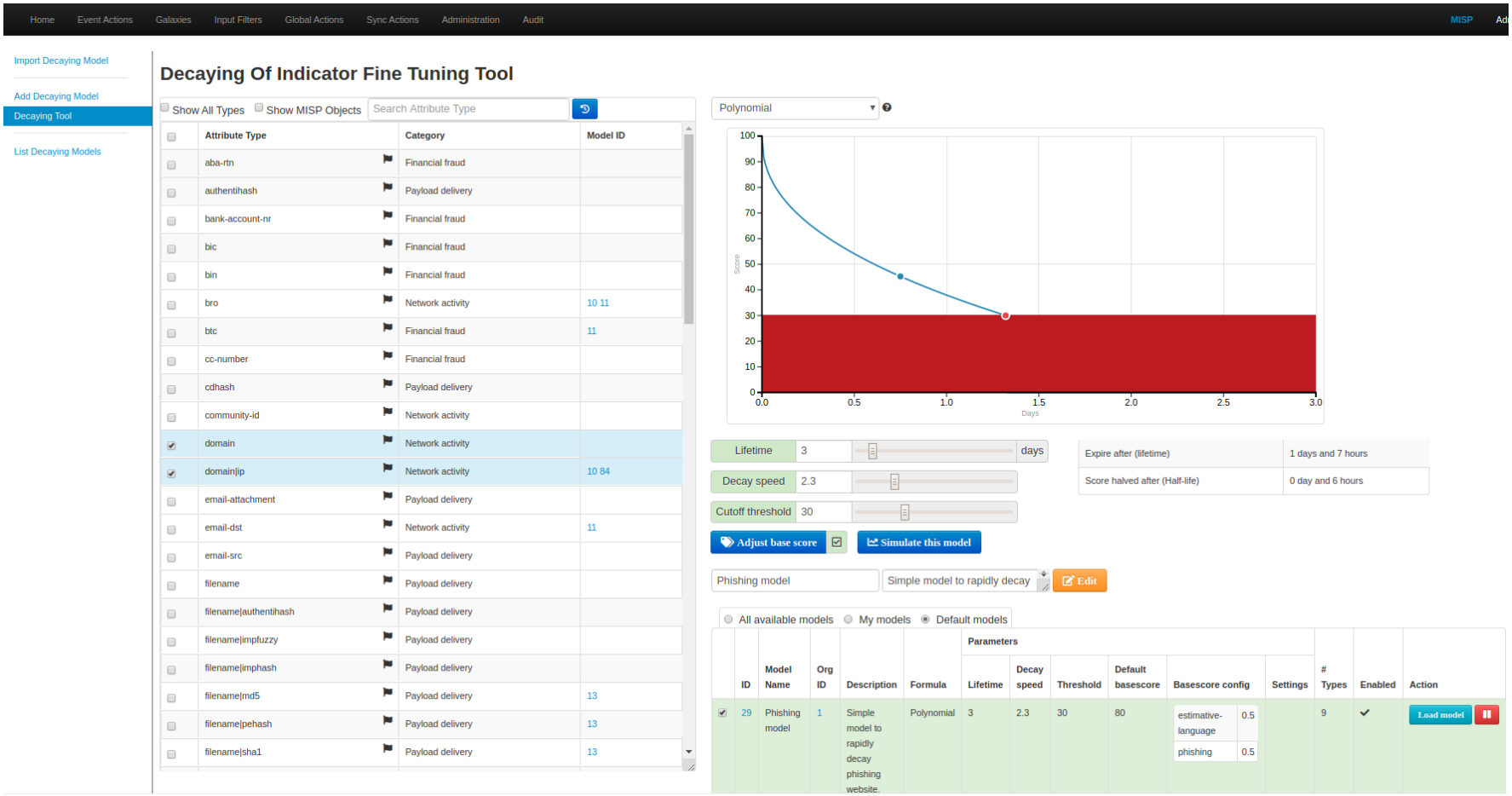
ID	Organization	Usable to everyone	Name	Description	Parameters { }	Formula	# Assigned Types	Version	Enabled	Actions
29	1	✔	Phishing model	Simple model to rapidly decay phishing website.	{ "lifetime": 3, "decay_speed": 2.3, "threshold": 30, "default_base_score": 80, "base_score_config": { "estimative-language": 0.5, "phishing": 0.5 } }	Polynomial	9	1	✔	
85	1	✘	NIDS Simple Decaying Model MISP	Simple decaying model for Network Intrusion Detection System (NIDS).	{ "lifetime": 120, "decay_speed": 2, "threshold": 30, "default_base_score": 80, "base_score_config": { "estimative-language": 0.25, "priority-level": 0.25, "retention": 0.25, "targeted-threat-index": 0.125, "false-positive": 0.125 } }	Polynomial	13	1	✔	

Page 1 of 1, showing 2 records out of 2 total, starting on record 1, ending on 2

« previous | next »

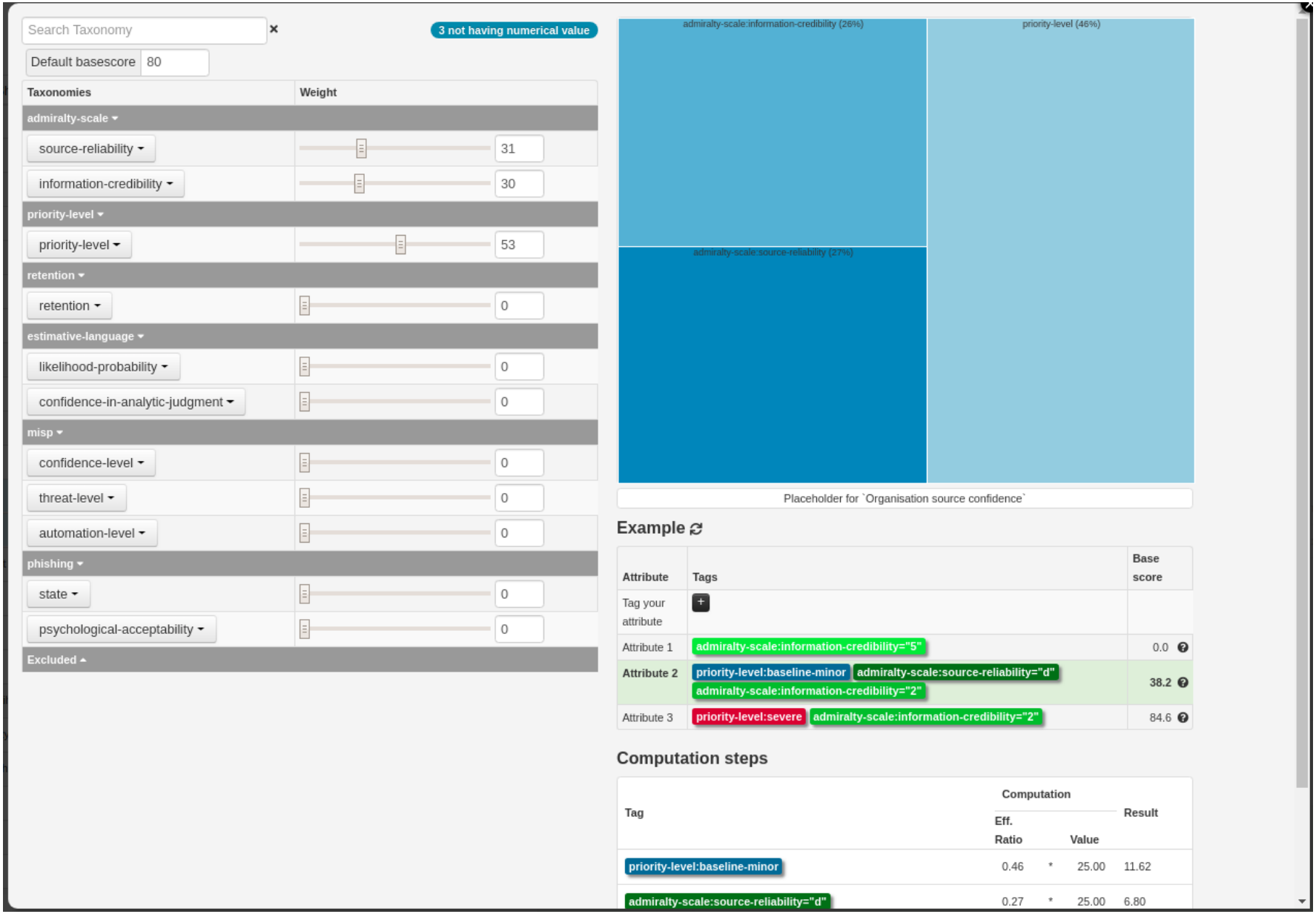
Standard CRUD operations: View, update, add, create, delete, enable, export, import

Implementation in MISP: Fine tuning tool



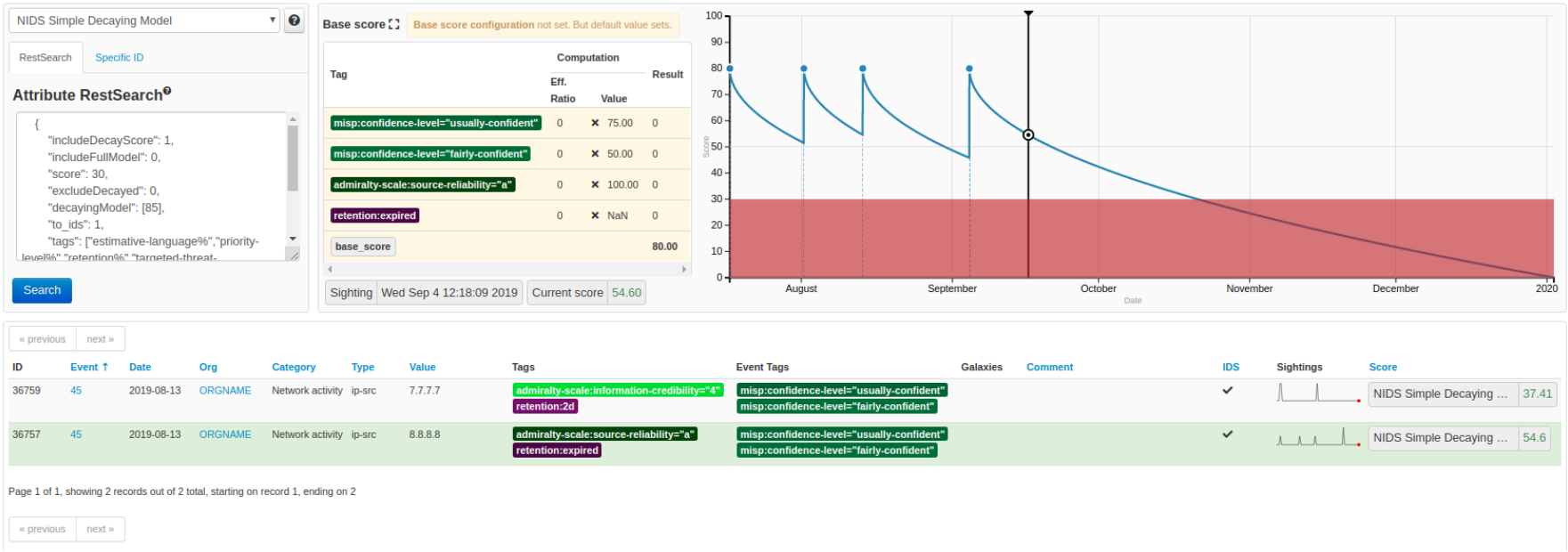
Configure models: Create, modify, visualise, perform mapping

Implementation in MISP: base_score tool



Adjust Taxonomies relative weights

Implementation in MISP: simulation tool



Simulate decay on *Attributes* with different *Models*

Implementation in MISP: API query body

/attributes/restSearch

```
{
    "includeDecayScore": 1,
    "includeFullModel": 0,
    "excludeDecayed": 0,
    "decayingModel": [85],
    "modelOverrides": {
        "threshold": 30
    }
    "score": 30,
}

```

Creating a new decay algorithm

```php
<?php
include_once 'Base.php';

class Polynomial extends DecayingModelBase
{
    public const DESCRIPTION = 'The description of your new decaying algorithm';

    public function computeScore($model, $attribute, $base_score, $elapsed_time)
    {
        // algorithm returning a numerical score
    }

    public function isDecayed($model, $attribute, $score)
    {
        // algorithm returning a boolean stating
        // if the attribute is expired or not
    }
}
?>
```

Decaying Models 2.0

- Improved support of *Sightings*
  - `False positive` *Sightings* should somehow reduce the score
  - `Expiration` *Sightings* should mark the attribute as decayed
- Potential *Model* improvements
  - Instead of resetting the score to `base_score` once a *Sighting* is set, the score should be increased additively (based on a defined coefficient); thus **prioritizing surges** rather than infrequent *Sightings*
  - Take into account related *Tags* or *Correlations* when computing score
- Increase *Taxonomy* coverage
  - Users should be able to manually override the `numerical_value` of *Tags*

MISP

INTELLIGENCE SHARING

Forensic support in MISP

Tools and visualization to support digital forensic expert

Team CIRCL

info@circl.lu

November 2, 2026

DFIR and MISP digital evidences

- **Share analyses and reports** of digital forensic evidences.
- **Propose changes** to existing analyses or reports.
- Extending existing events with additional evidences for local or use in limited distribution sharing (sharing can be defined at event level or attribute level).
- **Evaluate correlations**[1] of evidences against external or local attributes.
- **Report sightings** such as false-positive or true-positive (e.g. a partner/analyst has seen a similar indicator).

[1]MISP has a flexible correlation engine which can correlate on 1-to-1 value matches, but also on fuzzy hashing (e.g. ssdeep) or CIDR block matching.

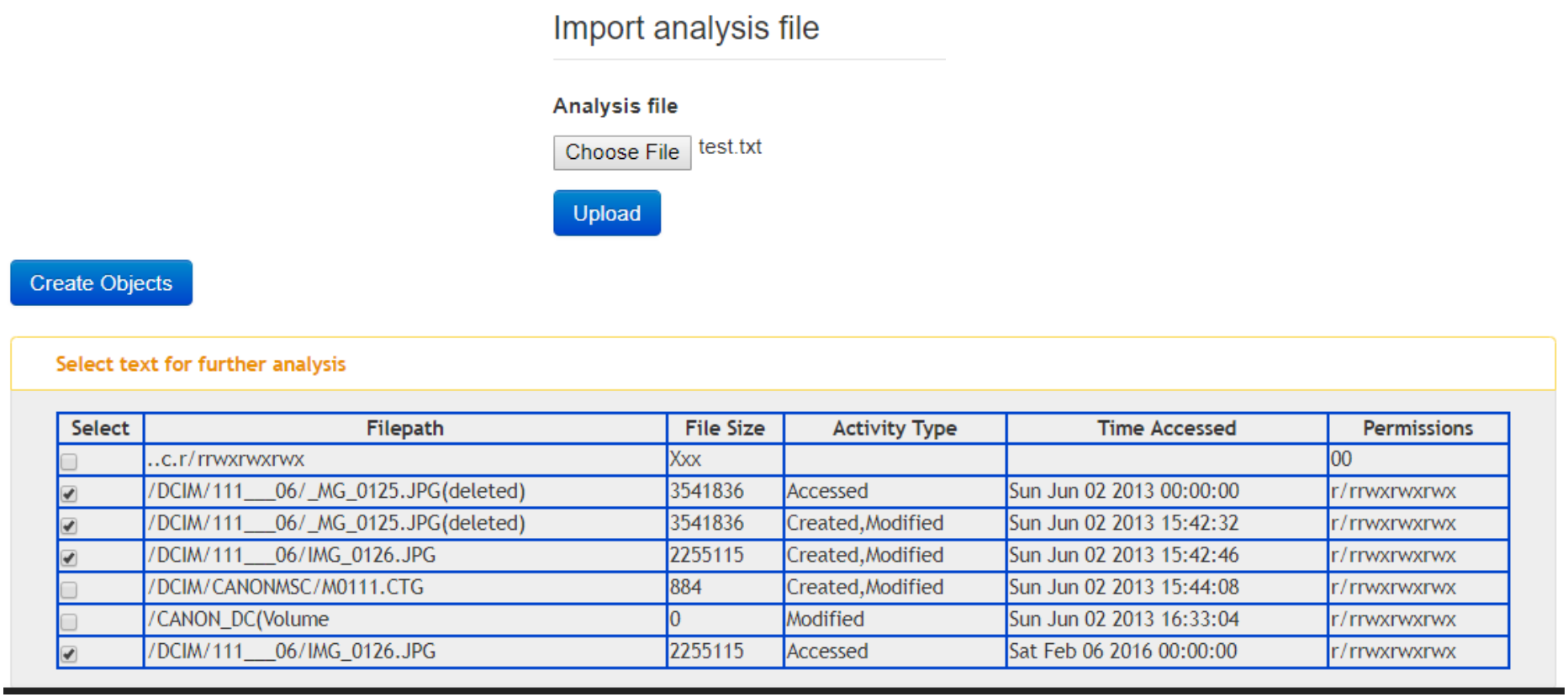
Benefits of using MISP

- LE can leverage the long-standing experience in information sharing and **bridge their use-cases** with MISP's information sharing mechanisms.
- **Accessing existing MISP information sharing communities** by receiving actionable information from CSIRT/CERT networks or security researchers.
- **Bridging LE communities with other communities**. Sharing groups can be created (and managed) cross-sectors to support specific use-cases.
- The **MISP standard** is a flexible format which can be extended by users using the MISP platform. A MISP object template can be created in under 30 minutes, allowing users to rapidly share information using their own data-models with existing communities.

Challenges and implementations

- Standard sharing mechanism for forensic cases
  - MISP allows for the efficient **collaborative** analysis of digital evidences
  - Correlation on certain attributes
- Importing disk images and file system data activity (`Mactime`)
  - Development of an adaptable import tool: From Mactime to MISP `Mactime object`
- Create, modify and visualise the timeline of events
  - Development of a flexible timeline system at the event level

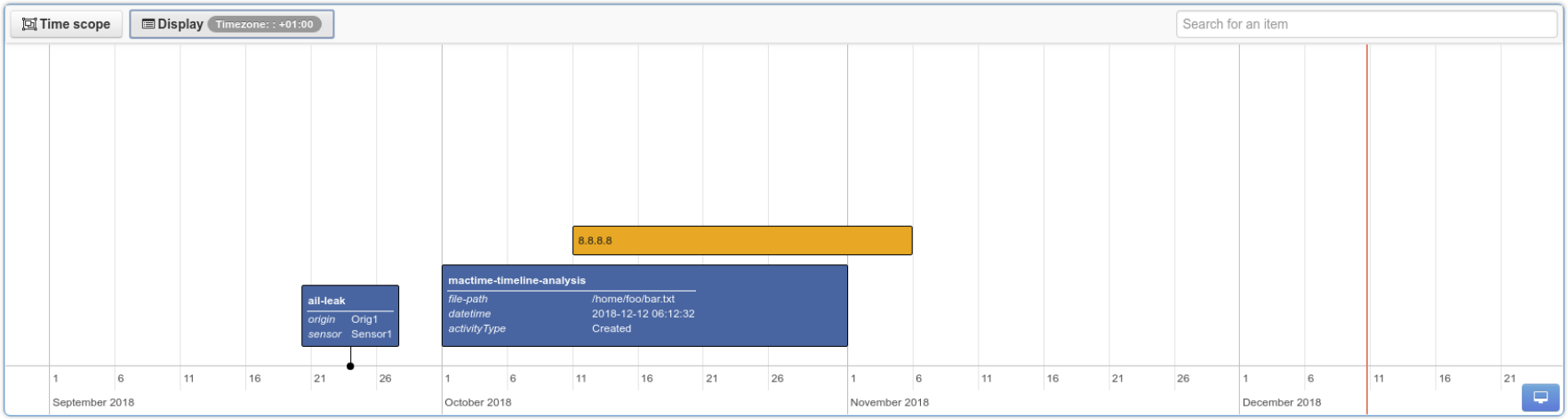
Forensic import (MISP 2.4.98)



Select	Filepath	File Size	Activity Type	Time Accessed	Permissions
☐	..c.r/rrwxrwxrwx	Xxx			00
☑	/DCIM/111___06/_MG_0125.JPG(deleted)	3541836	Accessed	Sun Jun 02 2013 00:00:00	r/rrwxrwxrwx
☑	/DCIM/111___06/_MG_0125.JPG(deleted)	3541836	Created,Modified	Sun Jun 02 2013 15:42:32	r/rrwxrwxrwx
☑	/DCIM/111___06/IMG_0126.JPG	2255115	Created,Modified	Sun Jun 02 2013 15:42:46	r/rrwxrwxrwx
☐	/DCIM/CANONMSC/M0111.CTG	884	Created,Modified	Sun Jun 02 2013 15:44:08	r/rrwxrwxrwx
☐	/CANON_DC(Volume	0	Modified	Sun Jun 02 2013 16:33:04	r/rrwxrwxrwx
☑	/DCIM/111___06/IMG_0126.JPG	2255115	Accessed	Sat Feb 06 2016 00:00:00	r/rrwxrwxrwx

- Possibility to import **Mactime** files [done]
- Pick only relevant files [done]
- `MISPObject` will be created [done]

Data visualization (MISP zoidberg branch)



- View: start-date only, spanning and search [dev-branch]
- Manipulate: Edit, Drag and Expand [dev-branch]
- Others: Timezone support [dev-branch]

→ For now [dev-branch], supports up to **micro-seconds** in the database and up to **milliseconds** in the web interface.

MISP

INTELLIGENCE SHARING

Working with the MISP 2.5 REST API

Query, interpret and automate

CIRCL / MISP Project

https://www.misp-project.org/

Reference snapshot: 29 October 2026

A useful integration asks a precise question

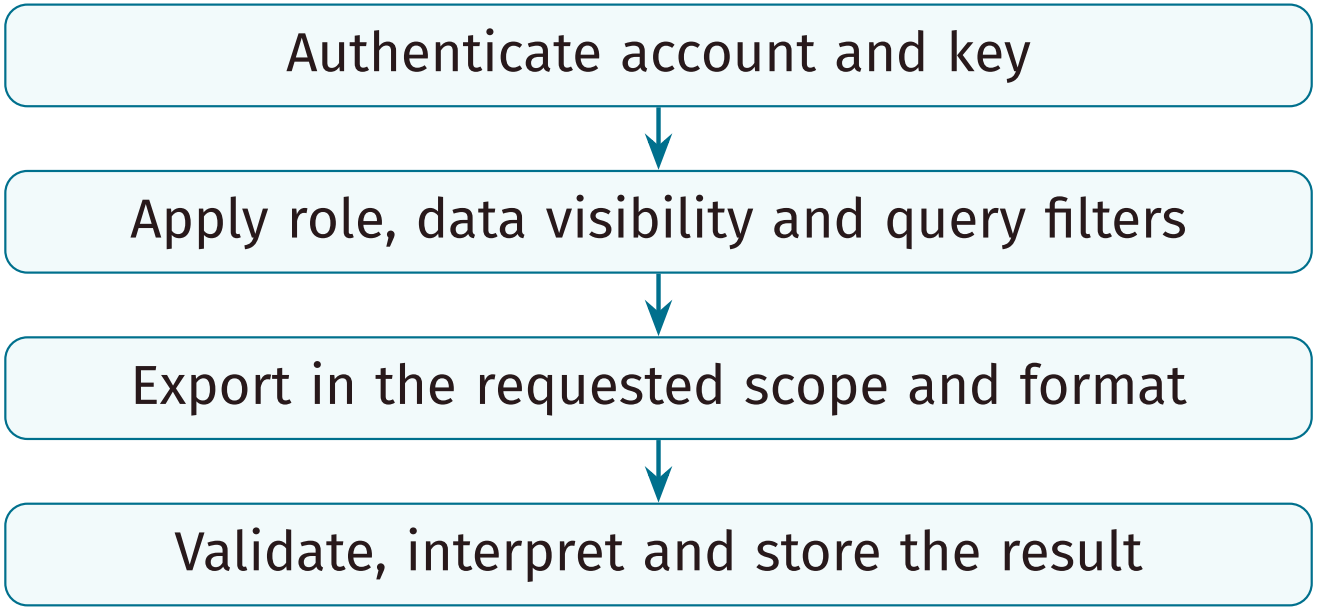
- Authenticate with the right account and key.
- Choose event or attribute scope.
- Set filters and export policy explicitly.
- Traverse pages without losing records.
- Handle updates, failures and removals deliberately.

Reference snapshot

MISP 2.5.50, reviewed 29 October 2026. Check your deployed version.

Make one request

The API uses your account’s visibility



Set up a key for the task

- The account needs API access: `perm_auth`.
- Send the raw key in `Authorization`.
- Advanced keys can restrict expiry, source IP and writes.
- Keep keys outside scripts, notebooks and shared collections.

Read-only does not mean GET-only

A read-only key can POST a search, while event creation is denied.

First request: inspect the instance

```
curl --fail-with-body --silent --show-error \
  --connect-timeout 5 --max-time 60 \
  -H "Authorization: $MISP_API_KEY" \
  -H 'Accept: application/json' \
  "$MISP_URL/servers/getVersion"
```

Read `version` and `pymisp_recommended_version`. Keep TLS verification enabled; supply your organisation's CA bundle if needed.

Choose the endpoint by the result you need

Endpoint	Result
`/events/index`	Event summaries
`/events/restSearch`	Full events or metadata
`/attributes/restSearch`	Matching attributes
`/events/view/{uuid}`	One known event

Use POST with a JSON body for search. Read each endpoint's contract.

A bounded attribute request

```
{
  "returnFormat": "json",
  "type": "domain",
  "value": "%.example",
  "published": [0, 1],
  "to_ids": [0, 1],
  "includeEventTags": true,
  "limit": 100,
  "page": 1
}
```

POST /attributes/restSearch

The response envelope depends on the scope

Attribute search

```
{"response": {"Attribute": [
  {"uuid": "...", "type": "domain", "value": "..."}
]}}
```

Event search

```
{"response": [
  {"Event": {"uuid": "...", "info": "..."}}
]}
```

Search with intent

A matching event can contain other attributes

Event scope

Find an event through a domain match; full JSON can also include the event's IP address and contextual attributes.

Attribute scope

Return the attributes that match the attribute query.

Use `metadata: true` for lighter event results.

Exact values and wildcard values are different

Filter value	Intent
`workshop.example`	Exact attribute value
`%.example`	Suffix match
`%REST API training%`	Event-info substring

Start with one known value or UUID. Add filters one at a time.

Select publication and detection flags explicitly

Raw JSON filter	Meaning
`published: 1`	Published events
`published: 0`	Unpublished events visible to the caller
`to_ids: 1`	Detection-marked attributes
`to_ids: 0`	Contextual attributes
`[0, 1]`	Either state, for these raw JSON filters

Express tag logic with explicit operators

```
{
  "tags": {
    "AND": ["tlp:clear"],
    "OR": ["training:reviewed", "training:verified"],
    "NOT": ["training:exclude"]
  }
}
```

Require the AND tag, one OR tag, and no NOT tag.

Tags may come from the event

- Event and attribute tags can both influence a match.
- `includeEventTags` adds inherited event tags to the result.
- Inspect the `inherited` marker before interpreting attachment.

Keep context when exporting

A tag-driven match does not mean that tag was attached directly to every returned attribute.

Pick the clock that answers your question

Field	Clock
`timestamp`	Change time in the selected scope
`event_timestamp`	Parent event change time
`publish_timestamp`, `last`	Event publication time
`from`, `to`	Event date bounds
`first_seen`, `last_seen`	Observation time

Changing the format can change the selection

JSON / CSV

Non-restrictive exporters in this snapshot. State selection explicitly when comparing results.

Detection text

Defaults to published events and detection-marked attributes when those filters are omitted.

An analysis result is not automatically a deployment policy.

A detection-policy exercise

```
{
  "returnFormat": "text",
  "type": "domain", "value": "%.example",
  "published": 1, "to_ids": 1,
  "tags": {"AND": ["tlp:clear"],
           "NOT": ["training:exclude"]},
  "enforceWarninglist": true,
  "limit": 100, "page": 1
}
```

Output is text. The unpublished workshop event correctly contributes nothing.

Warninglist annotations and filtering differ

Annotate

`includeWarninglistHits` adds matching-list information to JSON.

Filter

`enforceWarninglist` can remove rows after query pagination.

Empty formatted output is not always proof that later pages have no candidates.

Automate and verify

A role cap can make every page short

Requested	Returned	Continue?
Page 1, limit 100	2	Yes
Page 2, limit 100	2	Yes
Page 3, limit 100	1	Yes
Page 4, limit 100	0	Stop for this plain JSON query

Five records were available. Stopping at the first short page loses three.

Bound the traversal and reject incomplete results

- Start at page 1; request a deterministic order.
- Validate the envelope and attribute UUIDs.
- Continue through short pages.
- Fail on repeated UUIDs or a maximum-page bound.
- Publish output and checkpoints only after success.

Use the supplied read-only Python client

```
python examples/search_attributes.py \
  --page-size 100 --max-pages 20 \
  > attributes.ndjson.tmp \
  && mv attributes.ndjson.tmp attributes.ndjson
```

- Environment-provided URL and key; trusted TLS.
- Timeouts, no redirects, JSON/envelope checks.
- A failure leaves partial output unpublished.

PyMISP offers a typed view of the same task

```
rows = misp.search(
    controller="attributes", return_format="json",
    type_attribute="domain", value="%.example",
    include_event_tags=True, include_event_uuid=True,
    limit=100, page=1, pythonify=True,
)
```

Check for an error dictionary before treating the result as a list of objects.
This is one page, not a complete export.

Keep one fixed window for an incremental run

1. Capture an upper timestamp at run start.
2. Set the lower bound from the last successful run, with overlap.
3. Fetch every page using the same inclusive bounds.
4. Upsert by UUID and complete downstream writes.
5. Advance the checkpoint only when the whole run succeeds.

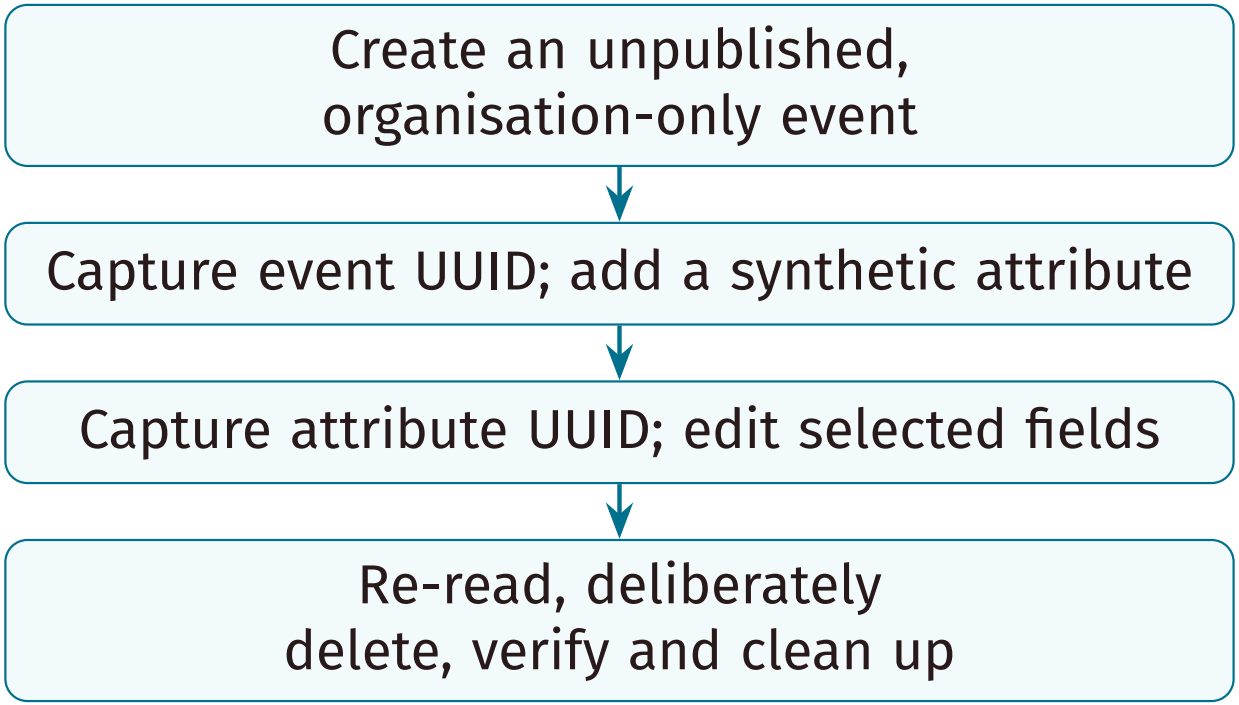
Plan for removals and lost visibility

- Normal searches omit soft-deleted attributes.
- Tombstone visibility depends on permissions and endpoint.
- Hard deletion and lost access may provide no visible tombstone.
- Consumers need reconciliation, expiry and removal rules.

An empty deletion search proves very little

Check the caller’s actual deletion-view contract.

Write a small, explicit change

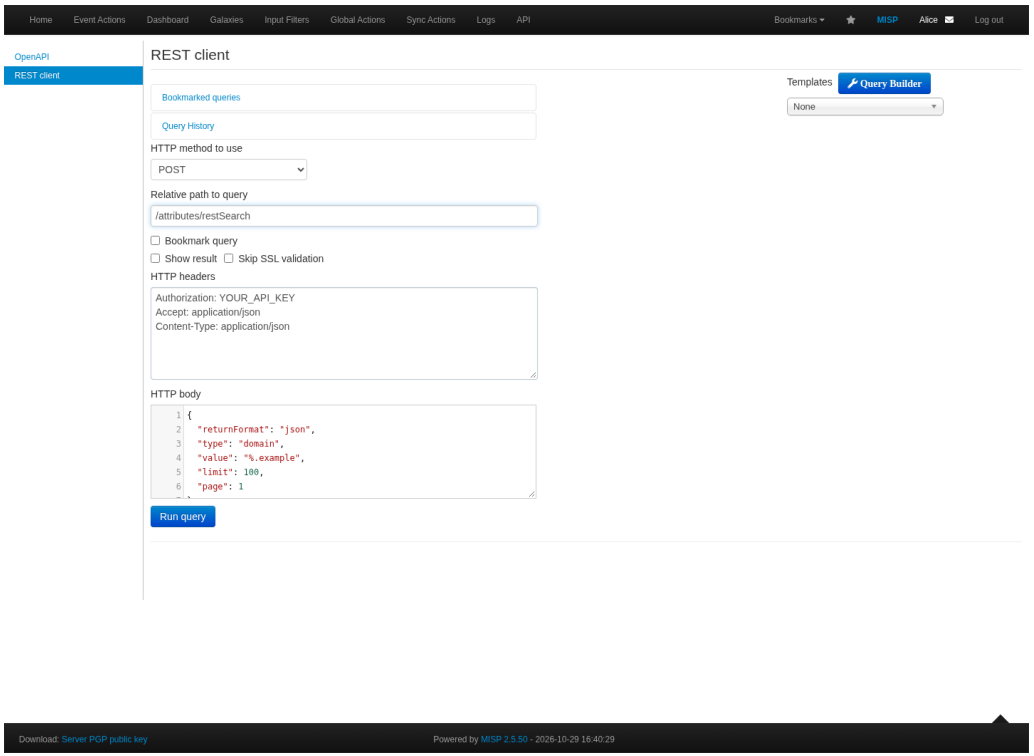


Update only the fields you intend

```
{
  "Attribute": {
    "comment": "Reviewed in the REST API workshop.",
    "to_ids": false
  }
}
```

`POST /attributes/edit/{attribute-uuid}` Re-read the attribute and verify the comment.

Use the built-in tools to inspect a request



- `/api/rest`
- Templates and query builder.
- History and bookmarks.
- Generated request code.

Try the read-only notebook and Postman collection

Notebook

Raw page, bounded traversal and typed PyMISP page. No saved keys or outputs.

Postman

Version, attribute JSON, event metadata, policy text and CSV: five requests, ten assertions.

Keep the filled Postman environment and notebook results local.

Classify failure before retrying

Symptom	Next check
401 / 403	Key, expiry, IP, role and endpoint permissions
404	Base path, UUID, existence and caller visibility
400 / 405	Method, JSON shape, types and validation
HTML / redirect	Login or proxy page; do not treat as empty data
429 / 5xx / timeout	Bounded read retry; reconcile uncertain writes
200, no rows	Scope, flags, tags, clocks, ACL and exporter defaults

Workshop: prove the boundaries

1. Create the private fixture and capture its UUIDs.
2. Query it as the owning organisation's reader.
3. Compare another organisation and a read-only key.
4. Traverse the role-capped pages.
5. Edit, re-read and deliberately soft-delete the attribute.
6. Inspect the owner deletion view; remove the lab event.

Evidence an integration reviewer can reproduce

- Fourteen offline client-boundary tests.
- Live MISP checks for ACLs, paging, filters and writes.
- Newman assertions and an executable read-only notebook.
- Pinned MISP/PyMISP sources and documented limits.

Continue from the repository

See `a.7-rest-API/README.md` for commands, source audit and the complete workshop.

MISP

INTELLIGENCE SHARING

Best Practices in Threat Intelligence

Gather, document, analyse and contextualise intelligence using MISP

CIRCL / Team MISP Project

MISP Project

https://www.misp-project.org/

CRI MISP 2024

Objectives

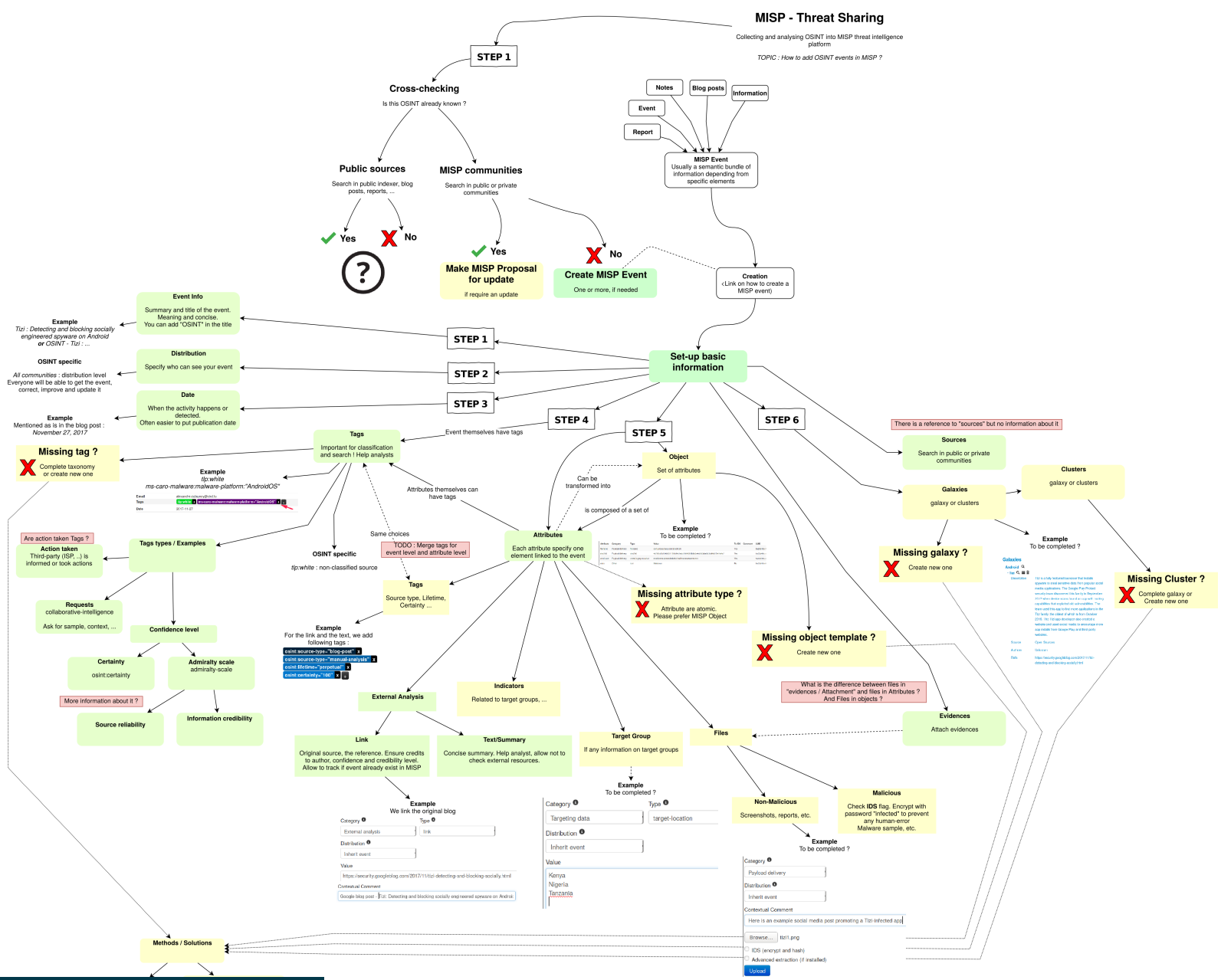
- Learn how to use MISP to support common OSINT gathering use-cases often used by SOC, CSIRTs and CERTs
  - Use practical exercise examples[1]
  - The exercises are based on **practical recent cases to model and structure intelligence** using the MISP standard
- Improve the data models available in MISP by exchanging live improvements and ideas
- Be able to share the results to the community at the end of this session

[1] https://gist.github.com/adulau/8c1de48060e259799d3397b83b0eec4f

(Threat) Intelligence

- **Cyber threat intelligence (CTI) is a vast concept** which includes different concepts, methods, and workflows
  - Intelligence is defined differently in the military than in the financial sector than in the intelligence community
- **MISP project doesn't want to lock an organisation or a user into a specific model**. Each model is useful depending on the objectives of an organisation
- A set of pre-defined knowledge base or data-models are available and organisations can select (or create) what they need
- During this session, an overview of the most used taxonomies, galaxies, and objects will be described

Overall process of collecting and analysing OSINT



Meta information and contextualisation 1/2

- Quality of indicators/attributes are important but **tagging and classification are also critical to ensure actionable information**
- Organizing intelligence is done in MISP by using tags, which often originate from MISP taxonomy libraries
- The scope can be classification (*tlp*, *PAP*), type (*osint*, *type*, *veris*), state (*workflow*), collaboration (*collaborative-intelligence*), or many other fields
- MISP taxonomy documentation is readily available[2]
- **Review existing practices of tagging in your sharing community, reuse practices, and improve context**

[2] https://www.misp-project.org/taxonomies.html

Meta information and contextualisation 2/2

- **When information cannot be expressed in triple tags format** (*namespace:predicate=value*), MISP use Galaxies
- **Galaxies** contain a huge set of common libraries[3] such as threat actors, malicious tools, tactics, target information, mitigations, and more
- When tagging or adding a Galaxy cluster, tagging at the event level is for the whole event (including attributes and objects). Tagging at the attribute level is for a more specific context

[3] https://www.misp-project.org/galaxy.html

Estimative Probability

- **Words of Estimative Probability**[4] propose clear wording while estimating probability of occurence from an event
- A MISP taxonomy called **estimative-language**[5] proposes an applied model to tag information in accordance with the concepts of Estimative Probability

[4] https://www.cia.gov/library/center-for-the-study-of-intelligence/csi-publications/books-and-monographs/sherman-kent-and-the-board-of-national-estimates-collected-essa6words.html

[5] https://www.misp-project.org/taxonomies.html

Reliability, credibility, and confidence

- The **Admiralty Scale**[6] (also called the **NATO System**) is used to rank the reliability of a source and the credibility of information
- A MISP taxonomy called admiralty-scale[7] is available
- US DoD **JP 2-0, Joint Intelligence**[8] includes an appendix to express confidence in analytic judgments
- A MISP predicate in estimative-language called confidence-in-analytic-judgment[9] is available

---

[6] https://www.ijlter.org/index.php/ijlter/article/download/494/234, *US Army Field Manual 2-22.3, 2006*

[7] https://www.misp-project.org/taxonomies.html

[8] http://www.jcs.mil/Portals/36/Documents/Doctrine/pubs/jp2_0.pdf, page 114

[9] https://www.misp-project.org/taxonomies.html

Adding attributes/objects to an event

- If the information is a **single atomic element**, using a single attribute is preferred
  - Choosing an attribute type is critical as this defines the automation/export rule (e.g. *url* versus *link* or ip-src/ip-dst?)
  - Enabling the IDS (automation) flag is also important, but *when you are in doubt*, don't set the IDS flag
- If the information is **composite** (ip/port, filename/hash, bank account/BIC), using an object is strongly recommended

How to select the right object?

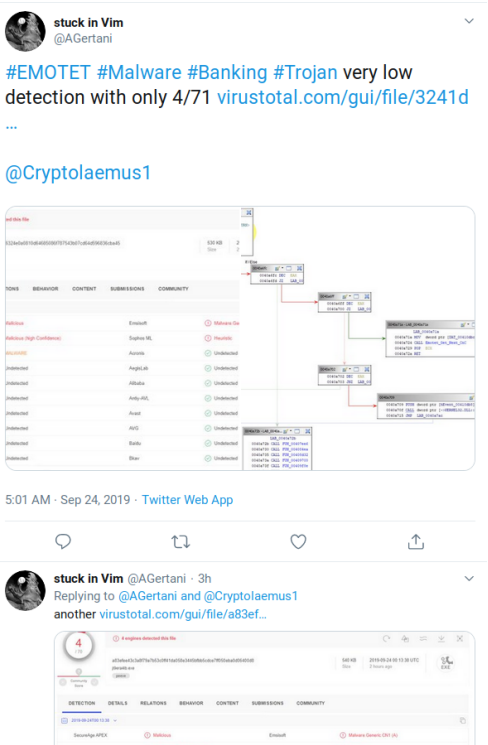
There are more than 150 MISP object[10] templates.
As an example, at CIRCL, we regularly use the following object templates *file*, *microblog*, *domain-ip*, *ip-port*, *coin-address*, *virustotal-report*, *paste*, *person*, *ail-leak*, *pe*, *pe-section*, *registry-key*.

[10] https://www.misp-project.org/objects.html

microblog object

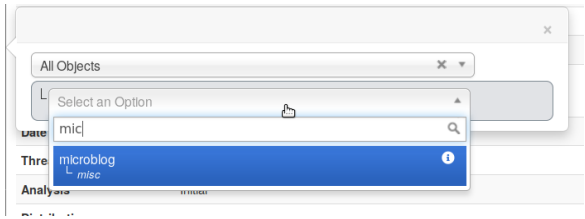
Use case

A series of OSINT tweets from a security researcher. To structure the thread, the information, and keep a history.



Object to use

The microblog object can be used for Tweets or any microblog post (e.g. Facebook). The object can be linked using *followed-by* to describe a series of post.



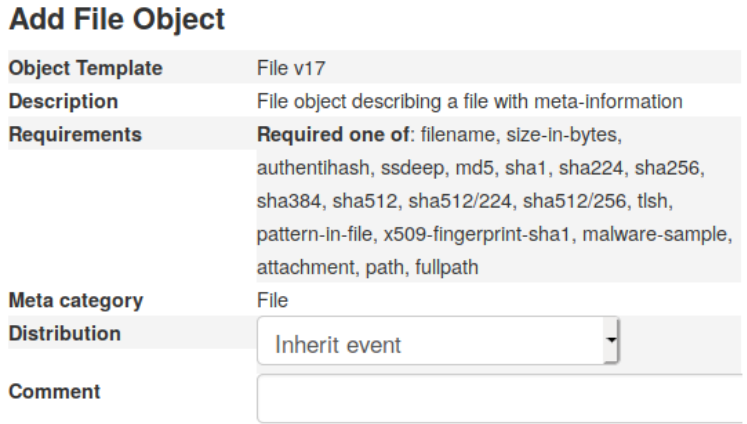
file object

Use case

- A file sample was received by email or extracted from VirusTotal
- A list of file hashes were included in a report
- A hash value was mentioned in a blog post

Object to use

The file object can be used to describe file. It's usual to have partial meta information such as a single hash and a filename.



References

- Graphical overview of OSINT collection using MISP https://github.com/adulau/misp-osint-collection
- MISP objects documentation https://www.misp-project.org/objects.html
- MISP taxonomies documentation https://www.misp-project.org/taxonomies.html
- MISP galaxy documentation https://www.misp-project.org/galaxy.html

MISP

INTELLIGENCE SHARING

MISP core development hands-on exercise

Building a small nifty feature for the MISP core

CIRCL / Team MISP Project

MISP Project

CRI MISP 2024

Some practical things first...

- If you'd like to take a peak at the main files already implemented:
  https://github.com/iglocska/misp-dev-training-cheat-sheet
- Full implementation:
  https://github.com/MISP/MISP/tree/dev_session/app

Let’s try to develop a feature together

- Idea: Users should have the option to set alert filters for the publish alert e-mails
- By default receive all alerts as before
- If a filter is set, check if the alert is interesting for us or not

How to ensure that the feature is useful for the community at large?

- Always try to think in reusable systems instead of fixing a single issue
  - Much higher chance of getting a PR merged if it doesn't just cover your specific use-case
  - Try to stay two steps ahead, see how your feature can be reused for other tasks

User settings - a long overdue feature

- Allow users to set preferences for certain views
- For high level users, all the technical details are sometimes wasted
- Simply not being interested in certain types of data points
- Non-standard MISP deployments (island only MISP instances, etc)
- User pre-sets for certain settings

Objectives of the feature

- User should be able to do the following with filter rules:
  - set
  - get
  - remove
  - index
- Filter rules should be flexible - we do not want to anticipate all possible settings in advance
- Ensure that the system is easy to extend and reuse

Before we start with anything...

- Update our MISP instance (git pull origin 2.4)
- Fork github.com/MISP/MISP (via the github interface)
- Add a new remote to our fork:
  - via username/password auth: git remote add my_fork https://github.com/iglocska/MISP
  - via ssh: git remote add my_fork gitgithub.com:iglocska/MISP.git
- Generally a good idea to work on a new branch: git checkout -b dev_exercise
- Enable debug in MISP

Implementation

- Storage:
  - Single key/value table for all settings
  - Each user should be able to set a single instance of a key
  - Values could possible become complex, let's use JSON!
  - Add timestamping for traceability
  - Consider which fields we might want to look-up frequently for indexing

The database changes we need

- The table structure:
  - id int(11) auto increment //primary key
  - key varchar(100) //add index!
  - value text //json
  - user_id int(11) //add index!
  - timestamp int(11) //add index!
- Tie it to into the upgrade system (app/Model/AppModel.php)
- Test our upgrade process! Check the output in the audit logs

Checklist

- Outline of the changes needed:
  - New Controller (UserSettingsController.php)
  - New Model (UserSetting.php)
  - New Views (setSetting, index)
  - Add new controller actions to ACL
  - Update the e-mail alert system to use the functionality

Create the new Model skeleton

- location: /var/www/MISP/app/Model/UserSetting.php
- Create basic skeleton
- Add model relationships (hasMany/BelongsTo)
- Use the hooking functionality to deal with the JSON field (beforeSave(), beforeFind())
- Add a function that can be used to check if a user should get an alert based on filters (checkPublishFilter())
- Add a function to check if a user can access/modify a setting (checkAccess())

Create the Controller skeleton

- location: /var/www/MISP/app/Model/UserSetting.php
- Create basic skeleton
- Set pagination rules
- Define CRUD functions (exceptionally, we diverge here from the norm)
  - setSetting()
  - getSetting()
  - index()
  - delete()

Start with an API only approach at first

- setSetting():
  - Accepted methods: ADD / POST
  - Separate handling of API / UI
  - POST should create/update an entry
  - GET should describe the API

getSetting / index

- getSetting():
  - Accepted methods: GET
  - Retrieves a single setting based on either ID or setting key and user_id
  - Encode the data depending on API/UI
  - Accepted methods: GET
  - List all settings
  - Filter user scope on demand
  - Filter available scopes based on role

delete

- delete():
  - Accepted methods: POST / DELETE
  - Deletes a single entry based on ID or setting key
  - Encode the data depending on API/UI

Add the ACL functionalities

- Tie functions into checkAccess():
  - Check if user is allowed to execute actions and throw exceptions if not
  - Add it to: setSetting() / getSetting() / delete()
- Consider that:
  - Site admins have full reign
  - Org admins can manage their own users
  - Everyone else can self-manage

Test the functionalities

- Use the REST client
- Expectations
  - GET on /setSetting and /delete describing our endpoints
  - POST /setSetting with "key": "publish_filter", "value": "Event.tags":"%sofacy%" should return newly added or modified filter
  - GET on /index should list our entries, GET on /getSetting should show an individual entry
  - DELETE on /delete should delete the entry

Start adding the UI components

- We now have a rudimentary CRUD, let's add some simple UI views
  - setSetting as a simple form
  - index should use the parametrised generators (IndexTable)
  - Add both views to the menu systems (side-menu, global menu)
  - Don't forget about sanitisation and translations!

Add the checkPublishFilter() function to the e-mailing

- Trace the code path of the e-mail sending to understand the process
- Decide on the best place to inject our check
- Don’t break the flow of the process!
- What do we have access to at this point? What format are they in?

Test if our code works correctly

- Do we see any notices / errors?
- Is our code easily accessible?
- Consider other roles! Can users/org admins do things we don’t want them to do?
- Is our code-base breaking the default behaviour?
- Is our update script working as expected?

Push our code to our fork and create a pull request

- git status to check what changed / got added
- git add /path/to/file to add files we want to commit
- git commit (format: is "new/fix/chg: [topic] My description"
- git push my_fork
- Create pull request from the github interface
- Wait for Travis to run, update the code if needed

MISP

INTELLIGENCE SHARING

MISP restSearch module development

Building a simple export module for the core

CIRCL / Team MISP Project

MISP Project

CRI MISP 2024

Building a native restSearch export

- Similar in scope to an **export module** of the MISP modules system
- Pros:
  - Can be used for composited data coming from a **filtered query**
  - Fast, **native approach**
  - Can be built to support **several scopes** (events, attributes, sightings)
- Cons...

Building a native restSearch export

- Similar in scope to an **export module** of the MISP modules system
- Pros:
  - Can be used for composited data coming from a **filtered query**
  - Fast, **native approach**
  - Can be built to support **several scopes** (events, attributes, sightings)
- Cons...

So how does restSearch work?

- Standardised way of collecting **parameters**
- Using the parameters, a loop is started to **chunk and gradually build** our export data
- The chunk size depends on memory envelopes
- Each chunk is **converted piece by piece**...
- ... and subsequently are concatenated into a temporary file
- Once no more elements are left, the file is sent in the response

Where does the module system come into play?

- The export modules handle 5 tasks:
  - Pass **meta-information** back to restSearch on the export format itself
  - Add a **start segment** to the exported data
  - Do the actual **conversion** from MISP's internal format to the desired export format
  - Provide a **separator** for data chunks
  - Have a **closing segment** for the returned data, based on the formatś conventions

Our little training module: Nibbler, the ever hungry IDS/IPS

Nibbler

- Simplistic tool with its **own proprietary format**
- Meant to mimic a typical **in-house tool**
- Lightweight scope, for simplicityś sake
- **pipe separated values**
- VALUE | TYPE | DESCRIPTION | REFERENCE | ACTION

Nibbler format - caveats

- Rules can be prepended by comments, each comment line starting with #
- Some characters have to be escaped in some custom, crazy ways
  - linebreaks: ##LINEBREAK##
  - commas: ##COMMA##
  - pipes: ##PIPE##

Nibbler format

- **Value**: The actual indicator value
- **Type**: The format of the indicator
- **Description**: A quick description for analysts investigating the alert, why is this relevant
- **Reference**: A backreference that the analyst can use to find out more about the alert
- **Action**: What should Nibbler do if it trips over the value?

Supported types

- IP
- Domain
- Hostname
- MD5
- SHA1
- SHA256
- Filename

Supported values

- ALERT - default behaviour, create an alert.
- BLOCK - block the action outright. Only set if the tag nibbler:block is present

Mapping the types to MISP

- Though we have types to map from MISP, in some cases several types map to a Nibbler type
- We've created a rough **mapping** (this is probably the most difficult task) in advance
- Some MISP types map to a Nibbler type directly
- **Composite** MISP types map to **2 Nibbler types** each

Mapping the types to MISP

- ip-dst :: IP
- ip-src :: IP
- domain :: Domain
- domain|ip :: Domain, IP
- hostname :: Hostname
- md5 :: MD5
- sha1 :: SHA1
- sha256 :: SHA256
- filename|md5 :: Filename, MD5
- malware-sample :: Filename, MD5
- filename|sha1 :: Filename, SHA1
- filename|sha256 :: Filename, SHA256

Export module skeleton

```
<?php
class NibblerExport
{
    public $additional_params = array();
    public function handler(
        $data, $options = array()
    ) {}
    public function header(
        $options = array()
    ) {}
    public function footer() {}
    public function separator() {}
}
```

Additional parameters

```
public $additional_params = array(
    'flatten' => 1
);
```

Adding our mapping

```
private $__mapping = array(
  'ip-dst' => 'IP',
  'ip-src' => 'IP',
  'domain' => 'Domain',
  'domain|ip' => ['Domain', 'IP'],
  'hostname' => 'Hostname',
  'md5' => 'MD5',
  'sha1' => 'SHA1',
  'sha256' => 'SHA256',
  'filename|md5' => array('Filename', 'MD5'),
  'malware-sample' => array('Filename', 'MD5'),
  'filename|sha1' => array('Filename', 'SHA1'),
  'filename|sha256' => array('Filename', 'SHA256')
);
```

Writing the start of the output

```
public function header($options = array())
{
    return sprintf(
        "# Nibbler rules generated by MISP at %s\n",
        date('Y-m-d H:i:s')
    );
}
```

Footer function - how should the output end?

```
public function footer()
{
    return "\n";
}
```

What separates the chunks?

```
public function separator()
{
    return "\n";
}
```

The actual legwork, the handler

```
public function handler($data, $options = array())
{
  if ($options['scope'] === 'Attribute') {
    $data['Attribute']['AttributeTag'] = $data['AttributeTag'];
    return $this->__convertAttribute($data['Attribute'], $data['Event']);
  }
  if ($options['scope'] === 'Event') {
    $result = array();
    foreach ($data['Attribute'] as $attribute) {
      $temp = $this->__convertAttribute($attribute, $data['Event']);
      if ($temp) $result[] = $temp;
    }
    return implode($this->separator(), $result);
  }
  return '';
}
```

Building an optional internal converter function

```
private function __convertAttribute($attribute, $eve
{
  if (empty($this->__mapping[$attribute['type']])) {
    // mapping not found - invalid type for nibbler
    return '';
  }
  if (is_array($this->__mapping[$attribute['type']])
    // handle mappings for composites - slide
  } else {
    // handle simple mappings - slide
  }
  // return 1 or 2 lines, separated by separator()
  return implode($this->separator(), $result);
}
```

Handling the simple case

```
$result[] = sprintf(
  '%s|%s|%s|%s|%s',
  $this->__escapeSpecialChars($attribute['value']),
  $this->__mapping[$attribute['type']],
  $event['uuid'],
  $this->__escapeSpecialChars($event['info']),
  'ALERT'
);
```

Handling the case for composites

```
$attribute['value'] = explode(
  '|', $attribute['value']
);
foreach (array(0,1) as $part) {
  $result[] = sprintf(
    '%s|%s|%s|%s|%s',
    $this->__escapeSpecialChars(
      $attribute['value'][$part]
    ),
    $this->__mapping[$attribute['type']][$part],
    $event['uuid'],
    $this->__escapeSpecialChars($event['info']),
    'ALERT'
  );
}
```

Putting it together

```
private function __convertAttribute($attribute, $event) {
  if (empty($this->__mapping[$attribute['type']])) return '';
  $result = array();
  $attributes = array();
  if (is_array($this->__mapping[$attribute['type']])) {
    $attribute['value'] = explode('|', $attribute['value']);
    foreach (array(0,1) as $part) {
      $result[] = sprintf(
        '%s|%s|%s|%s|%s',
        $this->__escapeSpecialChars($attribute['value'][$part]),
        $this->__mapping[$attribute['type']][$part],
        /events/view/ . $event['uuid'],
        $this->__escapeSpecialChars($event['info']),
        $this->__decideOnAction($attribute['AttributeTag'])
      );
    }
  } else {
    $result[] = sprintf(
      '%s|%s|%s|%s|%s',
      $this->__escapeSpecialChars($attribute['value']),
      $this->__mapping[$attribute['type']],
      /events/view/ . $event['uuid'],
      $this->__escapeSpecialChars($event['info']),
      $this->__decideOnAction($attribute['AttributeTag'])
    );
  }
  return implode($this->separator(), $result);
}
```

Adding the function that decides on the action

```
private function __decideOnAction($attributeTags)
{
  foreach($attributeTags as $attributeTag) {
    if (
      $attributeTag['Tag']['name'] ===
        'nibbler:block'
    ) {
      return 'BLOCK';
    }
  }
  return 'ALERT';
}
```

Finalising the export module... The escaping function

```
private function __escapeSpecialChars($value)
{
  $value = preg_replace(
    "/\r|\n/", "##LINEBREAK##", $value
  );
  $value = preg_replace(
    "/,/", "##COMMA##", $value
  );
  $value = preg_replace(
    "/\|/", "##PIPE##", $value
  );
  return $value;
}
```

Modifying the MISP core to know about the export module

- The **models** that we are targeting by scope (Event, Attribute) **need to be updated**
- They are located in **/var/www/MISP/app/Model/**
- The global variable **$validFormats** houses all mappings
- Simply add a new line such as the following:
- 'nibbler' => array('nibbler', 'NibblerExport', 'nibbler')

Let us test the module!

- Use the **rest client** to test it conveniently
- Both the event and attribute level restSearch function should work
- Simply set the **returnFormat** to nibbler, which should also show up as a valid export format

REST client

HTTP method to use

POST

Relative path to query

/events/restSearch

- [ ] Use full path - disclose my apikey
- [ ] Bookmark query
- [x] Show result
- [ ] Skip SSL validation

HTTP headers

```
Authorization: ArSxnHf20foSapnOSyxfrljMdl9oLDnvmqvHK97q
Accept: application/json
Content-Type: application/json
```

HTTP body

```
{
   "returnFormat": "nibbler",
   "page": 1,
   "limit": 4,
   "type": ["ip-dst", "ip-src", "domain|ip", "hostname", "domain"]
}
```

Run query

MISP

INTELLIGENCE SHARING

MISP - Galaxy 2.0

Method for sharing threat intelligence

Team CIRCL

info@circl.lu

November 2, 2026

Outline of the presentation

- Present the features available for Sharing *galaxy clusters*
- Look at the internals of what changed in the datamodel and MISP's behaviors

MISP Galaxy 2.0

Galaxy 2.0 introduces various new features for *Galaxies* and their *Clusters* allowing:

- Creation of **custom** *Clusters*
- **ACL** on *Clusters*
- **Connection** of *Clusters* via *Relations*
- **Synchronization** to connected instances.
- **Visualization** of forks and relationships

Default Galaxy clusters

Default *Galaxy cluster*

- Coming from the `misp-galaxy` repository[1]
- Cannot be edited
  - Only way to provide modification is to modify the stored JSON or to open a pull request
  - Are not synchronized
  - Source of trust
- Restrictions propagate to their children (`Galaxy cluster elements`, `Cluster relationships`)

Custom *Galaxy cluster*

- Can be created via the UI or API
- Belongs to an organisation
  - Fully editable
  - Are synchronized

[1] https://github.com/MISP/misp-galaxy

MISP Galaxy 2.0 - Comparison with prior version

Clusters and *Relations* can be edited.

- New *Clusters* fields
  - `distribution`, `sharing_group_id`
  - `org_id`, `orgc_id`
  - `locked`, `published`, `deleted`
  - `default`
    - *Clusters* coming from the `misp-galaxies` repository are marked as default
    - Not synchronized
    - Same purpose as *Event*'s `locked` field
  - `extends_uuid`
    - Point to the *Cluster* that has been forked
  - `extends_version`
    - Keep track of the *Cluster* version that has been forked

MISP Galaxy 2.0 - Others changes

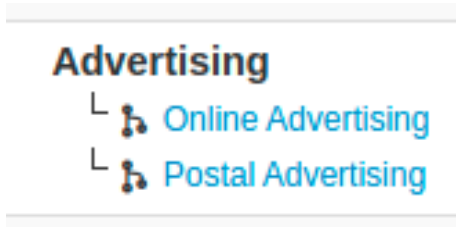
- *Role* `perm_galaxy_editor`
- Relations also have a `distribution` and can have *Tags*
- Synchronization servers have 2 new flags
  - `pull_galaxy_clusters`
  - `push_galaxy_clusters`
- Clusters `blocklist`

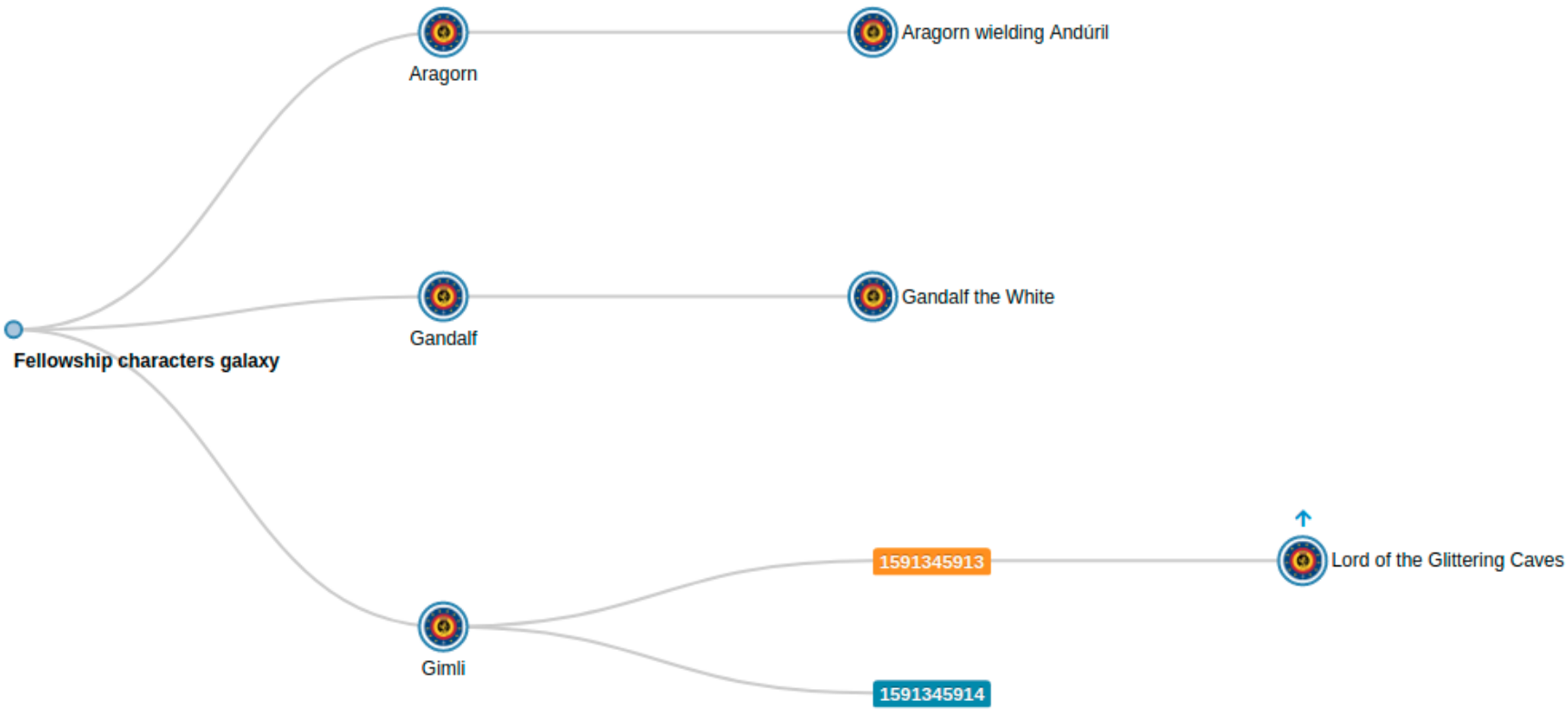
Features in depth: CRUD

- Standard CRUD
- Soft and Hard deletion
- Publishing
- Update forked cluster to keep it synchronized with its parent
- ACL on the *Cluster* itself, not on its tag
  - `misp-galaxy:galaxy-type="cluster UUID"`
  - `misp-galaxy:mitre-attack-pattern="e4932f21-4867-4de6-849a-1b11e48e2682"`

Features in depth: Visualization

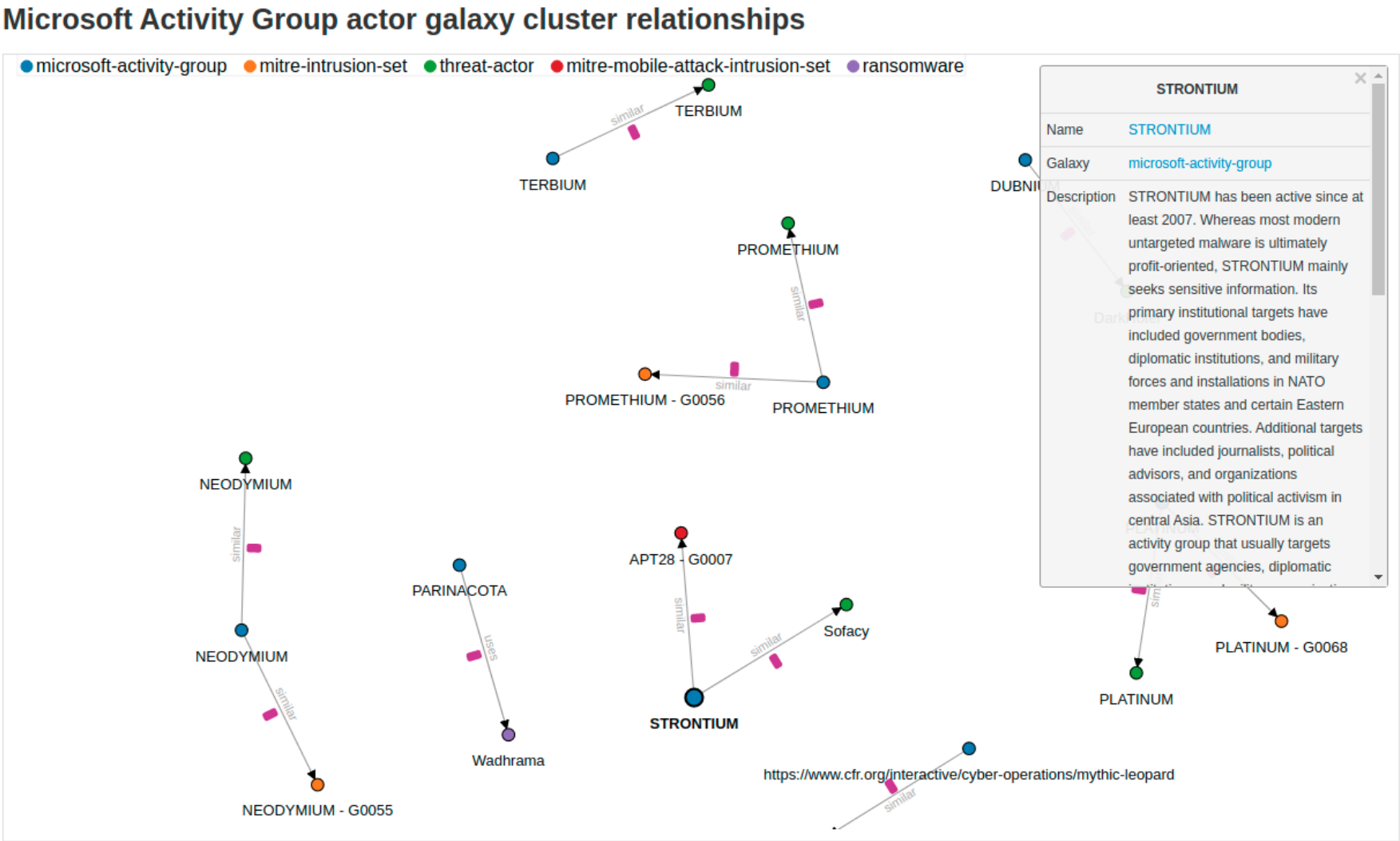
Tree view of forked Clusters





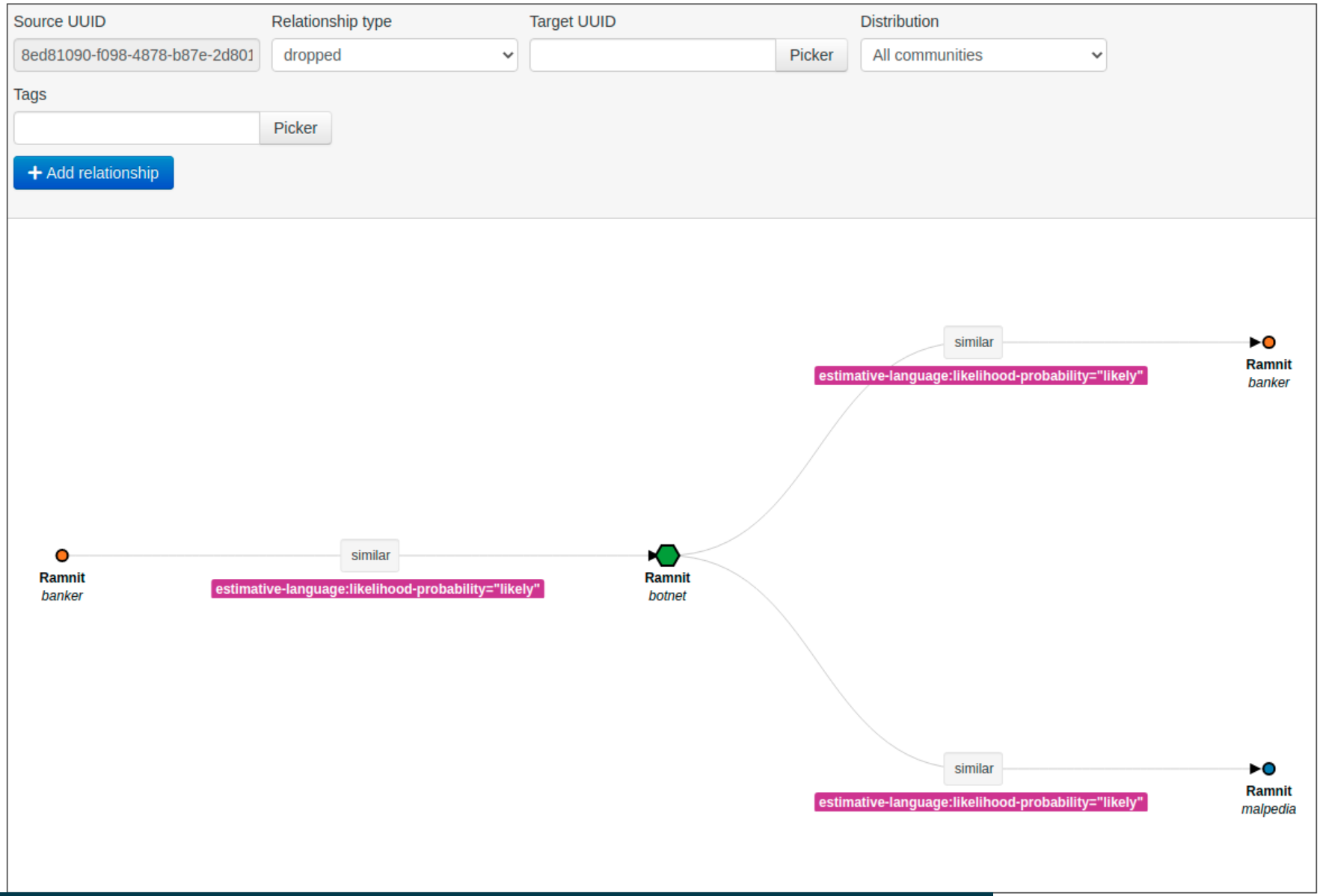
Features in depth: Visualization

Tree and network views for Relations between Clusters



Features in depth: Visualization

Tree and network views for Relations between Clusters

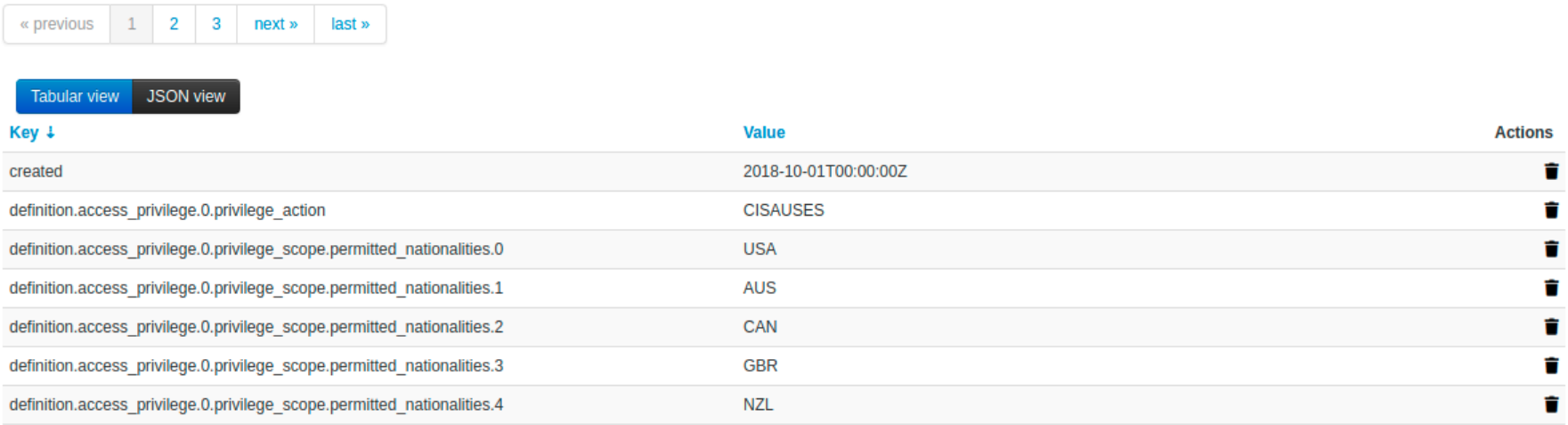


Galaxy cluster elements

Hasn’t been touched: Still a key-value stored. But new feature have been added[2]

Tabular view

- Allows you to browse **cluster elements** like before



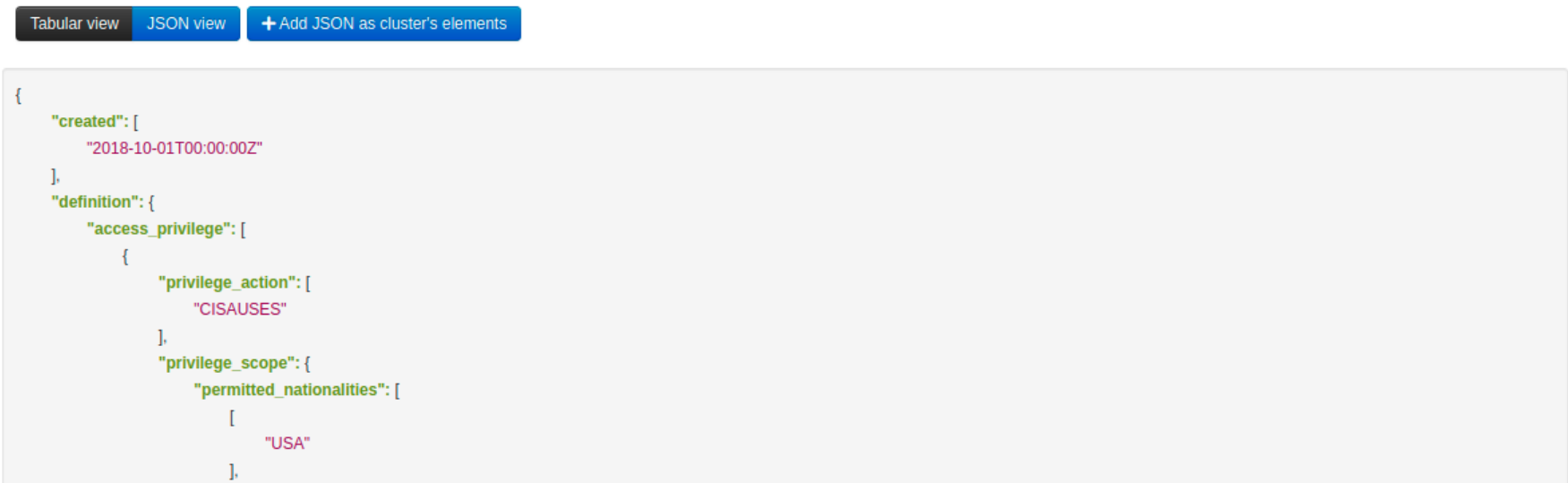
Key ↓	Value	Actions
created	2018-10-01T00:00:00Z	
definition.access_privilege.0.privilege_action	CISAUSES	
definition.access_privilege.0.privilege_scope.permitted_nationalities.0	USA	
definition.access_privilege.0.privilege_scope.permitted_nationalities.1	AUS	
definition.access_privilege.0.privilege_scope.permitted_nationalities.2	CAN	
definition.access_privilege.0.privilege_scope.permitted_nationalities.3	GBR	
definition.access_privilege.0.privilege_scope.permitted_nationalities.4	NZL	

[2]Will be included in next release

Galaxy cluster elements

JSON view

- Allows you to visualisation **cluster element** in a JSON structure
- Allows you to convert any JSON into **cluster elements** enabling searches and correlations



Synchronization in depth

Has its own synchronization mechanism which can be enabled with the `pull_galaxy_cluster` and `push_galaxy_cluster` flags

- **Pull All**: Pull all remote Clusters (similar to event's pull all)
- **Pull Update**: Update local Clusters (similar to event's pull update)
- **Pull Relevant**: Pull missing Clusters based on local Tags
- **Push**: Triggered whenever a Cluster is published or via standard push

MISP

INTELLIGENCE SHARING

An Introduction to Cybersecurity Information Sharing

MISP - Intelligence Sharing

CIRCL / Team MISP Project

MISP Project

https://www.misp-project.org/

CRI MISP 2024

Content of the presentation

- Data sharing in MISP
- Data models for the Data layer
- Data models for the Context layer

Layers of data model

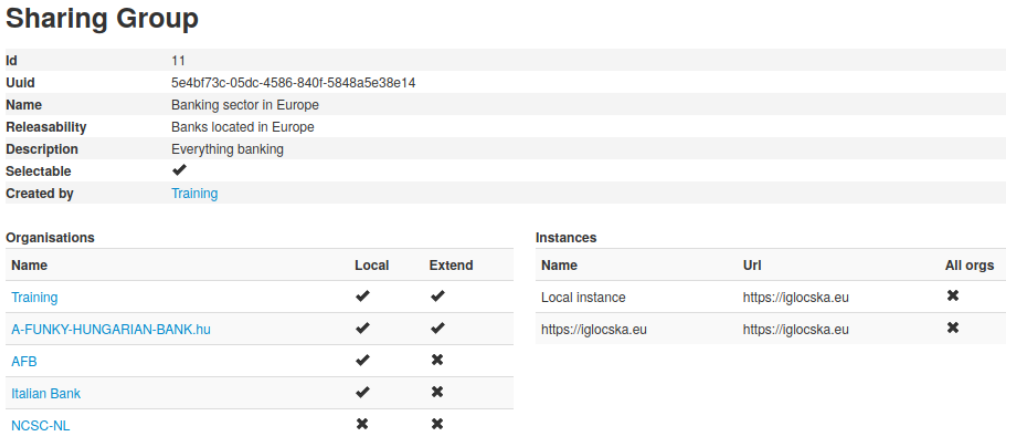
- Data layer
  - The raw data itself as well as element to link them together
  - Indicators, Observables and means to contextually link them
  - MISP terminology: Event, Attributes, misp-objects, ...
- Context layer
  - As important as the data layer, allow triage, false-positive management, risk-assessment and prioritisation
  - Latches on the data layer, usually referencing threat intelligence, concepts, knowledge base and vocabularies
  - Tags, Taxonomies, Galaxies, ...

Data sharing in MISP

Sharing in MISP: Distribution

MISP offers granulars distribution settings:

- `Organisation only`
- `This community`
- `Connected communities`
- `All communities`
- Distribution lists - aka **Sharing groups**



Sharing Group

Id	11
Uuid	5e4bf73c-05dc-4586-840f-5848a5e38e14
Name	Banking sector in Europe
Releasability	Banks located in Europe
Description	Everything banking
Selectable	✔
Created by	Training

Organisations

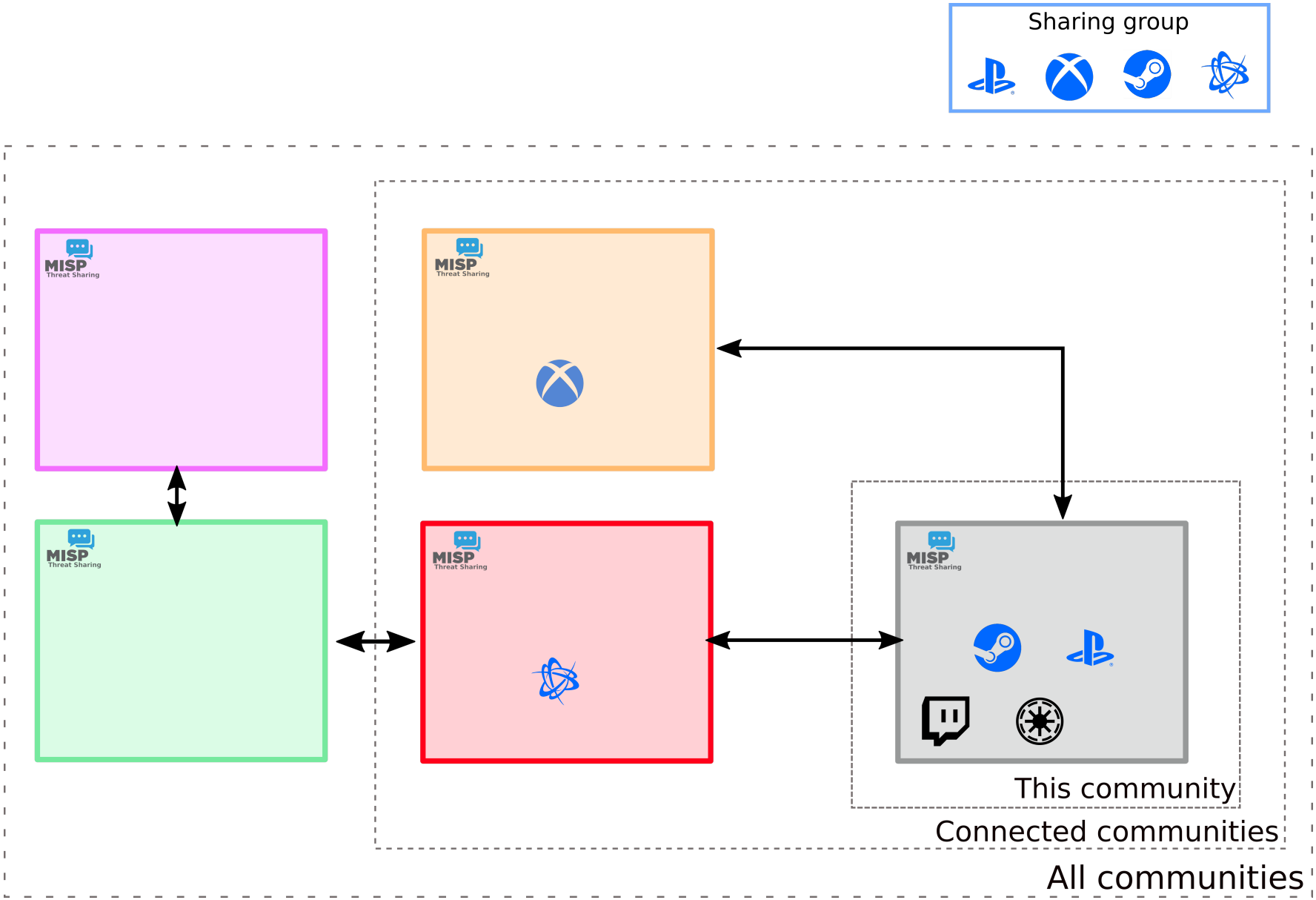
Name	Local	Extend
Training	✔	✔
A-FUNKY-HUNGARIAN-BANK.hu	✔	✔
AFB	✔	✖
Italian Bank	✔	✖
NCSC-NL	✖	✖

Instances

Name	Url	All orgs
Local instance	https://iglocska.eu	✖
https://iglocska.eu	https://iglocska.eu	✖

At multiple levels: **Events**, **Attributes**, **Objects** (and their **Attributes**) and **Galaxy-clusters**

Sharing in MISP: Distribution



Data layer

Data layer: Naming conventions

- Data layer
  - **Events** are encapsulations for contextually linked information
  - **Attributes** are individual data points, which can be indicators or supporting data.
  - **Objects** are custom templated Attribute compositions
  - **Object references** are the relationships between individual building blocks
  - **Shadow Attributes**/**Proposal** are suggestions made by users to modify an existing *attribute*
  - **Sightings** are a means to convey that a data point has been seen
  - **Event reports** are supporting materials for analysts to describe *events*, *processes*, etc

Data layer: Events

Events are encapsulations for contextually linked information

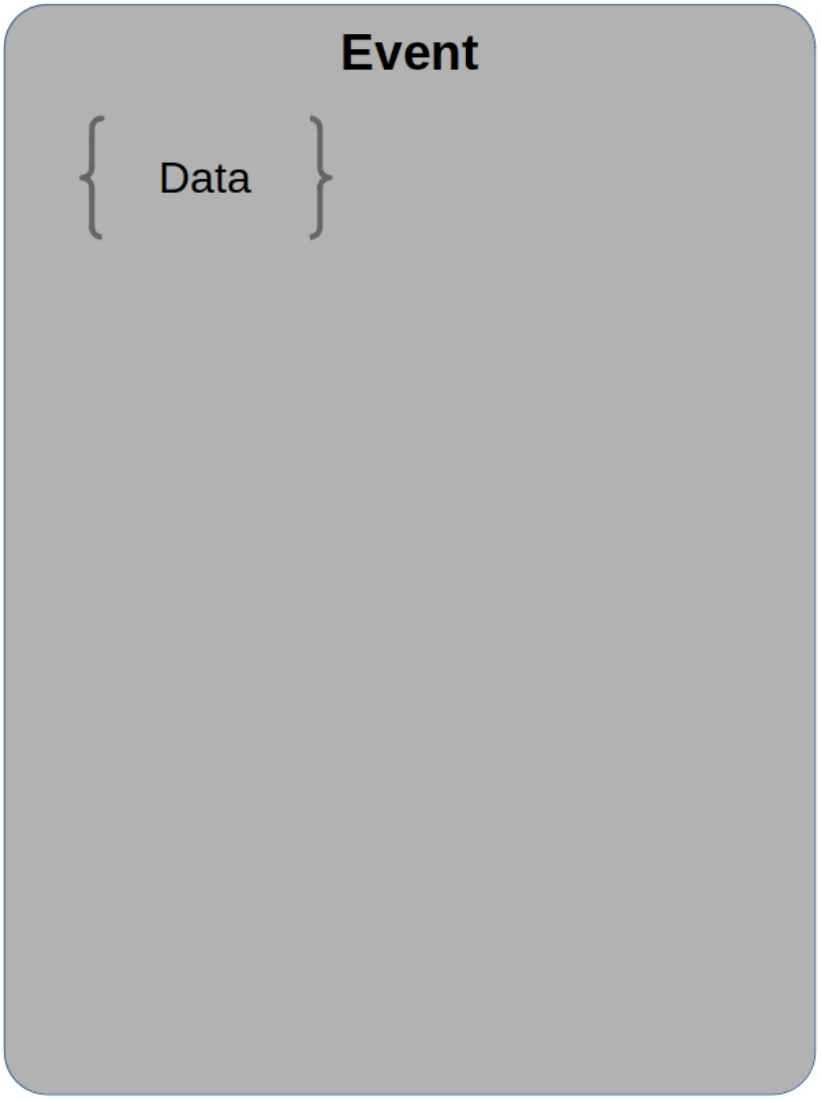
Purpose: Group datapoints and context together. Acting as an envelop, it allows setting distribution and sharing rules for itself and its children.

Usecase: Encode incidents / events / reports / ...

IoT malware - Gafgyt.Gen28 (active) - 20190220 - 20190222

Event ID	178
UUID	5c6d21e5-bb60-47b7-b892-42e6950d2111
Creator org	CIRCL
Owner org	Training
Creator user	andras.iklody@circl.lu
Tags	tlp:white x osint:source-type="automatic-collection" x circl:incident-classification="malware" x adversary:infrastructure-action="take-down" x
Date	2019-02-20
Threat Level	Low
Analysis	Completed
Distribution	All communities
Info	IoT malware - Gafgyt.Gen28 (active) - 20190220 - 20190222
Published	Yes (2020-11-28 07:53:39)
#Attributes	2601 (296 Objects)
First recorded change	2019-02-20 09:46:24
Last change	2020-10-10 07:36:28
Modification map	
Sightings	0 (0) - restricted to own organisation only.

Data layer: Event building blocks - Base



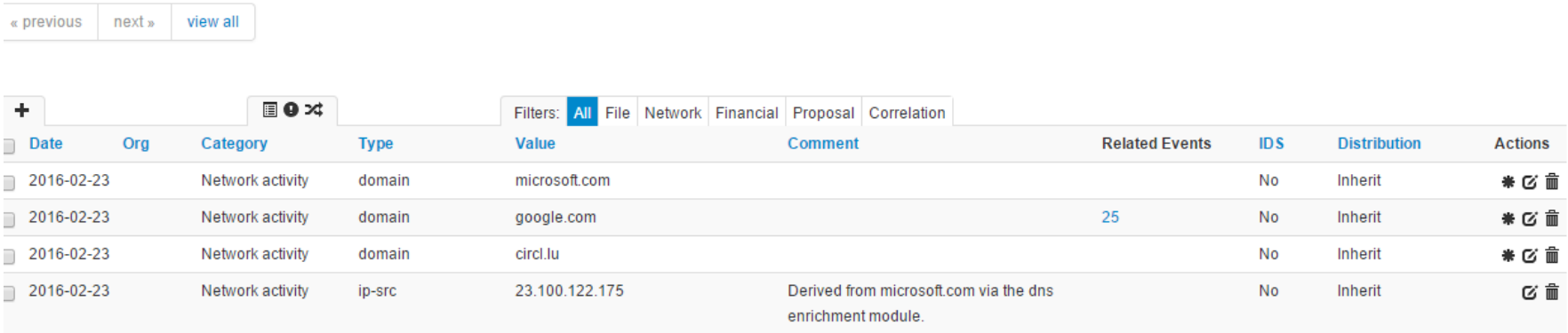
Data layer: Events

```
{
    "date": "2019-02-20",
    "info": "IoT malware - Gafgyt.Gen28 (active)",
    "uuid": "5c6d21e5-bb60-47b7-b892-42e6950d2111",
    "analysis": "2",
    "timestamp": "1602315388",
    "distribution": "3",
    "sharing_group_id": "0",
    "threat_level_id": "3",
    "extends_uuid": "",
    "Attribute": [...],
    "Object": [...],
    "EventReport": [...],
    "Tag": [...],
    "Galaxy": [...]
}
```

Data layer: Attributes

Attributes are individual data points, indicators or supporting data

> **Purpose**: Individual data point. Can be an indicator or supporting data.
>
> **Usecase**: Domain, IP, link, sha1, attachment, ...

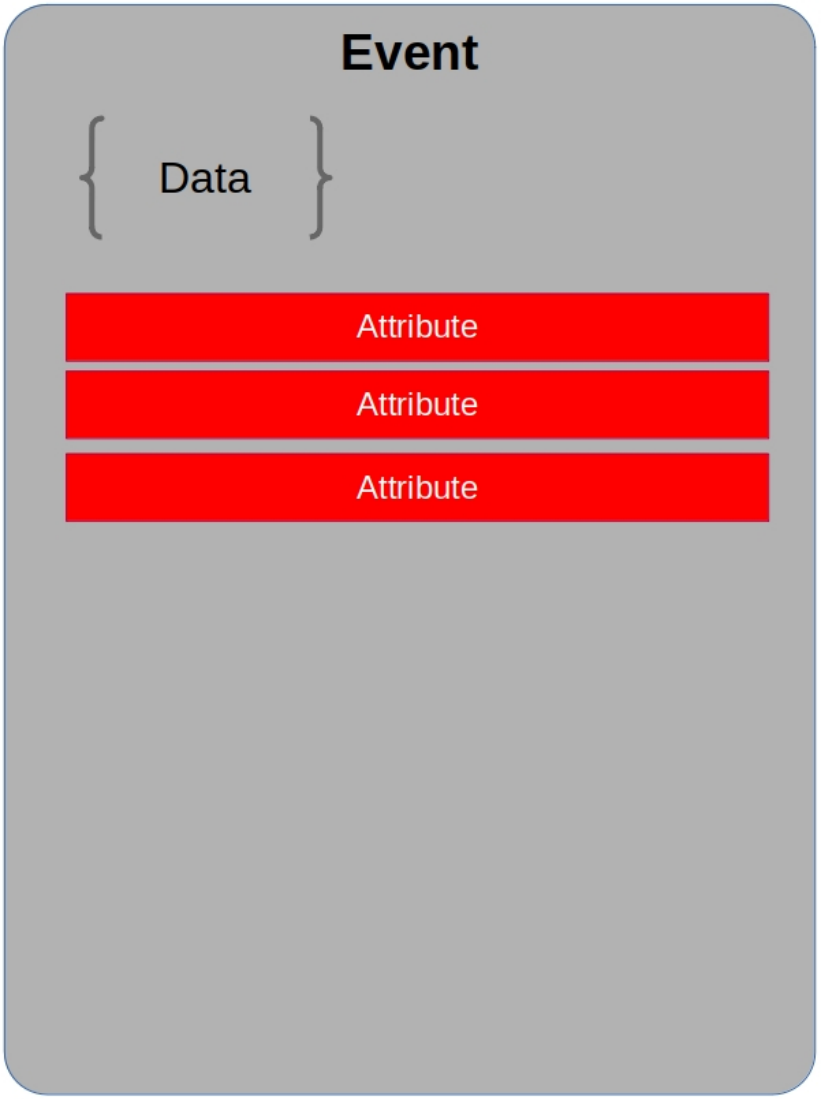


« previous | next » | view all

Filters: All | File | Network | Financial | Proposal | Correlation

Date	Org	Category	Type	Value	Comment	Related Events	IDS	Distribution	Actions
2016-02-23		Network activity	domain	microsoft.com			No	Inherit	
2016-02-23		Network activity	domain	google.com		25	No	Inherit	
2016-02-23		Network activity	domain	circl.lu			No	Inherit	
2016-02-23		Network activity	ip-src	23.100.122.175	Derived from microsoft.com via the dns enrichment module.		No	Inherit	

Data layer: Event building blocks - Raw data



Data layer: Attributes

```
{
    "type": "url",
    "category": "Network activity",
    "to_ids": true,
    "uuid": "5c6d24bd-d094-4dd6-a1b6-4fa3950d2111",
    "event_id": "178",
    "distribution": "5",
    "sharing_group_id": "0",
    "timestamp": "1550656701",
    "comment": "Delivery point for the malware",
    "object_id": "0",
    "object_relation": null,
    "first_seen": null,
    "last_seen": null,
    "value": "ftp://185.135.80.163/",
    "Tag": [...]
    "Galaxy": [...]
}
```

Data layer: MISP Objects

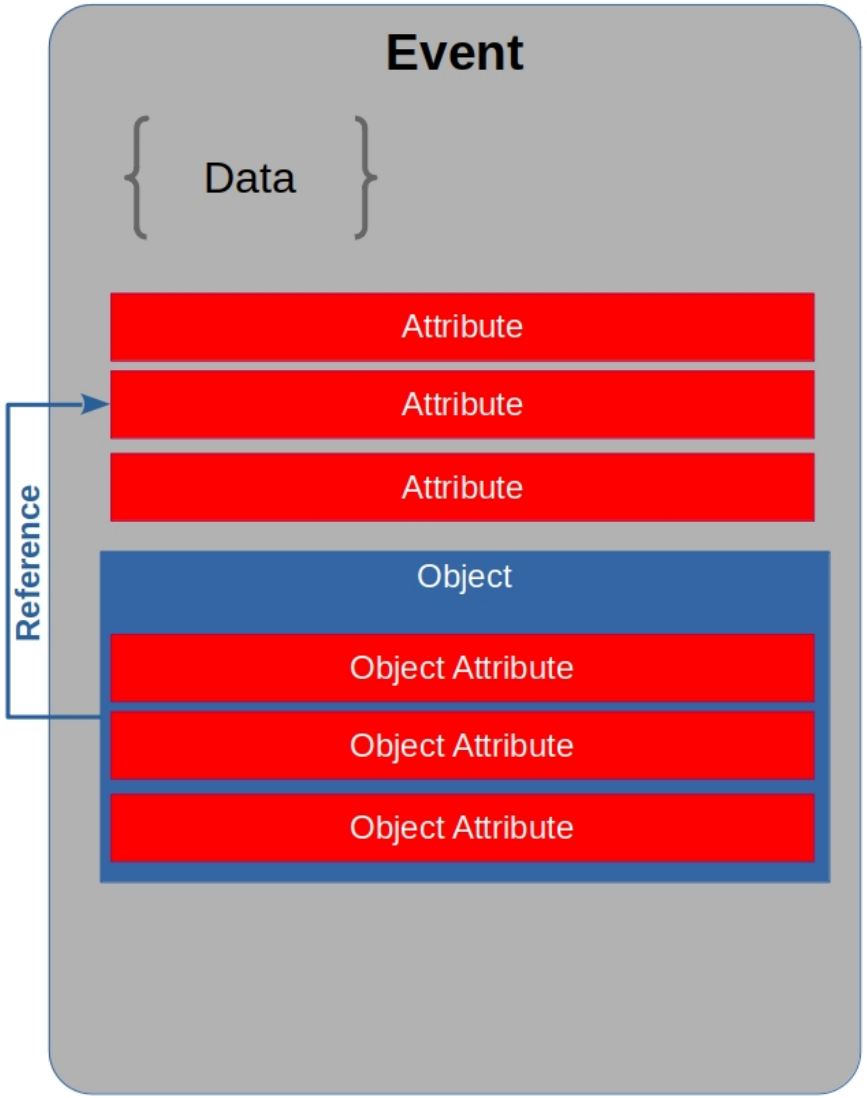
Objects are custom templated Attribute compositions

Purpose: Groups Attributes that are intrinsically linked together

Usecase: File, person, credit-card, x509, device, ...

☐	2018-03-27	**Name:** file ⤢ **References:** 1 ⤢ ⊕			
☐	2018-03-27	Payload delivery	**filename:** filename	putty.exe	+
☐	2018-03-27	Other	**size-in-bytes:** size-in-bytes	774200	+
☐	2018-03-27	Other	**entropy:** float	6.7264597226	+
☐	2018-03-27	Payload delivery	**md5:** md5	b6c12d88eeb910784d75a5e4df954001	+
☐	2018-03-27	Payload delivery	**sha1:** sha1	5ef9515e8fd92a254dd2dcdd9c4b50afa8007b8f	+
☐	2018-03-27	Payload delivery	**sha256:** sha256	81de43198730467613413870 5fc1c21188ad7f27edf6b77a6551aa693194485e	+
☐	2018-03-27	Payload delivery	**sha512:** sha512	e174ecf4fffb36d30c2cc66b37f82877d421244c924d5c9f39f2e0f37d85332b7d107d5ac5bd19cb7ffdcdbdd8b506d488faa30664ef610f62f3970c163cca76	+
☐	2018-03-27	Payload delivery	**malware-sample:**	putty.exe	+

Data layer: Event building blocks - Data composition



Data layer: MISP Objects

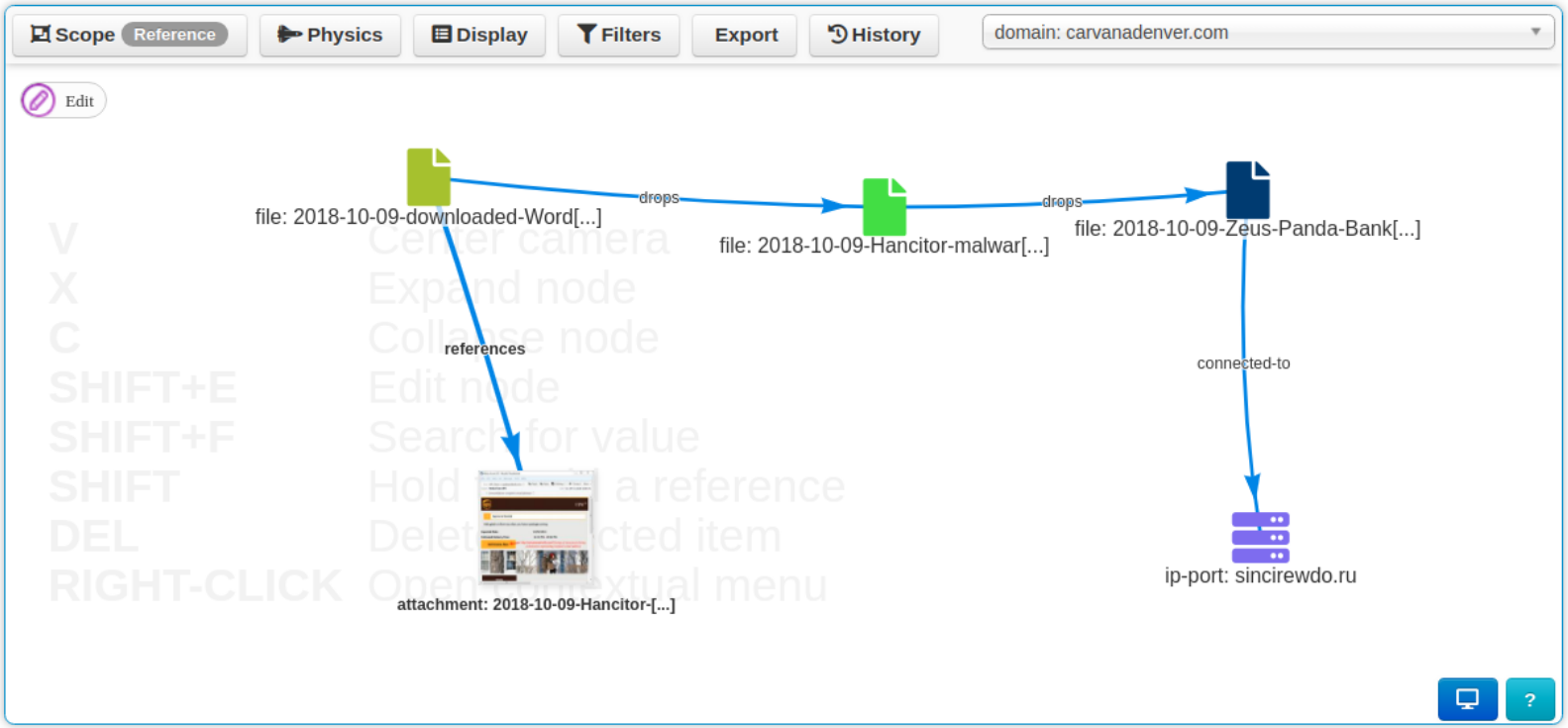
```
{
    "name": "elf-section",
    "meta-category": "file",
    "description": "Object describing a sect...",
    "template_uuid": "ca271f32-1234-4e87-b240-6b6e882de5de",
    "template_version": "4",
    "uuid": "ab5f0c85-5623-424c-bc03-d79841700d74",
    "timestamp": "1550655984",
    "distribution": "5",
    "sharing_group_id": "0",
    "comment": "",
    "first_seen": null,
    "last_seen": null,
    "ObjectReference": [],
    "Attribute": [...]
}
```

Data layer: Object references

Object references are the relationships between individual building blocks

Purpose: Allows to create relationships between entities, thus creating a graph where they are the edges and entities are the nodes.

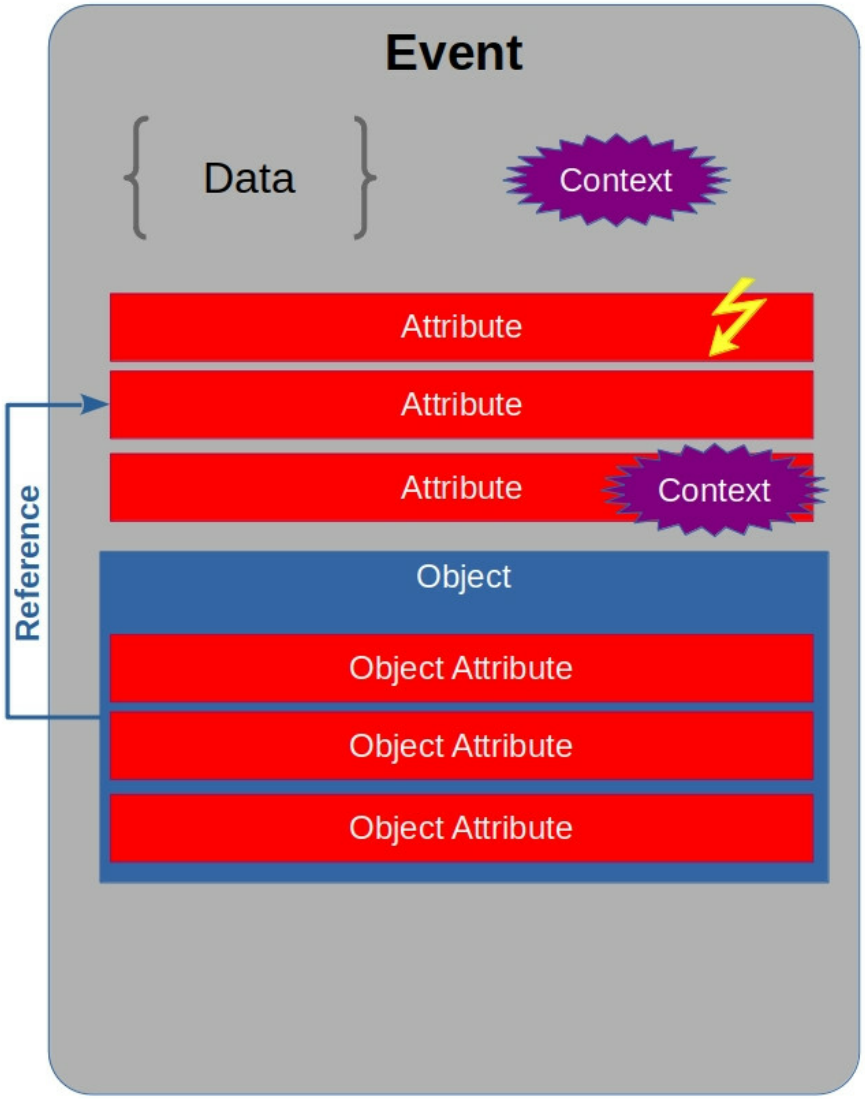
Usecase: Represent behaviours, similarities, affiliation, ...



Data layer: Object references

```
{
    "uuid": "5c6d21f9-0384-4bd2-b256-40de950d2111",
    "timestamp": "1602318569",
    "object_id": "1024",
    "source_uuid": "23275e05-c202-460e-aadf-819c417fb326",
    "referenced_uuid": "ab5f0c85-5623-424c-bc03-d79841700d74",
    "referenced_type": "1",
    "relationship_type": "included-in",
    "comment": "Section 0 of ELF"
}
```

Data layer: Event building blocks - Context

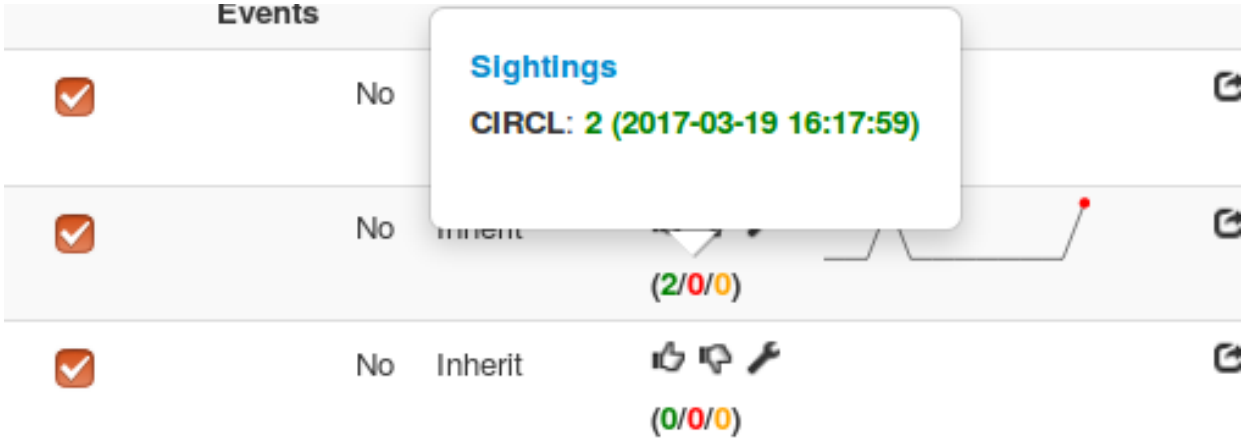


Data layer: Sightings

Sightings are a means to convey that a data point has been seen

Purpose: Allows to add temporality to the data.

Usecase: Record activity or occurence, perform IoC expiration, ...



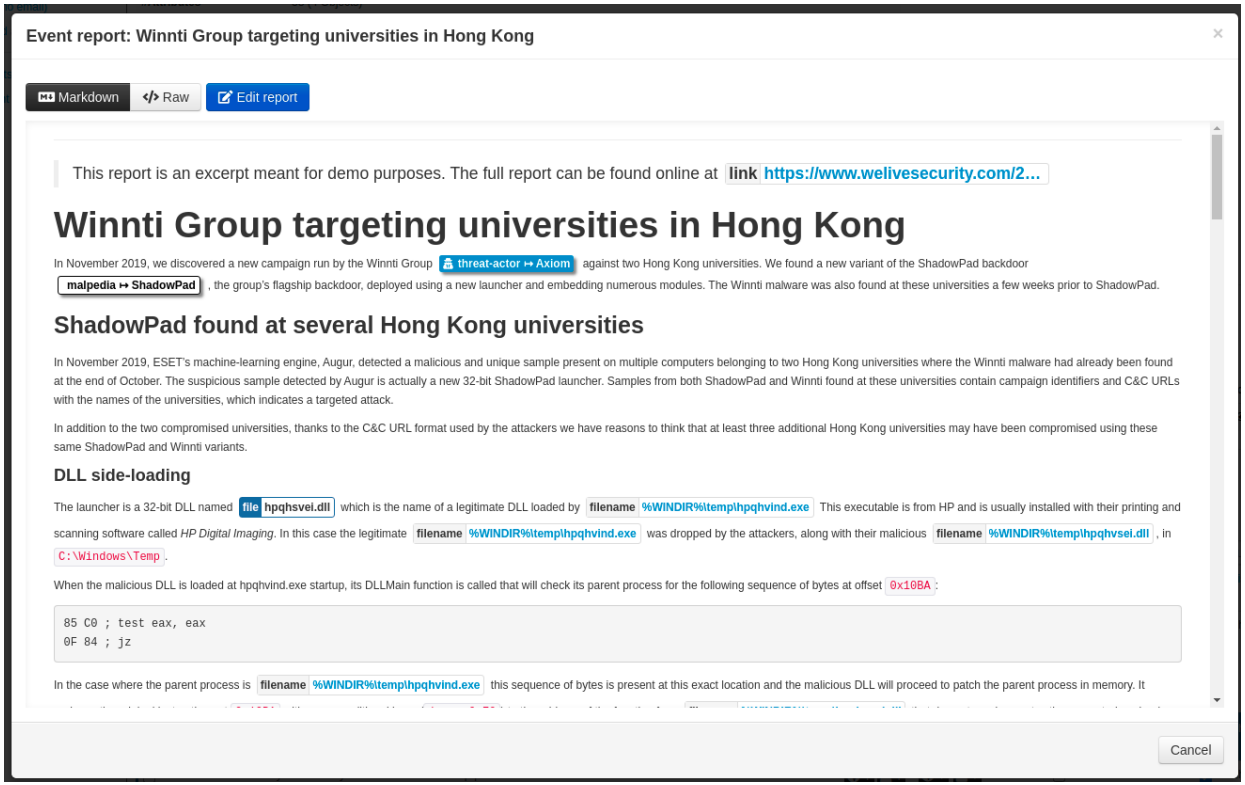

```
{
    "org_id": "1",
    "date_sighting": "1573722432",
    "uuid": "5dcd1940-5de8-4462-93dd-12a2a5e38e14",
    "source": "",
    "type": "0",
    "attribute_uuid": "5da97b59-9650-4be2-9443-2194a5e38e14"
}
```

Data layer: Event reports

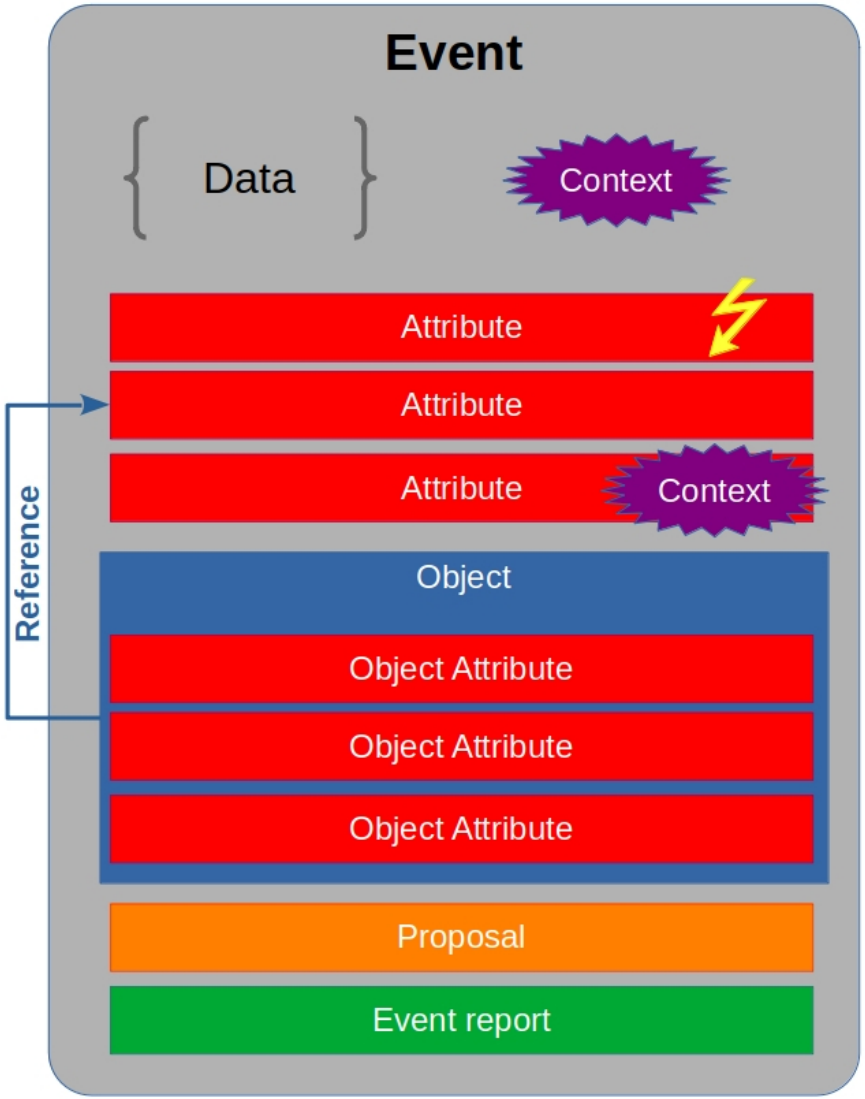
Event reports are supporting data for analysis to describe **events**, **processes**, ect

Purpose: Supporting data point to describe events or processes

Usecase: Encode reports, provide more information about the Event, ...



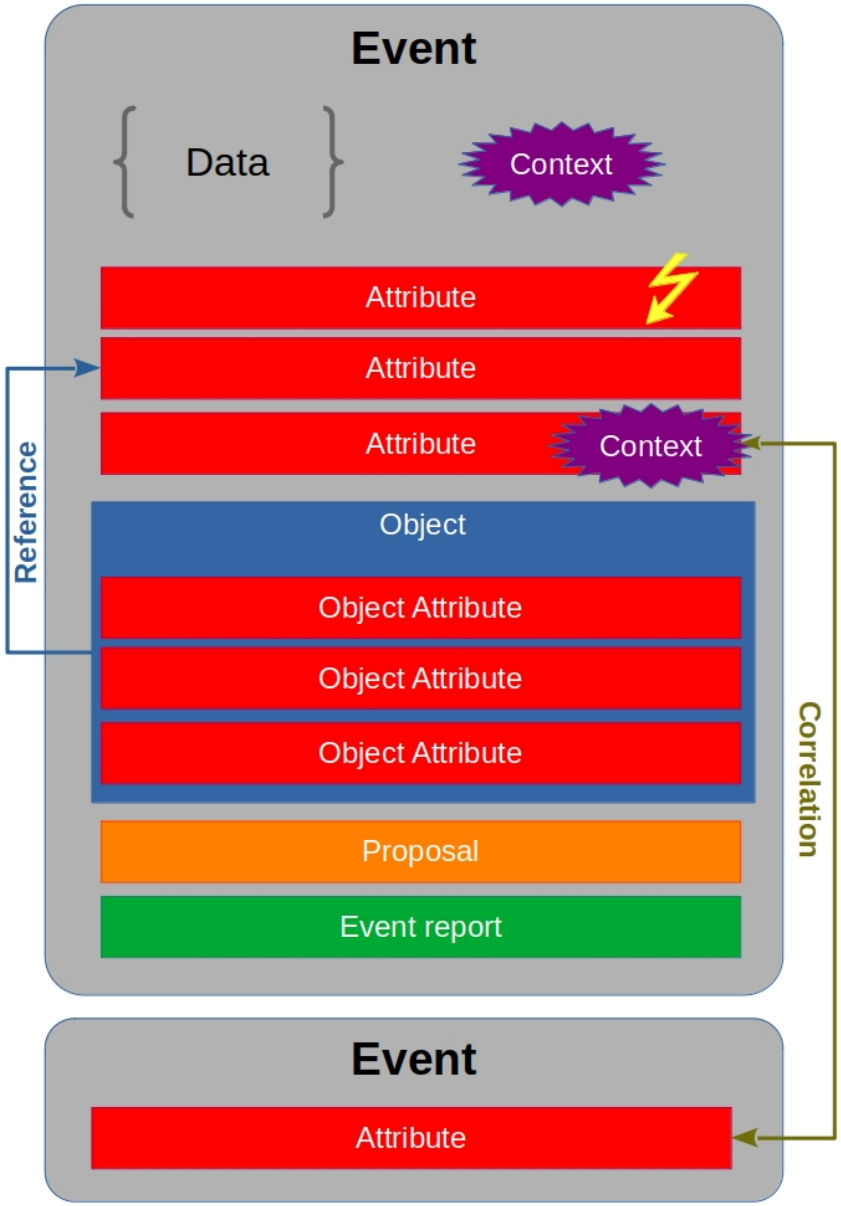
Data layer: Event building blocks - Collaboration & intelligence



Data layer: Event reports

```
{
    "uuid": "076e240b-5a76-4a8b-9eab-cfff551993dd",
    "event_id": "2127",
    "name": "Event report (1607362986)",
    "content": "...",
    "distribution": "5",
    "sharing_group_id": "0",
    "timestamp": "1607362986"
}
```

Data layer: Event building blocks - Full



Context layer

Context layer: Naming conventions

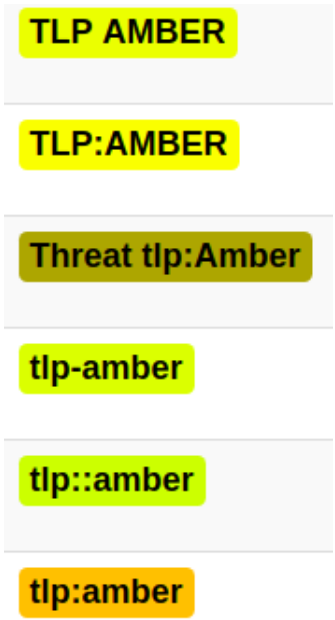
- Context layer
  - **Tags** are free-text labels attached to events/attributes and can come from **Taxonomies**
    - `Android Malware`, `C2`, ...
  - **Taxonomies** are a set of common classification allowing to express the same vocabulary among a distributed set of users and organisations
    - `tlp:green`, `false-positive:risk="high"`, `admiralty-scale:information-credibility="2"`

Context layer: Naming conventions

- Context layer
  - **Galaxies** are container copmosed of **Galaxy-clusters** that belongs to the same family
    - Similar to what **Events** are to **Attributes**
    - `Country`, `Threat actors`, `Botnet`, ...
  - **Galaxy-clusters** are knowledge base items coming from **Galaxies**.
    - Basically a taxonomy with additional meta-information
    - `misp-galaxy:threat-actor="APT 29"`, `misp-galaxy:country="luxembourg"`

Context layer: Tags

Simple free-text labels




```
{
    "name": "Android malware",
    "colour": "#22681c",
    "exportable": true,
    "numerical_value": null,
}
```

Context layer: Taxonomies

Simple label standardised on common set of vocabularies

Purpose: Enable efficent classification globally understood, easing consumption and automation.

Usecase: Provide classification such as: TLP, Confidence, Source, Workflows, Event type, ...

Tag	Events	Attributes	Tags
workflow:state="complete"	11	0	workflow:state="complete"
workflow:state="draft"	0	0	workflow:state="draft"
workflow:state="incomplete"	55	10	workflow:state="incomplete"
workflow:state="ongoing"	0	0	workflow:state="ongoing"

Context layer: Taxonomies

```
{
  "Taxonomy": {
    "namespace": "admiralty-scale",
    "description": "The Admiralty Scale or Ranking (also called
        the NATO System)...",
    "version": "6",
    "exclusive": false,
  },
  "entries": [
     {
       "tag": "admiralty-scale:information-credibility=\"1\"",
       "expanded": "Information Credibility: Confirmed by other
           sources",
       "numerical_value": 100,
       "exclusive_predicate": true,
     },
     ...
  ]
}
```

Context layer: Galaxies

Collections of **galaxy clusters**

Threat Actor galaxy

Galaxy ID	8
Name	Threat Actor
Namespace	misp
UUID	698774c7-8022-42c4-917f-8d6e4f06ada3
Description	Threat actors are characteristics of malicious actors (or adversaries) representing a cyber attack threat including presumed intent and historically observed behaviour.
Version	3

« previous | next »

All | Default | Custom 0 | My Clusters | Deleted | View Fork Tree | View Galaxy Relationships

apt29 Filter

ID	Published	Value	Synonyms	Owner Org	Creator Org	Default	Activity	#Events	#Relations	Description	Distribution	Actions
7059	N/A	APT 29	Dukes, Group 100, Cozy Duke, CozyDuke, EuroAPT, CozyBear, CozyCar, Cozer, Office Monkeys	MISP	MISP	✔		0	0 0	A 2015 report by F-Secure describe APT29 as: 'The Dukes are a well-resourced, highly dedicated and organized cyberespionage group that we believe has been working for the Russian Federation since at least 2008 to collect intelligence in support of foreign and security policy decision-making. The Dukes show unusual confidence in their ability to continue successfully compromising their	All communities	

Context layer: Galaxy clusters

Kownledge base items including a description, links, synonyms, meta-information and relationships

Purpose: Enable description of complex high-level information for classification

Usecase: Extensively describe elements such as threat actors, countries, technique used, ...

Threat Actor :: APT 29

Cluster ID	2805
Name	APT 29
Parent Galaxy	Threat Actor
Description	A 2015 report by F-Secure describe APT29 as: 'The Dukes are a well-resourced, highly dedicated and organized cyberespionage group that we believe has been working for the Russian Federation...
Published	No
Default	Yes
Version	190
UUID	b2056ff0-00b9-482e-b11c-c771daa5f28a
Collection UUID	7cdff317-a673-4474-84ec-4f1754947823
Source	MISP Project
Authors	Alexandre Dulaunoy, Florian Roth, Thomas Schreck, Timo Steffens, Various
Distribution	**All communities**
Owner Organisation	MISP
Creator Organisation	MISP
Connector tag	misp-galaxy:threat-actor="APT 29"
Events	0
Forked From	
Forked By	

Context layer: Galaxy clusters

Galaxy cluster elements: Tabular view

Tabular view | JSON view

Key ↓	Value	Actions
attribution-confidence	50	
cfr-suspected-state-sponsor	Russian Federation	
cfr-suspected-victims	United States	
cfr-suspected-victims	China	
cfr-suspected-victims	New Zealand	

Galaxy cluster elements: JSON view

Tabular view | JSON view | +Add JSON as cluster's elements


```
{
    "attribution-confidence": [
        "50"
    ],
    "cfr-suspected-state-sponsor": [
        "Russian Federation"
    ],
    "cfr-suspected-victims": [
        "United States",
        "China",
        "New Zealand",
        "Ukraine",
```

Context layer: Galaxy clusters

```
{
    "uuid": "5eda0a53-1d98-4d01-ae06-40da0a0002of",
    "type": "fellowship-characters",
    "value": "Aragorn wielding Anduril",
    "tag_name": "misp-galaxy:fellowship-characters=\"c3fe907a-6a36
        -4cd1-9456-dcdf35c3f907\"",
    "description": "The Aragorn character wielding Anduril",
    "source": "Middle-earth universe by J. R. R. Tolkien",
    "authors": null,
    "version": "1591347795",
    "distribution": "0",
    "sharing_group_id": null,
    "default": false,
    "extends_uuid": "5eda0117-1e14-4b0a-9e26-34aff331dc3b",
    "extends_version": "1591345431",
    "GalaxyElement": [...],
    "GalaxyClusterRelation": [...]
}
```

Context layer: Galaxies & Galaxy clusters

- MISP integrates MITRE's Adversarial Tactics, Techniques, and Common Knowledge (ATT&CK) and similar **Galaxy Matrix**
- MISP terminology of these matrixes: **Galaxy Matrix**

Pre Attack - Attack Pattern | **Enterprise Attack - Attack Pattern** | Mobile Attack - Attack Pattern

0 — 11 — Show all

Initial access	Execution	Persistence	Privilege escalation	Defense evasion	Credential access	Discovery	Lateral movement	Collection	Exfiltration	Command and control
Spearphishing Attachment	Scripting	Screensaver	File System Permissions Weakness	Process Hollowing	Securityd Memory	Password Policy Discovery	AppleScript	Data from Information Repositories	Exfiltration Over Alternative Protocol	Standard Application Layer Protocol
Spearphishing via Service	Command-Line Interface	Login Item	AppCert DLLs	Code Signing	Input Capture	System Network Configuration Discovery	Distributed Component Object Model	Data from Removable Media	Exfiltration Over Command and Control Channel	Communication Through Removable Media
Trusted Relationship	User Execution	Trap	Application Shimming	Rootkit	Bash History	Process Discovery	Pass the Hash	Man in the Browser	Data Compressed	Custom Command and Control Protocol
Replication Through Removable Media	Regsvcs/Regasm	System Firmware	Scheduled Task	NTFS File Attributes	Exploitation for Credential Access	Network Share Discovery	Exploitation of Remote Services	Data Staged	Automated Exfiltration	Multi-Stage Channels
Exploit Public-Facing Application	Trusted Developer Utilities	Registry Run Keys / Start Folder	Startup Items	Exploitation for Defense Evasion	Private Keys	Peripheral Device Discovery	Remote Desktop Protocol	Screen Capture	Scheduled Transfer	Remote Access Tools
Spearphishing Link	Windows Management Instrumentation	LC_LOAD_DYLIB Addition	New Service	Network Share Connection Removal	Brute Force	Account Discovery	Pass the Ticket	Email Collection	Data Encrypted	Uncommonly Used Port
Valid Accounts	Service Execution	LSASS Driver	Sudo Caching	Process Doppelgänging	Password Filter DLL	System Information Discovery	Windows Remote Management	Clipboard Data	Exfiltration Over Other Network Medium	Multilayer Encryption
Supply Chain Compromise	CMSTP	Rc.common	Process Injection	Disabling Security Tools	Two-Factor Authentication Interception	System Network Connections Discovery	Windows Admin Shares	Video Capture	Exfiltration Over Physical Medium	Domain Fronting
Drive-by Compromise	Control Panel Items	Authentication Package	Bypass User Account Control	Timestomp	LLMNR/NBT-NS Poisoning	Network Service Scanning	Remote Services	Audio Capture	Data Transfer Size Limits	Data Obfuscation
Hardware Additions	Dynamic Data Exchange	Component Firmware	Extra Window Memory Injection	Modify Registry	Credentials in Files	File and Directory Discovery	Taint Shared Content	Data from Network Shared Drive		Connection Proxy
	Source	Windows Management Instrumentation Event Subscription	Setuid and Setgid	Indicator Removal from Tools	Forced Authentication	Security Software Discovery	Application Deployment Software	Data from Local System		Commonly Used Port
	Space after Filename	Change Default File	Launch Daemon	Hidden Window	Keychain	System Service Discovery	Third-party Software	Automated Collection		Data Encoding

Galaxy JSON matrix-like

```
1  {
2    "description": "Universal Development and Security Guidelines as
         Applicable to Election Technology.",
3    "icon": "map",
4    "kill_chain_order": {                    \\Tab in the matrix
5          "example-of-threats": [             \\Column in the matrix
6          "setup | party/candidate-registration",
7          "setup | electoral-rolls",
8          "campaign | campaign-IT",
9          "all-phases | governement-IT",
10         "voting | election-technology",
11         "campaign/public-communication | media/press"
12       ]
13   },
14   "name": "Election guidelines",
15   "namespace": "misp",
16   "type": "guidelines",
17   "uuid": "c1dc03b2-89b3-42a5-9d41-782ef726435a",
18   "version": 1
19 }
```

Cluster JSON matrix-like

```
{
      "description": "DoS or overload of party/campaign
          registration, causing them to miss the deadline",
      "meta": {
        "date": "March 2018.",
         "kill_chain": [  \\Define in which column the cluster should be placed
           "example-of-threats:setup | party/candidate-registration"
         ],
        "refs": [
          "https://www.ria.ee/sites/default/files/content-editors/
              kuberturve/cyber_security_of_election_technology.pdf
              "
        ]
      },
      "uuid": "154c6186-a007-4460-a029-ea23163448fe",
      "value": "DoS or overload of party/campaign registration,
          causing them to miss the deadline"
}
```

Expressing relation between clusters

- Cluster can be related to one or more clusters using default relationships from MISP objects and a list of tags to classify the relation.

```
        "related": [
        {
          "dest-uuid": "5ce5392a-3a6c-4e07-9df3-9b6a9159ac45",
          "tags": [
            "estimative-language:likelihood-probability=\"likely\"
                "
          ],
          "type": "similar"
        }
      ],
      "uuid": "oca45163-e223-4167-b1af-f088ed14a93d",
      "value": "Putter Panda"
```

Acknowledgements

- Supported by the grant 2018-LU-IA-0148

Co-financed by the European Union

Connecting Europe Facility

MISP

INTELLIGENCE SHARING

Building dashboard widgets in MISP 2.5

Typed configuration, scoped data, reusable rendering

CIRCL / MISP Project

https://www.misp-project.org/

Reference snapshot: 29 October 2026

Build one widget, understand the whole path

- Discover and load a custom PHP class.
- Declare typed settings and a shared time filter.
- Query only the signed-in user’s own records.
- Return renderer data and choose a safe cache scope.
- Verify the contract through PHP, HTTP and the browser.

Reference environment

MISP 2.5.50, reviewed 29 October 2026. Patch level and theme matter.

Understand the model

A widget extends the integrated dashboard

The board owns

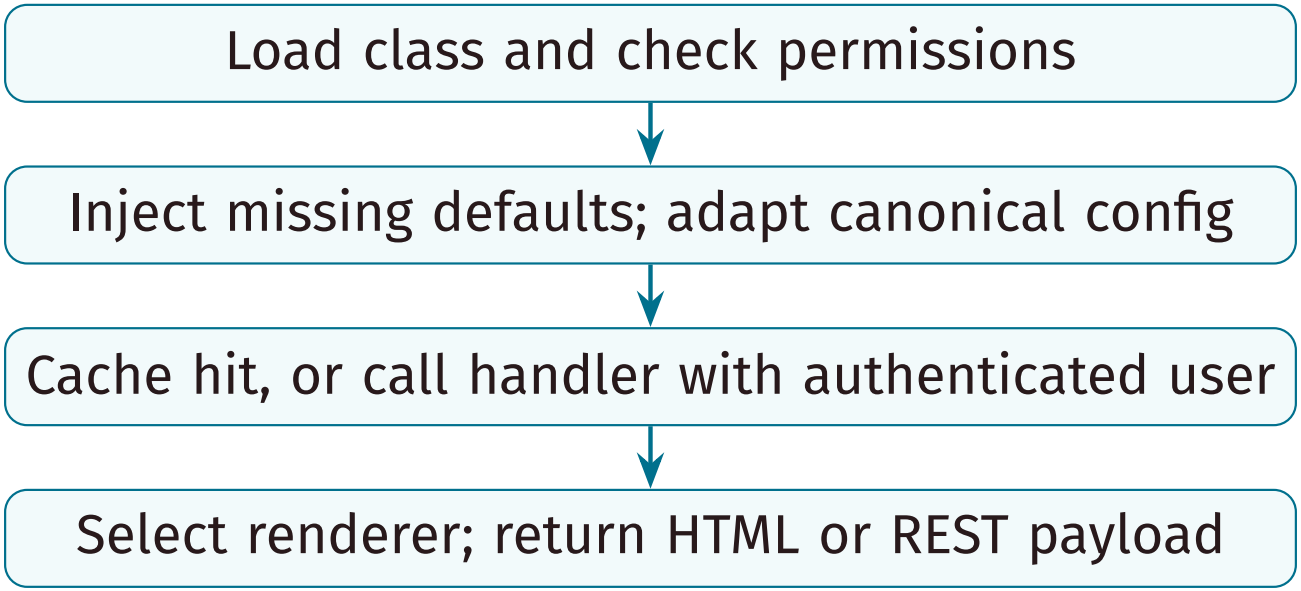
Layout, gallery, configuration, filters, scheduling and export.

Your widget owns

Metadata, input validation, data access and renderer-shaped output.

Reuse an existing renderer first. Add custom rendering only when the data needs a different visual form.

The render request has a defined pipeline



Source: DashboardsController::renderWidget

Discovery: filename, class and location

```
app/Lib/Dashboard/Custom/
  TrainingLoginSummaryWidget.php

class TrainingLoginSummaryWidget
{
    // Metadata and handler live here.
}
```

- Use a unique class ending in `Widget`.
- Match the filename exactly; no registration list.
- Built-in classes take precedence over Custom.

Metadata gives the gallery a useful entry

Property	Purpose
`title`, `description`	Explain the result and its data scope.
`category`, `thumbnail`	Help discovery; thumbnail is optional.
`render`	Name an existing renderer, here `SimpleList`.
`width`, `height`	Initial tile dimensions.
`schema`	Typed configuration and canonical filter declarations.
`params`, `placeholder`	Advanced option documentation and example JSON.

Two decisions: who may load it, what may they read?

Widget permission

`checkPermissions($user)` gates discovery and loading.

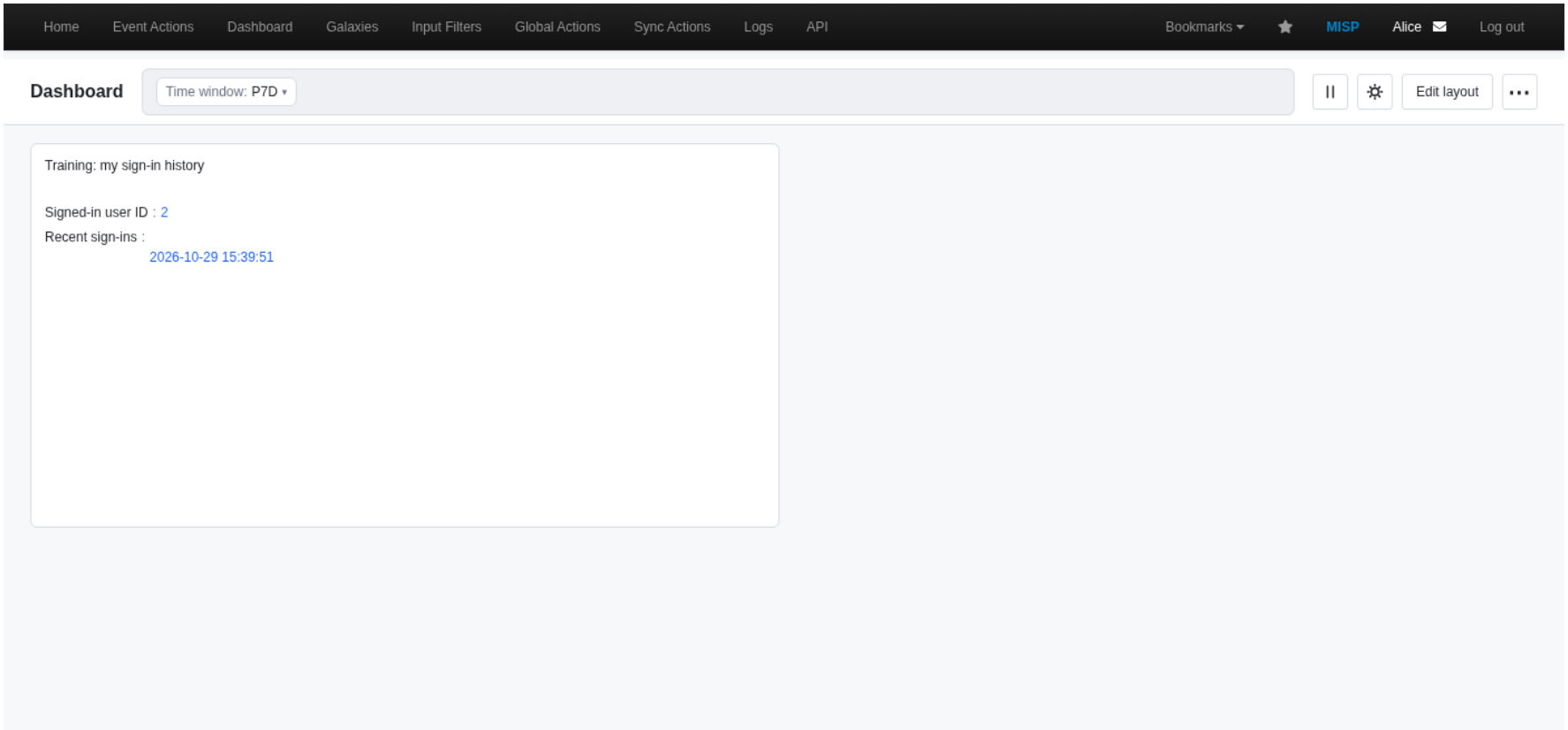
Data permission

The handler must apply the right user, organisation or event-access scope to its query.

A visible gallery card does not make every database row visible.

Build the example

Our exercise: my sign-in history



Declare the canonical time-window setting

```
public $schema = [
    'lookback' => [
        'type' => 'time_window',
        'default' => 'P7D',
        'help' => 'Your own sign-ins; at most 366 days.',
    ],
    // Scalar fields follow in the complete example.
];
```

The shared toolbar follows the declared **type**, not the name `lookback`.

Canonical inputs and scalar settings serve different jobs

Shared concepts

Time, tags, organisations, sharing groups, galaxy clusters, distribution, threat level and analysis state.

Widget-specific settings

`string`, `int`, `bool`, `enum`. Here: caption, result limit and organisation checkbox.

`attribute_type_filter` and `event_id_filter` are canonical, but are not toolbar controls in this snapshot.

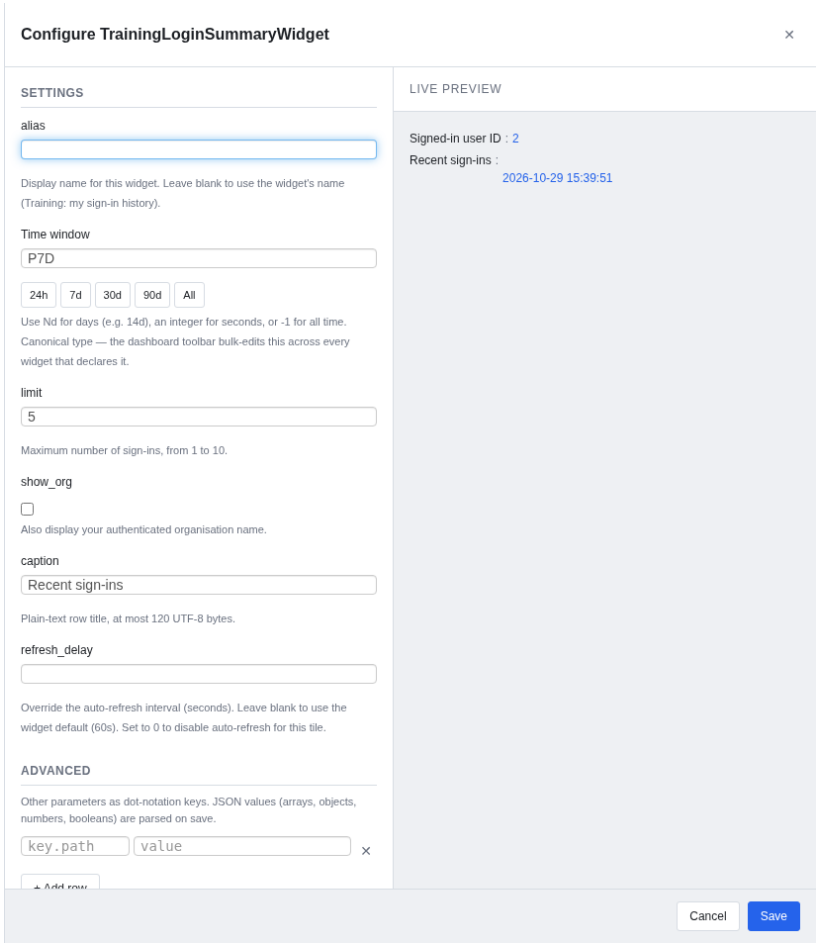
The adapter changes what the handler receives

Saved / wire value	Handler value
`P7D`	`7d`
`P2W`	`14d`
`PT6H`	`21600` seconds
`-1`	All-time sentinel, unchanged

Defaults do not mean validation

Missing keys get defaults. Explicit nulls, empty strings and unknown shapes may reach your handler.

Typed configuration, visible in the real panel



- Time-window control.
- Integer result limit.
- Boolean checkbox.
- Plain-text caption.

The panel saves configuration separately from layout editing.

Validate before querying

- `limit`: integer from 1 to 10.
- `show_org`: real boolean, not the string `"false"`.
- `caption`: nonempty UTF-8, at most 120 bytes, no controls.
- Reject unknown keys, including a supplied `user_id`.

Framework-owned settings

Accept `alias` and `refresh_delay`; do not let them change data.

Apply identity in the actual SQL conditions

```
$conditions = [
    'Log.user_id' => (int)$user['id'],
    'Log.action' => 'login',
];
if ($seconds !== -1) {
    $conditions['Log.created >='] =
        date('Y-m-d H:i:s', time() - $seconds);
}
```

Query identity comes from the authenticated user, never configuration.

Return the renderer's data, not a page

```
$rows = [
    ['title' => 'Signed-in user ID',
     'value' => (string)$user['id']],
    ['title' => $caption,
     'value' => $times ?: 'No matching sign-ins.'],
];
return $rows;
```

`SimpleList` accepts scalar or array values.
Empty history is a valid result. A database failure is an error.

Escaping belongs at the output sink

Use plain values

`<b>Plain text</b>` should appear as text in the widget.

- `SimpleList` escapes `title` and `value`.
- `html` and `html_title` deliberately bypass escaping.
- `drilldown` is checked by the dashboard URL validator.

Refresh interval and cache TTL are different

Example property	Meaning
`autoRefreshDelay = 60`	Request a refresh every 60 seconds
`cache_duration = 30`	Reuse a result for 30 seconds
`cache_scope = 'user'`	Separate cached results by user

Manual refresh can return a cache hit. Redis failure falls back to live work.

Choose cache scope from the output's audience

Scope	Appropriate when
Default	The same config yields identical data for every viewer.
`org`	An aggregate is identical within an organisation; site admins have a separate unrestricted bucket.
`user`	Personal identity, role or access changes the result.

This example needs user scope

Two users in the same organisation have different sign-in histories.

Verify and contribute

Test the wrapped REST contract

```
{
  "widget": "TrainingLoginSummaryWidget",
  "config": {
    "lookback": "P7D",
    "limit": 5,
    "show_org": false,
    "caption": "Recent sign-ins"
  }
}
```

`POST /dashboards/renderWidget`

- Response: widget, instance ID, translated config, renderer, data.
- `/exportjson:1`: bare handler data.
- `/exportcsv:1`: flattened CSV.

A layout identifies instances, not just classes

```
[
  {
    "instance_id": "training_signins",
    "widget": "TrainingLoginSummaryWidget",
    "config": {"lookback": "P7D", "limit": 5},
    "position": {"x": 0, "y": 0, "w": 6, "h": 4}
  }
]
```

Keep instance IDs stable. Importing a layout changes the personal board.

Saving layout does not save every kind of change

Layout staging

Add, move, resize and remove; then Save changes or Discard.

Configuration persistence

The configure panel and shared toolbar save settings separately.

A layout Discard is not a universal undo.

Three layers of evidence

Layer	Verify
PHP contracts	Real schema, adapter, cache keys and template; recorded database boundary.
Live HTTP	Actual SQL results, user A/B/A, invalid inputs, export and layout import/readback.
Browser	Discovery, typed settings, saved state, literal caption, resize and usable controls.

The supplied verifier runs **59 checks**; it is not an integration test.

Workshop: install, challenge, remove

1. Export your board; install the one custom PHP file.
2. Lint it and run `verify_widget.php` against MISP.
3. Add the widget; save a caption and checkbox setting.
4. Render as A, B, A within the cache TTL.
5. Try an invalid boolean, limit, null and supplied user ID.
6. Restore the board and remove the training PHP file.

Port the older lesson deliberately

Earlier pattern	Reviewed 2.5 model
Name collision / mismatched file	Unique matching class and filename
`params` only	Schema plus deliberate legacy support
`cacheLifetime`	Duration and appropriate cache scope
Renderer assumptions	Actual element and data contract
Size-only layout	Stable IDs and position coordinates
Ad hoc testing	Contracts, live requests and browser checks

If a new renderer is necessary

- Start with the PHP element’s precise data shape.
- Reuse theme tokens and accessible, responsive markup.
- Extend the existing dashboard chart modules where needed.
- Add a gallery preview for the renderer.
- Test empty data, failures, resizing, keyboard and themes.

Make the contribution reviewable

- State the question the widget answers and the audience.
- Document schema, data visibility, refresh and cache scope.
- Include focused checks and a reproducible lab procedure.
- Show real UI evidence with synthetic data.
- Explain saved-config compatibility and deployment needs.

Continue from the repository

Working example, request, layout, verifier and pinned source audit are in `a.a-widget-dev/README.md`.

MISP

INTELLIGENCE SHARING

Turning data into actionable intelligence

advanced features in MISP supporting your analysts and tools

CIRCL / Team MISP Project

MISP Project

CRI MISP 2024

about CIRCL

circl.lu
Computer Incident
Response Center
LUXEMBOURG

- The Computer Incident Response Center Luxembourg (CIRCL) is a government-driven initiative designed to provide a systematic response facility to computer security threats and incidents. CIRCL is the CERT for the private sector, communes and non-governmental entities in Luxembourg and is operated by securitymadein.lu g.i.e.

MISP and CIRCL

- CIRCL is mandated by the Ministry of Economy and acting as the Luxembourg National CERT for private sector.
- CIRCL leads the development of the Open Source MISP threat intelligence platform which is used by many military or intelligence communities, private companies, financial sector, National CERTs and LEAs globally.
- **CIRCL runs multiple large MISP communities performing active daily threat-intelligence sharing**.

The aim of this presentation

- To give some insight into what sort of an evolution of our various communities' have gone through as observed over the past 8 years
- Show the importance of **strong contextualisation**...
- ...and how that can be leveraged when trying to make our data **actionable**

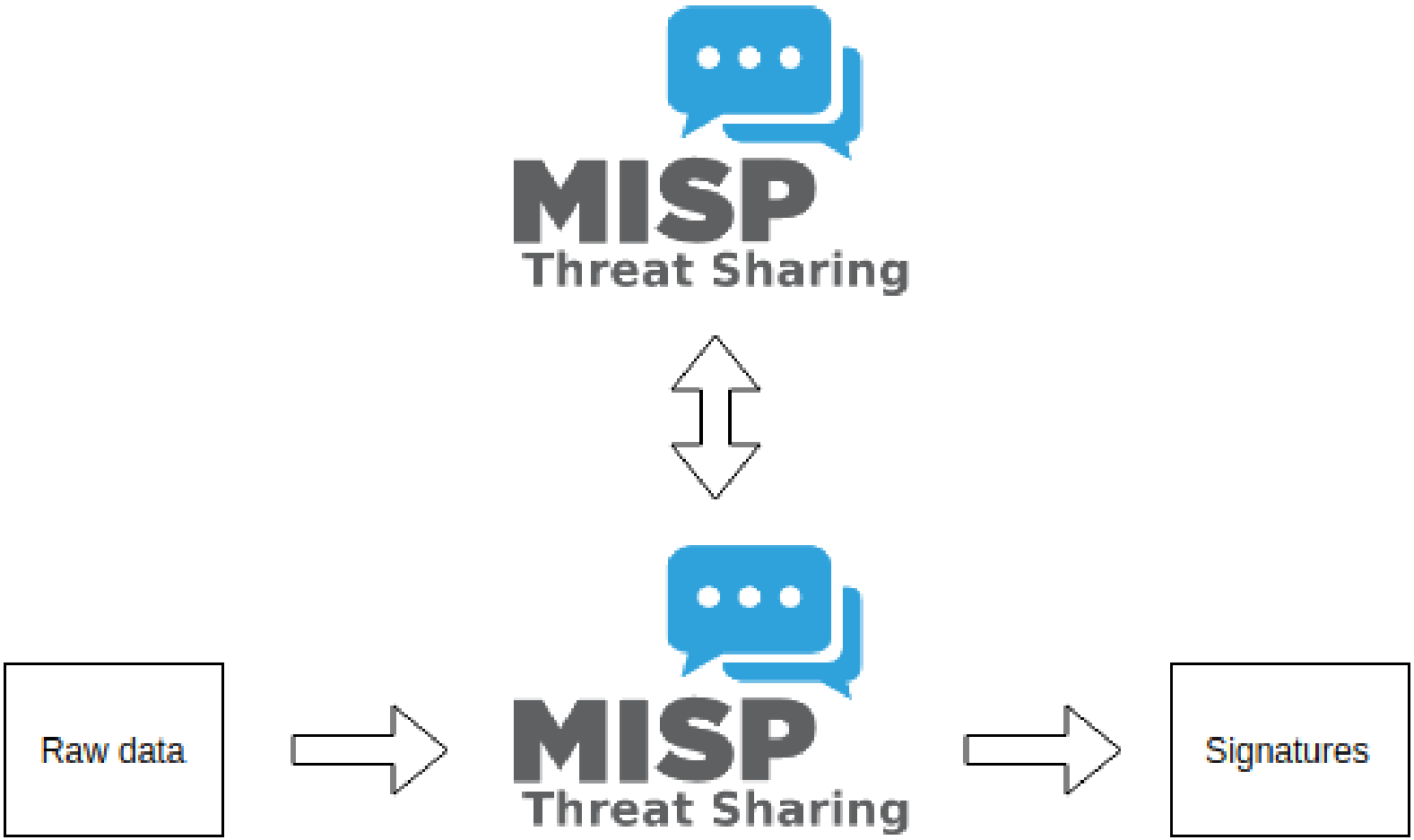
Development based on practical user feedback

- There are many different types of users of an information sharing platform like MISP:
  - **Malware reversers** willing to share indicators of analysis with respective colleagues.
  - **Security analysts** searching, validating and using indicators in operational security.
  - **Intelligence analysts** gathering information about specific adversary groups.
  - **Law-enforcement** relying on indicators to support or bootstrap their DFIR cases.
  - **Risk analysis teams** willing to know about the new threats, likelyhood and occurences.
  - **Fraud analysts** willing to share financial indicators to detect financial frauds.

The initial scope of MISP

- **Extract information** during the analysis process
- Store and **correlate** these datapoints
- **Share** the data with partners
- Focus on technical indicators: IP, domain, hostname, hashes, filename, pattern in file/memory/traffic
- Generate protective signatures out of the data: snort, suricata, OpenIOC

Initial workflow



Why was it so simplistic?

- This was both a reflection of our maturity as a community
  - Capabilities for **extracting** information
  - Capabilities for **utilising** the information
  - Lack of **willingness** to share context
  - Lack of **co-operation** between teams doing technical analysis/monitoring and threat-intel
- The more growth we saw in maturity, the more we tried to match it with our data-model, often against pushback

The growing need to contextualise data

- There were separate factors that made our data-sets less and less useful for detection/defense in general
  - **Growth of our communities**
  - Distinguish between information of interest and raw data
  - **False-positive** management
  - TTPs and aggregate information may be prevalent compared to raw data (risk assessment)
  - **Increased data volumes** leads to be able to prioritise

Our initial solution

- Allow users to **tag any information** created in MISP
- We wanted to be **lax with what we accept** in terms of data, but be **strict on what we fed to our tools**, with strong filter options
- We had some ideas on how to potentially move forward...

Our initial failures

- Try to capture different aspects of contextualisation into **normalised values** (threat level, source reliability, etc)
  - Didn't scale with needs other than our own
  - Incorporating new types of contextualisation would mean **the modification of the software**
  - Getting communities with **established naming conventions** to use anything but their go-to vocabularies was a pipe-dream
  - Heated arguments over numeric conversions

Human creativity

- We tried an alternate approach instead: Free tagging
  - Result was spectacularly painful, at least 7 different ways to spell tlp:amber
  - No canonisation for common terms lead to tagging ultimately becoming a highly flawed tool for filtering within a sharing community

How we ended up tackling the issue more successfuly

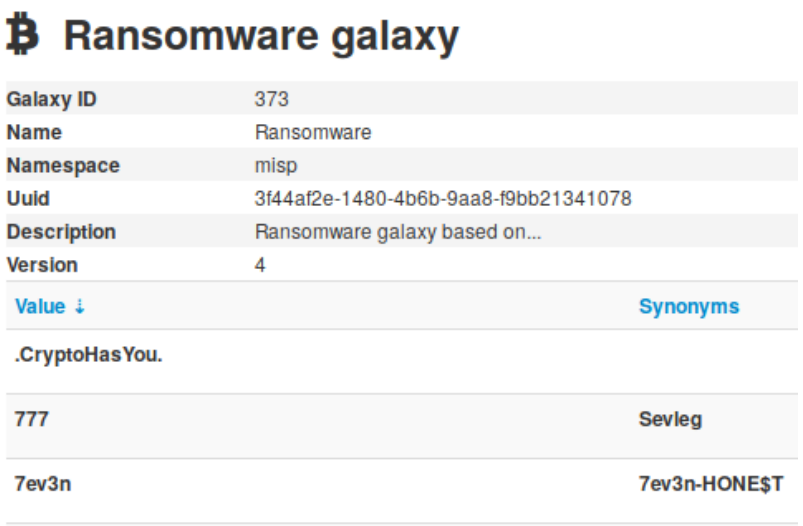
- We ended up with a mixed approach, currently implemented by the MISP-taxonomy system
  - Taxonomies are **vocabularies** of known tags
  - Tags would be in a **triple tag format**

    `namespace:predicate="value"`
  - Create your own taxonomies, recipients should be able to use data you tag with them without knowing it at the first place
  - Avoid any coding, stick to **JSON**
- Massive success, approaching 100 taxonomies
- Organisations can solve their own issues without having to rely on us

Tag	Events	Attributes	Tags
workflow:state="complete"	11	0	workflow:state="complete"
workflow:state="draft"	0	0	workflow:state="draft"
workflow:state="incomplete"	55	10	workflow:state="incomplete"
workflow:state="ongoing"	0	0	workflow:state="ongoing"

We were still missing something...

- Taxonomy tags often **non self-explanatory**
- Example: universal understanding of tlp:green vs APT 28
- For the latter, a single string was ill-suited
- So we needed something new in addition to taxonomies - **Galaxies**
  - Community driven **knowledge-base libraries used as tags**
  - Including descriptions, links, synonyms, meta information, etc.
  - Goal was to keep it **simple and make it reusable**
  - Internally it works the exact same way as taxonomies (stick to **JSON**)

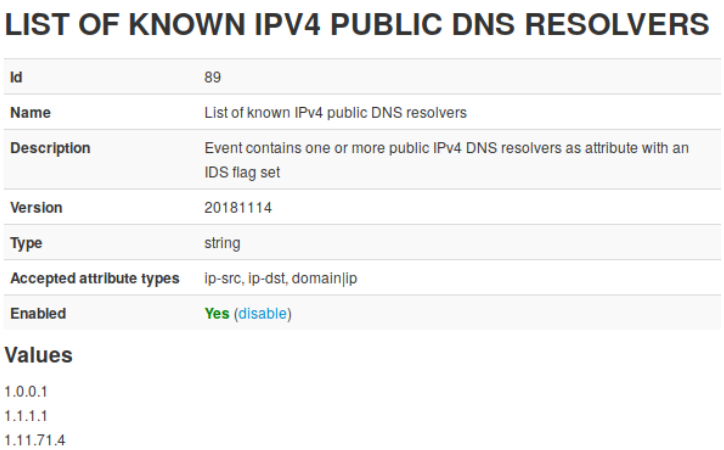


Broadening the scope of what sort of context we are interested in

- **Who** can receive our data? **What** can they do with it?
- **Data accuracy, source reliability**
- **Why** is this data relevant to us?
- **Who** do we think is behind it, **what tools** were used?
- What sort of **motivations** are we dealing with? Who are the **targets**?
- How can we **block/detect/remediate** the attack?
- What sort of **impact** are we dealing with?

Parallel to the contextualisation efforts: False positive handling

- Low quality / false positive prone information being shared
- Lead to **alert-fatigue**
- Exclude organisation xy out of the community?
- False positives are often obvious - **can be encoded**
- **Warninglist system**[1] aims to do that
- Lists of well-known indicators which are often false-positives like RFC1918 networks, ...



LIST OF KNOWN IPV4 PUBLIC DNS RESOLVERS

Id	89
Name	List of known IPv4 public DNS resolvers
Description	Event contains one or more public IPv4 DNS resolvers as attribute with an IDS flag set
Version	20181114
Type	string
Accepted attribute types	ip-src, ip-dst, domain|ip
Enabled	Yes (disable)

Values

1.0.0.1
1.1.1.1
1.11.71.4

Warning: Potential false positives

List of known IPv4 public DNS resolvers
Top 1000 website from Alexa
List of known google domains

[1] https://github.com/MISP/misp-warninglists

More complex data-structures for a modern age

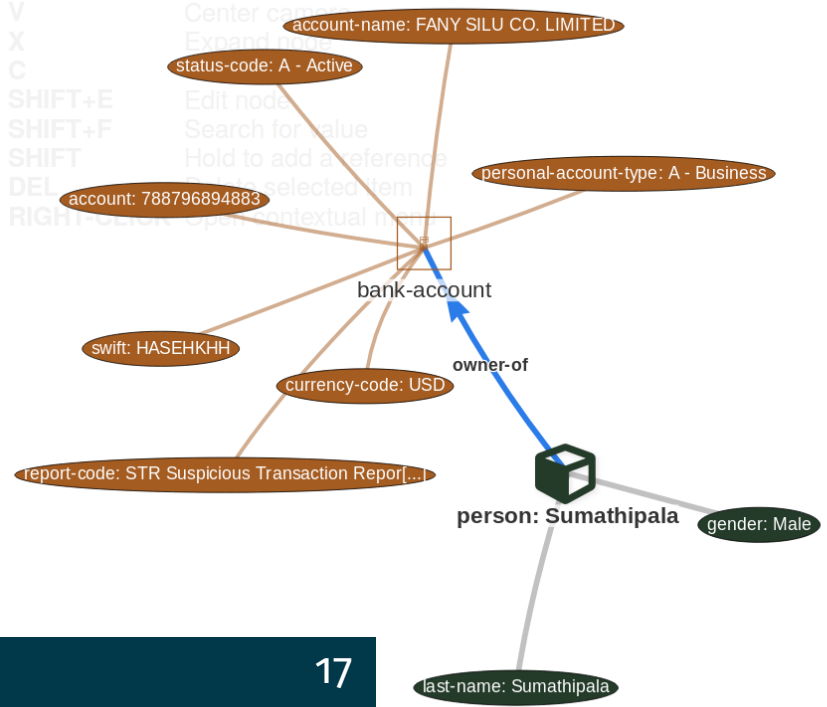
- Atomic attributes were a great starting point, but lacking in many aspects
- **MISP objects**[2] system
  - Simple **templating** approach
  - Use templating to build more complex structures
  - Decouple it from the core, allow users to **define their own** structures
  - MISP should understand the data without knowing the templates
  - Massive caveat: **Building blocks have to be MISP attribute types**
  - Allow **relationships** to be built between objects

[2] https://github.com/MISP/misp-objects

Supporting specific datamodel

Filters: All | File | Network | Financial | Proposal | Correlation | Warnings | Include deleted attributes | Show context fields

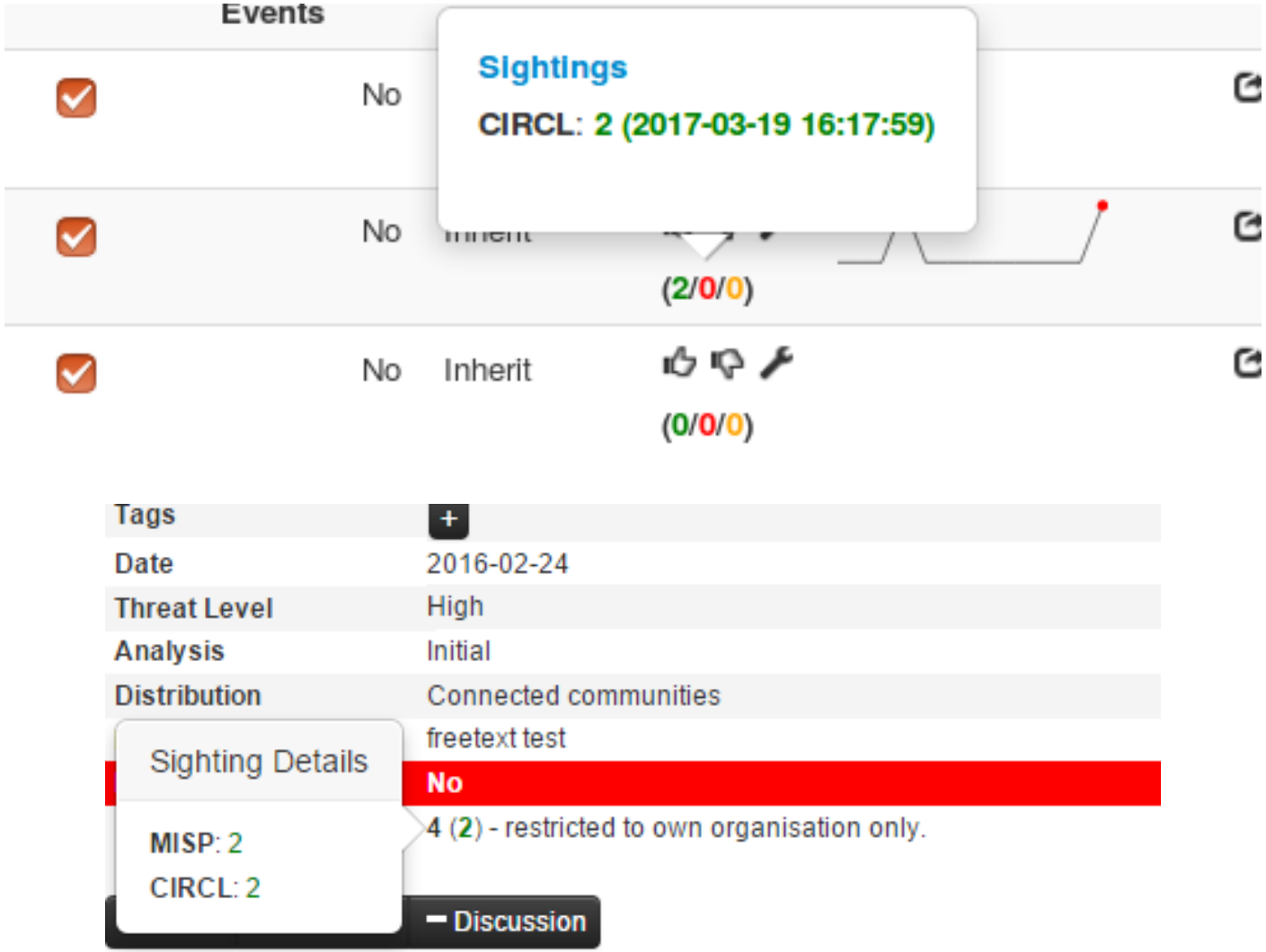
Date	Org	Category	Type	Value	Tags	Galaxies	Comment	Correlate	Related Events
2018-09-28		**Name:** bank-account **References:** 0							
2018-09-28		Other	**status-code:** text	A - Active	+	Add		☐	
2018-09-28		Other	**report-code:** text	STR Suspicious Transaction Report	+	Add		☐	
2018-09-28		Other	**personal-account-type:** text	A - Business	+	Add		☐	
2018-09-28		Financial fraud	**swift:** bic	HASEHKHH	+	Add		☑	3849 11320 11584
2018-09-28		Financial fraud	**account:** bank-account-nr	788796894883	+	Add		☑	
2018-09-28		Other	**account-name:** text	FANY SILU CO. LIMITED	+	Add		☑	
2018-09-28		Other	**currency-code:** text	USD	+	Add		☐	



Continuous feedback loop

- Data ingested by MISP was in a sense frozen in time
- We had a creation data, but lacked a way to use the output of our detection
- Lead to the introduction of the **Sighting system**
- The community could sight indicators and convey the time of sighting
- Potentially powerful tool for IoC lifecycle management, clumsy query implementation default

Supporting specific datamodel



Making use of all this context

- Most obvious goal: Improve the way we query data
  - Unified all export APIs
  - Incorporate all contextualisation options into **API filters**
  - Allow for an **on-demand** way of **excluding potential false positives**
  - Allow users to easily **build their own** export modules feed their various tools

Example query

/attributes/restSearch

```
{
    "returnFormat": "netfilter",
    "enforceWarninglist": 1,
    "tags": {
      "NOT": [
        "tlp:white",
        "type:OSINT"
      ],
      "OR": [
        "misp-galaxy:threat-actor=\"Sofacy\"",
        "misp-galaxy:sector=\"Chemical\""
      ],
    }
}
```

Synchronisation filters

- Make decisions on whom to share data with based on context
  - MISP by default decides based on the information creator's decision who data gets shared with
  - Community hosts should be able to **act as a safety net** for sharing
    - **Push filters** - what can I push?
    - **Pull filters** - what am I interested in?
    - **Local tags** allow for information flow control

The emergence of ATT&CK and similar galaxies

- Standardising on high-level **TTPs** was a solution to a long list of issues
- Adoption was rapid, tools producing ATT&CK data, familiar interface for users
- A much better take on kill-chain phases in general
- Feeds into our **filtering** and **situational awareness** needs extremely well
- Gave rise to other, ATT&CK-like systems tackling other concerns
  - **attck4fraud** [3] by Francesco Bigarella from ING
  - **Election guidelines** [4] by NIS Cooperation Group

[3] https://www.misp-project.org/galaxy.html#_attck4fraud

[4] https://www.misp-project.org/galaxy.html#_election_guidelines

Example query to generate ATT&CK heatmaps

```
/events/restSearch
{
    "returnFormat": "attack",
    "tags": [
        "misp-galaxy:sector=\"Chemical\""
    ],
    "timestamp": "365d"
}
```

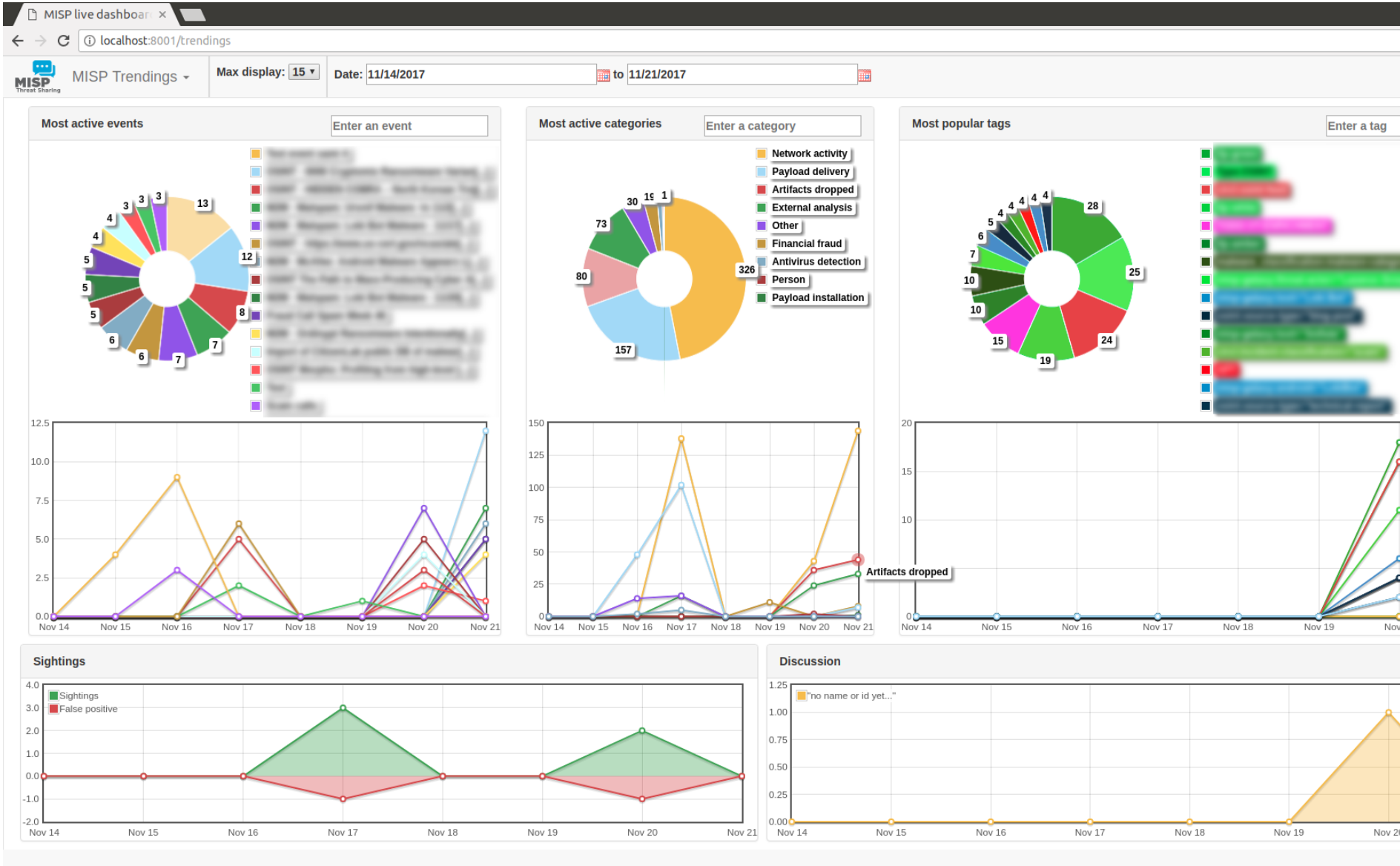
A sample result for the above query

Pre Attack - Attack Pattern | **Enterprise Attack - Attack Pattern** | Mobile Attack - Attack Pattern

0 — 11 — Show all

Initial access	Execution	Persistence	Privilege escalation	Defense evasion	Credential access	Discovery	Lateral movement	Collection	Exfiltration	Command and control
Spearphishing Attachment	Scripting	Screensaver	File System Permissions Weakness	Process Hollowing	Securityd Memory	Password Policy Discovery	AppleScript	Data from Information Repositories	Exfiltration Over Alternative Protocol	Standard Application Layer Protocol
Spearphishing via Service	Command-Line Interface	Login Item	AppCert DLLs	Code Signing	Input Capture	System Network Configuration Discovery	Distributed Component Object Model	Data from Removable Media	Exfiltration Over Command and Control Channel	Communication Through Removable Media
Trusted Relationship	User Execution	Trap	Application Shimming	Rootkit	Bash History	Process Discovery	Pass the Hash	Man in the Browser	Data Compressed	Custom Command and Control Protocol
Replication Through Removable Media	Regsvcs/Regasm	System Firmware	Scheduled Task	NTFS File Attributes	Exploitation for Credential Access	Network Share Discovery	Exploitation of Remote Services	Data Staged	Automated Exfiltration	Multi-Stage Channels
Exploit Public-Facing Application	Trusted Developer Utilities	Registry Run Keys / Start Folder	Startup Items	Exploitation for Defense Evasion	Private Keys	Peripheral Device Discovery	Remote Desktop Protocol	Screen Capture	Scheduled Transfer	Remote Access Tools
Spearphishing Link	Windows Management Instrumentation	LC_LOAD_DYLIB Addition	New Service	Network Share Connection Removal	Brute Force	Account Discovery	Pass the Ticket	Email Collection	Data Encrypted	Uncommonly Used Port
Valid Accounts	Service Execution	LSASS Driver	Sudo Caching	Process Doppelgänging	Password Filter DLL	System Information Discovery	Windows Remote Management	Clipboard Data	Exfiltration Over Other Network Medium	Multilayer Encryption
Supply Chain Compromise	CMSTP	Rc.common	Process Injection	Disabling Security Tools	Two-Factor Authentication Interception	System Network Connections Discovery	Windows Admin Shares	Video Capture	Exfiltration Over Physical Medium	Domain Fronting
Drive-by Compromise	Control Panel Items	Authentication Package	Bypass User Account Control	Timestomp	LLMNR/NBT-NS Poisoning	Network Service Scanning	Remote Services	Audio Capture	Data Transfer Size Limits	Data Obfuscation
Hardware Additions	Dynamic Data Exchange	Component Firmware	Extra Window Memory Injection	Modify Registry	Credentials in Files	File and Directory Discovery	Taint Shared Content	Data from Network Shared Drive		Connection Proxy
	Source	Windows Management Instrumentation Event Subscription	Setuid and Setgid	Indicator Removal from Tools	Forced Authentication	Security Software Discovery	Application Deployment Software	Data from Local System		Commonly Used Port
	Space after Filename	Change Default File	Launch Daemon	Hidden Window	Keychain	System Service Discovery	Third-party Software	Automated Collection		Data Encoding

Monitor trends outside of MISP (example: dashboard)



Decaying of indicators

- We were still missing a way to use all of these systems in combination to decay indicators
- Move the decision making **from complex filter options to** complex **decay models**
- Decay models would take into account various **taxonomies**, **sightings**, the **type** of each indicator **Sightings** and **Creation date**
- The first iteration of what we have in MISP now took:
  - 2 years of research
  - 3 published research papers
  - A lot of prototyping

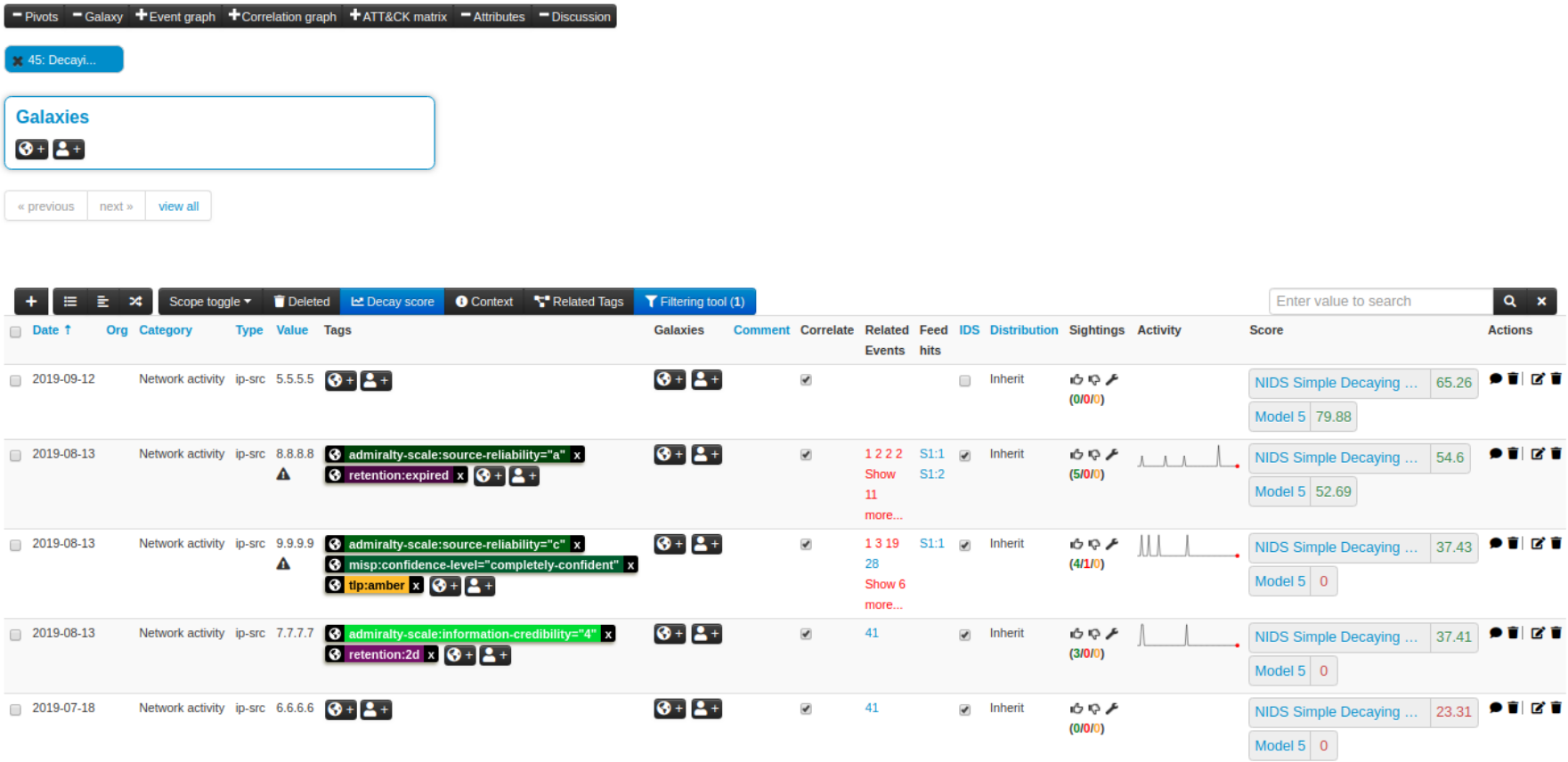
Scoring Indicators: Our solution

$$\texttt{score}(\texttt{Attribute}) = \texttt{base_score}(\texttt{Attribute}, \texttt{Model}) \bullet \texttt{decay}(\texttt{Model}, \texttt{time})$$

Where,

- $\texttt{score} \in [0, 100]$
- $\texttt{base_score} \in [0, 100]$
- `decay` is a function defined by model's parameters controlling decay speed
- `Attribute` Contains *Attribute*'s values and metadata (*Taxonomies*, *Galaxies*, ...)
- `Model` Contains the *Model*'s configuration

Implementation in MISP: `Event/view`



- `Decay score` toggle button
  - Shows Score for each *Models* associated to the *Attribute* type

Implementation in MISP: API result

/attributes/restSearch


```
"Attribute": [
  {
    "category": "Network activity",
    "type": "ip-src",
    "to_ids": true,
    "timestamp": "1565703507",
    [...]
    "value": "8.8.8.8",
    "decay_score": [
      {
        "score": 54.4752238495444456,
        "decayed": false,
        "DecayingModel": {
          "id": "85",
          "name": "NIDS Simple Decaying Model"
        }
```

Implementation in MISP: Index

Decaying Models

« previous | next »

All Models | My Models | Shared Models | Default Models

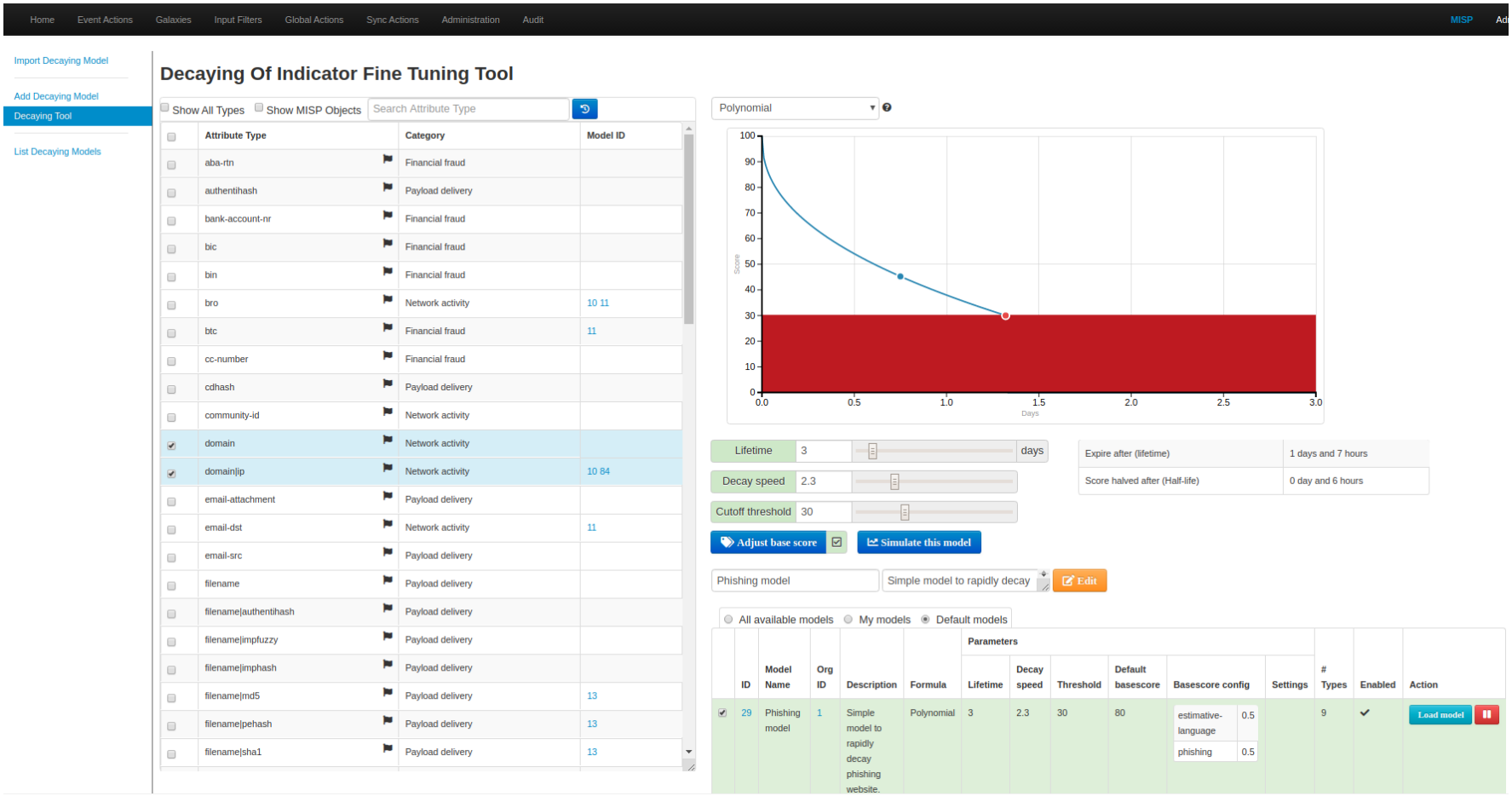
ID	Organization	Usable to everyone	Name	Description	Parameters { }	Formula	# Assigned Types	Version	Enabled	Actions
29	1	✔	Phishing model	Simple model to rapidly decay phishing website.	{ "lifetime": 3, "decay_speed": 2.3, "threshold": 30, "default_base_score": 80, "base_score_config": { "estimative-language": 0.5, "phishing": 0.5 } }	Polynomial	9	1	✔	
85	1	✘	NIDS Simple Decaying Model MISP	Simple decaying model for Network Intrusion Detection System (NIDS).	{ "lifetime": 120, "decay_speed": 2, "threshold": 30, "default_base_score": 80, "base_score_config": { "estimative-language": 0.25, "priority-level": 0.25, "retention": 0.25, "targeted-threat-index": 0.125, "false-positive": 0.125 } }	Polynomial	13	1	✔	

Page 1 of 1, showing 2 records out of 2 total, starting on record 1, ending on 2

« previous | next »

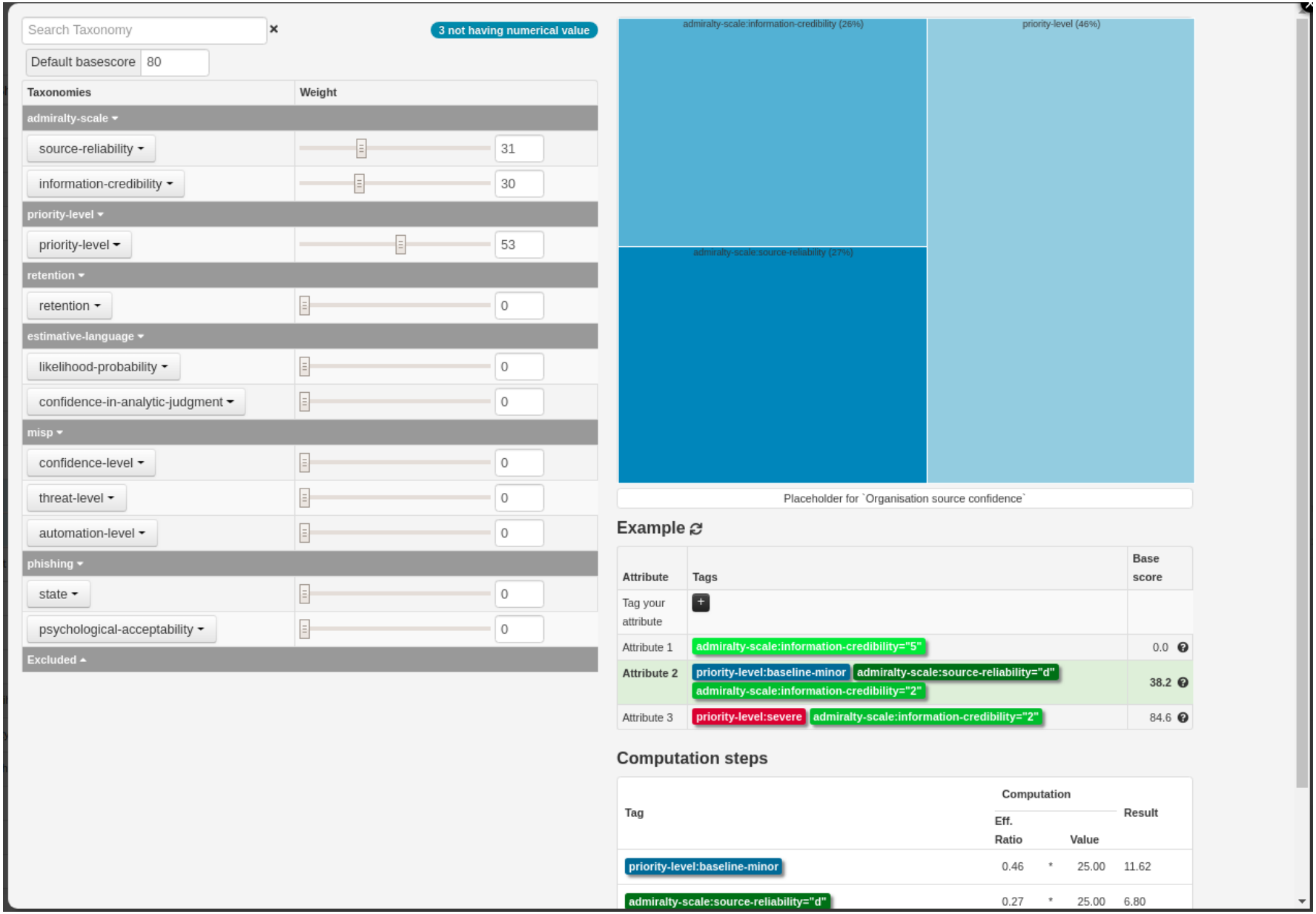
View, update, add, create, delete, enable, export, import

Implementation in MISP: Fine tuning tool



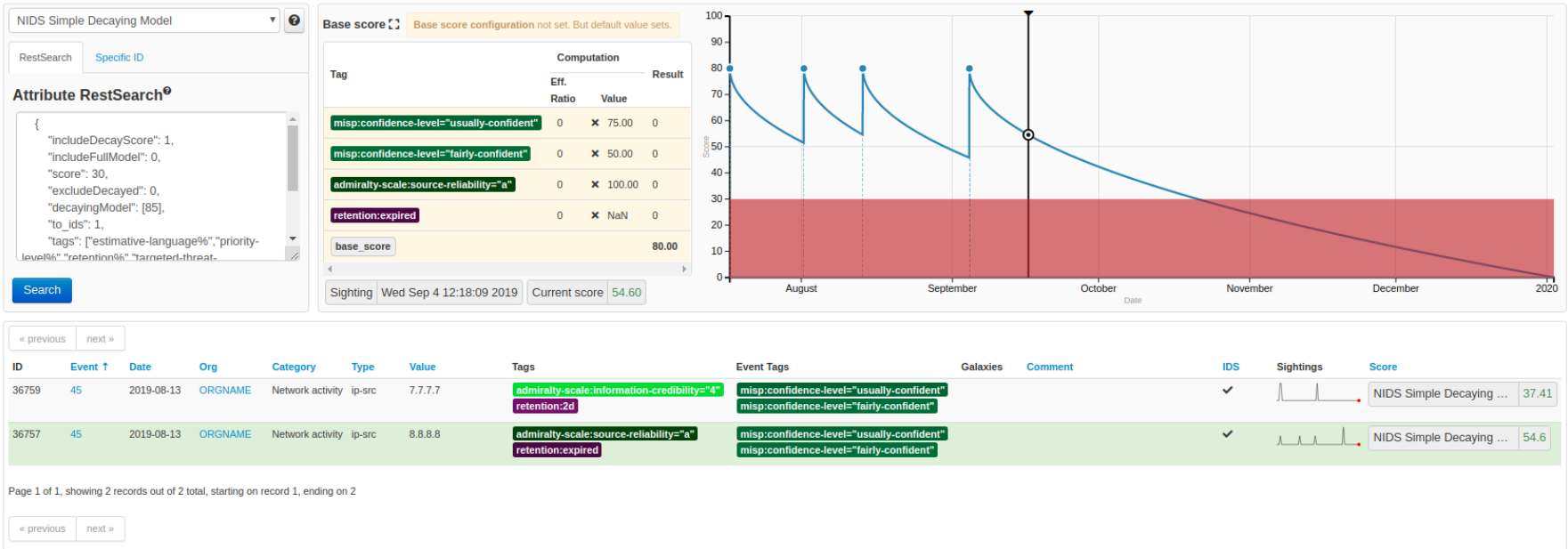
Create, modify, visualise, perform mapping

Implementation in MISP: base_score tool



Adjust Taxonomies relative weights

Implementation in MISP: simulation tool



Simulate *Attributes* with different *Models*

Implementation in MISP: API query body

```
/attributes/restSearch
{
    "includeDecayScore": 1,
    "includeFullModel": 0,
    "excludeDecayed": 0,
    "decayingModel": [85],
    "modelOverrides": {
        "threshold": 30
    }
    "score": 30,
}
```

To sum it all up...

- Massive rise in **user capabilities**
- Growing need for truly **actionable threat intel**
- Lessons learned:
  - **Context is king** - Enables better decision making
  - **Intelligence and situational awareness** are natural by-products of context
  - Don’t lock users into your **workflows**, build tools that enable theirs

Get in touch if you have any questions

- Contact us
  - https://twitter.com/mokaddem_sami
  - https://twitter.com/iglocska
- Contact CIRCL
  - info@circl.lu
  - https://twitter.com/circl_lu
  - https://www.circl.lu/
- Contact MISPProject
  - https://github.com/MISP
  - https://gitter.im/MISP/MISP
  - https://twitter.com/MISPProject

MISP

INTELLIGENCE SHARING

Turning data into actionable intelligence

advanced features in MISP supporting your analysts and tools

CIRCL / Team MISP Project

MISP Project

CRI MISP 2024

The aim of this presentation

- Why is **contextualisation** important?
- What options do we have in MISP?
- How can we **leverage** this in the end?

The growing need to contextualise data

- Contextualisation became more and more important as we as a community matured
  - **Growth and diversification** of our communities
  - Distinguish between information of interest and raw data
  - **False-positive** management
  - TTPs and aggregate information may be prevalent compared to raw data (risk assessment)
  - **Increased data volumes** leads to a need to be able to prioritise
- These help with filtering your TI based on your **requirements**...
- ...as highlighted by Pasquale Stirparo *Your Requirements Are Not My Requirements*

Objectives

- Some main objectives we want to achieve when producing data
  - Ensure that the information is **consumable** by everybody
  - That it is **useful** to the entire target audience
  - The data is **contextualised** for it to be understood by everyone
- What we ideally want from our data
  - We want to be able to **filter** data for different use-cases
  - We want to be able to get as much knowledge out of the data as possible
  - We want to know where the data is from, how it got there, why we should care

Different layers of context

- Context added by analysts / tools
- Data that tells a story
- Encoding analyst knowledge to automatically leverage the above

Context added by analysts / tools

Expressing why data-points matter

- An IP address by itself is barely ever interesting
- We need to tell the recipient / machine why this is relevant
- All data in MISP has a bare minimum required context
- We differentiate between indicators and supporting data

Broadening the scope of what sort of context we are interested in

- **Who** can receive our data? **What** can they do with it?
- **Data accuracy, source reliability**
- **Why** is this data relevant to us?
- **Who** do we think is behind it, **what tools** were used?
- What sort of **motivations** are we dealing with? Who are the **targets**?
- How can we **block/detect/remediate** the attack?
- What sort of **impact** are we dealing with?

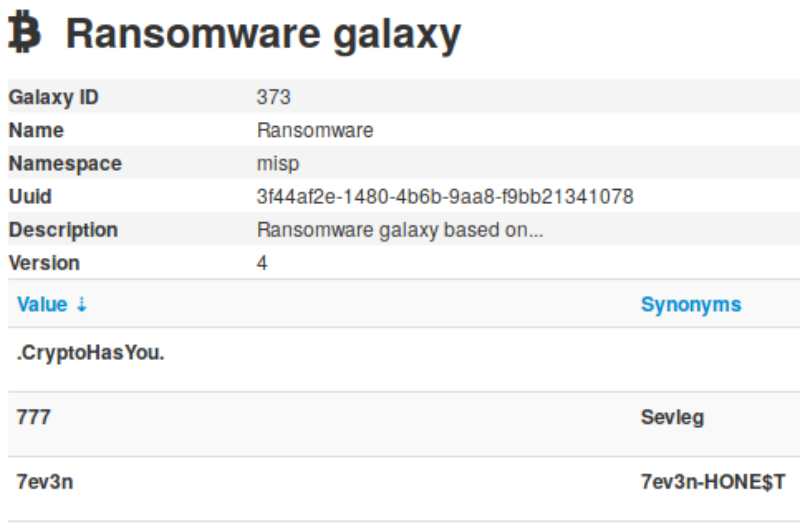
Tagging and taxonomies

- Simple labels
- Standardising on vocabularies
- Different organisational/community cultures require different nomenclatures
- Triple tag system - taxonomies
- JSON libraries that can easily be defined without our intervention

Tag	Events	Attributes	Tags
workflow:state="complete"	11	0	workflow:state="complete"
workflow:state="draft"	0	0	workflow:state="draft"
workflow:state="incomplete"	55	10	workflow:state="incomplete"
workflow:state="ongoing"	0	0	workflow:state="ongoing"

Galaxies

- Taxonomy tags often **non self-explanatory**
  - Example: universal understanding of tlp:green vs APT 28
- For the latter, a single string was ill-suited
- So we needed something new in addition to taxonomies - **Galaxies**
  - Community driven **knowledge-base libraries used as tags**
  - Including descriptions, links, synonyms, meta information, etc.
  - Goal was to keep it **simple and make it reusable**
  - Internally it works the exact same way as taxonomies (stick to **JSON**)



Ransomware galaxy

Galaxy ID	373
Name	Ransomware
Namespace	misp
Uuid	3f44af2e-1480-4b6b-9aa8-f9bb21341078
Description	Ransomware galaxy based on...
Version	4

Value ↓	Synonyms
.CryptoHasYou.	
777	Sevleg
7ev3n	7ev3n-HONE$T

The emergence of ATT&CK and similar galaxies

- Standardising on high-level **TTPs** was a solution to a long list of issues
- Adoption was rapid, tools producing ATT&CK data, familiar interface for users
- A much better take on kill-chain phases in general
- Feeds into our **filtering** and **situational awareness** needs extremely well
- Gave rise to other, ATT&CK-like systems tackling other concerns
  - **attck4fraud** [1] by Francesco Bigarella from ING
  - **Election guidelines** [2] by NIS Cooperation Group

[1] https://www.misp-project.org/galaxy.html#_attck4fraud
[2] https://www.misp-project.org/galaxy.html#_election_guidelines

Data that tells a story

More complex data-structures for a modern age

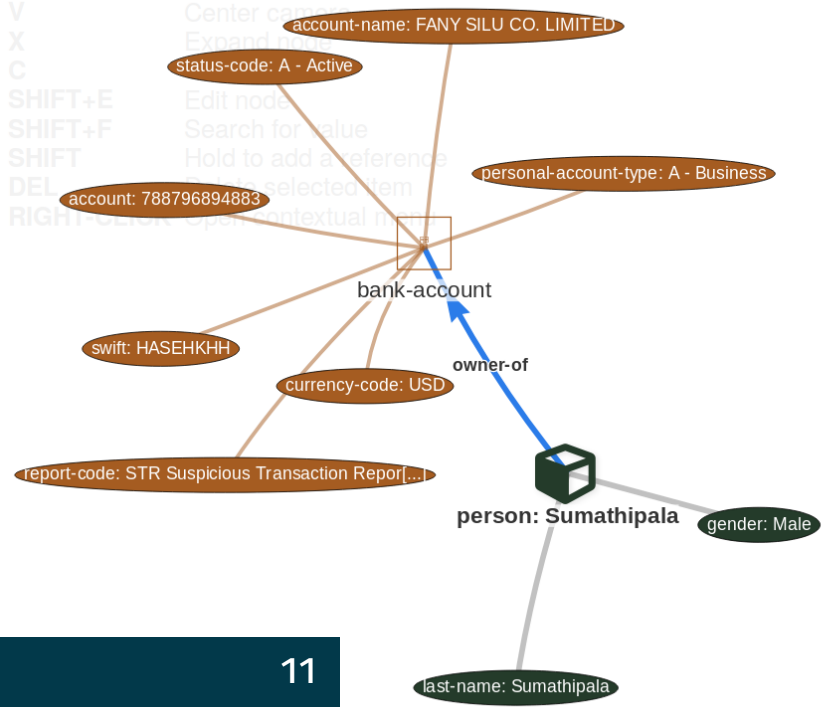
- Atomic attributes were a great starting point, but lacking in many aspects
- **MISP objects**[3] system
  - Simple **templating** approach
  - Use templating to build more complex structures
  - Decouple it from the core, allow users to **define their own** structures
  - MISP should understand the data without knowing the templates
  - Massive caveat: **Building blocks have to be MISP attribute types**
  - Allow **relationships** to be built between objects

[3] https://github.com/MISP/misp-objects

Supporting specific datamodels

Filters: All | File | Network | Financial | Proposal | Correlation | Warnings | Include deleted attributes | Show context fields

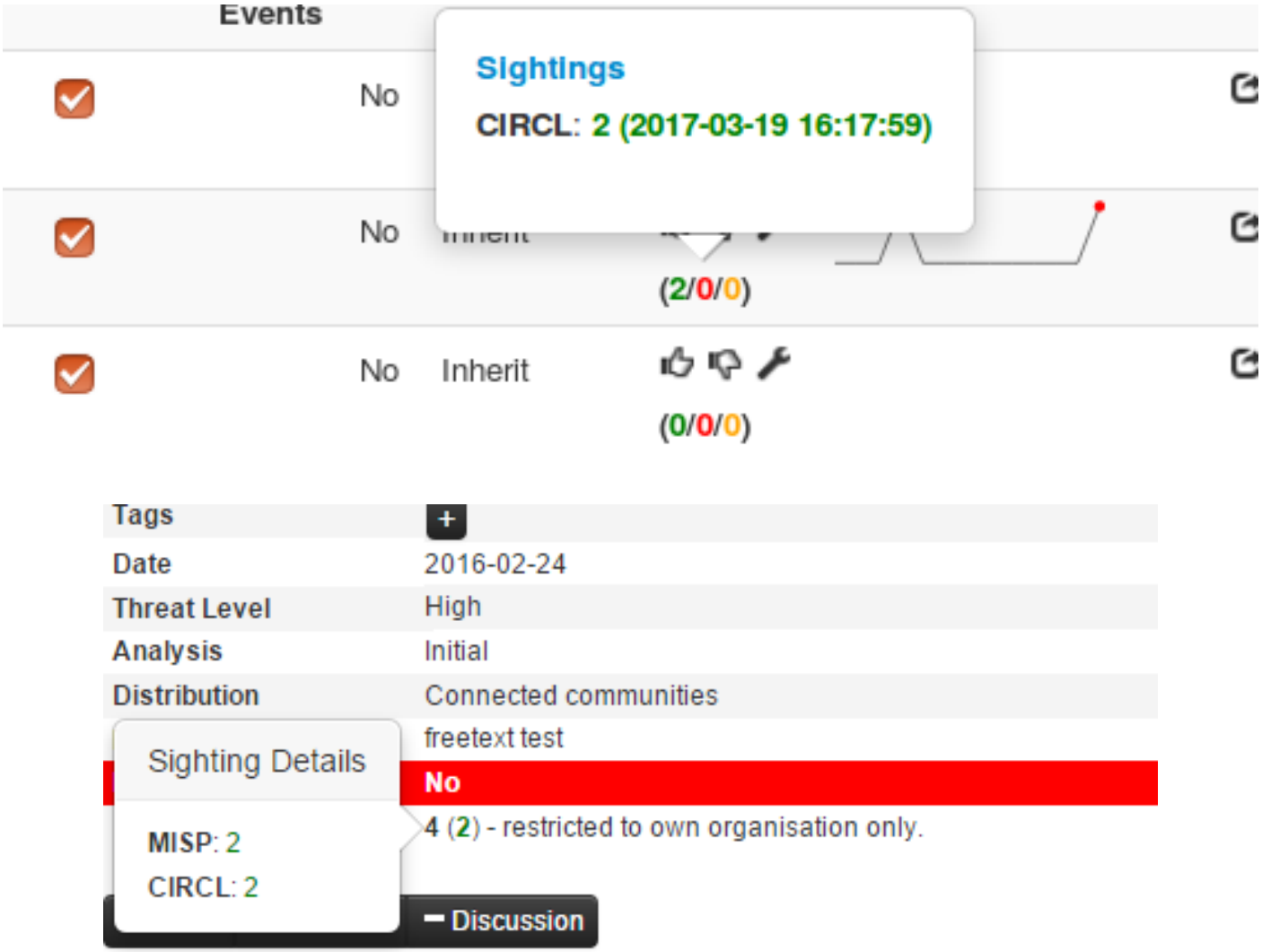
Date	Org	Category	Type	Value	Tags	Galaxies	Comment	Correlate	Related Events
2018-09-28		**Name:** bank-account **References:** 0							
2018-09-28		Other	**status-code:** text	A - Active	+	Add		☐	
2018-09-28		Other	**report-code:** text	STR Suspicious Transaction Report	+	Add		☐	
2018-09-28		Other	**personal-account-type:** text	A - Business	+	Add		☐	
2018-09-28		Financial fraud	**swift:** bic	HASEHKHH	+	Add		☑	3849 11320 11584
2018-09-28		Financial fraud	**account:** bank-account-nr	788796894883	+	Add		☑	
2018-09-28		Other	**account-name:** text	FANY SILU CO. LIMITED	+	Add		☑	
2018-09-28		Other	**currency-code:** text	USD	+	Add		☐	



Continuous feedback loop

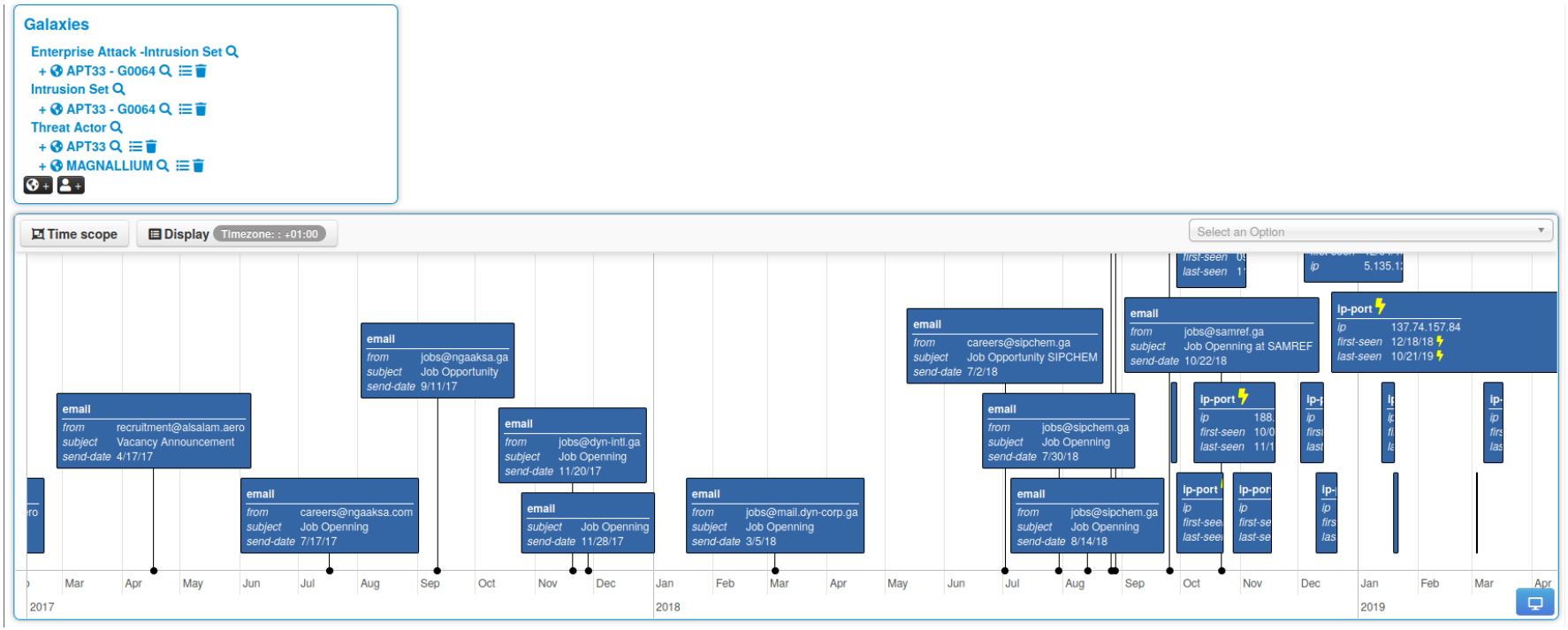
- Data shared was **frozen in time**
- All we had was a creation/modification timestamp
- Improved tooling and willingness allowed us to create a **feedback loop**
- Lead to the introduction of the **Sighting system**
- Signal the fact of an indicator sighting...
- ...as well as **when** and **where** it was sighted
- Vital component for IoC **lifecycle management**

Continuous feedback loop (2)



A brief history of time - Adding temporality to our data

- As Andreas said - no time based aspect was painful
- Recently introduced **`first_seen`** and **`last_seen`** data points
- Along with a complete integration with the **UI**
- Enables the **visualisation** and **adjustment** of indicators timeframes



The various ways of encoding analyst knowledge to automatically leverage our TI

False positive handling

- Low quality / false positive prone information being shared
- Lead to **alert-fatigue**
- Exclude organisation xy out of the community?
- FPs are often obvious - **can be encoded**
- **Warninglist system**[4] aims to do that
- Lists of well-known indicators which are often false-positives like RFC1918 networks, ...

LIST OF KNOWN IPV4 PUBLIC DNS RESOLVERS

Id	89
Name	List of known IPv4 public DNS resolvers
Description	Event contains one or more public IPv4 DNS resolvers as attribute with an IDS flag set
Version	20181114
Type	string
Accepted attribute types	ip-src, ip-dst, domain|ip
Enabled	**Yes** (disable)

Values

1.0.0.1
1.1.1.1
1.11.71.4

Warning: Potential false positives

List of known IPv4 public DNS resolvers
Top 1000 website from Alexa
List of known google domains

[4] https://github.com/MISP/misp-warninglists

Making use of all this context

- Providing advanced ways of querying data
  - Unified export APIs
  - Incorporating all contextualisation options into **API filters**
  - Allowing for an **on-demand** way of **excluding potential false positives**
  - Allowing users to easily **build their own** export modules feed their various tools

Example query

/attributes/restSearch

```
{
    "returnFormat": "netfilter",
    "enforceWarninglist": 1,
    "tags": {
      "NOT": [
        "tlp:white",
        "type:OSINT"
      ],
      "OR": [
        "misp-galaxy:threat-actor=\"Sofacy\"",
        "misp-galaxy:sector=\"Chemical\""
      ],
    }
}
```

Example query to generate ATT&CK heatmaps

```
/events/restSearch
{
    "returnFormat": "attack",
    "tags": [
        "misp-galaxy:sector=\"Chemical\""
    ],
    "timestamp": "365d"
}
```

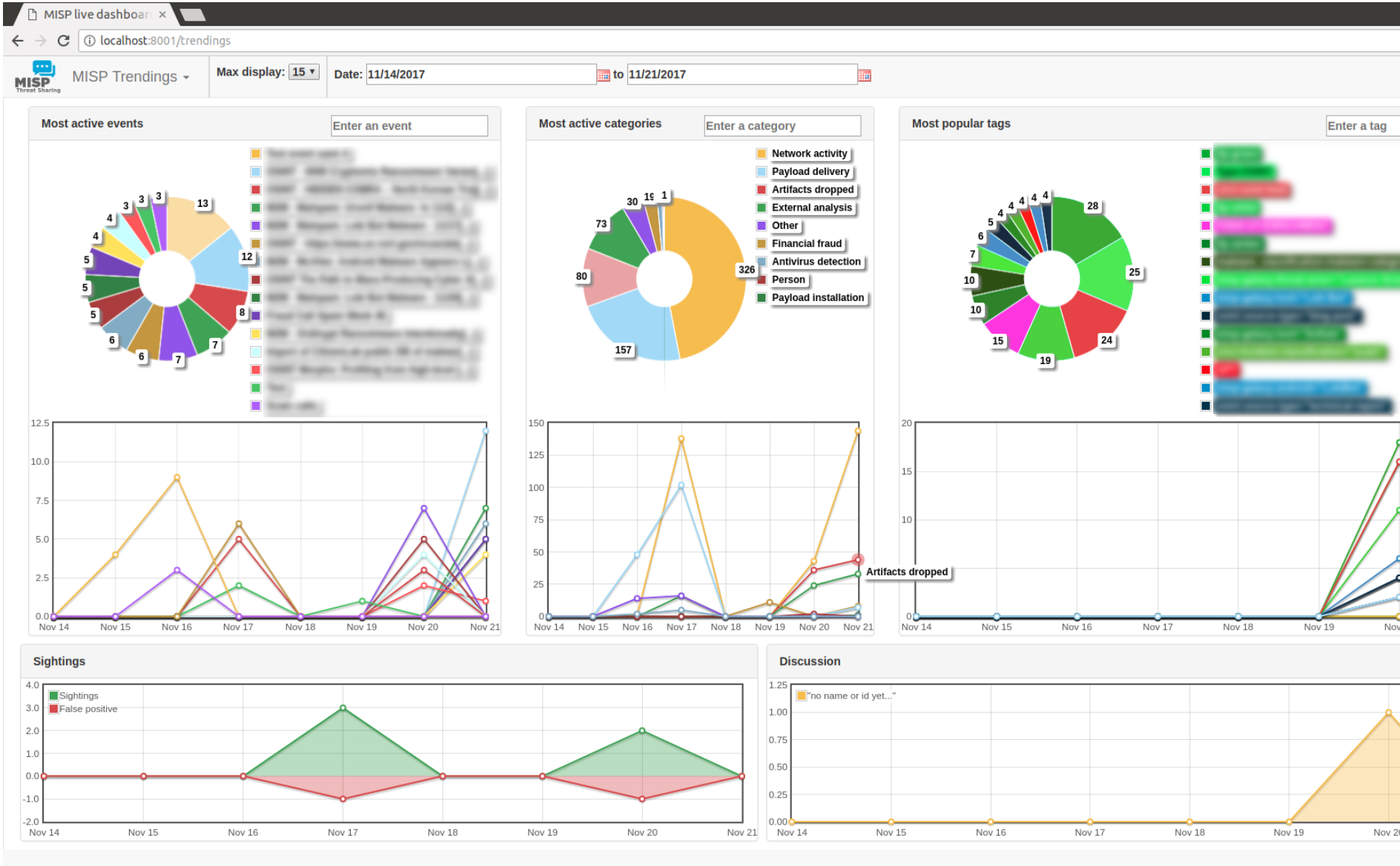
A sample result for the above query

Pre Attack - Attack Pattern | **Enterprise Attack - Attack Pattern** | Mobile Attack - Attack Pattern

0 — 11 — Show all

Initial access	Execution	Persistence	Privilege escalation	Defense evasion	Credential access	Discovery	Lateral movement	Collection	Exfiltration	Command and control
Spearphishing Attachment	Scripting	Screensaver	File System Permissions Weakness	Process Hollowing	Securityd Memory	Password Policy Discovery	AppleScript	Data from Information Repositories	Exfiltration Over Alternative Protocol	Standard Application Layer Protocol
Spearphishing via Service	Command-Line Interface	Login Item	AppCert DLLs	Code Signing	Input Capture	System Network Configuration Discovery	Distributed Component Object Model	Data from Removable Media	Exfiltration Over Command and Control Channel	Communication Through Removable Media
Trusted Relationship	User Execution	Trap	Application Shimming	Rootkit	Bash History	Process Discovery	Pass the Hash	Man in the Browser	Data Compressed	Custom Command and Control Protocol
Replication Through Removable Media	Regsvcs/Regasm	System Firmware	Scheduled Task	NTFS File Attributes	Exploitation for Credential Access	Network Share Discovery	Exploitation of Remote Services	Data Staged	Automated Exfiltration	Multi-Stage Channels
Exploit Public-Facing Application	Trusted Developer Utilities	Registry Run Keys / Start Folder	Startup Items	Exploitation for Defense Evasion	Private Keys	Peripheral Device Discovery	Remote Desktop Protocol	Screen Capture	Scheduled Transfer	Remote Access Tools
Spearphishing Link	Windows Management Instrumentation	LC_LOAD_DYLIB Addition	New Service	Network Share Connection Removal	Brute Force	Account Discovery	Pass the Ticket	Email Collection	Data Encrypted	Uncommonly Used Port
Valid Accounts	Service Execution	LSASS Driver	Sudo Caching	Process Doppelgänging	Password Filter DLL	System Information Discovery	Windows Remote Management	Clipboard Data	Exfiltration Over Other Network Medium	Multilayer Encryption
Supply Chain Compromise	CMSTP	Rc.common	Process Injection	Disabling Security Tools	Two-Factor Authentication Interception	System Network Connections Discovery	Windows Admin Shares	Video Capture	Exfiltration Over Physical Medium	Domain Fronting
Drive-by Compromise	Control Panel Items	Authentication Package	Bypass User Account Control	Timestomp	LLMNR/NBT-NS Poisoning	Network Service Scanning	Remote Services	Audio Capture	Data Transfer Size Limits	Data Obfuscation
Hardware Additions	Dynamic Data Exchange	Component Firmware	Extra Window Memory Injection	Modify Registry	Credentials in Files	File and Directory Discovery	Taint Shared Content	Data from Network Shared Drive		Connection Proxy
	Source	Windows Management Instrumentation Event Subscription	Setuid and Setgid	Indicator Removal from Tools	Forced Authentication	Security Software Discovery	Application Deployment Software	Data from Local System		Commonly Used Port
	Space after Filename	Change Default File	Launch Daemon	Hidden Window	Keychain	System Service Discovery	Third-party Software	Automated Collection		Data Encoding

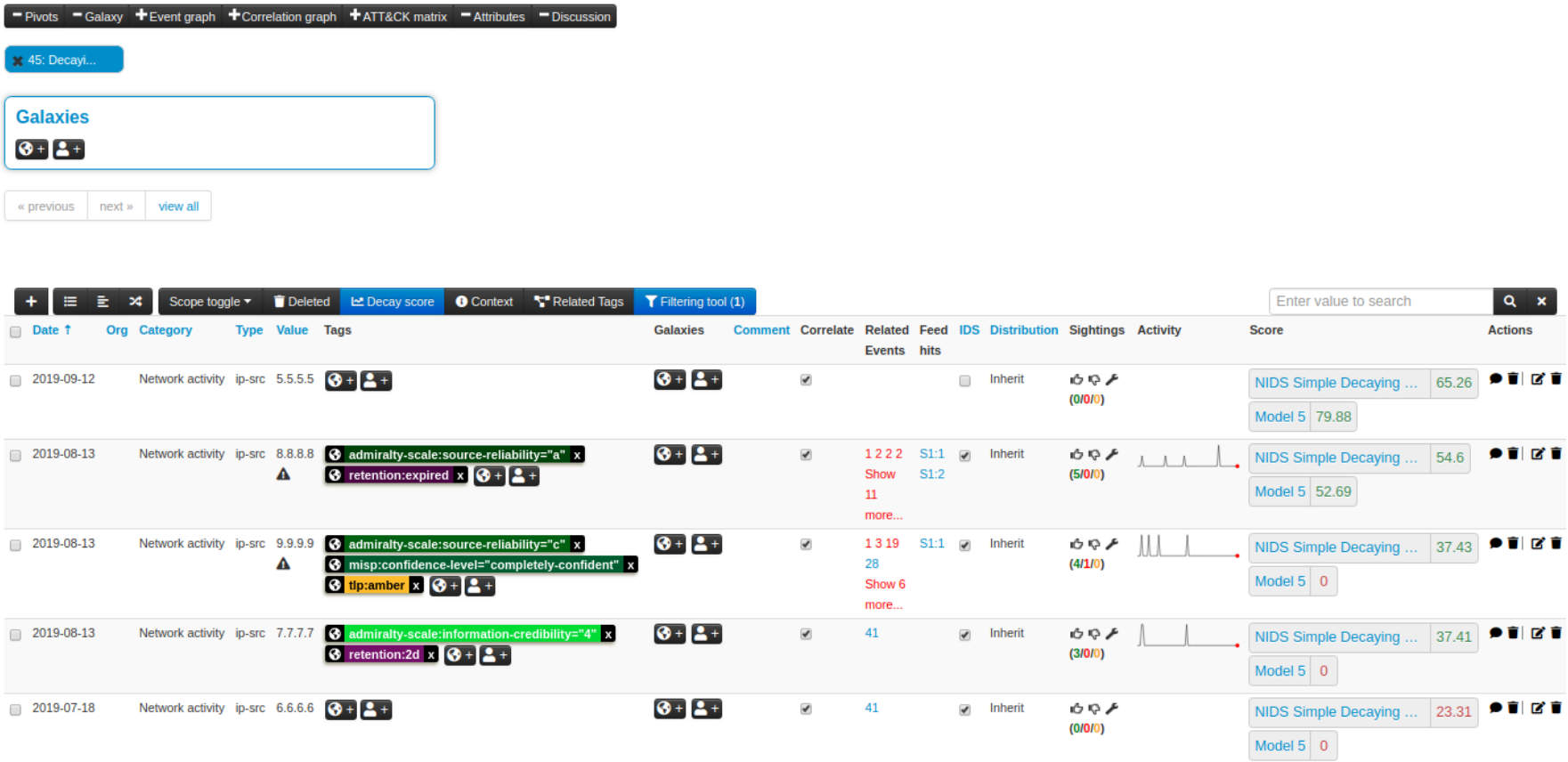
Monitor trends outside of MISP (example: dashboard)



Decaying of indicators

- We were still missing a way to use all of these systems in combination to decay indicators
- Move the decision making **from complex filter options to** complex **decay models**
- Decay models would take into account various available **context**
  - Taxonomies
  - Sightings
  - type of each indicator
  - Creation date
  - ...

Implementation in MISP: Event/view



- Decay score toggle button
  - Shows Score for each *Models* associated to the *Attribute* type

Implementation in MISP: API result

/attributes/restSearch


```
"Attribute": [
  {
    "category": "Network activity",
    "type": "ip-src",
    "to_ids": true,
    "timestamp": "1565703507",
    [...]
    "value": "8.8.8.8",
    "decay_score": [
      {
        "score": 54.4752238495444456,
        "decayed": false,
        "DecayingModel": {
          "id": "85",
          "name": "NIDS Simple Decaying Model"
        }
```

To sum it all up...

- Massive rise in **user capabilities**
- Growing need for truly **actionable threat intel**
- Lessons learned:
  - **Context is king** - Enables better decision making
  - **Intelligence and situational awareness** are natural by-products of context
  - Don’t lock users into your **workflows**, build tools that enable theirs

Get in touch if you have any questions

- Contact us
  - https://twitter.com/mokaddem_sami
  - https://twitter.com/iglocska
- Contact CIRCL
  - info@circl.lu
  - https://twitter.com/circl_lu
  - https://www.circl.lu/
- Contact MISPProject
  - https://github.com/MISP
  - https://gitter.im/MISP/MISP
  - https://twitter.com/MISPProject

MISP

INTELLIGENCE SHARING

MISP Standard

The collaborative intelligence standard powering intelligence and information exchange, sharing and modeling.

CIRCL / Team MISP Project

http://www.misp-standard.org/

Twitter: *@MISPProject*

CRI MISP 2024

MISP Standard

- Following the grow of organisations relying on MISP, the **JSON format used by MISP are standardised under the misp-standard.org umbrella**
- The goal is to provide a flexible set of standards to support information exchange and data modeling in the following field:
  - Cybersecurity intelligence
  - Threat intelligence
  - Financial fraud
  - Vulnerability information
  - Border control information
  - Digital Forensic and Incident Response
  - and intelligence at large

Standard - MISP core format

This standard describes the **MISP core format** used to exchange indicators and threat information between MISP instances. The **JSON format includes the overall structure along with the semantics associated for each respective key**. The format is described to support other implementations, aiming to reuse the format and ensuring the interoperability with the existing MISP software and other Threat Intelligence Platforms.

MISP object template format

This standard describes the **MISP object** template format which describes a simple JSON format to represent the various templates used to construct MISP objects. A **public directory of common MISP object templates and relationships** is available and relies on the MISP object reference format.

MISP galaxy format

This standard describes the **MISP galaxy format which describes a simple JSON format to represent galaxies and clusters** that can be attached to MISP events or attributes. A public directory of MISP galaxies is available and relies on the MISP galaxy format. MISP galaxies are used to attach additional information structures such as MISP events or attributes. **MISP galaxy is a public repository of known malware, threats actors and various other collections of data that can be used to mark, classify or label data in threat information sharing**.

SightingDB format

This standard describes the format used by SightingDB to give automated context to a given Attribute by **counting occurrences and tracking times of observability**. SightingDB was designed to provide to MISP and other tools an interoperable, scalable and fast way to store and retrieve attributes sightings.

Internet-Draft - IETF for MISP formats and MISP standard

- If you want to contribute to our IETF Internet-Draft for the MISP standard, misp-rfc[1] is the repository where to contribute.
- **Update only the markdown file**, the XML and ASCII for the IETF I-D are automatically generated.
- If a major release or updates happen in the format, we will publish the I-D to the IETF[2].
- The process is always MISP implementation $\rightarrow$ IETF I-D updates.
- Then published standards in misp-standard.org.

[1] https://github.com/MISP/misp-rfc

[2] https://datatracker.ietf.org/doc/search/?name=misp&activedrafts=on&rfcs=on

MISP

INTELLIGENCE SHARING

MISP CLI

Automate all the things

CIRCL / Team MISP Project

MISP Project

CRI MISP 2024

MISP CLI functionalities

- The MISP API is great for remotely executing administrative tasks
- But sometimes we want to simplify the process / avoid having to deal with authentication
- MISP also has an extensive CLI sub-system for this reason

Types of objectives for the scripts

- Automating recurring tasks
- Recovery from loss of access
- Updates / initialisation
- Background worker management

CLI documentation

- https://path.to.your.misp/events/automation

Administering the background workers via the API.

You can start/stop and view the bacground workers via the API.

Add worker: `http://localhost:5001/servers/startWorker/[queue_name]`

Stop worker: `http://localhost:5001/servers/stopWorker/[worker_pid]`

Get worker info: `http://localhost:5001/servers/getWorkers`

Administering MISP via the CLI

Certain administrative tasks are exposed to the API, these help with maintaining and configuring MISP in an automated way / via external tools.:

Get Setting: `MISP/app/Console/cake Admin getSetting [setting]`

Set Setting: `MISP/app/Console/cake Admin setSetting [setting] [value]`

Get Authkey: `MISP/app/Console/cake Admin getAuthkey [email]`

Set Baseurl: `MISP/app/Console/cake Baseurl [baseurl]`

Change Password: `MISP/app/Console/cake Password [email] [new_password] [--override_password_change]`

Clear Bruteforce Entries: `MISP/app/Console/cake Admin clearBruteforce [user_email]`

Run Database Update: `MISP/app/Console/cake Admin updateDatabase`

Update All JSON Structures: `MISP/app/Console/cake Admin updateJSON`

Update Galaxy Definitions: `MISP/app/Console/cake Admin updateGalaxies`

Update Taxonomy Definitions: `MISP/app/Console/cake Admin updateTaxonomies`

Update Object Templates: `MISP/app/Console/cake Admin updateObjectTemplates`

Update Warninglists: `MISP/app/Console/cake Admin updateWarningLists`

Usage

`/var/www/MISP/app/Console/cake [Shell] [Command] [parameters]`

- Example:
  - `/var/www/MISP/app/Console/cake Password "andras.iklody@gmail.com" "Nutella"`
  - Change password to "Nutella" for my user
  - Some shells are single use and don't need a command parameter
- Also used by the background processing
- Automation is meant to be used via cron jobs

Automation via crontab

- Edit crontab of www-data user
- `crontab -u www-data -e`
- `0 3,9,15,21 * * * /var/www/MISP/app/Console/cake Server pull 1 30 full`
- Pull server ID #30 as user #1 every 6 hours
- `@hourly /var/www/MISP/app/Console/cake Server cacheFeed 1 csv full`
- Cache all csv feeds as user #1 every hour

MISP

INTELLIGENCE SHARING

MISP Deployment

Some basic guidelines

CIRCL / Team MISP Project

MISP Project

CRI MISP 2024

MISP deployment considerations

- **Deployment types**
- **Distro** choice
- **Hardware specs**
- **Authentication**
- Other considerations - **settings**, **gotchas**

Deployment types

- Native install
  - Manual
  - One-liner script - `INSTALL.sh`: https://github.com/MISP/MISP/tree/2.4/INSTALL
- MISP VM: https://www.circl.lu/misp-images/latest/
- Docker: https://github.com/MISP/misp-docker
- RPM maintained by `amuehlem`: https://github.com/misp/misp-rpm
- Cloud provider images: https://github.com/MISP/misp-cloud
- LXC images via `misp-airgap`: https://github.com/MISP/misp-airgap/

Docker options

- There are several MISP Docker installations available.
- The official MISP Docker installation is maintained by `ostefano` and can be found at `https://github.com/MISP/misp-docker`.
- The official MISP Docker image also includes one for the MISP modules.

Distro options

- Ubuntu 22.04 (20.04 will also work)
  - Our target platform
  - Our CI target
  - Use this unless you are absolutely forced not to
  - This is the platform we can support you with!
- CentOS 7
  - Annoying to operate
  - Less tested, though used by many
  - CentOS is dead. Consider other options
- RHEL 7
  - Same annoyance as CentOS in general
  - We test against CentOS in general, some assembly may be required

Hardware specs

- No firm recommendations, it's highly usage dependent
- It's better to go a bit over what you need than under
- **SSDs** are massively beneficial
- Let's look at what affects specs and some sample configurations

Hardware considerations

- What are the factors that can impact my performance?
  - Clustering of the data (how many datapoints / event?) (RAM, disk speed)
  - Correlation (RAM, disk speed, disk space)
    - Consider blocking overtly correlating values from doing so
    - Feed ingestion strategy is crucial
  - Over-contextualisation (RAM, disk speed)
    - Tag/attach galaxies to the event instead of each attribute when possible

Hardware considerations - continues

- What are the factors that can impact my performance?
  - Number of users that are active at any given time (RAM, CPU, disk speed)
  - Logging strategy (Disk space)
  - API users especially with heavy searches (substring searches for example) (RAM, CPU, Disk speed)

Hardware considerations - continues

- What are the factors that generally do **NOT** impact my performance as much as expected?
  - Warninglist usage
  - Number of raw attributes on the instance
  - Number of sync connections / recurring syncs (with measure)
  - Tools feeding off the automation channels (ZMQ, kafka, syslog)

Authentication options

- Username/password is the default
- Some built in modules by 3rd parties (LDAP, Shibboleth, x509, OpenID, Azure Active Directory)
- CustomAuth system for more flexibility
- Additionally, consider Email OTP

Other considerations - tuning

- PHP tuning
  - Maximum memory usage (per process)
  - Timeout settings
  - Consider setting it per role!
  - Background processes are exempt
- MySQL: key buffer size is important
- Generally, tune for few heavy requests rather than many light ones

Other considerations - high availability

- Clustering
  - Load balanced apache servers with MISP
  - Replicating / mirrored database backends
- Careful about session pinning
- Attachment storage can be abstracted / network attached
- An example implementation for AWS
  https://github.com/0xtf/HAMISPA

MISP

INTELLIGENCE SHARING

An Introduction to Workflows in MISP

MISP - Intelligence Sharing

CIRCL / Team MISP Project

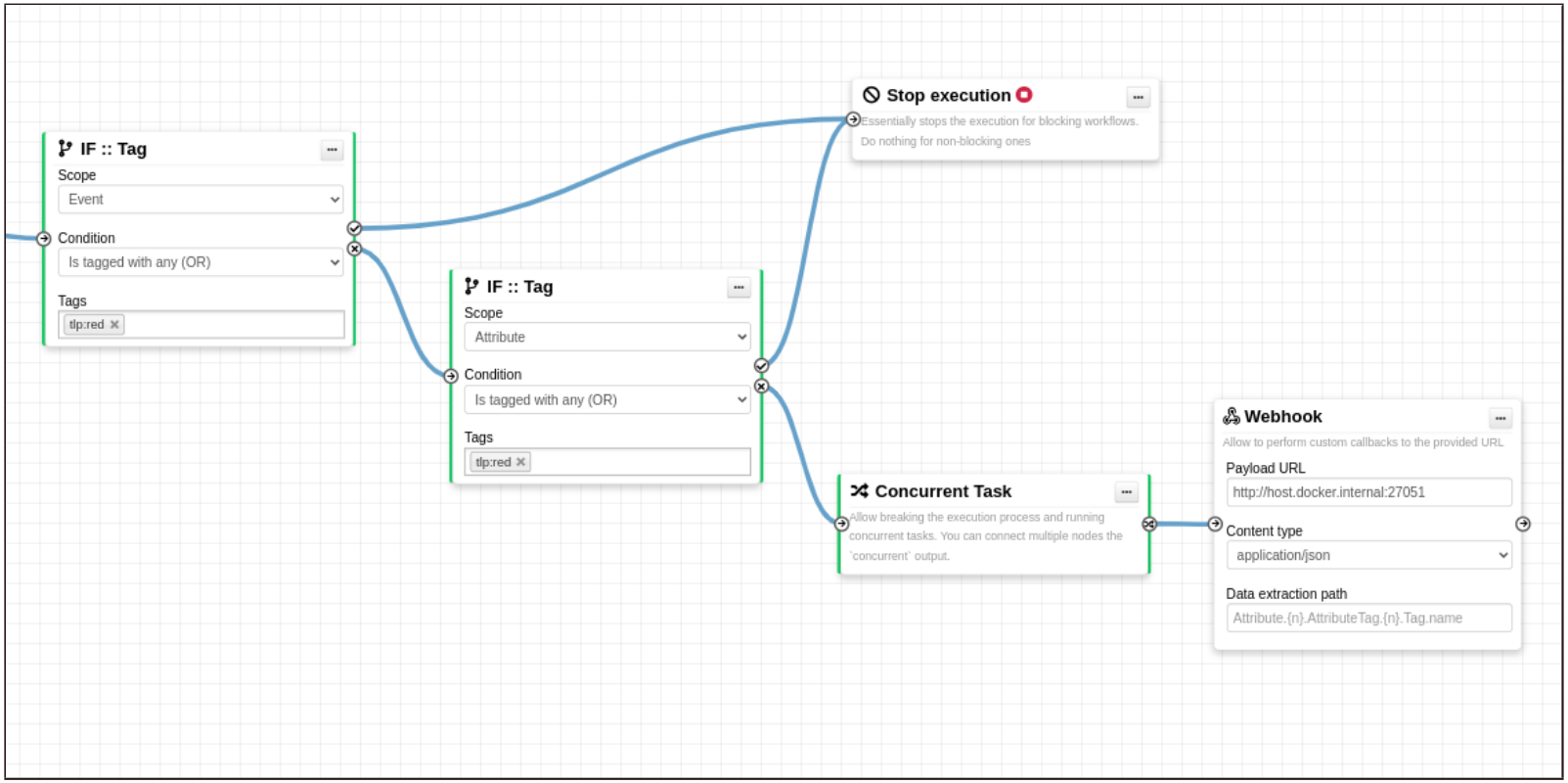
MISP Project

https://www.misp-project.org/

CRI MISP 2024

Content of the presentation

- MISP Workflows fundamentals
- Getting started
- Design of the system & how it can be extended



What problems are we trying to tackle

- Initial idea came during GeekWeek7.5[1]
- Needs:
  - Prevent default MISP behaviors
  - Hook specific actions to run callbacks
- Use-cases:
  - Prevent publication of events not meeting some criterias
  - Prevent querying thrid-party services (e.g. virustotal) with sensitive information
  - Send notifications in a chat rooms
  - And much much more..

[1] Workshop organized by the Canadian Cyber Center

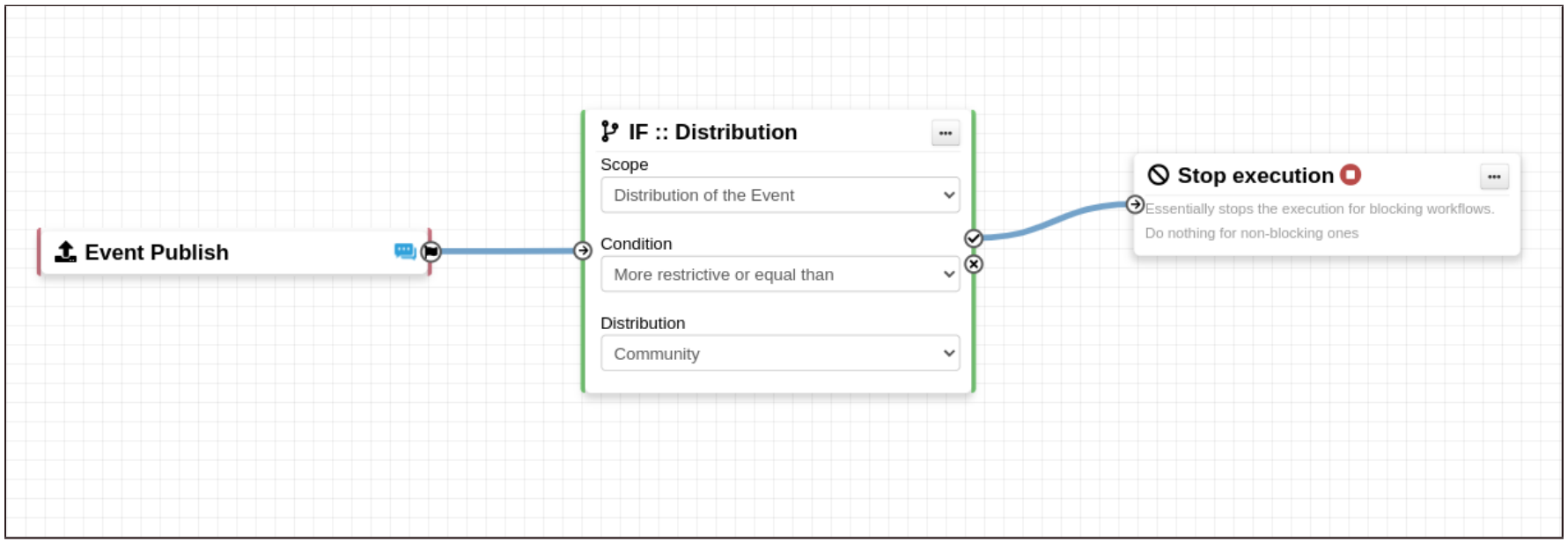
Workflow - Fundamentals

Simplistic overview of a Workflow in action

1. An **action** happens in MISP
2. If there is an **enabled** Workflow for that **action**, run it
3. If all went fine, MISP **continue** to perform the action
   - The operation can potentially be cancelled by `blocking` modules

Terminology

- **workflow**: Sequence of all operations (nodes) to be executed. Basically the whole graph.
- **execution path**: A path composed of nodes
- **trigger**: Starting point of a workflow. Triggers are called when specific actions happen in MISP
  - A trigger can only have one workflow and vice-versa



Workflow execution process

Typical execution process:

1. An action happens in MISP
2. The workflow associated to the trigger is ran
3. Execution result?
   - `success`: Continue the action
   - `failure` | `blocked`: Cancel the action

Example for Event publish:

1. An Event is about to be published
2. MISP executes the workflow listening to the `event-publish` trigger
   - **success**: Continue the publishing action
   - **failure** | `blocked`: Stop publishing and log the reason

Blocking and non-blocking Workflows

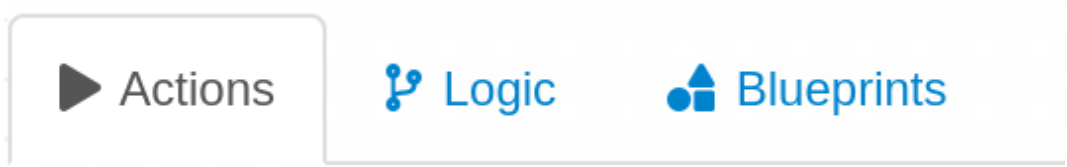
Currently 2 types of workflows:

- **Blocking**: Completion of the action can be prevented
  - If a **blocking module** blocks the action
  - If a **blocking module** raises an exception
- **Non-blocking**: Workflow execution outcome has no impact
  - **Blocking modules** can still stop the execution

Execution context

- Workflows can be triggered by **any users**
- Workflows can be triggered by actions done via the **UI** or **API**
- However, the user for which the workflow executes has:
  - The `site-admin` permission
  - Is from the `MISP.host_org_id`
- Ensures data is processed regardless of ownership and access: **no ACL**

Classes of Workflow modules



3 classes of modules

- **action**: Allow to executes functions, callbacks or scripts
  - Can stop execution
  - e.g. Webhook, block the execution, perform enrichments, ...
- **logic**: Allow to redirect the execution flow.
  - IF condition, fork the blocking execution into a non-blocking one, ...
- **blueprint**: Allow to reuse composition of modules
  - Can save subworkflows and its module's configuration

Sources of Workflow modules

3 sources of action modules

- Built-in **default** modules
  - Part of the MISP codebase
  - **app/Model/**`WorkflowModules/action/[module_name].php`
- User-defined **custom** modules
  - Written in PHP
  - Can extend existing default modules
  - Can use MISP's built-in functionalities (restsearch, enrichment, push to zmq, ...)
  - Faster and easier to implement new complex behaviors
  - **app/Lib/**`WorkflowModules/action/[module_name].php`

Sources of Workflow modules

3 sources of action modules

- Modules from the **enrichment service**
  - **Default** and **custom** modules
  - From the *misp-module*

  - Written in Python
  - Can use any python libraries
  - New *misp-module* module type: `action`

→ Both the PHP and Python systems are **plug-and-play**

Triggers currently available

Currently 8 triggers can be hooked. 3 being **blocking**.

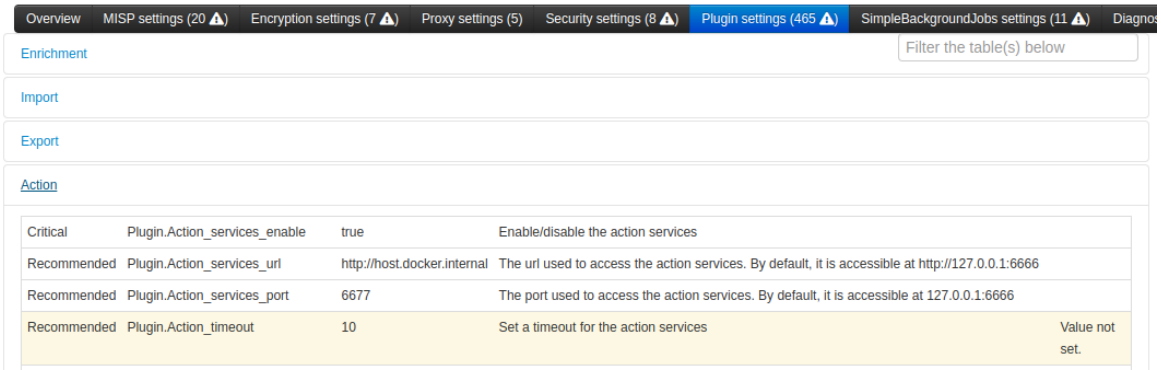
Trigger name	Scope	Trigger overhead	Description	Run counter	Blocking Workflow	MISP Core format	Workflow ID	Last Update	Enabled	Actions
Attribute After Save	attribute	high	This trigger is called after an Attribute has been saved in the database	58	×	✓	160	2022-07-29 06:58:11	✓	
Enrichment Before Query	others	low	This trigger is called just before a query against the enrichment service is done	841	✓	✓	162	2022-07-29 08:32:32	✓	
Event After Save	event	medium	This trigger is called after an Event has been saved in the database	11	×	✓	175	2022-07-29 08:37:23	✓	
Event Publish	event	low	This trigger is called just before a MISP Event starts the publishing process	1	✓	✓	180	2022-07-29 12:14:10	✓	
Object After Save	object	high	This trigger is called after an Object has been saved in the database	35	×	✓	161	2022-07-28 13:59:37	×	
Post After Save	post	low	This trigger is called after a Post has been saved in the database	36	×	×	176	2022-07-28 13:59:51	✓	
User After Save	user	low	This trigger is called after a user has been saved in the database	55	×	×	159	2022-07-28 14:00:03	✓	
User Before Save	user	low	This trigger is called just before a user is save in the database	42	✓	×	158	2022-07-28 14:00:32	✓	

Workflow - Getting started

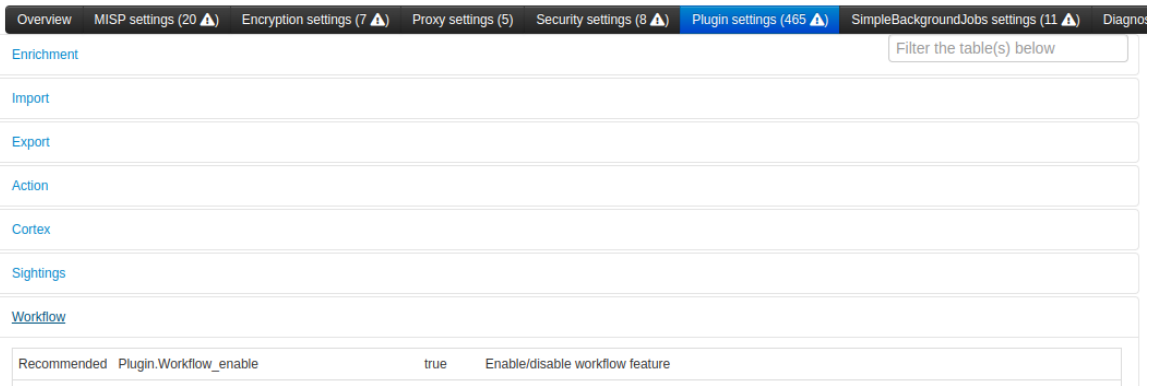
Getting started with workflows (1)

Review MISP settings:

1. Make sure `MISP.background_jobs` is turned on
2. Make sure workers are up-and-running and healthy
3. Turn the setting `Plugin.Workflow_enable` on



4. [optional:misp-module] Turn the setting `Plugin.Action_services_enable` on



Getting started with workflows (2)

If you wish to use action modules from `misp-module`, make sure to have:

- The latest update of `misp-module`
  - There should be an `action_mod` module type in `misp-modules/misp_modules/modules`
- Restarted your `misp-module` application

```
1 # This command should show all ‘action‘ modules
2 $ curl -s http://127.0.0.1:6666/modules | \
3 jq '.[] | select(.meta."module-type"[] | contains("action")) |
4 {name: .name, version: .meta.version}'
```

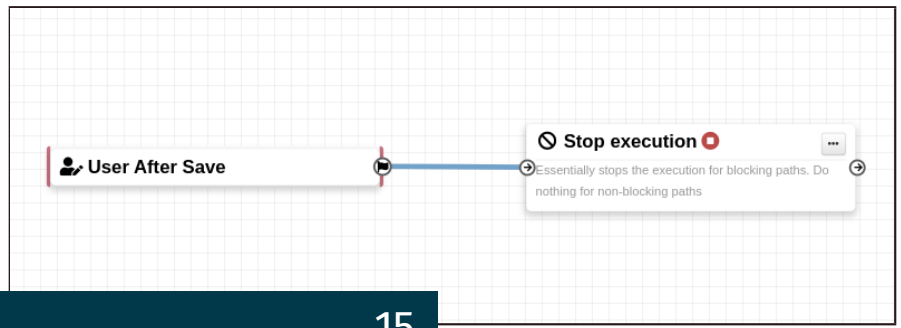
Getting started with workflows (3)

1. Go to the list of modules
   - `Administration > Workflows > List Modules`
   - or `/workflows/moduleIndex`
2. Make sure **default** modules are loaded
3. [optional:misp-module] Make sure **misp-module** modules are loaded

Creating a workflow with the editor

1. Go to the list of triggers `Administration > Workflows`
2. Enable and edit a `trigger` from the list
3. Drag an `action` module from the side panel to the canvas
4. From the `trigger` output, drag an arrow into the `action`'s input (left side)
5. Execute the action that would run the trigger and observe the effect!

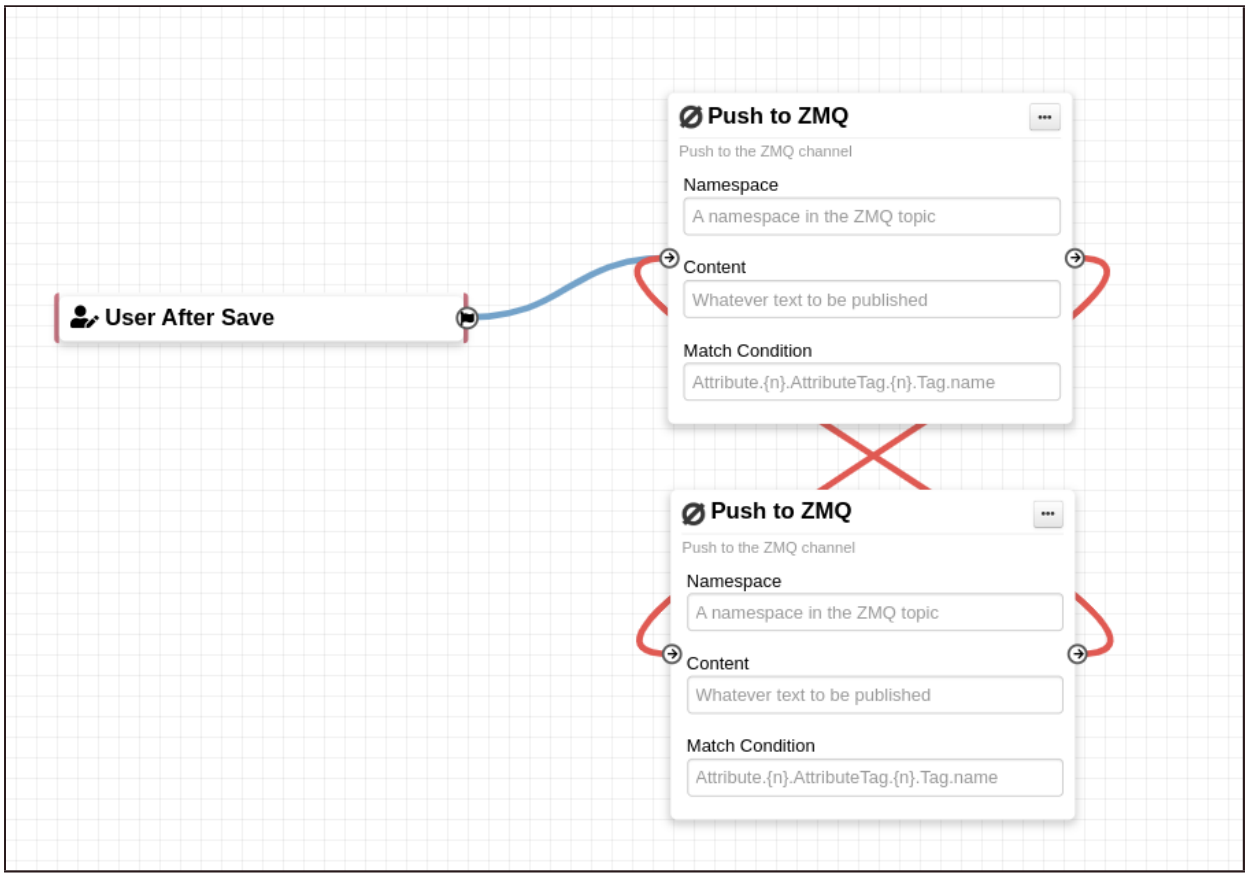
Trigger name	Scope	Trigger overhead	Description	Run counter	Blocking Workflow	MISP Core format	Workflow ID	Last Update	Enabled	Actions
Attribute After Save	attribute	high	This trigger is called after an Attribute has been saved in the database	58	×	✓	160	2022-07-29 06:58:11	✓	
Enrichment Before Query	others	low	This trigger is called just before a query against the enrichment service is done	841	✓	✓	162	2022-07-29 08:32:32	✓	
Event After Save	event	medium	This trigger is called after an Event has been saved in the database	11	×	✓	175	2022-07-29 08:37:23	✓	
Event Publish	event	low	This trigger is called just before a MISP Event starts the publishing process	1	✓	✓	180	2022-07-29 12:14:10	✓	
Object After Save	object	high	This trigger is called after an Object has been saved in the database	35	×	✓	161	2022-07-28 13:59:37	×	
Post After Save	post	low	This trigger is called after a Post has been saved in the database	36	×	×	176	2022-07-28 13:59:51	✓	
User After Save	user	low	This trigger is called after a user has been saved in the database	55	×	×	159	2022-07-28 14:00:03	✓	
User Before Save	user	low	This trigger is called just before a user is save in the database	42	✓	×	158	2022-07-28 14:00:32	✓	



Working with the editor

Operations not allowed:

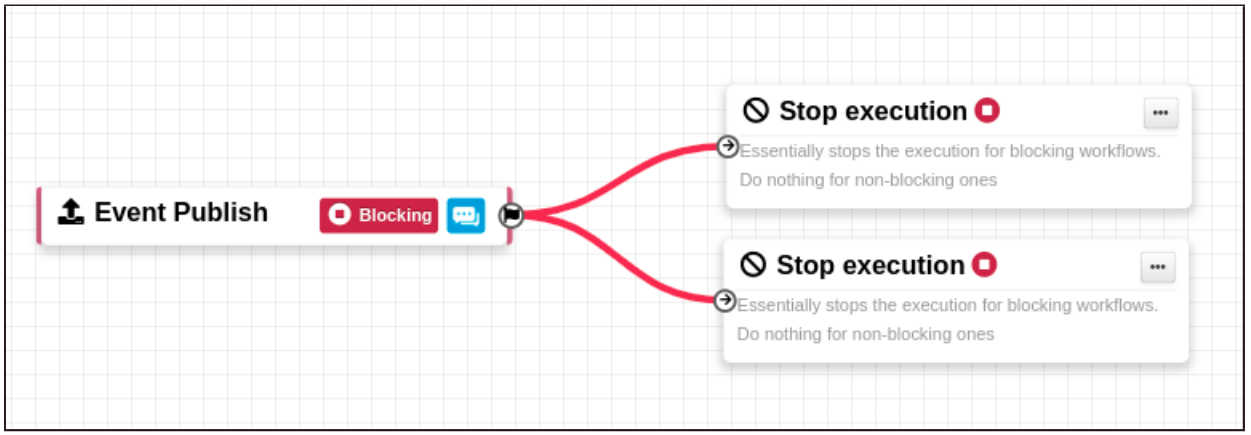
- Execution loop are not authorized
  - Current caveat: If an action re-run the workflow in any way



Working with the editor

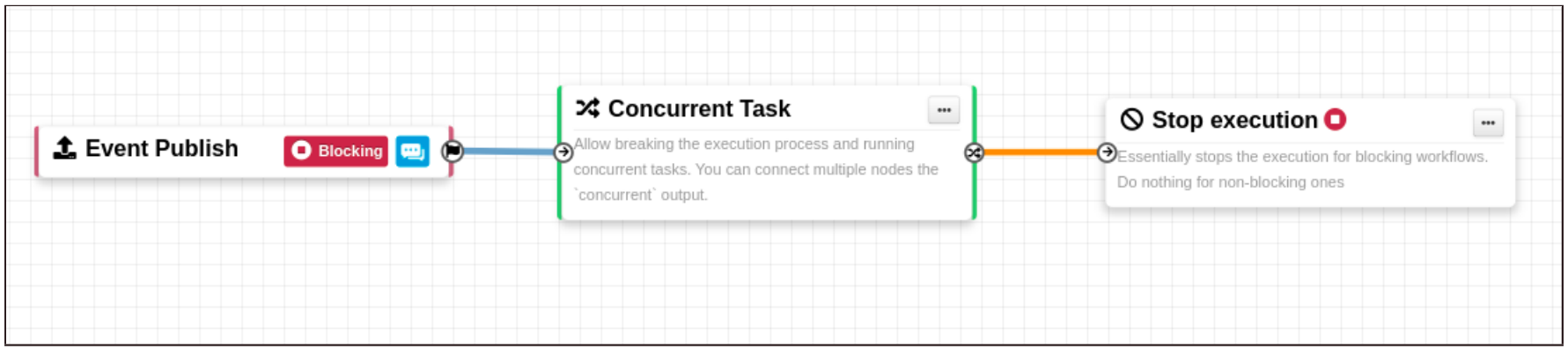
Operations not allowed:

- Multiple connections from the same output
  - Execution order not guaranted and confusing for users



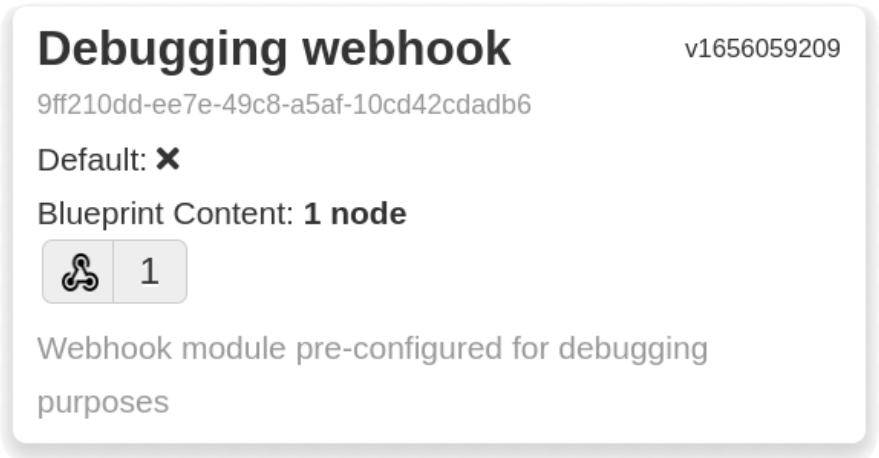
Operations showing a warning:

- **Blocking** modules after a **concurrent tasks** module
- **Blocking** modules in a **non-blocking** workflow



Workflow blueprints

1. Blueprints allow to **re-use parts** of a workflow in another one
2. Blueprints can be saved, exported and **shared**

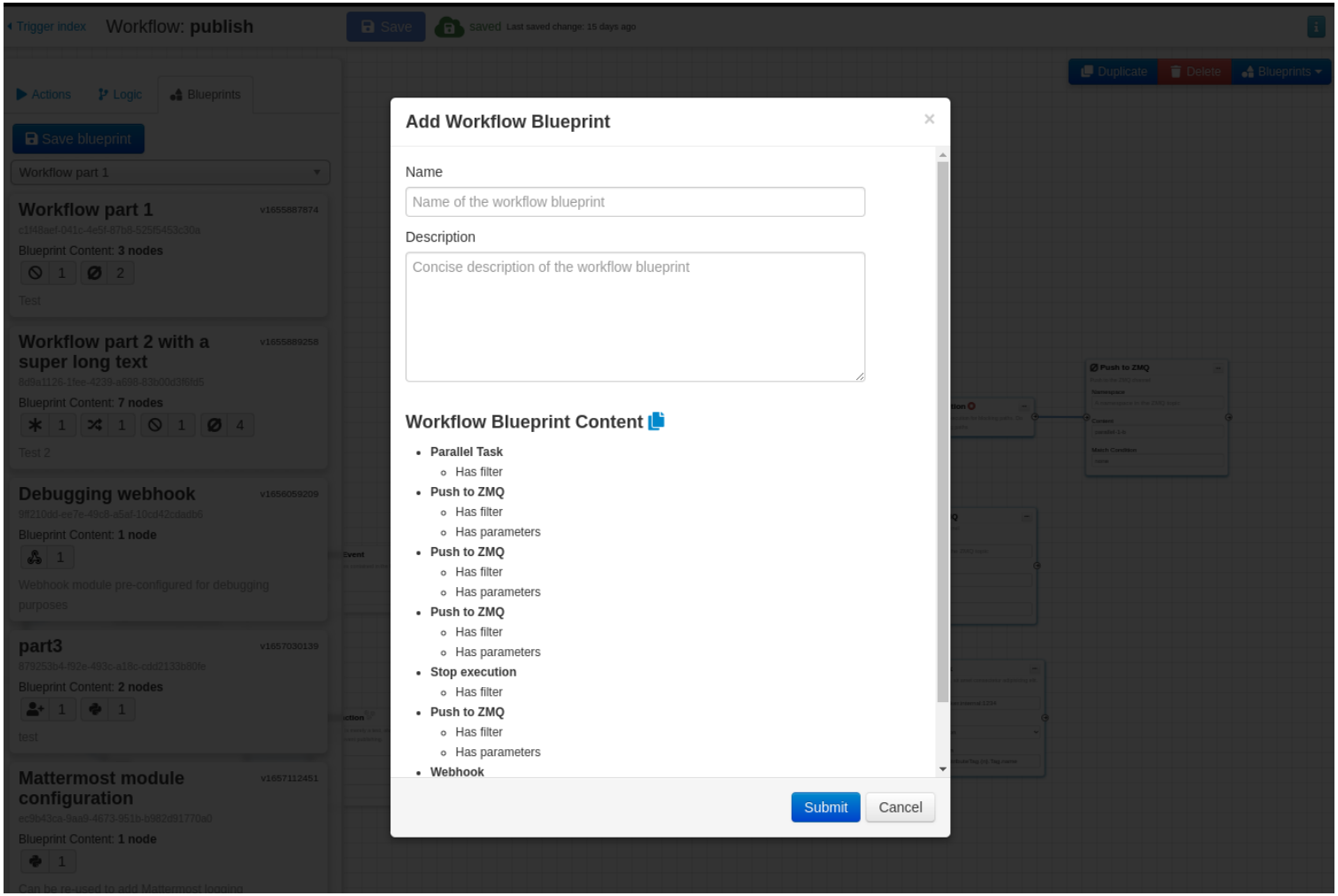


Blueprints origins:

1. From the "official" `misp-workflow-blueprints` repository
2. Created or imported by users

Workflow blueprints: Create

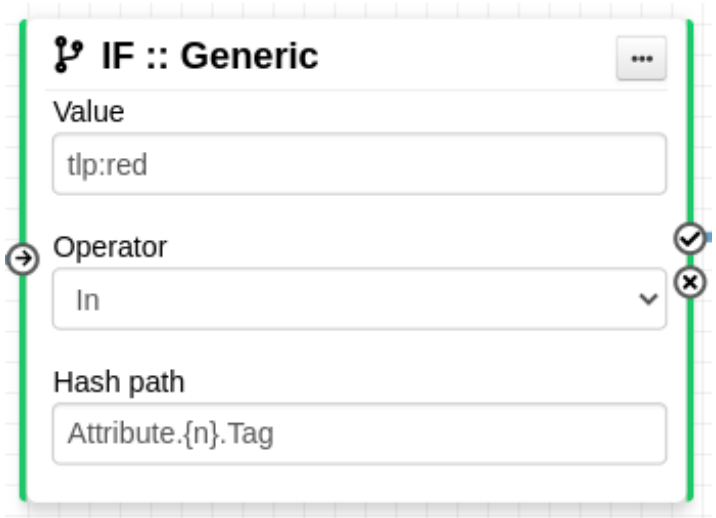
Select one or more modules to be saved as blueprint then click on the `save blueprint` button



Hash path filtering

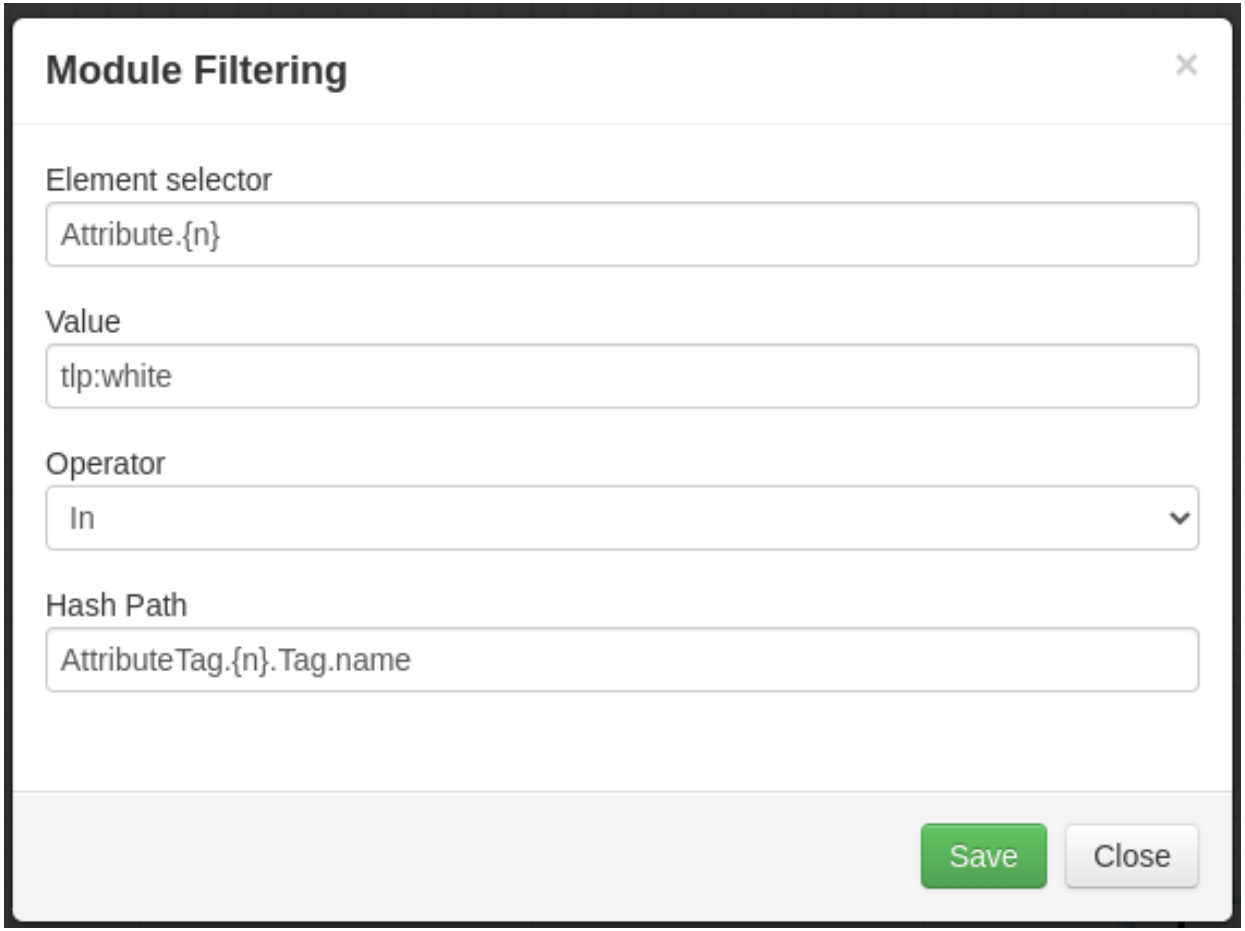
- Some modules have the possibility to filter or check conditions using CakePHP's path expression.

```
1 $path_expression = '{n}[name=fred].id';
2 $users = [
3     {'id': 123, 'name': 'fred', 'surname': 'bloggs'},
4     {'id': 245, 'name': 'fred', 'surname': 'smith'},
5     {'id': 356, 'name': 'joe', 'surname': 'smith'},
6 ];
7 $ids = Hash::extract($users, $path_expression);
8 // => $ids will be [123, 245]
```



Module filtering

- Some action modules accept **filtering** conditions
- E.g. the `enrich-event` module will only perform the enrichment on Attributes having a `tlp:white` Tag

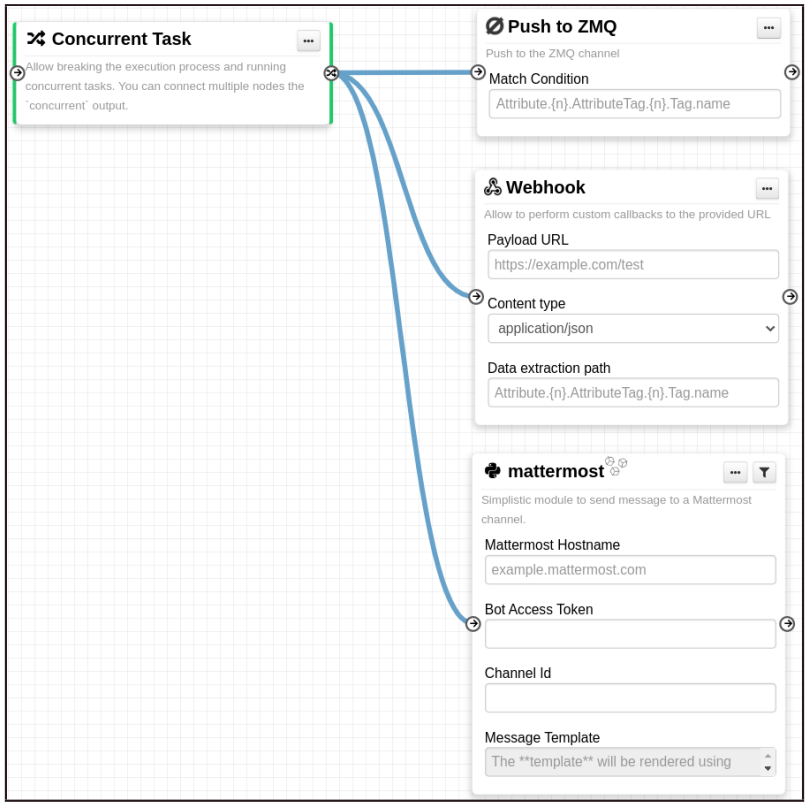


Data format in Workflows

- All triggers will inject data in a workflow
- In some cases, there is no format (e.g. User after-save)
- In others, the format is **compliant with the MISP Core format**
- In addition to the RFC, the passed data has **additional properties**
  - Attributes are **always encapsulated** in the Event or Object
  - Additional key **`_AttributeFlattened`**
  - Additional key **`_allTags`**
  - Additional key **`inherited`** for Tags

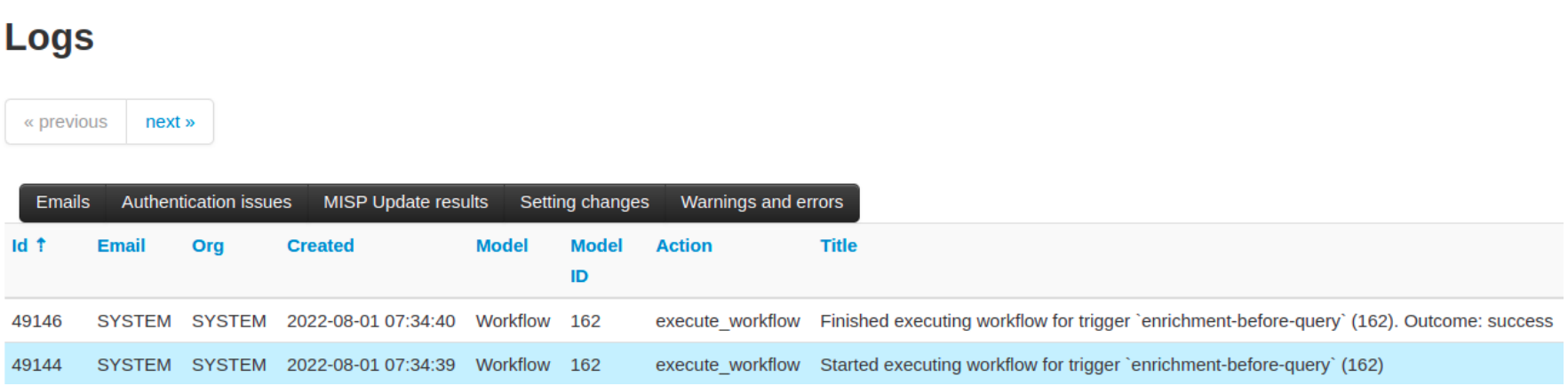
Logic module: Concurrent Task

- Special type of **logic** module allowing multiple connections
- Allows **breaking the execution** flow into a concurrent tasks to be executed later on by a background worker
- As a side effect, blocking modules **cannot cancel** ongoing operations



Debugging Workflows: Log Entries

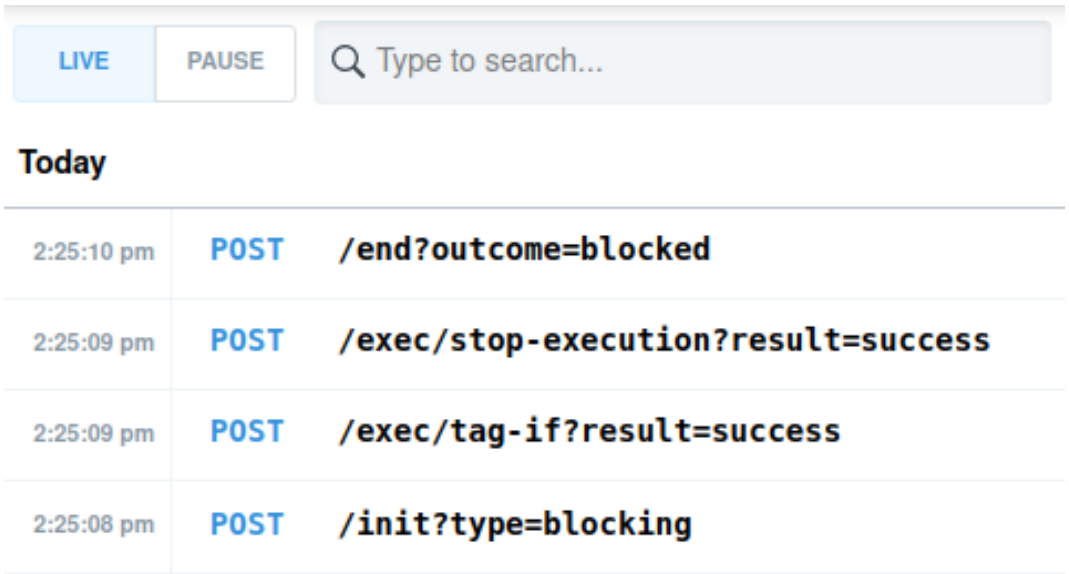
- Workflow execution is logged in the application logs:
  - `/admin/logs/index`
- Or stored on disk in the following file:
  - `/app/tmp/logs/workflow-execution.log`
- Use the `webhook-listener.py` tool
  - `/tools/misp-workflows/webhook-listener.py`



Debugging Workflows: Debug mode

- The

can be turned on for each workflows
- Each nodes will send data to the provided URL
  - Configure the setting: `Plugin.Workflow_debug_url`
- Result can be visualized in
  - **offline**: `tools/misp-workflows/webhook-listener.py`
  - **online**: `requestbin.com` or similar websites

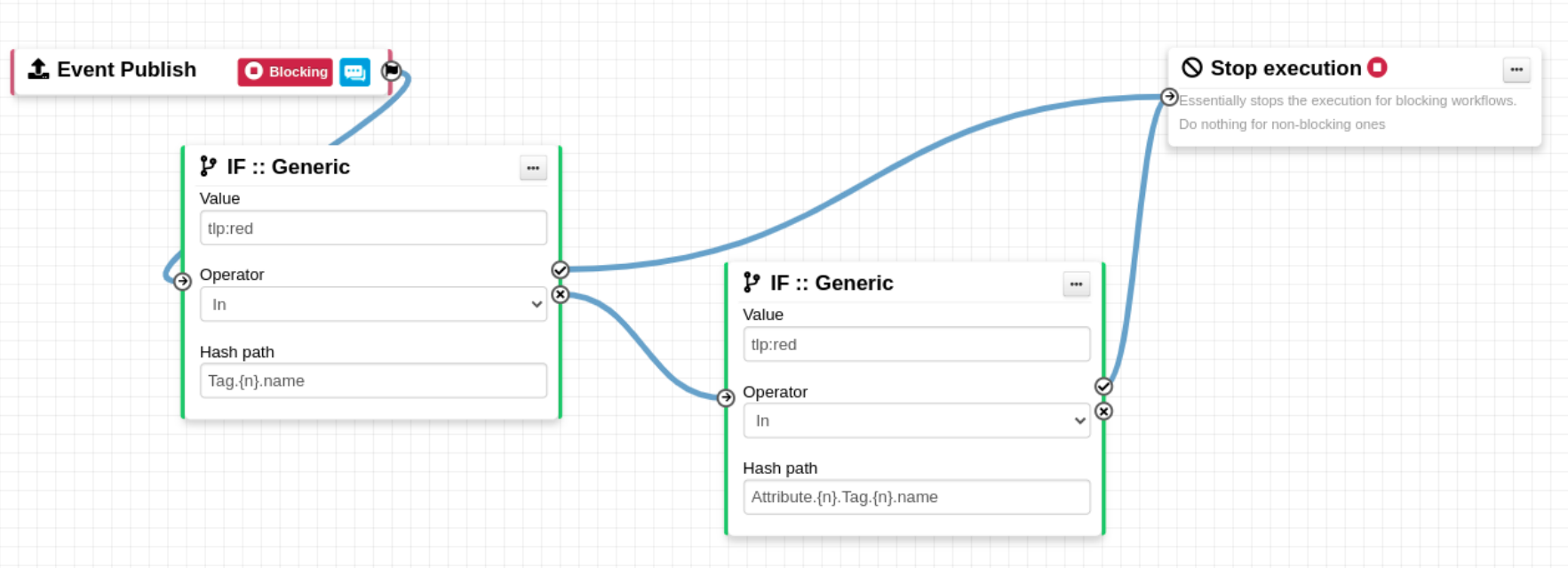


Learning by examples

Workflow example 1

1. The `Event-Publish` trigger uses the MISP core format
2. The `IF::Tag` module checks if at least one of the Attribute has the `tlp:white` tag
3. If it does, the `Push-to-ZMQ` module will be executed

Workflow example 2



- If an event has the `tlp:red` tag or any of the attribute has it, the publish process will be cancelled

Extending the system

Creating a new module in PHP

app > Lib > WorkflowModules > action > Module_blueprint_action_module.php > ...

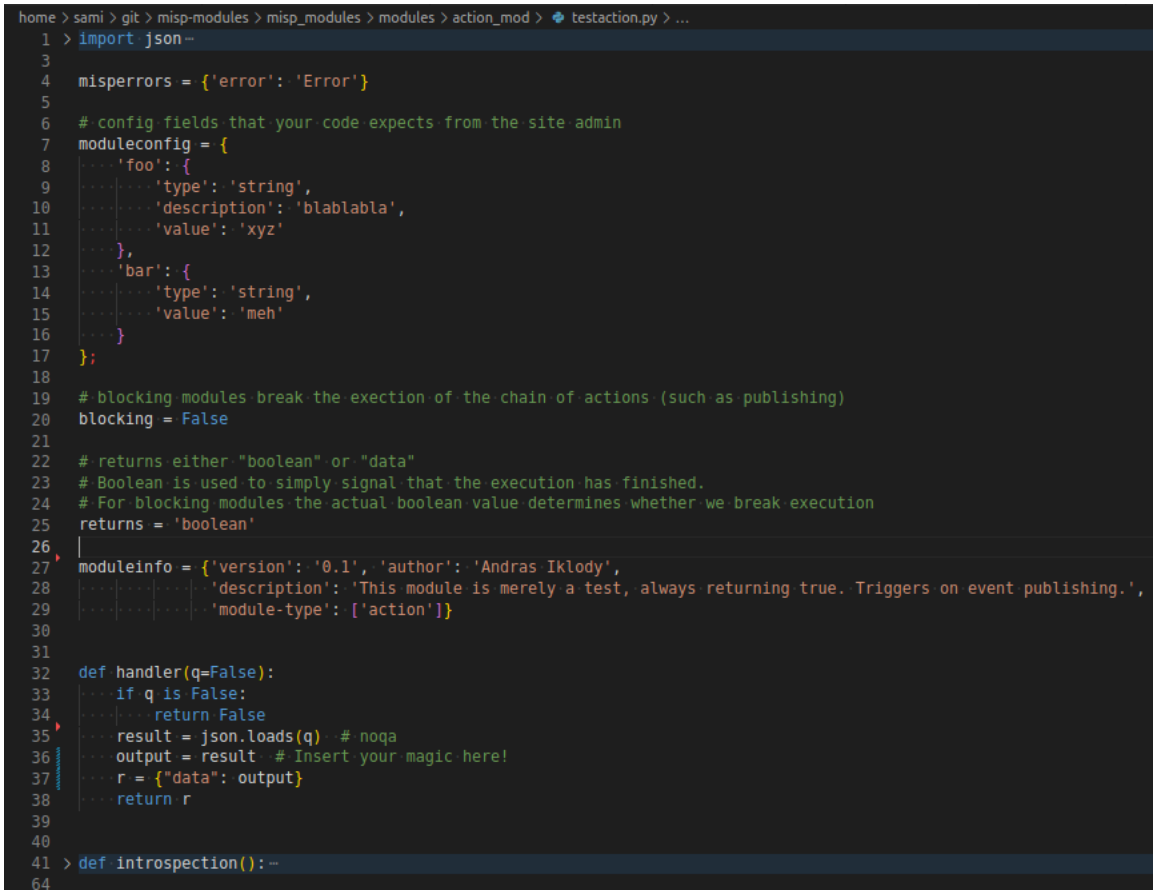
```php
<?php
include_once APP . 'Model/WorkflowModules/WorkflowBaseModule.php';

class Module_blueprint_action_module extends WorkflowBaseModule
{
    public $is_blocking = false;
    public $disabled = true;
    public $id = 'blueprint-action-module';
    public $name = 'Blueprint action module';
    public $description = 'Lorem ipsum dolor, sit amet consectetur adipisicing elit.';
    public $icon = 'shapes';
    public $inputs = 1;
    public $outputs = 1;
    public $params = [];

    public function exec(array $node, WorkflowRoamingData $roamingData, array &$errors = [])
    : bool
    {
        parent::exec($node, $roamingData, $errors);
        // If $this->is_blocking == true, returning `false` will stop the execution.
        $errors[] = __('Execution stopped');
        return false;
    }
}
```

- **app/Lib/**`WorkflowModules/action/[module_name].php`
- Module configuration are defined as public variables
- The `exec` function has to be implemented.
  - If it returns **true**, execution will proceed
  - If it returns **false**
    - And the module is blocking, the execution will stop and the operation will be blocked

Creating a new module in Python



- Module configuration are defined in the `moduleinfo` and `moduleconfig` variables
- The `handler` function has to be implemented.
- Blocking logic is the same as other modules

MISP

INTELLIGENCE SHARING

Automation with Workflows in MISP

Short version

Sami Mokaddem

MISP Project

https://www.misp-project.org/

Content of the presentation

1. Automation in MISP
2. MISP Workflows
   - Fundamentals
   - Demo with examples
   - Using the system
   - How it can be extended

Automation in MISP: What already exists?

MISP API / PyMISP

- Needs CRON Jobs in place
- Potentially heavy for the server
- Not realtime

PubSub channels

- After the actions happen: No feedback to MISP
- Tougher to put in place & to share
- Full integration amounts to develop a new tool

→ No way to **prevent** behavior
→ Difficult to setup **hooks** to execute callbacks

Simple automation in MISP made easy

- **Visual** dataflow programming
- **Drag & Drop** editor
- Flexible **Plug & Play** system
- **Share** workflows, **debug** and **replay**

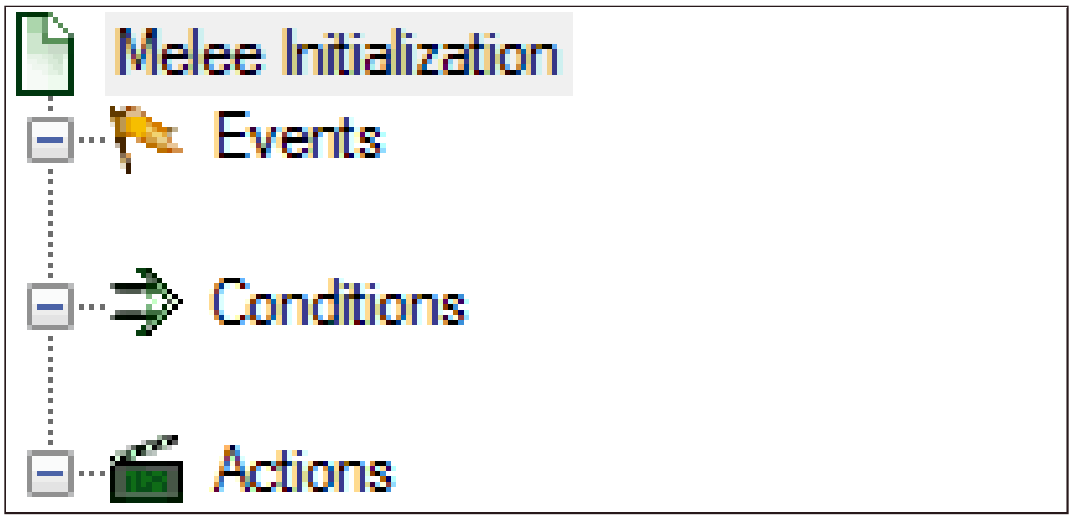
Example of use-cases

- **Notification** on specifc actions
  - New events matching criteria
  - New users
  - Automated alerts for high-priority IOCs
- **Extend** existing MISP behavior
  - Push data to another system
  - Automatic enrichment
  - Sanity check to block publishing / sharing
  - Curation pipelines
- **Hook** capabilities
  - Assign tasks and notify incident response team members
- ...

Workflow - Fundamentals

Objective: Start with the foundation to understand the basics

How does it work



1. An **event** happens in MISP
2. *(optional)* Check if all **conditions** are satisfied
3. Execute all **actions**
   - May prevent MISP to complete its original event

What kind of events?

Events

- New MISP Event
- Attribute has been saved
- New discussion post
- New user created
- Query against third-party services
- ...

Supported events in MISP are called **Triggers**

A **Trigger** is associated with **1-and-only-1 Workflow**

Triggers currently available

Currently 11 triggers can be hooked. 3 being Blocking.

Triggers

List the available triggers that can be listened to by workflows.
Missing a trigger? Feel free to open a Github issue!
Documentation and concepts

« previous | next »

All | attribute | event | log | object | others | post | user | Blocking | Enabled | Disabled

Trigger name	Scope	Trigger overhead	Run counter	Blocking Workflow	MISP Core format	Workflow ID	Last Update	Debug enabled	Enabled	Actions
Attribute After Save	attribute	high	110	✗	✓	160	2023-09-14 06:54:37	☐	✗	
Enrichment Before Query	others	low	2226	✓	✓	162	2023-10-09 07:56:42	☐	✓	
Event After Save	event	high	191	✗	✓	175	2023-10-02 14:55:19	☐	✗	
Event After Save New	event	low	7	✗	✓	182	2023-03-16 14:05:07	☐	✗	
Event After Save New From Pull	event	low	6	✗	✓	183	2023-10-09 07:57:02	☐	✗	
Event Publish	event	low	2	✓	✓	188	2023-10-09 07:56:25	☐	✓	
Log After Save	log	high	0	✗	✗	185	2023-06-05 13:26:50	☐	✗	
Object After Save	object	high	35	✗	✓	161	2023-06-05 13:27:00	☐	✗	
Post After Save	post	low	36	✗	✗	176	2022-07-28 13:59:51	☐	✗	
User After Save	user	low	0	✗	✗	181	2022-08-05 07:19:46	☐	✗	
User Before Save	user	low	42	✓	✗	158	2023-06-05 13:27:25	☐	✗	

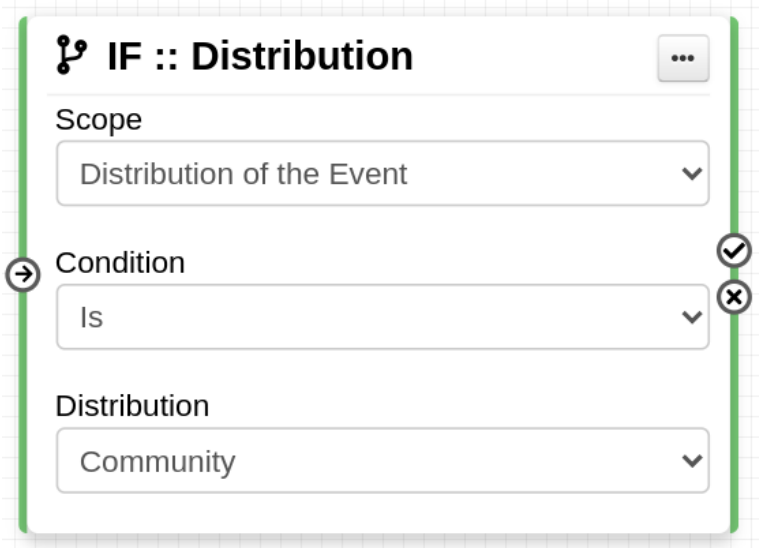
Page 1 of 1, showing 1 records out of 11 total, starting on record 1, ending on 11

What kind of conditions?

Conditions

- A MISP Event is tagged with `tlp:red`
- The distribution of an Attribute is a sharing group
- The creator organisation is `circl.lu`
- Or any other **generic** conditions

❓ These are also called **Logic modules**



Workflow - Logic modules

- **logic** modules: Allow to redirect the execution flow.
  - IF conditions
  - Delay execution

All | Action | Logic | misp-module | Custom | Blocking | Enabled | Disabled | Enter value to search | Filter

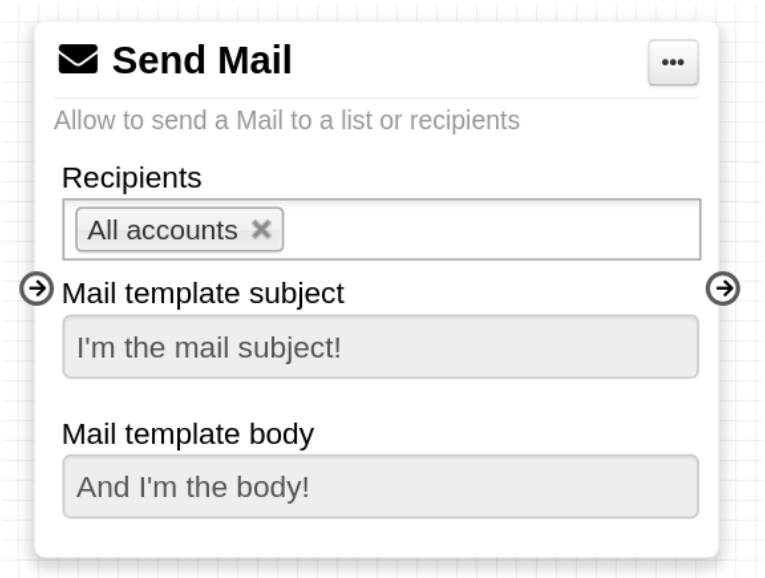
Module name	Type	Blocking	MISP Core format	misp-module	Custom	Enabled	Actions
Blueprint logic module	logic	✗	✗	✗	✓	✗	
Concurrent Task	logic	✗	✗	✗	✗	✓	
IF :: Distribution	logic	✗	✓	✗	✗	✓	
Filter :: Generic	logic	✗	✗	✗	✗	✗	
Filter :: Remove filter	logic	✗	✗	✗	✗	✗	
IF :: Generic	logic	✗	✗	✗	✗	✓	
IF :: Organisation	logic	✗	✓	✗	✗	✓	
IF :: Published	logic	✗	✓	✗	✗	✓	
IF :: Tag	logic	✗	✓	✗	✗	✓	
IF :: Threat Level	logic	✗	✗	✗	✗	✗	

What kind of actions?

Actions

- Send an email notification
- Perform enrichments
- Send a chat message on MS Teams
- Attach a local tag
- ...

❓ These are also called **Action modules**



Workflow - Action modules

- **action** modules: Allow to executes operations
  - Tag operations
  - Send notifications
  - Webhooks & Custom scripts

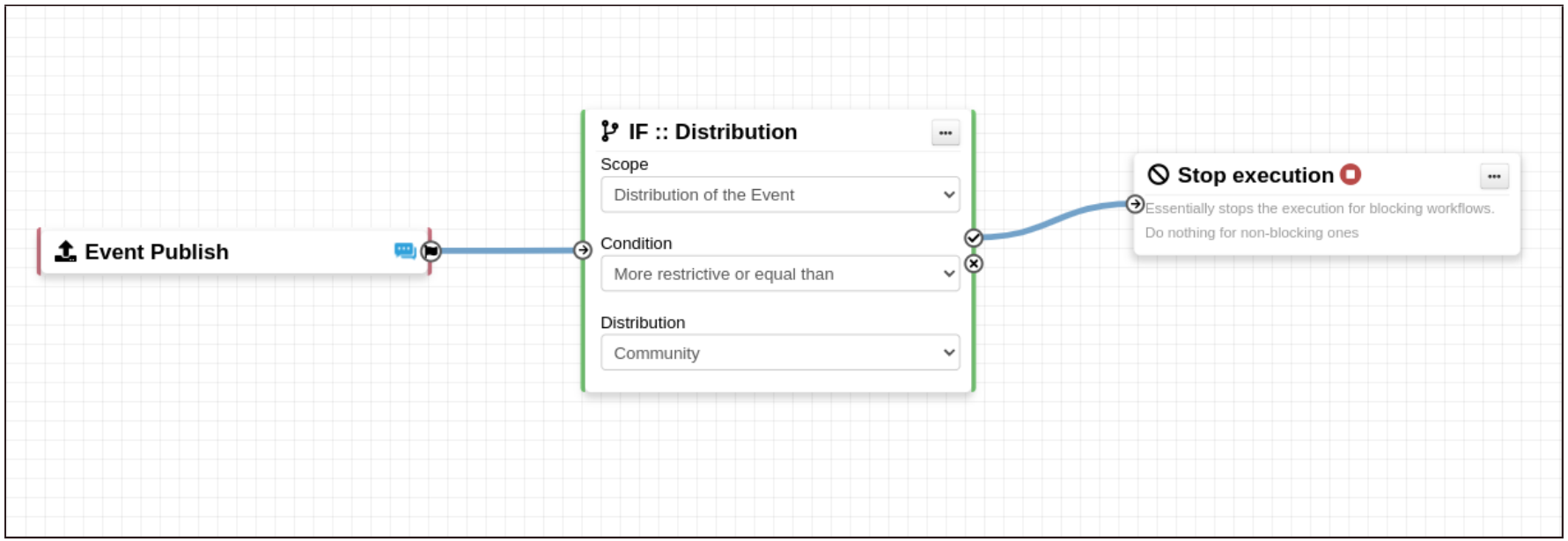
All | Action | Logic | misp-module | Custom | Blocking | Enabled | Disabled

Enter value to search | Filter

Module name	Type	Blocking	MISP Core format	misp-module	Custom	Enabled	Actions
Attach enrichment	action	✗	✓	✗	✗	✓	
Attribute edition operation	action	✗	✓	✗	✗	✓	
Attribute IDS Flag operation	action	✗	✓	✗	✗	✓	
Blueprint action module	action	✗	✗	✗	✓	✓	
Enrich Event	action	✗	✓	✗	✗	✓	
mattermost	action	✗	✗	✓	✗	✓	
MS Teams Webhook	action	✗	✗	✗	✗	✓	
Push to ZMQ	action	✗	✗	✗	✗	✓	
Send Log Mail	action	✗	✗	✗	✗	✗	
Send Mail	action	✗	✗	✗	✗	✓	
Splunk HEC export	action	✗	✓	✗	✗	✗	
Stop execution	action	✓	✗	✗	✗	✓	
Tag operation	action	✗	✓	✗	✗	✓	
testaction	action	✗	✗	✓	✗	✓	
Webhook	action	✗	✗	✗	✗	✓	

What is a MISP Workflow?

- Sequence of all nodes to be executed in a specific order
- Workflows can be enabled / disabled
- A Workflow is associated to **1-and-only-1 trigger**



Sources of Workflow modules (0)

Currently 36 built-in modules.

- **Trigger** module (11): built-in **only**
  - Get in touch if you want more
- **Logic** module (10): built-in & **custom**
- **Action** module (20): built-in & **custom**

Sources of Workflow modules (1)

- Built-in **default** modules
  - Part of the MISP codebase
  - Get in touch if you want us to increase the selection (or merge PR!)

Sources of Workflow modules (2)

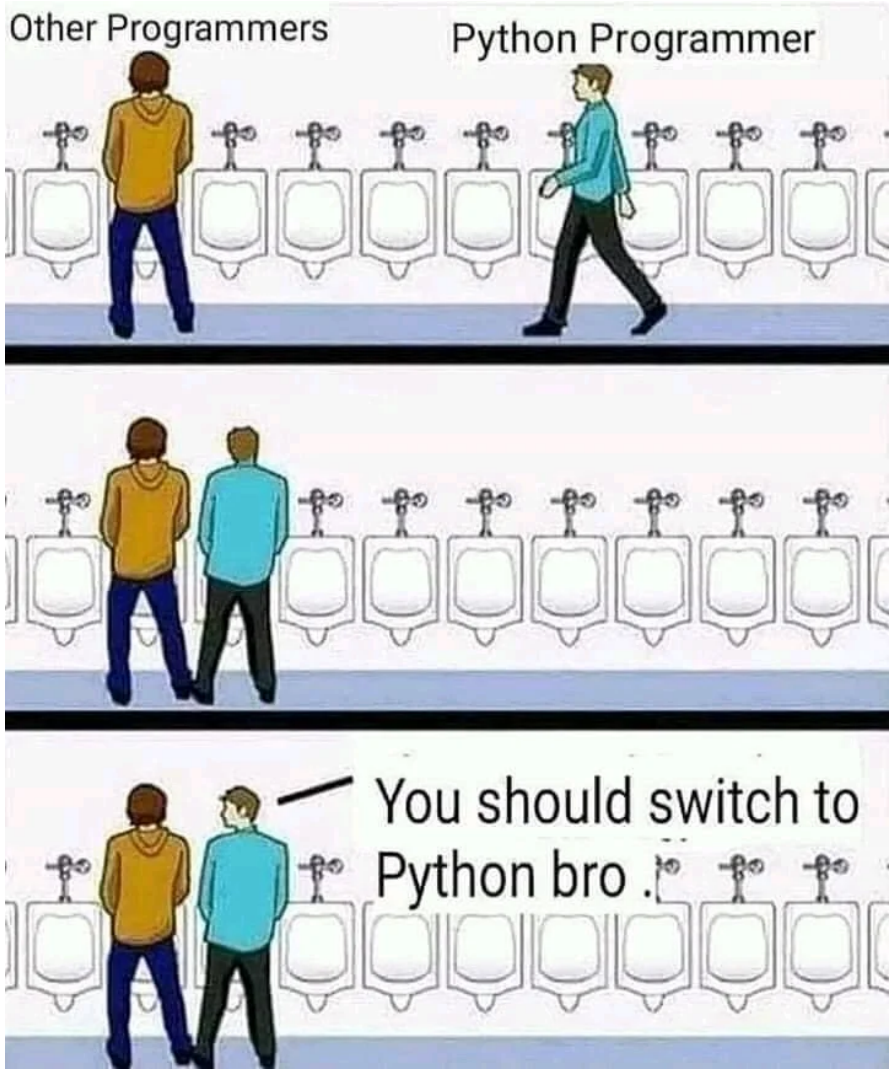
User-defined **custom** modules

- Written in PHP
- Extend existing modules
- MISP code reuse

Sources of Workflow modules (3)

Modules from the misp-module **enrichment service**

- Written in Python
- Can use any python libraries
- Plug & Play



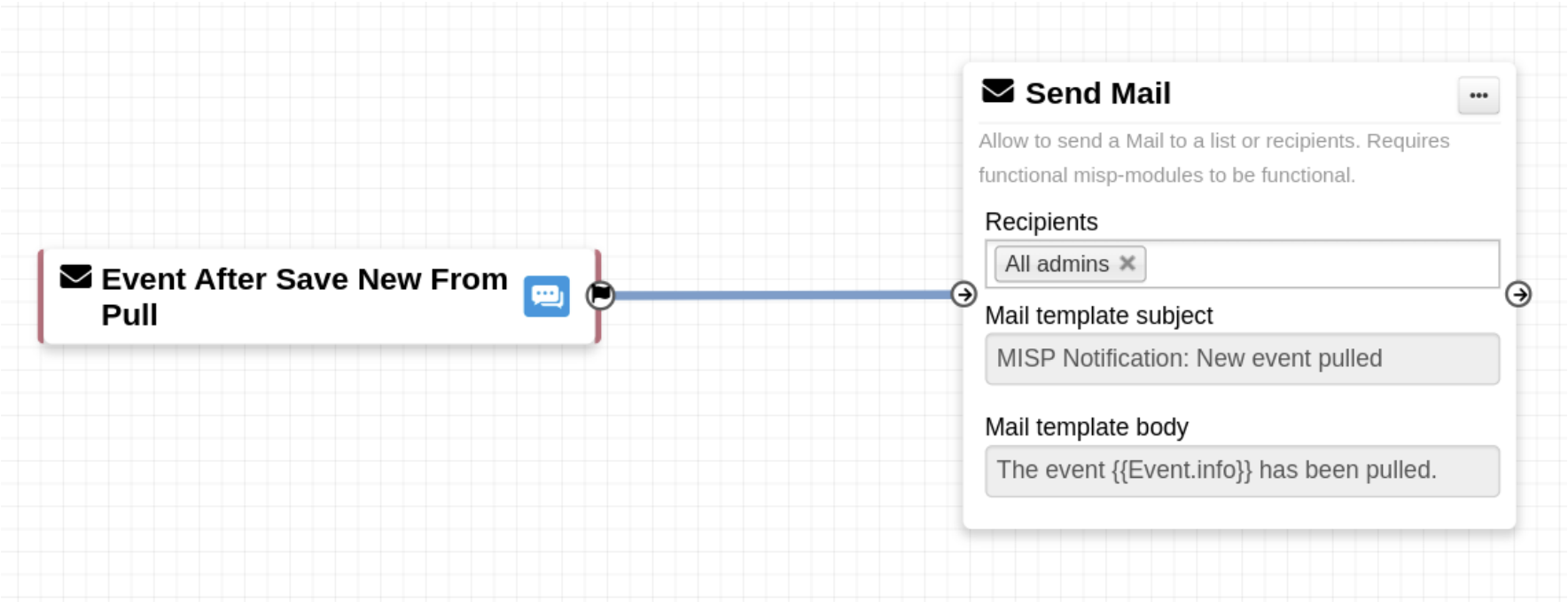
Demo by examples

WF-1. Send an email to **all admins** when a new event has been pulled

WF-2. Block queries on 3rd party services when **tlp:red** or **PAP:red**

- **tlp:red**: For the eyes and ears of individual recipients only
- **PAP:RED**: Only passive actions that are not detectable from the outside

Demo WF-1: Send an email to all admins when a new event has been pulled



Demo WF-2: Block queries on 3rd party services when tlp:red or PAP:red

- **tlp:red**: For the eyes and ears of individual recipients only
- **PAP:RED**: Only passive actions that are not detectable from the outside

Everything is ready?

Let’s see how to build a workflow!

Creating a workflow with the editor

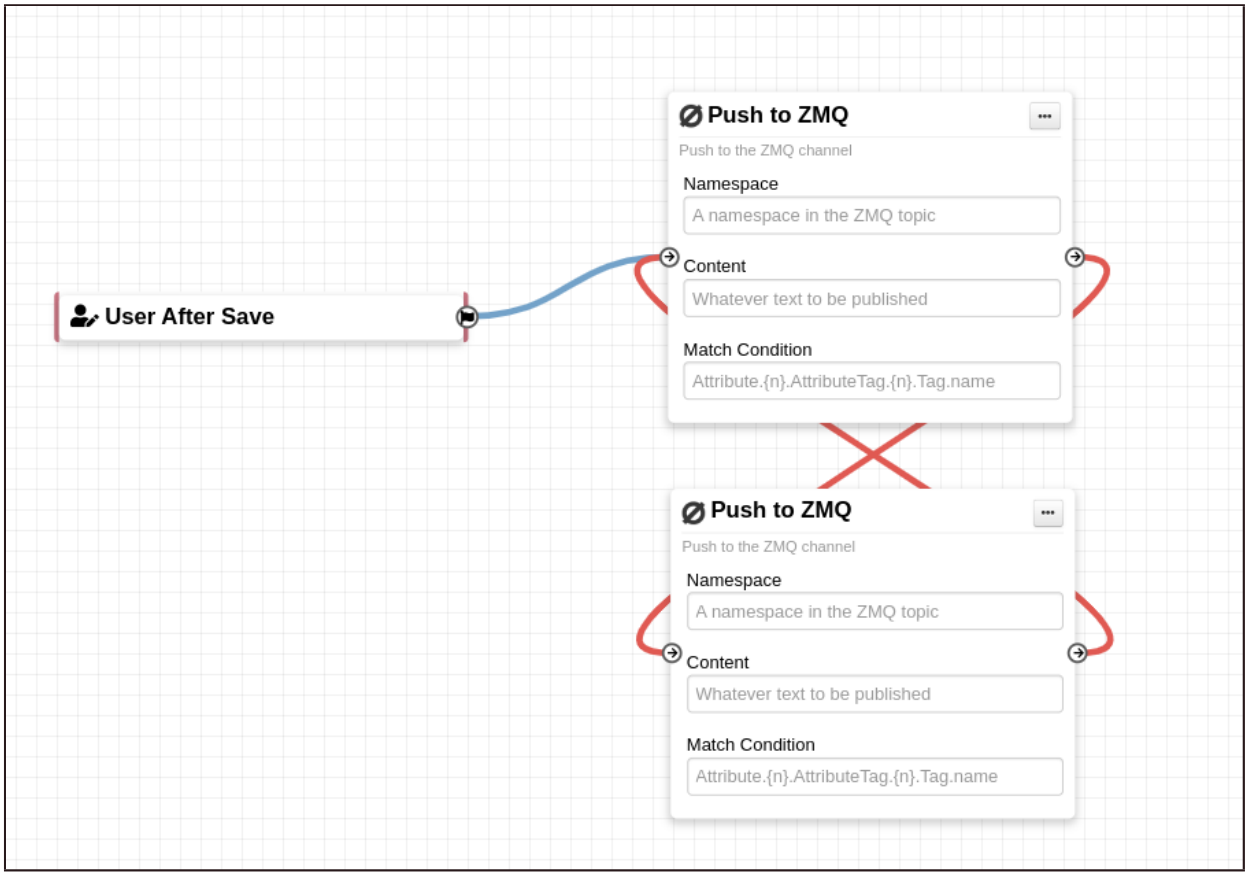
1. Prevent event publication **if tlp:red** tag
   - Send a mail to `admin@admin.test` about potential data leak
2. **else**, send a notification on Mattermost

Considerations when working with workflows

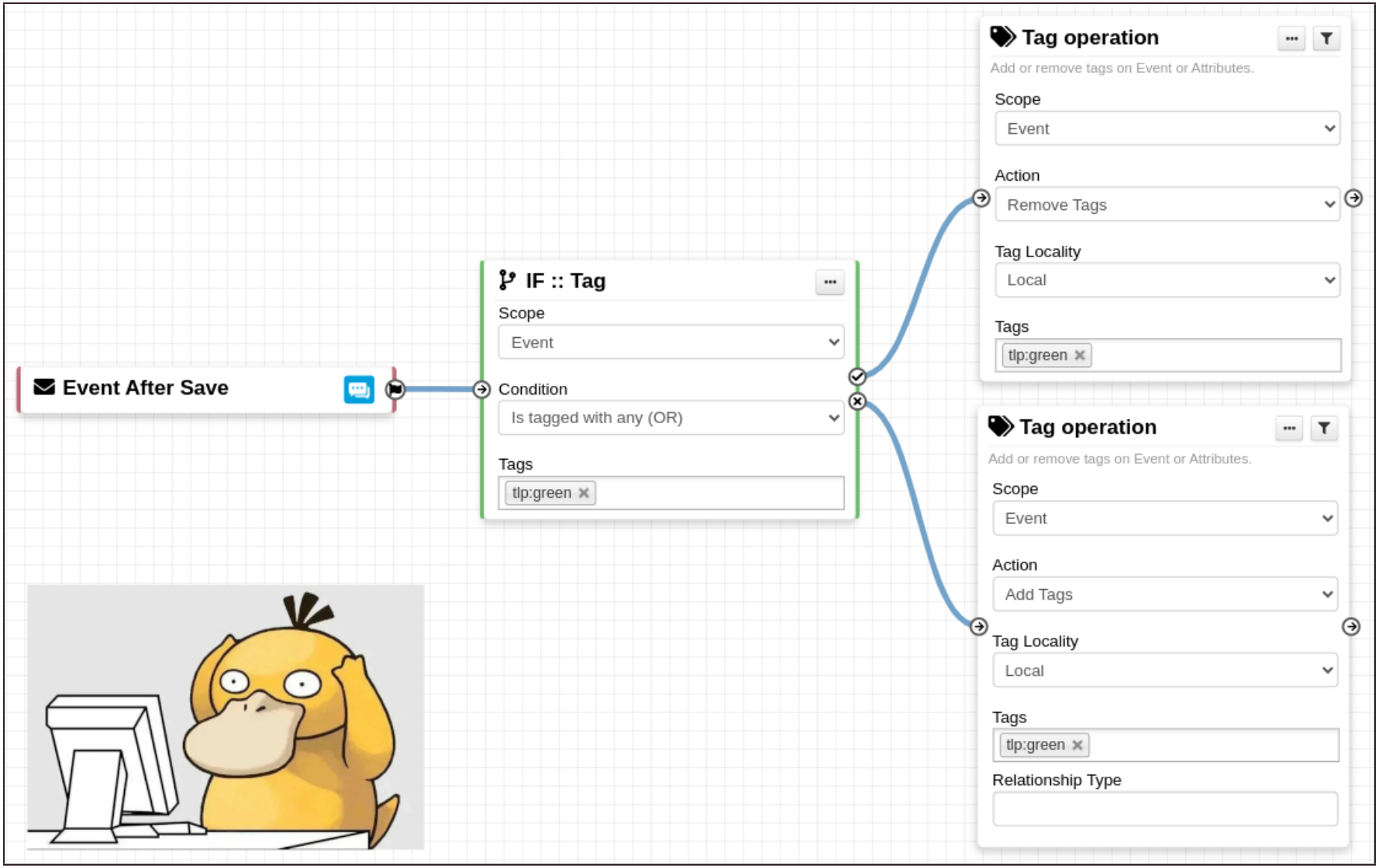
Objective: Overview of some common pitfalls

Working with the editor - Operations not allowed

Execution loop are not authorized



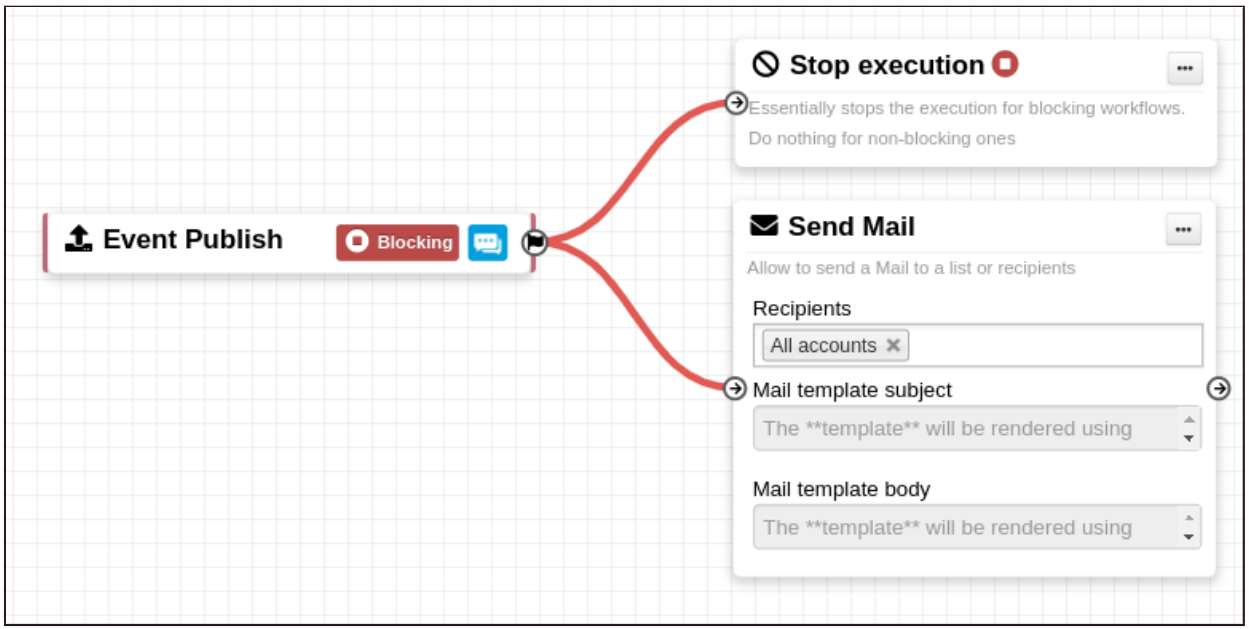
Recursive workflows



⚠Recursion: If an action re-run the workflow

Working with the editor - Operations not allowed

Multiple connections from the same output



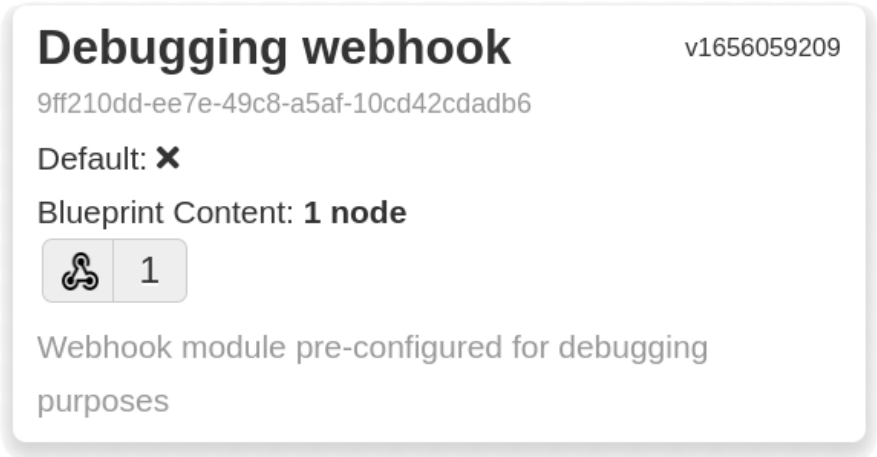
- Execution order not guaranted
- Confusing for users

Advanced usage

Objective: Overview of Blueprints, Data format and Filtering

Workflow blueprints

1. Blueprints allow to **re-use parts** of a workflow in another one
2. Blueprints can be saved, exported and **shared**



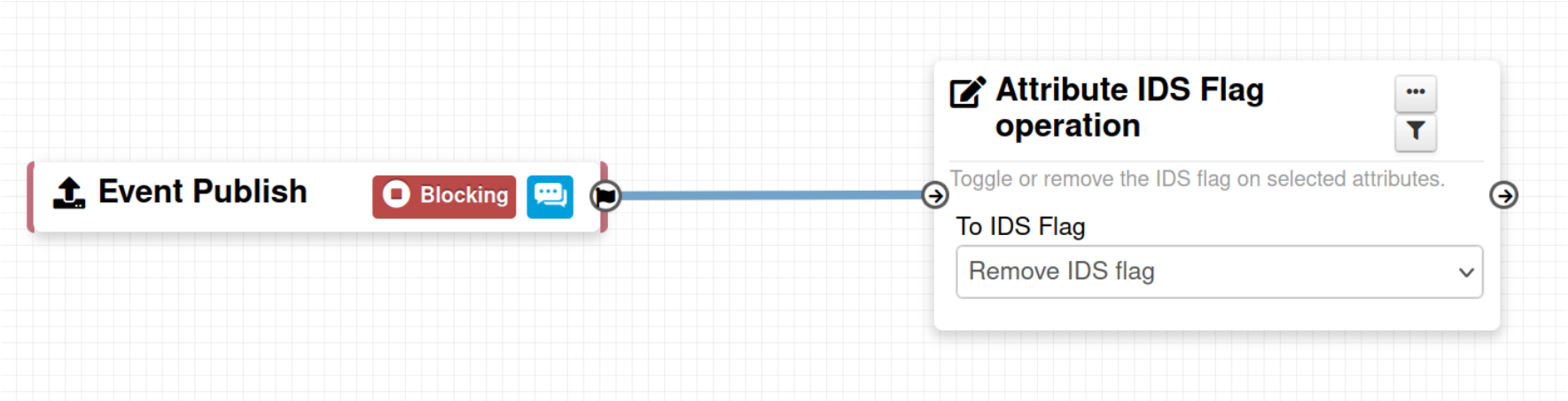
Blueprints sources: `MISP/misp-workflow-blueprints` repository[1]

- Block actions if any attributes have the `PAP:RED` or `tlp:red` tag
- Curation pipeline
- Enrich data from 3rd-party

[1] https://github.com/MISP/misp-workflow-blueprints

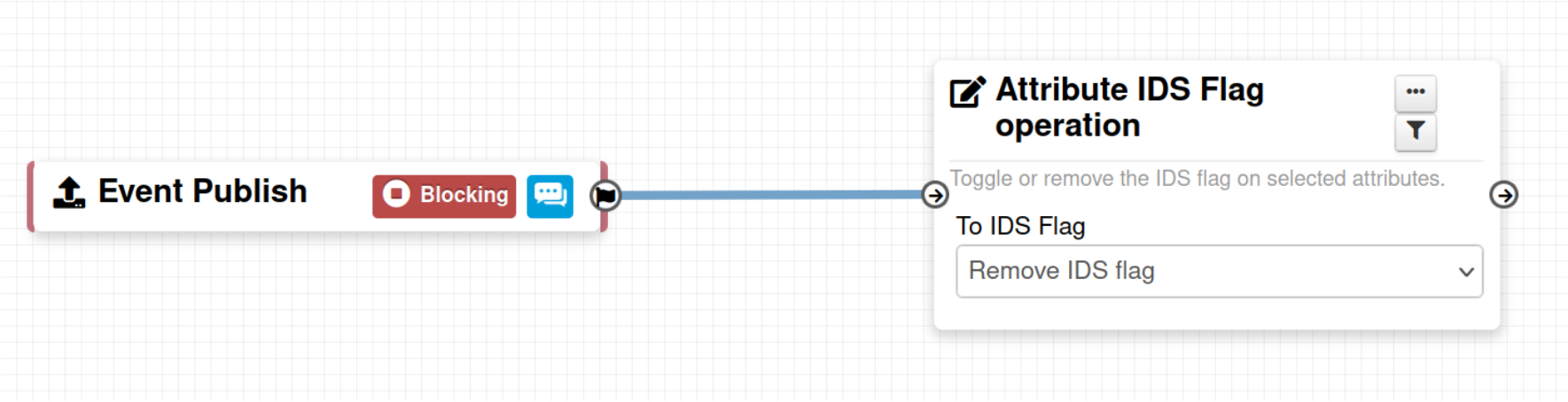
Fitlering data on which to apply a module

What is the outcome of executing this workflow?



Fitlering data on which to apply a module

What is the outcome of executing this workflow?

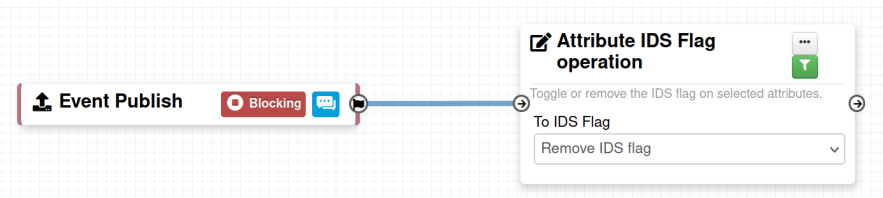


All Attributes get their `to_ids` turned off.

How could we force that action only on Attribute of type `comment`?

→ Hash path filtering!

Fitlering data on which to apply a module



Node Filtering

Element selector

Event._AttributeFlattened.{n}

Value

comment

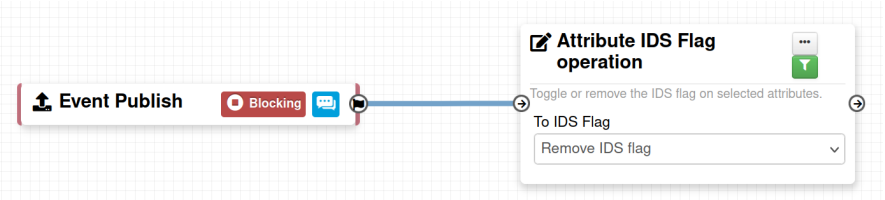
Operator

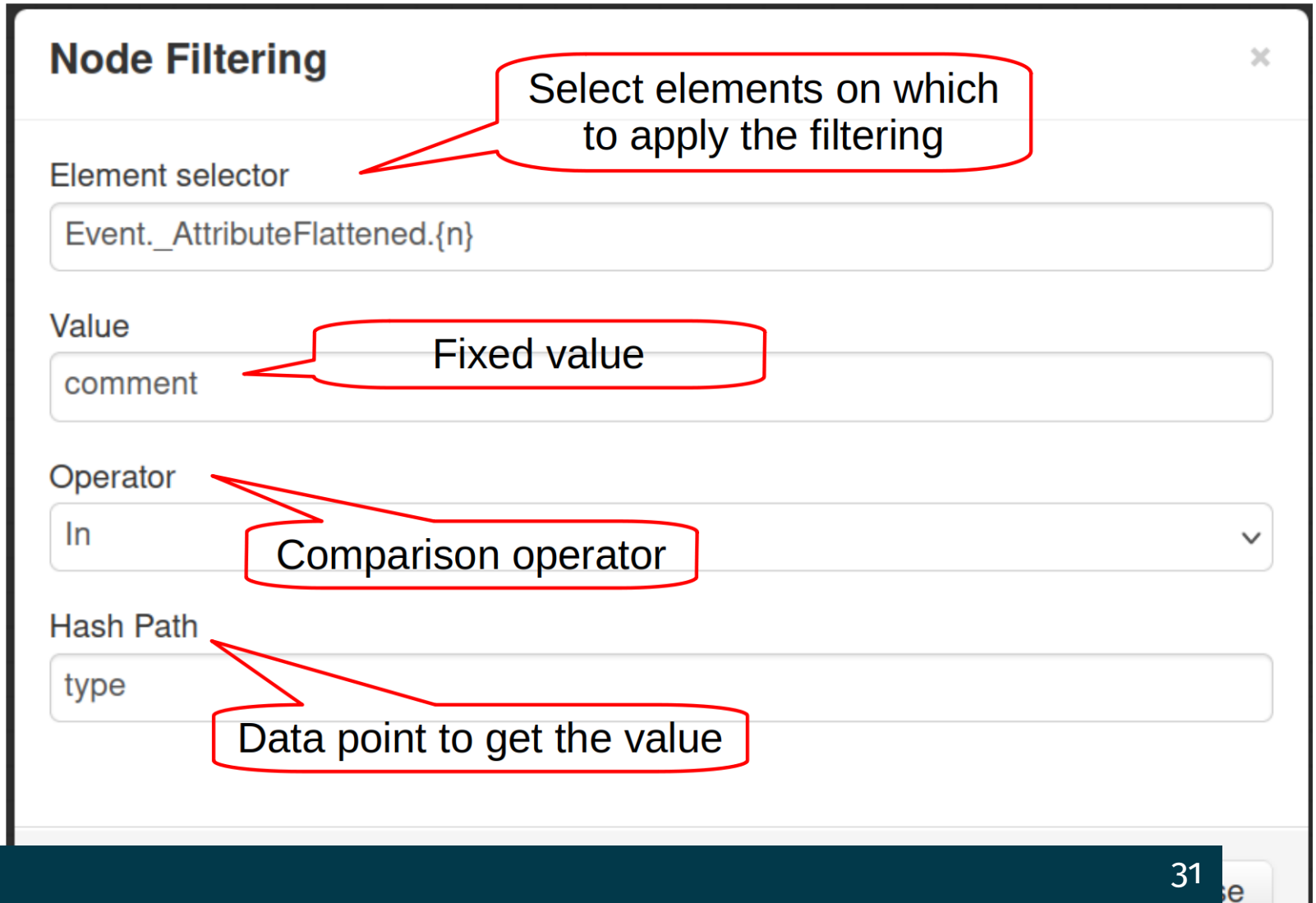
In

Hash Path

type

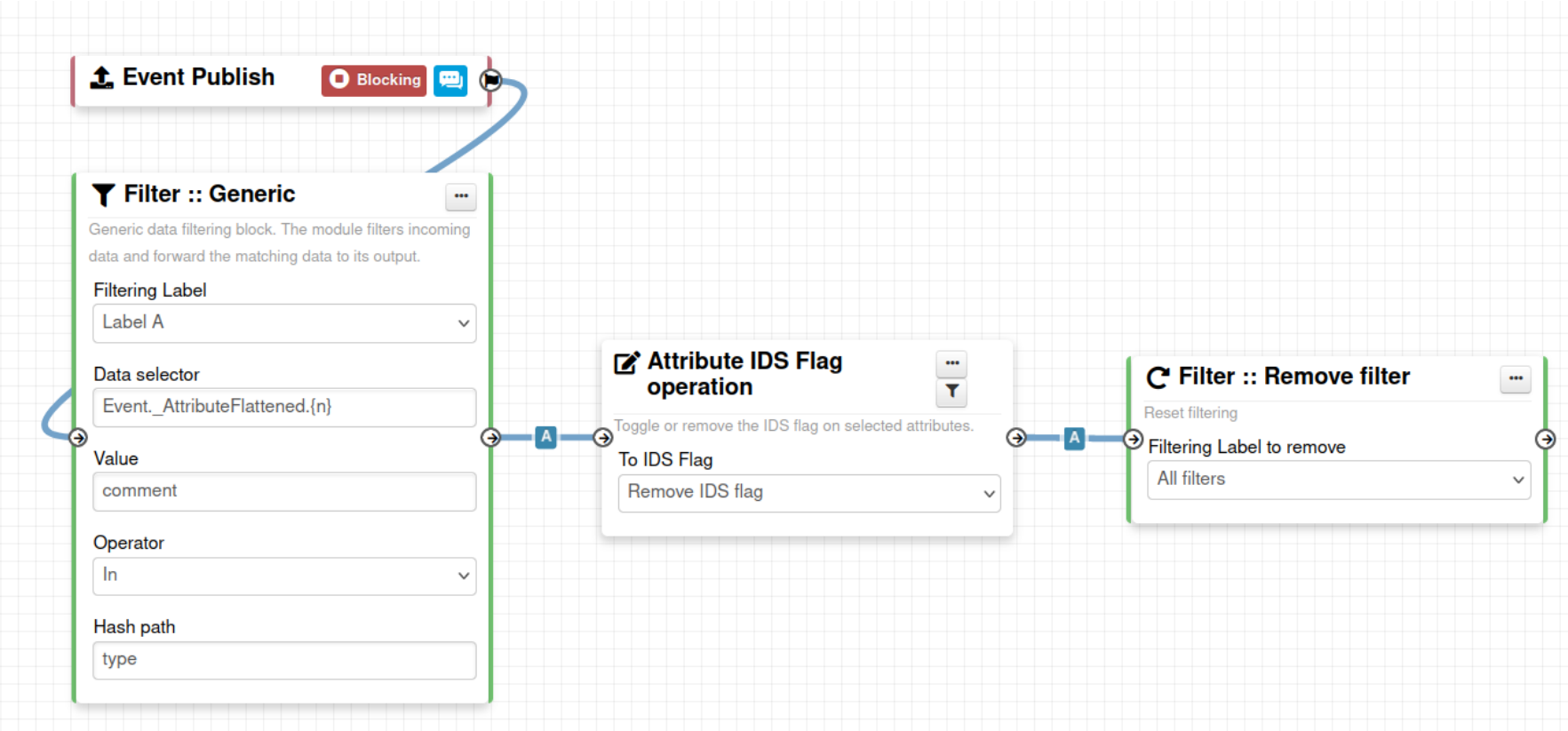
Fitlering data on which to apply a module





Fitlering data on which to apply on multiple modules

New feature as of **v2.4.171** allows setting filters on a path.



Should I migrate to MISP Workflows

I have automation in place using the API/ZMQ. Should I move to Workflows?

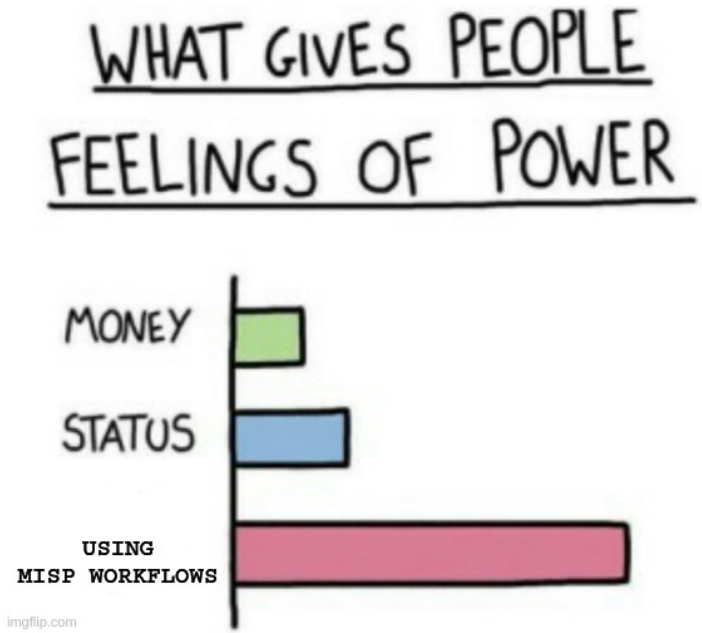
- I have a curation pipeline using the API, should I port it to workflows?
  - **No** in general, but WF can be used to start the curation process or perform simple pre-processing
- What if I want to **block** some actions
  - Put the blocking logic in the WF, keep the remaining outside
- Bottom line is **Keep it simple** for you to maintain

Future works

- More modules
- More modules
- More triggers
- Recursion prevention system

Final words

- Designed to **quickly** and **cheaply** integrate MISP in CTI pipelines
- **Beta** Feature unlikely to change. But still..
- Waiting for feedback!
  - New triggers?
  - New modules?



MISP
INTELLIGENCE SHARING

MISP-STIX Project

Python library to convert MISP <-> STIX

MISP core team
TLP:WHITE

MISP Project
https://www.misp-project.org/
MISP Training

MISP & STIX

- **Built-in integration**
  - Available from the UI
  - Accessible via restSearch
- Export & Import features
  - Export MISP data collections
  - Import STIX files
- Supported version
  - STIX 1.1.1 & 1.2
  - STIX 2.0 & 2.1

misp-stix - Key features

- MISP $\Longleftrightarrow$ STIX conversion
  - Used by MISP core to handle the conversion ability
  - Preserve as much content & context as possible
- Support all the STIX versions
  - **STIX 2.1 Support**
  - 1.1.1, 1.2, 2.0 Support enhanced

- **Mapping documentation**[1]
- Package available on PyPI[2]

[1] https://github.com/misp/misp-stix/tree/main/documentation#readme
[2] https://pypi.org/project/misp-stix/

Handling the conversion with a python library

- Integration in python code
  - Automation made easier by a close coupling with PyMISP
    - Export content from MISP

```
In [1]: import json
   ...: from misp_stix_converter import MISPtoSTIX21Parser
   ...: from pymisp import PyMISP
   ...: with open('tmp/config.json', 'r') as f:
   ...:     url, api_key, verify_cert = json.load(f)
   ...: misp = PyMISP(url, api_key, verify_cert)
   ...: misp.toggle_global_pythonify()
   ...: collection = misp.search(
   ...:     controller='attributes', page=1,
   ...:     type_attribute='ip-src', limit=10,
   ...:     tags=['tlp:white', 'tlp:clear']
   ...: )
   ...: parser = MISPtoSTIX21Parser()
   ...: parser.parse_misp_attributes(collection)
   ...: print(parser.bundle.serialize())
{"type": "bundle", "id": "bundle--a421897a-cafe-45ad-97ef-58761f6fac54", "objects": [{"type": "identity", "spec_ve
, "id": "identity--55f6ea65-aa10-4c5a-bf01-4f84950d210f", "created": "2015-09-14T15:40:21.000Z", "modified": "2015
21.000Z", "name": "MISP", "identity_class": "organization"}, {"type": "indicator", "spec_version": "2.1", "id": "i
e4cfe-21ac-46a7-9d82-06b3950d210b", "created_by_ref": "identity--55f6ea65-aa10-4c5a-bf01-4f84950d210f", "created":
07:15:10.000Z", "modified": "2014-10-03T07:15:10.000Z", "pattern": "[network-traffic:src_ref.type = 'ipv4-addr' AN
ffic:src_ref.value = '1.48.209.68']", "pattern_type": "stix", "pattern_version": "2.1", "valid_from": "2014-10-03T
kill_chain_phases": [{"kill_chain_name": "misp-category", "phase_name": "Network activity"}], "labels": ["misp:typ
, "misp:category=\"Network activity\"", "misp:to_ids=\"True\""]}, {"type": "indicator", "spec_version": "2.1", "id
--542e4cfe-05f4-46ab-b5b8-06b3950d210b", "created_by_ref": "identity--55f6ea65-aa10-4c5a-bf01-4f84950d210f", "crea
0-03T07:15:10.000Z", "modified": "2014-10-03T07:15:10.000Z", "pattern": "[network-traffic:src_ref.type = 'ipv4-add
k-traffic:src_ref.value = '1.73.227.172']", "pattern_type": "stix", "pattern_version": "2.1", "valid_from": "2014-
0Z", "kill_chain_phases": [{"kill_chain_name": "misp-category", "phase_name": "Network activity"}], "labels": ["mi
src\"", "misp:category=\"Network activity\"", "misp:to_ids=\"True\""]}, {"type": "indicator", "spec_version": "2.1
icator--542e4cfe-81c4-45f2-9e67-06b3950d210b", "created_by_ref": "identity--55f6ea65-aa10-4c5a-bf01-4f84950d210f",
2014-10-03T07:15:10.000Z", "modified": "2014-10-03T07:15:10.000Z", "pattern": "[network-traffic:src_ref.type = 'ip
network-traffic:src_ref.value = '1.162.58.214']", "pattern_type": "stix", "pattern_version": "2.1", "valid_from":
7:15:10Z", "kill_chain_phases": [{"kill_chain_name": "misp-category", "phase_name": "Network activity"}], "labels"
=\"ip-src\"", "misp:category=\"Network activity\"", "misp:to_ids=\"True\""]}, {"type": "indicator", "spec_version"
```

Handling the conversion with a python library

- Integration in python code
  - Automation made easier by a close coupling with PyMISP
    - Export content from MISP
    - Using the STIX return format directly

```
In [2]: import json
   ...: from misp_stix_converter import MISPtoSTIX21Parser
   ...: from pymisp import PyMISP
   ...: with open('tmp/config.json', 'r') as f:
   ...:     url, api_key, verify_cert = json.load(f)
   ...: misp = PyMISP(url, api_key, verify_cert)
   ...: misp.toggle_global_pythonify()
   ...: body = {
   ...:     'returnFormat': 'stix2', 'stix-version': '2.1',
   ...:     'type': 'ip-src', 'tags': ['tlp:white', 'tlp:clear'],
   ...:     'page': 1, 'limit': 10
   ...: }
   ...: print(misp.direct_call('/attributes/restSearch', body))
{'type': 'bundle', 'id': 'bundle--b8a39b06-219a-4f49-b46a-1ba30051a9bc', 'objects': [{'type': 'identity', 'spec_ve
, 'id': 'identity--55f6ea65-aa10-4c5a-bf01-4f84950d210f', 'created': '2015-09-14T15:40:21.000Z', 'modified': '2015
21.000Z', 'name': 'MISP', 'identity_class': 'organization'}, {'type': 'indicator', 'spec_version': '2.1', 'id': 'i
e4cfe-21ac-46a7-9d82-06b3950d210b', 'created_by_ref': 'identity--55f6ea65-aa10-4c5a-bf01-4f84950d210f', 'created':
07:15:10.000Z', 'modified': '2014-10-03T07:15:10.000Z', 'pattern': "[network-traffic:src_ref.type = 'ipv4-addr' AN
ffic:src_ref.value = '1.48.209.68']", 'pattern_type': 'stix', 'pattern_version': '2.1', 'valid_from': '2014-10-03T
kill_chain_phases': [{'kill_chain_name': 'misp-category', 'phase_name': 'Network activity'}], 'labels': ['misp:typ
'misp:category="Network activity"', 'misp:to_ids="True"']}, {'type': 'indicator', 'spec_version': '2.1', 'id': 'in
4cfe-05f4-46ab-b5b8-06b3950d210b', 'created_by_ref': 'identity--55f6ea65-aa10-4c5a-bf01-4f84950d210f', 'created':
7:15:10.000Z', 'modified': '2014-10-03T07:15:10.000Z', 'pattern': "[network-traffic:src_ref.type = 'ipv4-addr' AND
fic:src_ref.value = '1.73.227.172']", 'pattern_type': 'stix', 'pattern_version': '2.1', 'valid_from': '2014-10-03T
kill_chain_phases': [{'kill_chain_name': 'misp-category', 'phase_name': 'Network activity'}], 'labels': ['misp:typ
'misp:category="Network activity"', 'misp:to_ids="True"']}, {'type': 'indicator', 'spec_version': '2.1', 'id': 'in
4cfe-81c4-45f2-9e67-06b3950d210b', 'created_by_ref': 'identity--55f6ea65-aa10-4c5a-bf01-4f84950d210f', 'created':
7:15:10.000Z', 'modified': '2014-10-03T07:15:10.000Z', 'pattern': "[network-traffic:src_ref.type = 'ipv4-addr' AND
fic:src_ref.value = '1.162.58.214']", 'pattern_type': 'stix', 'pattern_version': '2.1', 'valid_from': '2014-10-03T
kill_chain_phases': [{'kill_chain_name': 'misp-category', 'phase_name': 'Network activity'}], 'labels': ['misp:typ
'misp:category="Network activity"', 'misp:to_ids="True"']}, {'type': 'indicator', 'spec_version': '2.1', 'id': 'in
                                                    d_by_ref': 'identity--55f6ea65-aa10-4c5a-bf01-4f84950d210f', 'cre
                                                    15:10.000Z', 'pattern': "[network-traffic:src_ref.type = 'ipv4-ad
```

Handling the conversion with a python library

- Integration in python code
  - Automation made easier by a close coupling with PyMISP
    - Converting STIX content and adding the resulting Event

```
In [1]: import json
   ...: from misp_stix_converter import ExternalSTIX2toMISPParser
   ...: from pathlib import Path
   ...: from pymisp import PyMISP
   ...: with open('tmp/config.json', 'r') as f:
   ...:     url, api_key, verify_cert = json.load(f)
   ...: misp = PyMISP(url, api_key, verify_cert)
   ...: misp.toggle_global_pythonify()
   ...: parser = ExternalSTIX2toMISPParser()
   ...: parser.parse_stix_content(
   ...:     'tmp/AA23-263A_#StopRansomware_Snatch_Ransomware.stix21.json'
   ...: )
   ...: event = misp.add_event(parser.misp_event)
   ...: event.id
Out[1]: 1424
```

    - Using the API endpoint directly

```
In [2]: params = {'galaxies_as_tags': 0, 'debug': 1}
   ...: response = misp.upload_stix(
   ...:     'tmp/AA23-187A.stix21.json', kw_params=params
   ...: )
   ...: response.json()['Event']['id']
Out[2]: '1425'
```

Handling the conversion with a python library

- Addressing the limitations of a MISP built-in integration
  - Export & import features available as a command-line application

```
oui chrisr3d ~/git/MISP/MISP-STIX-Converter
$ (git::dev) poetry run misp_stix_converter export -h
usage: misp_stix_converter export [-h] -f FILE [FILE ...] -v {1.1.1,1.2,2.0,2.1} [-s] [-m] [--output_dir OUTPUT_DIR] [-o OUTPUT_NAME] [--level {attribute,event}]
                                  [--format {json,xml}] [-n NAMESPACE] [-org ORG]

options:
  -h, --help            show this help message and exit
  -f FILE [FILE ...], --file FILE [FILE ...]
                        Path to the file(s) to convert.
  -v {1.1.1,1.2,2.0,2.1}, --version {1.1.1,1.2,2.0,2.1}
                        STIX specific version.
  -s, --single_output   Produce only one result file (in case of multiple input file).
  -m, --in_memory       Store result in memory (in case of multiple result files) instead of storing it in tmp files.
  --output_dir OUTPUT_DIR
                        Output path - used in the case of multiple input files when the `single_output` argument is not used.
  -o OUTPUT_NAME, --output_name OUTPUT_NAME
                        Output file name - used in the case of a single input file or when the `single_output` argument is used.

STIX 1 specific arguments:
  --level {attribute,event}
                        MISP data structure level.
  --format {json,xml}   STIX 1 format.
  -n NAMESPACE, --namespace NAMESPACE
                        Namespace to be used in the STIX 1 header.
  -org ORG              Organisation name to be used in the STIX 1 header.
oui chrisr3d ~/git/MISP/MISP-STIX-Converter
$ (git::dev) poetry run misp_stix_converter import -h
usage: misp_stix_converter import [-h] -f FILE [FILE ...] -v {1,2} [-s] [-o OUTPUT_NAME] [--output_dir OUTPUT_DIR] [-d DISTRIBUTION] [-sg SHARING_GROUP] [--galaxies_as_tags]

options:
  -h, --help            show this help message and exit
  -f FILE [FILE ...], --file FILE [FILE ...]
                        Path to the file(s) to convert.
  -v {1,2}, --version {1,2}
                        STIX major version.
  -s, --single_output   Produce only one MISP event per STIX file(in case of multiple Report, Grouping or Incident objects).
  -o OUTPUT_NAME, --output_name OUTPUT_NAME
                        Output file name - used in the case of a single input file or when the `single_output` argument is used.
  --output_dir OUTPUT_DIR
                        Output path - used in the case of multiple input files when the `single_output` argument is not used.
  -d DISTRIBUTION, --distribution DISTRIBUTION
                        Distribution level for the imported MIPS content.
  -sg SHARING_GROUP, --sharing_group SHARING_GROUP
                        Sharing group ID when distribution is 4.
  --galaxies_as_tags    Import MISP Galaxies as tag names instead of the standard Galaxy format.
oui chrisr3d ~/git/MISP/MISP-STIX-Converter
$ (git::dev)
```

Handling the conversion with a python library

- Addressing the limitations of a MISP built-in integration
  - Export & import features available as a command-line application

```
oui chrisr3d ~/git/MISP/MISP-STIX-Converter
$ (git::dev) poetry run misp_stix_converter import -v 2 -f tmp/debug/STIX/playbook_json/*.json
Failed parsing the following - and the related error message:
 - tmp/debug/STIX/playbook_json/automated-libra.json -  Invalid value for Indicator 'pattern': FAIL: Error found at line 1:0. input is missing s
 brackets
Successfully processed your files. Results available in:
 - tmp/debug/STIX/playbook_json/adept-libra.json.out
 - tmp/debug/STIX/playbook_json/agedlibra.json.out
 - tmp/debug/STIX/playbook_json/agent-tesla.json.out
 - tmp/debug/STIX/playbook_json/alloytaurus.json.out
 - tmp/debug/STIX/playbook_json/api-hammering-technique.json.out
 - tmp/debug/STIX/playbook_json/atlassian-confluence-CVE-2022-26134.json.out
 - tmp/debug/STIX/playbook_json/avoslocker-ransomware.json.out
 - tmp/debug/STIX/playbook_json/blackbasta-ransomware.json.out
 - tmp/debug/STIX/playbook_json/blackcat-ransomware.json.out
 - tmp/debug/STIX/playbook_json/bluesky-ransomware.json.out
 - tmp/debug/STIX/playbook_json/boggyserpens.json.out
 - tmp/debug/STIX/playbook_json/brute-ratel.json.out
 - tmp/debug/STIX/playbook_json/chromeloader.json.out
 - tmp/debug/STIX/playbook_json/clean-ursa.json.out
 - tmp/debug/STIX/playbook_json/cloaked-ursa.json.out
 - tmp/debug/STIX/playbook_json/clop-ransomware.json.out
 - tmp/debug/STIX/playbook_json/conti-ransomware.json.out
 - tmp/debug/STIX/playbook_json/crawling-taurus.json.out
 - tmp/debug/STIX/playbook_json/crooked-pisces.json.out
 - tmp/debug/STIX/playbook_json/darkside-ransomware.json.out
 - tmp/debug/STIX/playbook_json/dearcry-ransomware.json.out
 - tmp/debug/STIX/playbook_json/egregor-ransomware.json.out
 - tmp/debug/STIX/playbook_json/ekans-ransomware.json.out
 - tmp/debug/STIX/playbook_json/emotet.json.out
 - tmp/debug/STIX/playbook_json/evasive-serpens.json.out
 - tmp/debug/STIX/playbook_json/f5-big-ip-cve-2022-1388.json.out
 - tmp/debug/STIX/playbook_json/fighting-ursa.json.out
 - tmp/debug/STIX/playbook_json/golfing-taurus.json.out
 - tmp/debug/STIX/playbook_json/granite-taurus.json.out
 - tmp/debug/STIX/playbook_json/hellokitty-ransomware.json.out
 - tmp/debug/STIX/playbook_json/hermeticwiper.json.out
 - tmp/debug/STIX/playbook_json/hive-ransomware.json.out
```

Continuous Work in Progress & Improvement

- **Improve the import feature**
  - Handle different content design from different sources
  - Support of existing STIX objects libraries[3]
  - Support custom STIX format
  - **Handle validation issues**
- Continuous MISP $\Longleftrightarrow$ STIX mapping improvement
- More tests to avoid edge case issues

- Participating in Oasis CTI TC

[3] https://github.com/mitre/cti

How to report bugs/issues

- Github issues
  - **https://github.com/MISP/misp-stix/issues**
  - https://github.com/MISP/MISP/issues
- Please provide details
  - How did the issue happen
  - **Recommendation**: provide samples
- Any feedback welcome

To get in touch with us

- https://github.com/MISP/misp-stix
- https://github.com/MISP/misp-stix/tree/main/documentation

- https://github.com/MISP
- https://www.misp-project.org/
- https://twitter.com/MISPProject
- https://twitter.com/chrisred_68

MISP

INTELLIGENCE SHARING

Contributing MISP modules with an AI agent

From a precise brief to a tested pull request

CIRCL / MISP Project

https://www.misp-project.org/

Reference snapshot: 29 October 2026

What a successful contribution contains

- A module that fits existing discovery and execution.
- Correct MISP inputs, outputs, configuration and provenance.
- Tests that demonstrate behavior, not just Python syntax.
- Useful metadata, documentation and a focused pull request.

The agent assists development

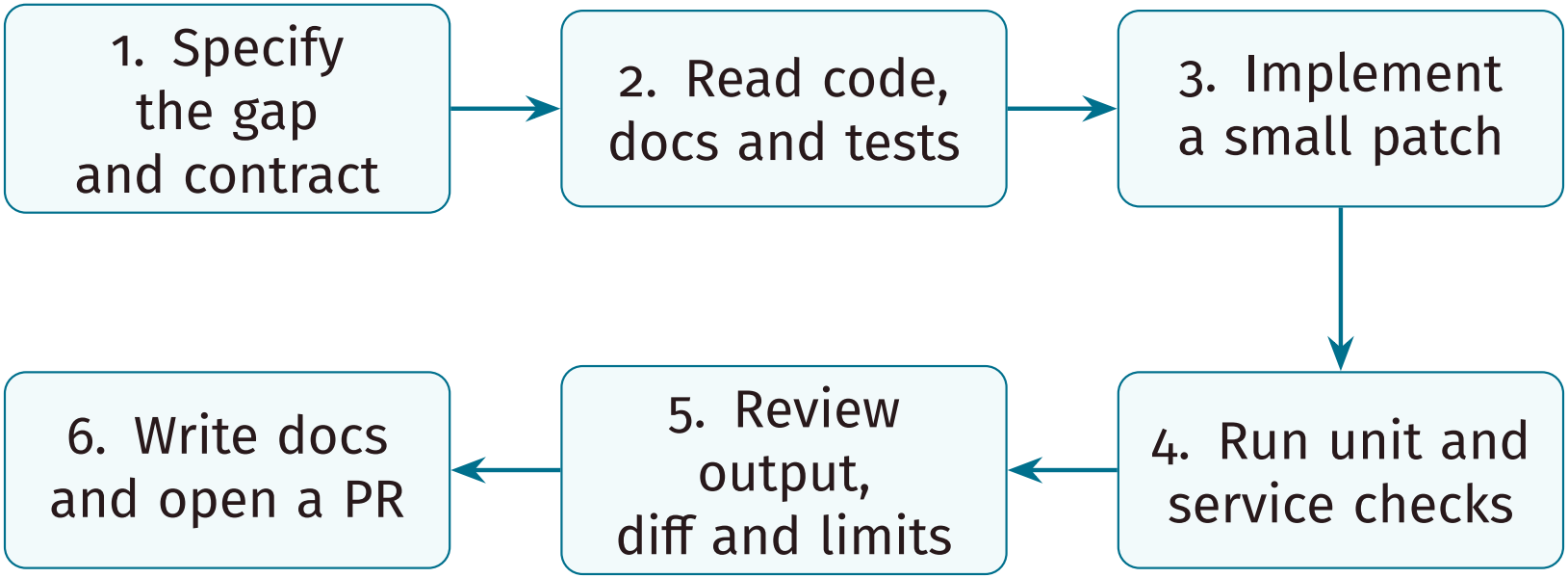
It is not a new module family or a reason to add a model service to the runtime implementation.

Keep ownership and evidence explicit

Contributor	Coding agent	Maintainer
Defines the need and boundaries.	Inspects code and prepares a plan and patch.	Assesses fit and maintenance cost.
Checks contracts and reviews results.	Runs permitted checks and reports actual outcomes.	Reviews the patch and evidence.
Owns the submitted contribution.	Identifies assumptions and limits.	Decides whether to merge.

A confident explanation from an agent is not execution evidence.

A repeatable development loop



Reference snapshot: misp-modules 3a903305cf37, reviewed 29 October 2026.

Read the existing contract

Give the agent the right repository context

- Contributor instructions, dependency declarations and CI.
- Loader and dispatcher: `misp_modules/__init__.py` and `__main__.py`.
- A recent module of the **same family**, plus its tests.
- Existing shared libraries and target-format builders.
- Official API/schema documentation, version and examples.

Is a new module needed?

Search existing capabilities before implementing a duplicate integration.

Select a family before writing code

Directory	Metadata type	Typical contract
`expansion`	expansion / hover	Observable or attribute to enrichment results.
`import_mod`	import	File, paste or report to parsed MISP data.
`export_mod`	export	Supplied MISP data to an encoded artifact.
`action_mod`	action	Workflow data and parameters to an operation/result.

Match the specific module's schema; this is not one universal request format.

Preserve the module interface

- `introspection()`: supported input/output or family-specific setup.
- `version()`: accurate capabilities, configuration and metadata.
- `dict_handler(request=...)`: dictionary execution path.
- `handler(q=...)`: supported legacy JSON execution path.
- Return fresh result/error objects; keep request state local.

Read the fallback behavior

The reviewed dispatcher catches `AttributeError` for legacy fallback. Do not use that exception as input validation.

Write down the request and result schema

- Supported attributes, event envelopes or file inputs?
- Settings in `config`, `params` or import options?
- Simplified expansion or `format: misp_standard`?
- Attributes, objects, an encoded file or action status?
- Missing data, empty results and partial failures?

Use existing MISP semantics

Map the provider's response into valid types, object templates and the selected family's contract.

Keep provenance and detection intent

- Preserve the source UUID and original evidence.
- Use PyMISP to build and serialize structured objects.
- Link derived objects to the correct source.
- Choose valid types and object relations.
- Do not silently promote contextual data to IDS indicators.

Reference exercise

Keep the original attribute; add a normalized `domain-ip` object with a `derived-from` reference.

Dependencies and configuration are not incidental

- Use the service's actual Python environment and dependency workflow.
- Add necessary dependencies without unrelated lockfile upgrades.
- Metadata describing a dependency does not install it.
- Distinguish request config, service defaults and module-specific environment variables.
- Keep credentials and private incidents out of prompts and fixtures.

Direct the agent and test behavior

A useful brief needs an explicit contract

1. Purpose, gap and unique identifier.
2. Family, supported inputs and expected outputs.
3. Official provider/schema version and configuration keys.
4. Side effects and permitted execution boundaries.
5. Acceptance tests, error cases and required evidence.

Reusable task prompt

Start with `AGENT_BRIEF.md`. It is a prompt to adapt, not an automatically installed agent policy.

Worked exercise: normalize a DNS name locally

- Input: `domain/hostname`, value and source UUID.
- Normalize case, one root dot and standard-library IDNA.
- Reject malformed labels, unsupported types, IP literals and bad UUIDs.
- Preserve the input; add a referenced MISP-standard object.
- No DNS lookup, provider account, model call or runtime agent framework.

Training scope

This is a contract exercise, not a proposal to add another normalizer upstream. A contribution must address a real project gap.

An acceptance request the agent can test

```
{
  "module": "training_domain_normalize",
  "attribute": {
    "type": "domain",
    "value": "B\u00dcCHER.Example.",
    "uuid": "13d8d18b-784a-42a0-9a58-a9cddbfda561",
    "to_ids": true,
    "comment": "Synthetic training source"
  }
}
```

Expected: `xn--bcher-kva.example` in a `domain-ip` object linked to the original source UUID.

Ask for small, reviewable implementation steps

1. Metadata and introspection.
2. Pure parsing/normalization and validation.
3. MISP result construction with references.
4. A thin provider boundary, if the task needs one.
5. Tests and documentation for the actual contract.

Inspect the diff

Reject unrelated framework changes, dependency churn, import-time networking and invented endpoint behavior.

If a module calls a provider, test the boundary

- Mock the provider client or HTTP boundary where it is used.
- Use documented fixtures: valid, empty and malformed responses.
- Test authentication errors, rate limits and bounded timeouts.
- Preserve references across multiple and partial results.
- Verify scope and acknowledgements for operations that write.

Keep output construction real

Mock external I/O, not the PyMISP objects that assertions need to inspect.

Make tests challenge the implementation

Dimension	What can go wrong
Input/schema	Missing fields, unsupported types, Unicode and length boundaries.
MISP semantics	Wrong relations, lost source UUIDs, changed detection flags.
State isolation	Caller or request mutates another result or shared metadata.
Errors	Exceptions hidden as empty success; errors ignored at HTTP 200.
Deployment	Service imports another package or skips the new file.

Report test levels honestly

- **Unit**: validation, parsing, serialization and isolation.
- **Service**: discovery and real `/query` requests.
- **MISP**: family settings, user access and UI ingestion.
- **Provider/target**: authorization, acknowledgements and activation.

Keep evidence scoped

Record commands, exit codes and counts. Distinguish passed, failed, skipped and unrun checks.

Verify and contribute

Check what the service actually imports

- Use an editable package install or explicit custom root.
- Install dependencies in the service environment.
- Restart the correct process.

```
# From this companion deck's directory:
misp-modules -l 127.0.0.1 -p 6666 \
  --custom "$PWD/examples/custom"
```

The root contains `expansion/training_domain_normalize.py`.

Verify discovery and dispatch

```
curl --fail --silent --show-error \
  http://127.0.0.1:6666/modules |
  jq '.[] | select(.name == "training_domain_normalize")'

python3 examples/verify_module.py \
  --url http://127.0.0.1:6666 -v
```

- Check family, capabilities, input format and config schema.
- Verify valid results and invalid-input errors through HTTP.
- Inspect `/openapi.json`, not just the handler.

HTTP 200 is not the acceptance criterion

- Inspect `error`/`errors` and the actual output envelope.
- Check references and source fields after serialization.
- Decode exports and inspect the target format.
- Check action outcomes appropriate to the operation.
- Verify MISP enablement separately from service discovery.

Use the project's quality and documentation workflow

- Read current CI, Poetry groups, extras and native prerequisites.
- Run relevant tests and formatting/lint under project configuration.
- Broaden checks when changes or failures warrant it.
- Supply complete metadata and user documentation.
- Keep dependency/lock edits tied to the implementation.

The brief supplies command templates; substitute actual paths and family.

What the pull request should explain

1. Integration gap and resulting behavior.
2. Input/output examples and configuration, without secrets.
3. Provider/schema version and side effects.
4. Actual validation, commands, counts and limitations.
5. Dependencies and documentation needed by users.

Keep it focused

Include the implementation, tests and documentation a maintainer needs. Remove incidental edits and generated noise.

Human review is part of the workflow

- Compare mappings with official API/schema documentation.
- Read validation, errors, references and config behavior.
- Inspect tests for missing cases, not only their exit status.
- Check for unexpected target writes or runtime model calls.
- Reconsider a duplicate or unnecessary upstream integration.

Workshop: brief, implement, verify

1. Adapt `AGENT_BRIEF.md` to the exercise.
2. Ask for a contract and acceptance plan.
3. Implement in a separate local directory; use the reference as an answer key.
4. Run unit and service checks and review source/object relationships.
5. Draft a PR description stating what was and was not tested.

The logic to preserve

1. Existing loader, family and dispatch contracts.
2. Types, object templates, source references and detection intent.
3. Explicit configuration, bounded I/O and useful failures.
4. Repeatable tests and installation/discovery evidence.
5. A contribution that a maintainer can review.

Resources for the next contribution

- `README.md`: setup, exercise specification and source audit.
- `AGENT_BRIEF.md`: reusable task prompt.
- `examples/`: reference, request and verifier.
- MISP/misp-modules and its current contribution guide.
- Module and installation documentation.
- The companion *MISP modules* introduction and local API lab.

MISP
INTELLIGENCE SHARING

Automation in MISP

Tutorial and Hands-On

Alexandre Dulaunoy & Sami Mokaddem

MISP Project
https://www.misp-project.org/

Content of the presentation

1. Automation in MISP
2. MISP API / PyMISP
3. PubSub channels (ZeroMQ)
4. MISP Workflows
   - Fundamentals
   - Demo with examples
   - Using the system
   - How it can be extended

Automation in MISP: What is available?

MISP API / PyMISP

- Needs CRON Jobs in place
- Potentially heavy for the server
- Not realtime

PubSub channels

- After the actions happen: No feedback to MISP
- Tougher to put in place & to share
- Full integration amounts to develop a new tool

MISP Workflow

- Realtime and allows feedback
- Easy to use and share but **can be dangerous**

MISP API / PyMISP - Fundamentals

Objective: Get to know how to use the MISP API PyMISP

MISP API / PyMISP - Demo

- Generate an API key
- RestClient overview
- MISP API Overview notebook[1]
- PyMISP Overview notebook[2]

[1] https://github.com/MISP/misp-training/blob/main/a.7-rest-API/Training%20-%20Using%20the%20API%20in%20MISP.ipynb

[2] https://github.com/MISP/PyMISP/blob/main/docs/tutorial/FullOverview.ipynb

PubSub channels (ZeroMQ) - Fundamentals

Objective: Learn how to setup realtime automation using the ZeroMQ channel

ZeroMQ channel - Demo

- What is ZeroMQ?
  - *N-to-N Asynchronous message-processing tasks*
  - *Publisher (MISP) and consumer (scripts)*
- Configuring ZeroMQ in MISP
- Integrating with the ZeroMQ of MISP

MISP Workflows - Fundamentals

Objective: Learn how to use the MISP Worklfow feature

Automation in MISP: What already exists?

MISP API / PyMISP

- Needs CRON Jobs in place
- Potentially heavy for the server
- Not realtime

PubSub channels

- After the actions happen: No feedback to MISP
- Tougher to put in place & to share
- Full integration amounts to develop a new tool

→ No way to **prevent** behavior
→ Difficult to setup **hooks** to execute callbacks

What type of use-cases are we trying to support?

- **Prevent** default MISP behaviors to happen
  - Prevent **publication of events** not passing sanity checks
  - Prevent **querying** thrid-party **services** with sensitive information
  - …
- **Hook** specific actions to run callbacks
  - **Automatically run** enrichment services
  - Modify data on-the-fly: False positives, enable CTI-Pipeline
  - Send notifications in a chat rooms
  - …

Simple automation in MISP made easy

- Why?
  - Everyone loves **simple automation**
  - **Visual** dataflow programming
  - Users want **more control**
- How?
  - **Drag & Drop** editor
  - Prevent actions **before they happen**
  - Flexible **Plug & Play** system
  - **Share** workflows, **debug** and **replay**

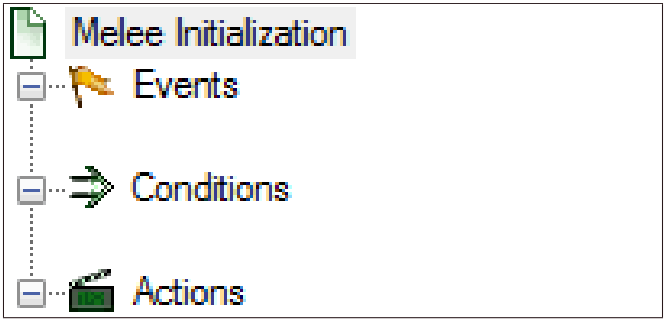
Example of use-cases

- **Notification** on specifc actions
  - New events matching criteria
  - New users
  - Automated alerts for high-priority IOCs
- **Extend** existing MISP behavior
  - Push data to another system
  - Automatic enrichment
  - Sanity check to block publishing / sharing
- **Hook** capabilities
  - Assign tasks and notify incident response team members
  - Run curation pipeline
- ...

Workflow - Fundamentals

Objective: Start with the foundation to understand the basics

How does it work



1. An **event** happens in MISP
2. Check if all **conditions** are satisfied
3. Execute all **actions**
   - May prevent MISP to complete its original event

What kind of events?

Events

- New MISP Event
- Attribute has been saved
- New discussion post
- New user created
- Query against third-party services
- ...

❓ Supported events in MISP are called **Triggers**
❓ A **Trigger** is associated with **1-and-only-1 Workflow**

Triggers currently available

Currently 17 triggers can be hooked. 3 being Blocking.

Triggers

List the available triggers that can be listened to by workflows.
Missing a trigger? Feel free to open a Github issue!
Documentation and concepts

« previous next »

All | attribute | event | object | others | post | user | Blocking | Enabled | Disabled

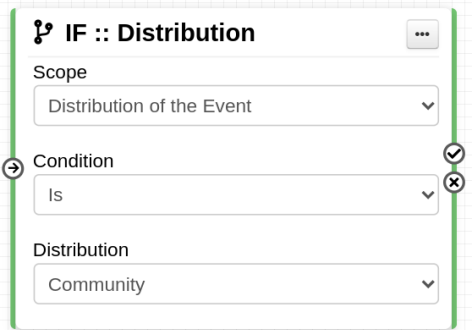
Trigger name	Scope	Trigger overhead	Run counter	Blocking Workflow	MISP Core format	Workflow ID	Last Update	Debug enabled	Enabled	Actions
Attribute After Save	attribute	high	83	×	✓	160	2022-08-03 09:00:41	☐	×	
Enrichment Before Query	others	low	1154	✓	✓	162	2022-10-17 12:35:57	☐	✓	
Event After Save	event	high	49	×	✓	175	2022-10-14 13:32:01	☐	✓	
Event After Save New	event	low	5	×	✓	182	2022-10-17 09:12:14	☑	✓	
Event After Save New From Pull	event	low	6	×	✓	183	2022-10-17 09:01:36	☑	✓	
Event Publish	event	low	126	✓	✓	180	2022-10-13 10:42:53	☑	✓	
Object After Save	object	high	35	×	✓	161	2022-08-05 07:12:52	☐	×	
Post After Save	post	low	36	×	×	176	2022-07-28 13:59:51	☐	×	
User After Save	user	low	0	×	×	181	2022-08-05 07:19:46	☐	×	
User Before Save	user	low	42	✓	×	158	2022-07-28 14:00:32	☐	×	

What kind of conditions?

Conditions

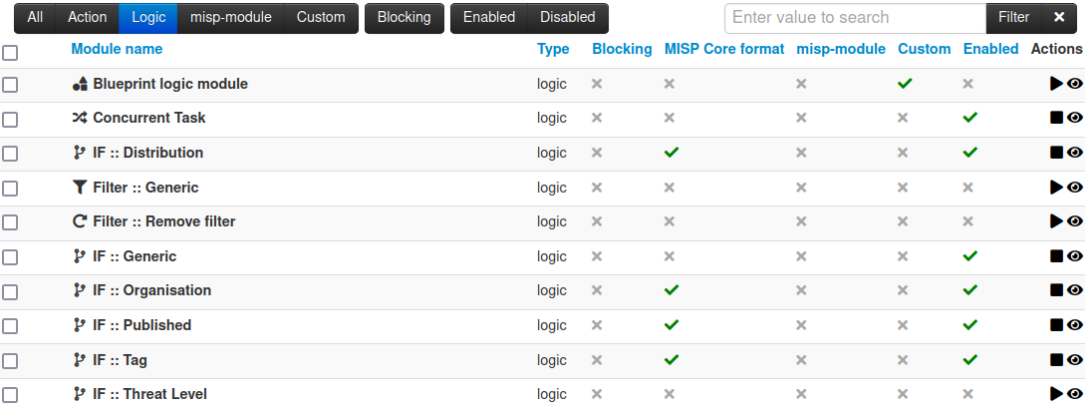
- A MISP Event is tagged with `tlp:red`
- The distribution of an Attribute is a sharing group
- The creator organisation is `circl.lu`
- Or any other **generic** conditions

These are also called **Logic modules**



Workflow - Logic modules

- **logic** modules: Allow to redirect the execution flow.
  - IF conditions
  - Delay execution



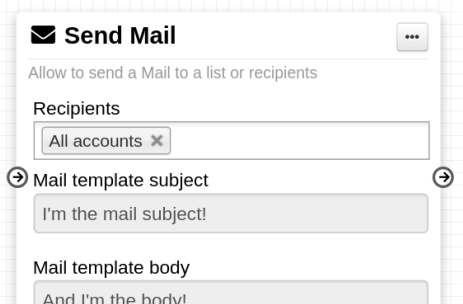
Module name	Type	Blocking	MISP Core format	misp-module	Custom	Enabled	Actions
Blueprint logic module	logic	✗	✗	✗	✓	✗	▶ 👁
Concurrent Task	logic	✗	✗	✗	✗	✓	■ 👁
IF :: Distribution	logic	✗	✓	✗	✗	✓	■ 👁
Filter :: Generic	logic	✗	✗	✗	✗	✗	▶ 👁
Filter :: Remove filter	logic	✗	✗	✗	✗	✗	▶ 👁
IF :: Generic	logic	✗	✗	✗	✗	✓	■ 👁
IF :: Organisation	logic	✗	✓	✗	✗	✓	■ 👁
IF :: Published	logic	✗	✓	✗	✗	✓	■ 👁
IF :: Tag	logic	✗	✓	✗	✗	✓	■ 👁
IF :: Threat Level	logic	✗	✗	✗	✗	✗	▶ 👁

What kind of actions?

Actions

- Send an email notification
- Perform enrichments
- Send a chat message on MS Teams
- Attach a local tag
- ...

These are also called **Action modules**



Workflow - Action modules

- **action** modules: Allow to executes operations
  - Tag operations
  - Send notifications
  - Webhooks & Custom scripts

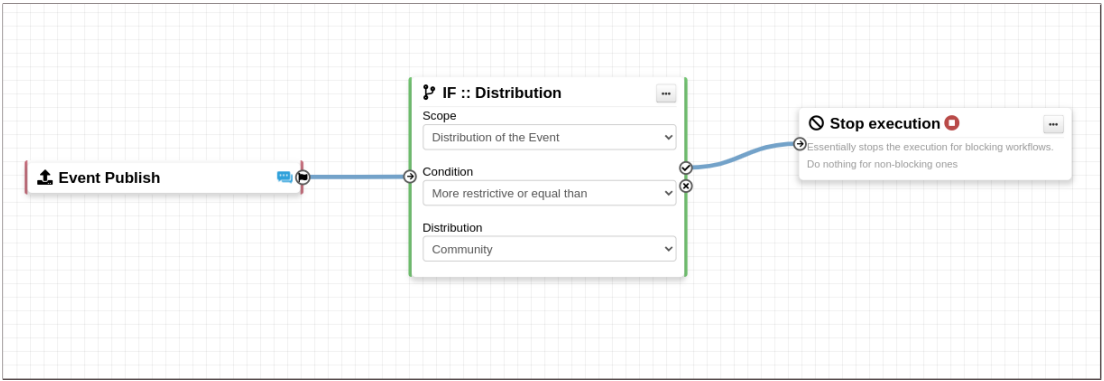
All | Action | Logic | misp-module | Custom | Blocking | Enabled | Disabled

Enter value to search | Filter

Module name	Type	Blocking	MISP Core format	misp-module	Custom	Enabled	Actions
Attach enrichment	action	×	✓	×	×	✓	
Attribute edition operation	action	×	✓	×	×	✓	
Attribute IDS Flag operation	action	×	✓	×	×	✓	
Blueprint action module	action	×	×	×	✓	✓	
Enrich Event	action	×	✓	×	×	✓	
mattermost	action	×	×	✓	×	✓	
MS Teams Webhook	action	×	×	×	×	✓	
Push to ZMQ	action	×	×	×	×	✓	
Send Log Mail	action	×	×	×	×	×	
Send Mail	action	×	×	×	×	✓	
Splunk HEC export	action	×	✓	×	×	×	
	action	✓	×	×	×	✓	

What is a MISP Workflow?

- Sequence of all nodes to be executed in a specific order



Workflow execution for Event publish

- An Event is about to be published
  - The workflow for the `event-publish` trigger starts
- Conditions are evaluated
  - They might change the path taken during the execution
- Actions are executed
  - **success**: Continue the publishing action

execute_workflow	Finished executing workflow for trigger `event-publish` (180). Outcome: success

  - **failure** | `blocked`: Stop publishing and log the reason

execute_workflow	Execution stopped. Node `stop-execution` (8) from Workflow `Workflow for trigger event-publish` (180) returned the following error: Execution stopped

Blocking and non-blocking

Two types of workflows:

Blocking Workflows

- Can prevent / block the original event to happen
- If a **blocking module** blocks the action

Non blocking Workflows execution doesn't block the request

- Executed in the background **after the request**:
  No way to prevent something that happened in the past

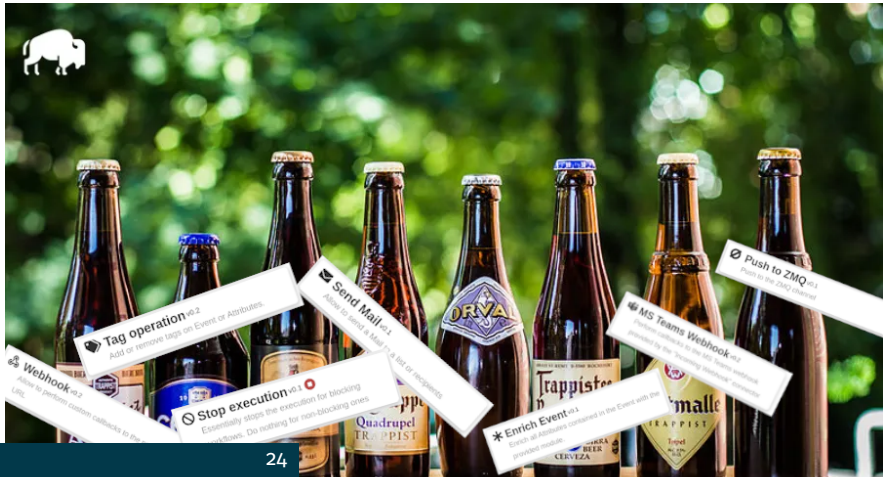
Sources of Workflow modules (0)

Currently 43 built-in modules.

- **Logic** module (11): built-in & **custom**
- **Action** module (32): built-in & **custom**

Sources of Workflow modules (1)

- Built-in **default** modules
  - Part of the MISP codebase
  - Get in touch if you want us to increase the selection (or merge PR!)



Sources of Workflow modules (2)

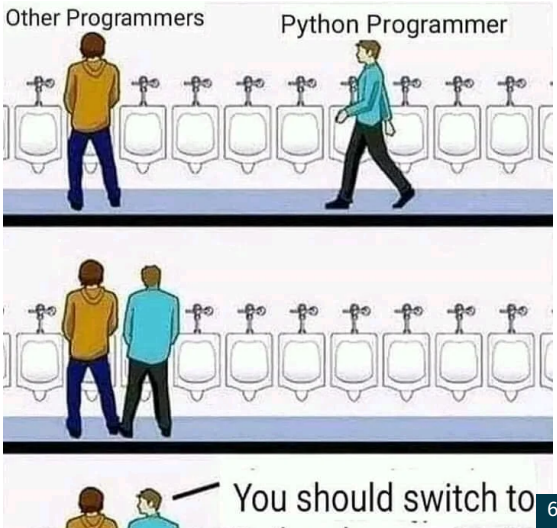
User-defined **custom** modules

- Written in PHP
- Extend existing modules
- MISP code reuse

Sources of Workflow modules (3)

Modules from the misp-module **enrichment service**

- Written in Python
- Can use any python libraries
- Plug & Play



Demo by examples

WF-1. Send an email to **all** when a new event has been pulled

WF-2. Block queries on 3rd party services when **tlp:red** or **PAP:red**

- **tlp:red**: For the eyes and ears of individual recipients only
- **PAP:RED**: Only passive actions that are not detectable from the outside

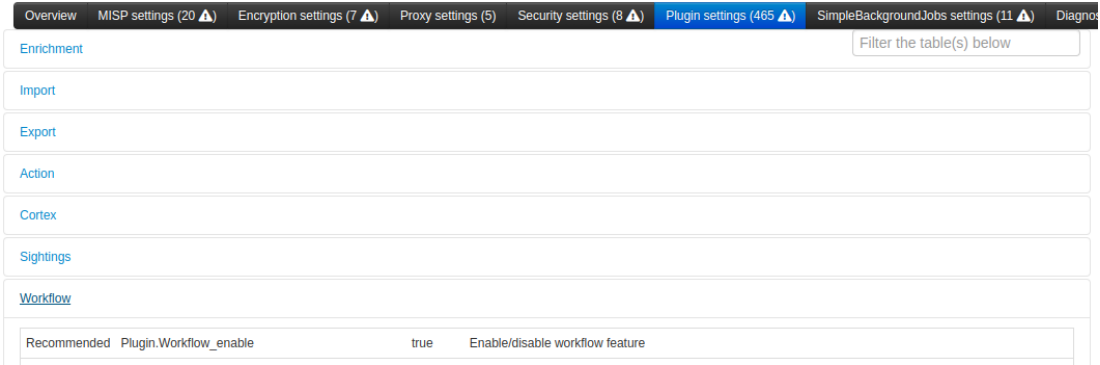
Workflow - Getting started

Objective: How to install & configure workflows

Getting started with workflows (2)

Review MISP settings:

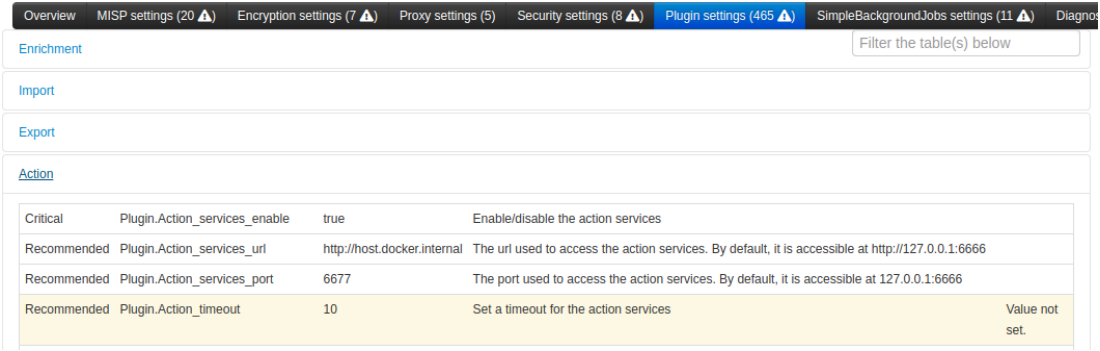
1. Make sure `MISP.background_jobs` is turned on
2. Make sure workers are up-and-running and healthy
3. Turn the setting `Plugin.Workflow_enable` on



Getting started with workflows (3)

Review MISP settings:

4. [optional:misp-module] Turn the setting `Plugin.Action_services_enable` on



Everything is ready?

Let’s see how to build a workflow!

Creating a workflow with the editor

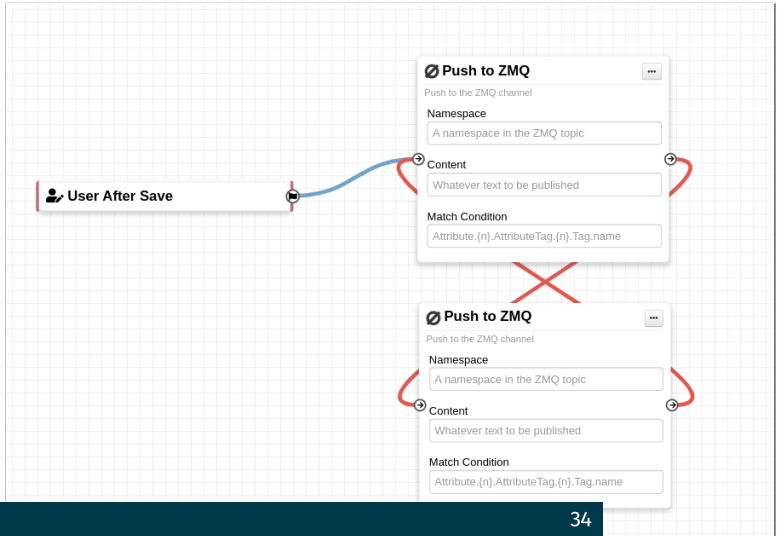
1. Prevent event publication if **tlp:red** tag
2. Send a mail to `admin@admin.test` about potential data leak
3. Otherwise, send a notification on **Mattermost**, **MS Teams**, **Telegram**, ...

Considerations when working with work-flows

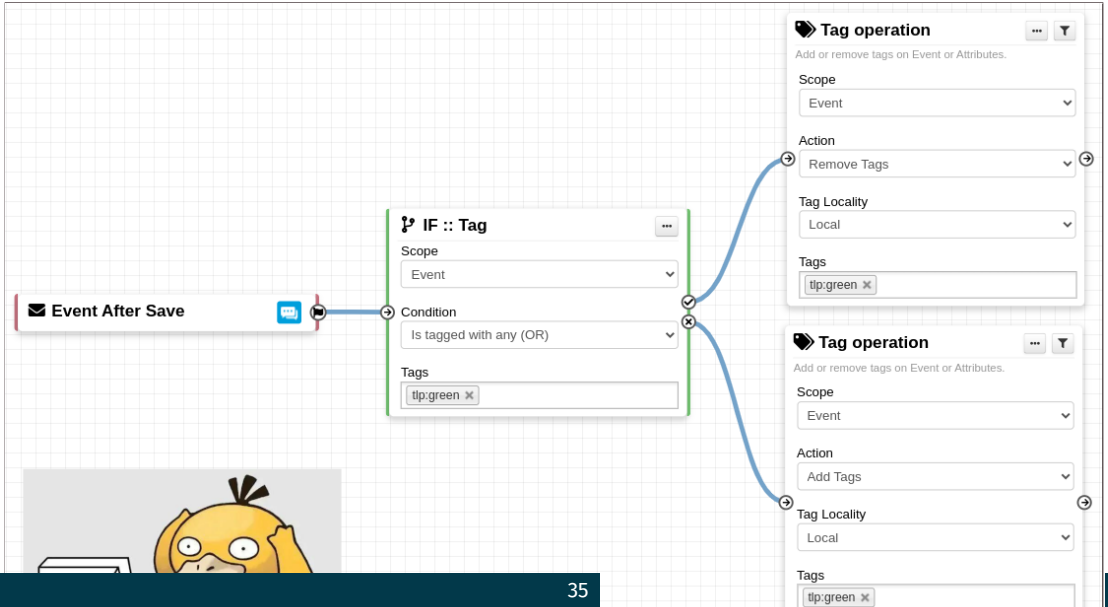
Objective: Overview of some common pitfalls

Working with the editor - Operations not allowed

Execution loop are not authorized

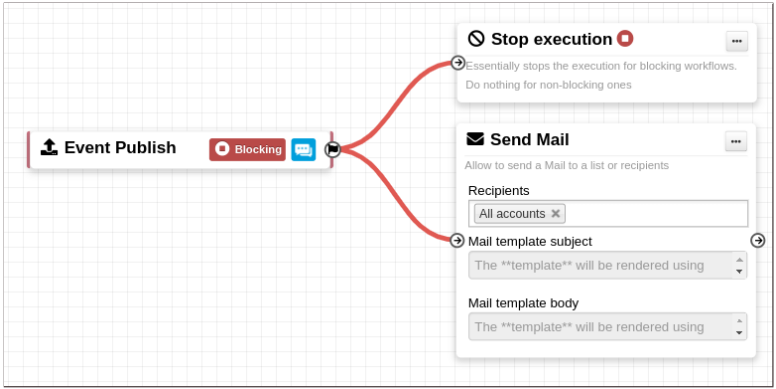


Recursive workflows



Working with the editor - Operations not allowed

Multiple connections from the same output



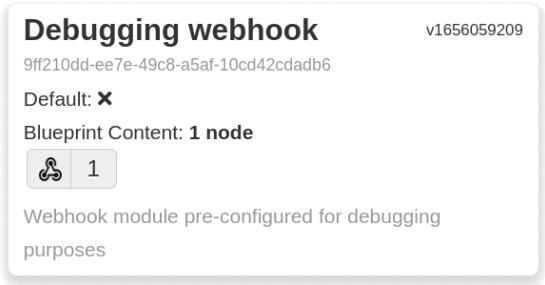
- Execution order not guaranted
- Confusing for users

Advanced usage

Objective: Overview of Blueprints, Data format and Filtering

Workflow blueprints

1. Blueprints allow to **re-use parts** of a workflow in another one
2. Blueprints can be saved, exported and **shared**



Blueprints sources:

1. Created or imported by users
2. From the `MISP/misp-workflow-blueprints` repository[3]

[3] https://github.com/MISP/misp-workflow-blueprints

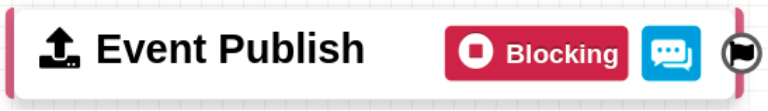
Workflow blueprints

Currently, 13 blueprints available in the official repo:

- Attach the `tlp:clear` tag on elements having the `tlp:white` tag
- Block actions if any attributes have the `PAP:RED` or `tlp:red` tag
- Disable `to_ids` flag for existing hash in *hashlookup*
- Set tag based on *BGP Ranking* maliciousness level
- ...

Built something cool? **Share you blueprints!**

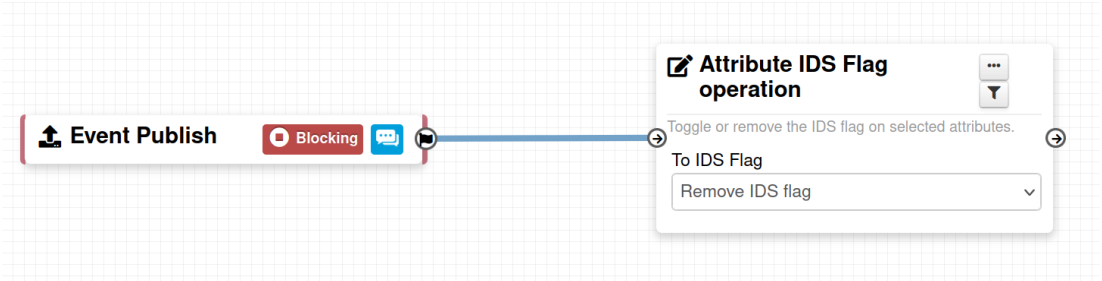
Data format in Workflows



- In most cases, the format is the **MISP Core format**
  - Attributes are **always encapsulated** in the Event or Object
  - ⚠ Event data might be partial
- And has **additional properties**
  - Additional key **`_AttributeFlattened`**
  - Additional key **`_allTags`**
  - Additional key **`inherited`** for Tags

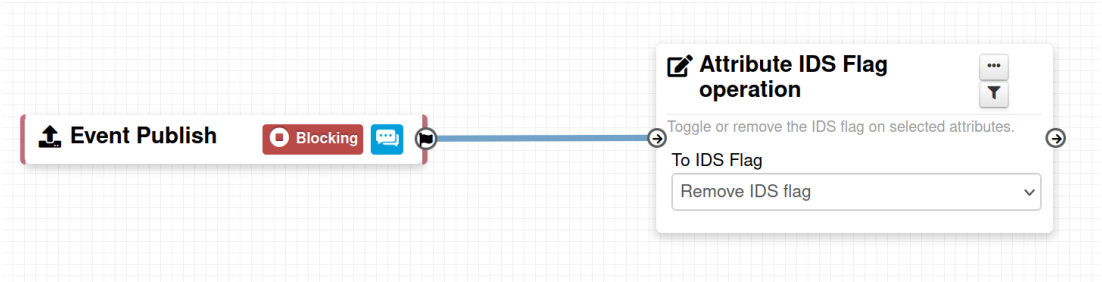
Fitlering data on which to apply a module

What happens when an Event is about to be published?



Fitlering data on which to apply a module

What happens when an Event is about to be published?



All Attributes get their `to_ids` turned off.

How could we force that action, only on Attribute of type `comment`?

→ Hash path filtering!

Hash path filtering (1)

Filtering and checking conditions using hash path expression.

```
{
    "person": { "name": "John" }
}
Hash Path 1 = 'person.name';
Result will be 'John'

[
    {'id': 123, 'name': 'fred', 'surname': 'bloggs'},
    {'id': 245, 'name': 'fred', 'surname': 'smith'},
    {'id': 356, 'name': 'joe', 'surname': 'smith'},
]
Hash Path 2 = '{n}.name';
Result will be ['fred', 'fred', 'joe']
```

Hash path filtering can be used to **filter** data **on the node** it is passed to or on the **execution path**.

Hash path filtering - Example

```
{
    "Event": {
        "uuid": ...
        "timestamp": ...
        "distribution": 1,
        "published": false,
        "Attribute": [
            {
                "type": "ip-src",
                "value": "8.8.8.8", ...
            },
            ...
        ],
        ...
    }
}
```

1. Access Event distribution
   - `Event.distribution`

Hash path filtering - Exercise (1)

```
{
    "Event": {
        "uuid": ...
        "distribution": 1,
        "published": false,
        "Attribute": [
            {
                "type": "ip-src",
                "value": "8.8.8.8", ...
            },
            ...
        ],
        ...
    }
}
```

2. Access Event published state

Hash path filtering - Exercise (1)

```
{
    "Event": {
        "uuid": ...
        "distribution": 1,
        "published": false,
        "Attribute": [
            {
                "type": "ip-src",
                "value": "8.8.8.8", ...
            },
            ...
        ],
        ...
    }
}
```

2. Access Event published state
   - `Event.published`

Hash path filtering - Exercise (2)

```
{
    "Event": {
        "uuid": ...
        "Attribute": [
            {
                "type": "ip-src",
                "value": "8.8.8.8", ...
            },
            {
                "type": "domain",
                "value": "misp-project.org", ...
            }
        ],
        ...
    }
}
```

3. Access all Attribute types
   - Hint: Use **{n}** to loop

Hash path filtering - Exercise (2)

```
{
    "Event": {
        "uuid": ...
        "Attribute": [
            {
                "type": "ip-src",
                "value": "8.8.8.8", ...
            },
            {
                "type": "domain",
                "value": "misp-project.org", ...
            }
        ],
        ...
    }
}
```

3. Access all Attribute types
   - Hint: Use **{n}** to loop
   - `Event.Attribute.{n}.type`

Hash path filtering - Exercise (3)

```
{
    "Event": {
        "Attribute": [
            {
                "value": "8.8.8.8",
                "Tag": [
                    { "name": "PAP:AMBER", ... },
                    { "name": "tlp:amber", ... },
                ], ...
            },
        ],
        ...
    }
}
```

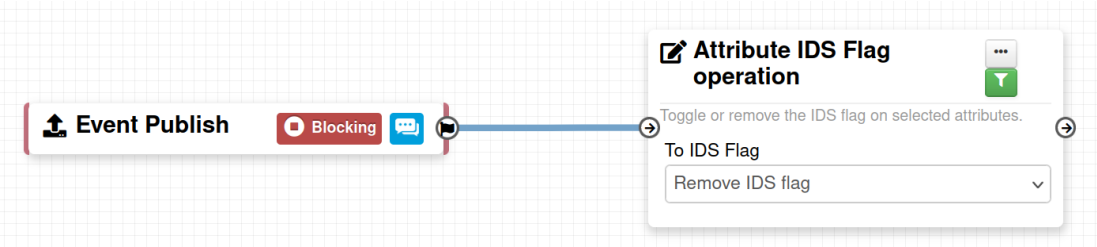
3. Access all Tag names attached to Attributes

Hash path filtering - Exercise (3)

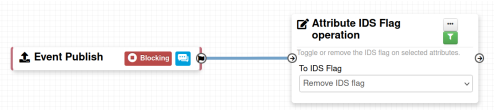
```
{
    "Event": {
        "Attribute": [
            {
                "value": "8.8.8.8",
                "Tag": [
                    { "name": "PAP:AMBER", ... },
                    { "name": "tlp:amber", ... },
                ], ...
            },
        ],
        ...
    }
}
```

3. Access all Tag names attached to Attributes
   - `Event.Attribute.{n}.Tag.{n}.name`

Fitlering data on which to apply a module



Fitlering data on which to apply a module



Node Filtering

Element selector

Event._AttributeFlattened.{n}

Value

comment

Operator

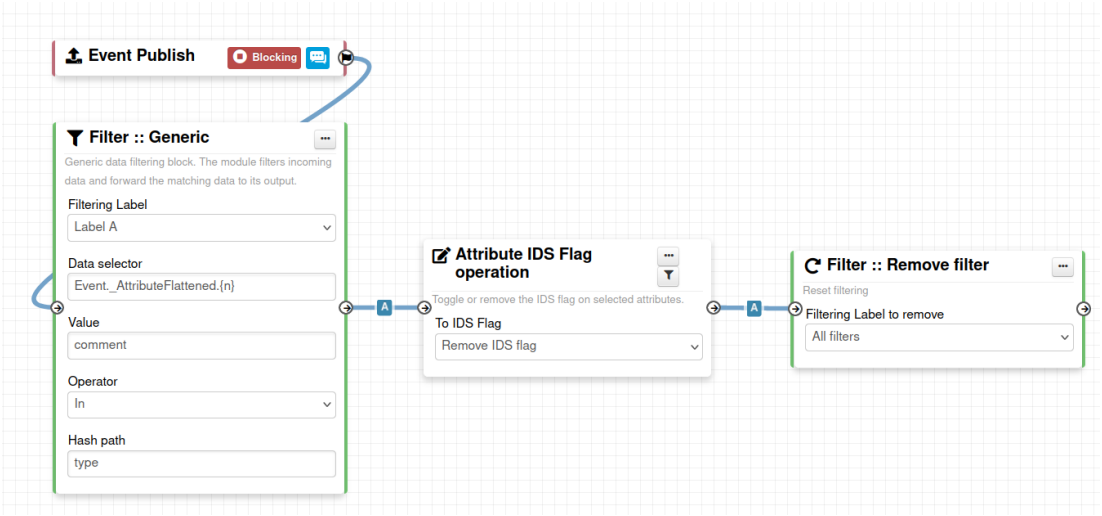
In

Hash Path

type

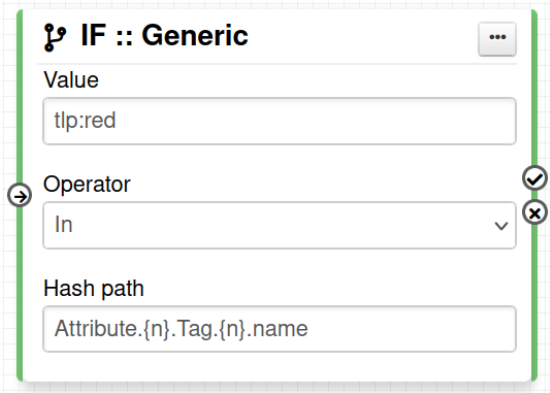
Fitlering data on which to apply on multiple modules

New feature as of **v2.4.171** allows setting filters on a path.

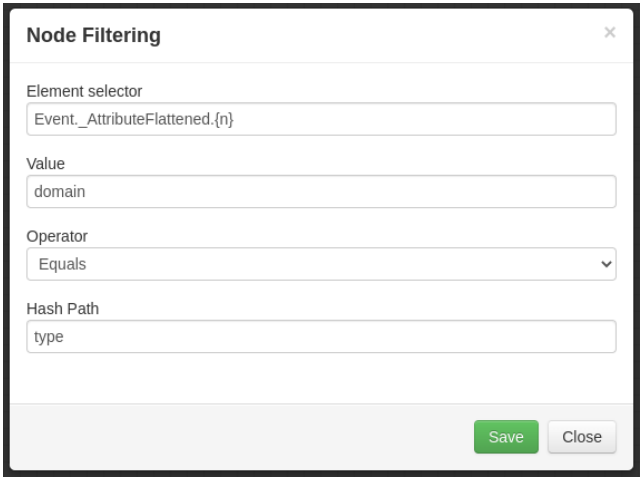


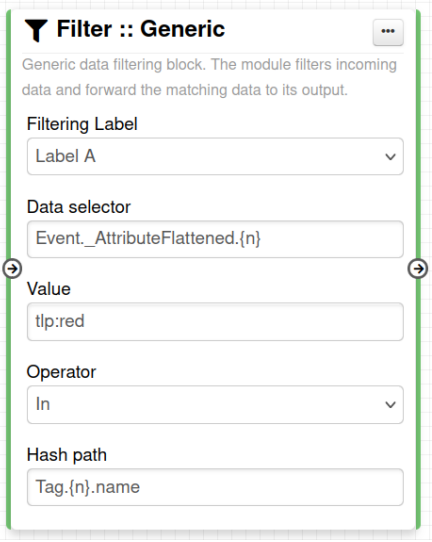
Hash path filtering (2)

```
{
    "Attribute": [
        {
            "type": "domain",
            "value": "cti-summit.org",
            "Tag": [
                {
                    "name": "tlp:red",
                    "colour": "#CC0033"
                }
            ]
        }
    ]
}
```



Hash path filtering (3)





Exercices

Debugging

Debugging Workflows: Log Entries

- Workflow execution is logged in the application logs:
  - `/admin/logs/index`
  - Note: Might be phased out as its too verbose
- Or stored on disk in the following file:
  - `/app/tmp/logs/workflow-execution.log`

Logs

« previous | next »

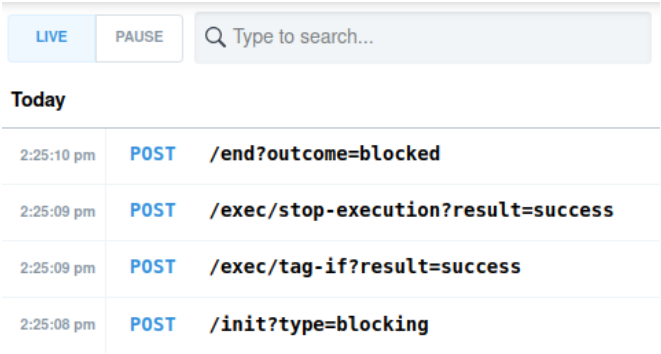
Emails | Authentication issues | MISP Update results | Setting changes | Warnings and errors

Id ↑	Email	Org	Created	Model	Model ID	Action	Title
49146	SYSTEM	SYSTEM	2022-08-01 07:34:40	Workflow	162	execute_workflow	Finished executing workflow for trigger `enrichment-before-query` (162). Outcome: success
49144	SYSTEM	SYSTEM	2022-08-01 07:34:39	Workflow	162	execute_workflow	Started executing workflow for trigger `enrichment-before-query` (162)

Debugging Workflows: Debug mode

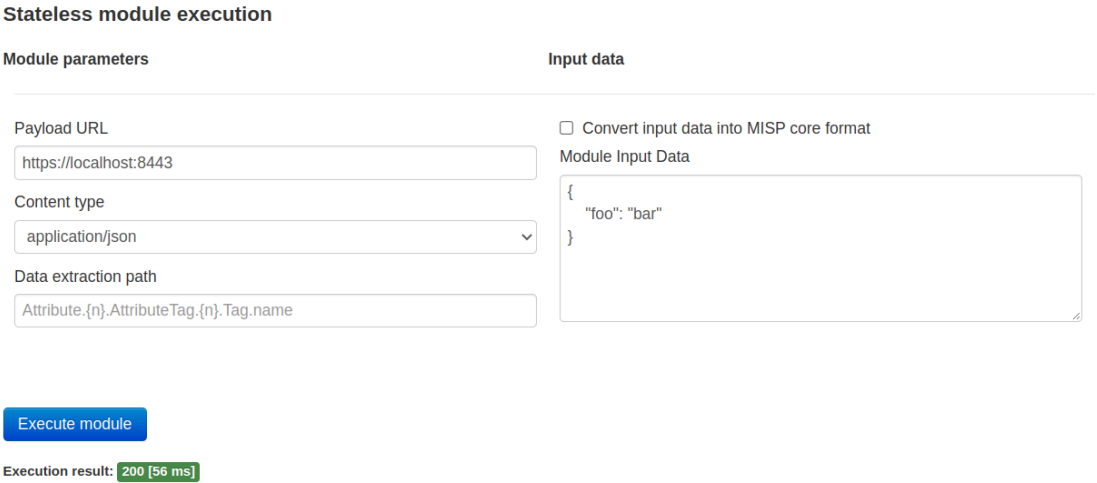
- The

can be turned on for each workflows
- Each nodes will send data to the provided URL
  - Configure the setting: `Plugin.Workflow_debug_url`
- Result can be visualized in
  - **offline**: `tools/misp-workflows/webhook-listener.py`
  - **online**: `requestbin.com` or similar websites



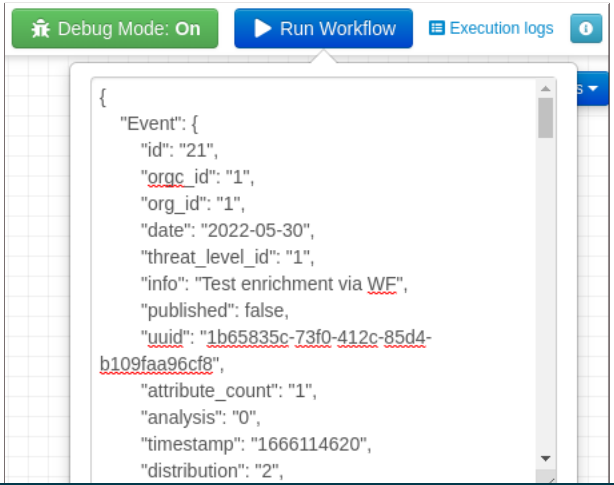
Debugging modules: Stateless execution

- Test custom modules with custom input



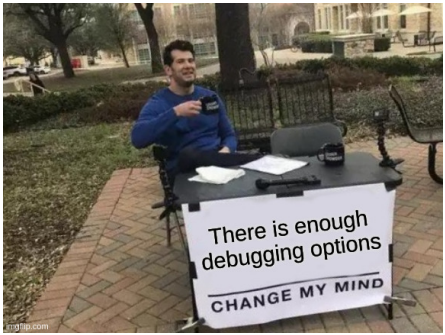
Debugging modules: Re-running workflows

- Try workflows with custom input
- Re-run workflows to ease debugging



Debugging options

- Workflow **execution and outcome**
- Module **execution and outcome**
- **Live** workflow debugging with module inspection
- **Re-running/testing** workflows with custom data
- **Stateless** module execution



Extending the system

Creating a new module in PHP

- **app/Lib/**`WorkflowModules/action/[module_name].php`
- Designed to be easilty extended
  - Helper functions
  - Module configuration as variables
  - Implement runtime logic
- Main benefits
  - Fast
  - Re-use existing functionalities
  - No need for misp-modules

Creating a new module in PHP

app > Lib > WorkflowModules > action > Module_blueprint_action_module.php > ...

```php
<?php
include_once APP . 'Model/WorkflowModules/WorkflowBaseModule.php';

class Module_blueprint_action_module extends WorkflowBaseModule
{
    public $is_blocking = false;
    public $disabled = true;
    public $id = 'blueprint-action-module';
    public $name = 'Blueprint action module';
    public $description = 'Lorem ipsum dolor, sit amet consectetur adipisicing elit.';
    public $icon = 'shapes';
    public $inputs = 1;
    public $outputs = 1;
    public $params = [];

    public function exec(array $node, WorkflowRoamingData $roamingData, array &$errors = [])
    : bool
    {
        parent::exec($node, $roamingData, $errors);
        // If $this->is_blocking == true, returning `false` will stop the execution.
        $errors[] = __('Execution stopped');
        return false;
```

Creating a new module in Python

- Similar to how other `misp-modules` are implemented
  - Helper functions
  - Module configuration as variables
  - Implement runtime logic
- Main benefits
  - Easier than PHP
  - Lots of libraries for integration

Creating a new module in Python

home > sami > git > misp-modules > misp_modules > modules > action_mod > testaction.py > ...

```python
import json

misperrors = {'error': 'Error'}

# config fields that your code expects from the site admin
moduleconfig = {
    'foo': {
        'type': 'string',
        'description': 'blablabla',
        'value': 'xyz'
    },
    'bar': {
        'type': 'string',
        'value': 'meh'
    }
};

# blocking modules break the exection of the chain of actions (such as publishing)
blocking = False

# returns either "boolean" or "data"
# Boolean is used to simply signal that the execution has finished.
# For blocking modules the actual boolean value determines whether we break execution
returns = 'boolean'

moduleinfo = {'version': '0.1', 'author': 'Andras Iklody',
              'description': 'This module is merely a test, always returning true. Triggers on event publishing.',
```

Should I migrate to MISP Workflows

I have automation in place using the API / ZMQ. Should I move to Workflows?

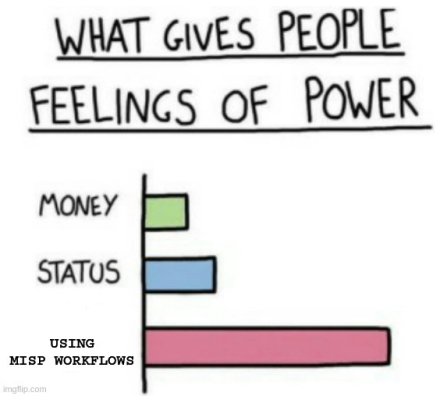
- I (have/am planning to create) a curation pipeline using the API, should I port them to workflows?
  - **No** in general, but WF can be used to start the curation process
- What if I want to **block** some actions
  - Put the blocking logic in the WF, the remaining outside
- Currently, workflows with **lots of node are not encouraged**
- Bottom line is **Keep it simple**

Future works

- More modules
- More modules
- More triggers
- More documentation
- Recursion prevention system

Final words

- Designed to **quickly** and **cheaply** integrate MISP in CTI pipelines
- **Beta** Feature unlikely to change. But still..
- Waiting for feedback!
  - New triggers?
  - New modules?
  - What's acheivable



MISP Concepts Cheat sheet

Glossary

Correlations: Links created automatically whenever an `Attribute` is created or modified. They allow interconnection between `Events` based on their attributes.

Correlation Engine: Is the system used by MISP to create correlations between `Attribute` 's value. It currently supports strict string comparison, SSDEEP and CDIR blocks matches.

Caching: Is the process of *fetching* data from a MISP instance or feed but only storing hashes of the collected values for correlation and look-up purposes.

Delegation: Act of transfering the ownership of an `Event` to another organisation while hidding the original creator, thus providing anonymity.

Deletion (hard/soft): *Hard deletion* is the act of removing the element from the system; it will not perform revocation on other MISP instances. *Soft deletion* is the act flagging an element as deleted and propagating the revocation among the network of connected MISP instances.

Extended Event: `Event` that extends an existing `Event` , providing a combined view of the data contained in both `Events` . The owner of the extending `Event` is the organisation that created the extension. This allows anyone to extend any `Events` and have total control over them.

`Galaxy` **Matrix**: Matrix derived from `Galaxy Clusters` belonging to the same `Galaxy` . The layout (pages and columns) is defined at the `Galaxy` level and its content comes from the `Galaxy Clusters` meta-data themselves.

Indicators: `Attribute` containing a pattern that can be used to detect suspicious or malicious activity. These `Attributes` usually have their `to_ids` flag enabled.

Orgc / Org: *Creator Organisation* (**Orgc**) is the organisation that created the data and the one allowed to modify it. *Owner Organisation* (**Org**) is the organisation owning the data on a given instance and is allowed to view it regardless of the distribution level. The two are not necessarily the same.

Publishing: Action of declaring that an `Event` is ready to be synchronised. It may also send e-mail notifications and makes it available to some export formats.

Pulling: Action of using a user on a remote instance to fetch the accessible data and storing it locally.

Pushing: Action of using an uplink connection via a *sync. user* to send data to a remote instance.

Synchronisation: Is the exchange of data between two (or more) MISP instances throught the *pull* or *push* mechanisms.

Sync. filtering rule: Can be applied on a synchronisation link for both the *pull* and *push* mechanisms to block or allow data to be transfered.

Sync. User: Special role of a user granting addional sync permissions. The recommanded way to setup *push* synchronisation is to use *sync users*.

Proposals: Are a mechanism to propose modications to the creating organisations (**Orgc**). If a path of connected MISP instances exists, the `Proposal` will be synchronised allowing the creator to accept or discard it.

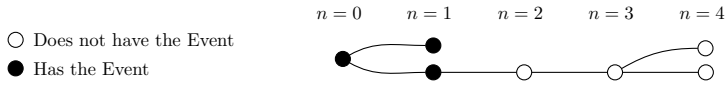
Distribution

Controls who can see the data and how it should be synchronised.

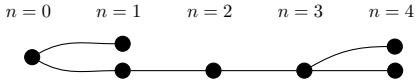
Organisation only: Only members of your organisation

This community: Organisations on this MISP instance

Connected Communities: Organisations on this MISP instance and those on MISP instances synchronising with this one. Upon receiving data, the distribution will be downgraded to `This community` to avoid further propagation. ($n \leq 1$)



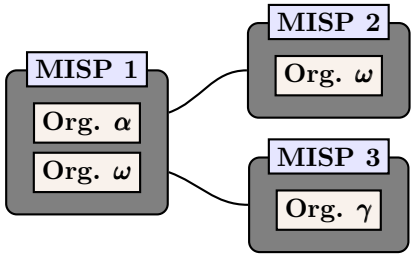
All Communities: Anyone having access. Data will be freely propagated in the network of connected MISP instances. ($n = \infty$)



Sharing Groups: Distribution list that exhaustively keeps track of which organisations can access the data and how it should be synchronised.

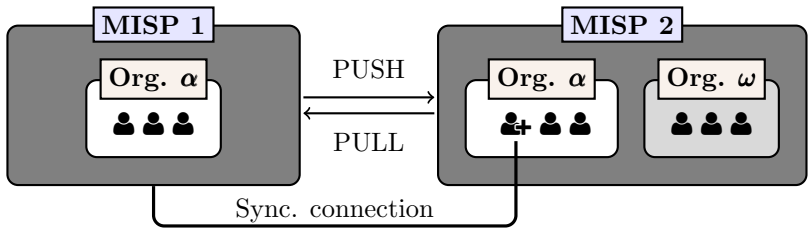
`Sharing Group` configuration	
Organisations	Org. α Org. ω Org. γ
Instances*	MISP 1 MISP 2 MISP 3

*Or enable roaming mode instead



Synchronisation

*The act of **sharing** where everyone can be a consumer and/or a producer.* A one way synchronisation link between two MISP instances. Organisation α created a *sync user* on MISP 2 and noted down the generated API Key. A synchronisation link can be created on MISP 1 using the API Key and the organisation of the *sync user*. At that point, MISP 1 can *pull* data from MISP 2 and *push* data to MISP 2.



MISP Data Model Cheat Sheet

- Context such as `Taxonomies` or `Galaxy Clusters` can be attached to the element
- Has a distribution level
- ⇄ Can be synchronised to/from other instances

Event

Encapsulations for contextually linked information.

Purpose: Group datapoints and context together. Acting as an envelop, it allows setting distribution and sharing rules for itself and its children.
Usecase: Encode incidents/events/reports/...
▶ `Events` can contain other elements such as `Attributes` , `MISP Objects` and `Event Reports` .
▶ The distribution level and any context added on an `Event` (such as `Taxonomies`) are propagated to its underlying data.

Attribute

Basic building block to share information.

Purpose: Individual data point. Can be an indicator or supporting data.
Usecase: Domain, IP, link, sha1, attachment, ...
▶ `Attributes` cannot be duplicated inside the same `Event` and can have `Sightings` .
▶ The difference between an indicator or supporting data is usualy indicated by the state of the attribute's `to_ids` flag.

MISP Object

Advanced building block providing `Attribute` compositions via templates.

Purpose: Groups `Attributes` that are intrinsically linked together.
Usecase: File, person, credit-card, x509, device, ...
▶ `MISP Objects` have their attribute compositions described in their respective template. They are instanciated with `Attributes` and can `Reference` other `Attributes` or `MISP Objects` .
▶ MISP is not required to know the template to save and display the object. However, *edits* will not be possible as the template to validate against is unknown.

↗ Object Reference

Relationships between individual building blocks.

Purpose: Allows to create relationships between entities, thus creating a graph where they are the edges and entities are the nodes.
Usecase: Represent behaviours, similarities, affiliation, ...
▶ `References` can have a textual relationship which can come from MISP or be set freely.

Sightings

Means to convey that an `Attribute` has been seen.

Purpose: Allows to add temporality to the data.
Usecase: Record activity or occurence, perform IoC expiration, ...
▶ `Sightings` are the best way to express that something has been seen. They can also be used to mark *false positives*.

Event Report

Advanced building block containing formated text.

Purpose: Supporting data point to describe events or processes.
Usecase: Encode reports, provide more information about the `Event` , ...
▶ `Event Reports` are markdown-aware and include a special syntax to reference data points or context.

Proposals

Clone of an `Attribute` containing information about modification to be done.

Purpose: Allow the correction or the creation of `Attributes` for `Events` your organisation does not own.
Usecase: Disable the IDS flag, Correct errors
▶ As `Proposals` are sync., if the creator organisation is connected to the MISP instance from where the `Proposal` has been created, it will be able to either *accept* or *discard* it.

𝒯 Taxonomies

Machine and human-readable labels standardised on a common set of vocabularies.

Purpose: Enable efficent classification globally understood, easing consumption and automation.
Usecase: Provide classification such as: TLP, Confidence, Source, Workflows, Event type, ...
▶ Even though MISP allows the creation of free-text tags, it's always preferable to use those coming from `Taxonomies` , if they exists.

Galaxies

Act as a container to group together context described in `Galaxy Clusters` by their type.

Purpose: Bundle `Galaxy Clusters` by their type to avoid confusion and to ease searches.
Usecase: Bundle types: Exploit-Kit, Preventive Measures, ATT&CK, Tools, Threat-actors, ...

Galaxies Clusters

Kownledge base items used as tags with additional complex meta-data aimed for human consumption.

Purpose: Enable description of complex high-level information for classification.
Usecase: Extensively describe elements such as: threat actors, countries, technique used, ...
▶ `Galaxy Clusters` can be seen as an enhanced `Taxonomy` as they can have meta-data and relationships with other `Galaxy Clusters` .
▶ Any `Galaxy Clusters` can contain the following:

- `Cluster Elements`: Key-Value pair forming the meta-data.

Example: `Country:LU`, `Currency:Dollar`, `Synonym:APT28`, `refs:https://*`, ...

- `Cluster Relations` (⇄): Enable the creation of relationships between one or more `Galaxy Clusters` .

Example: Threat actor `X is similar` to threat actor `Y` with `high-likelyhood`.

Analyst Notes

Text element that can be attached to many element

Purpose: Share and add an analysis to any MISP data
Usecase: Describe information about specific details, annotate elements
▶ Any user can attach `Analyst Notes` to data they don't own. For example: `Events` , `Attributes` , `Galaxy Clusters` , ⋯
▶ The note is actually attached to the target's UUID

Analyst Opinions

Text element with a numerical opinion that can be attached to many element

Purpose: Share and add an opinion to any MISP data
Usecase: Provide feedback to third-parties, Coordinate and Collaborate
▶ Basically the same as a `Analyst Note`
▶ The numerical value of the `Analyst Opinion` is $\in [0, 100]$. where 50 is the neutral point. Any values < 50 are considered negatives, values > 50 are considered positives.

Analyst Relationships

Link between two entities using a verb

Purpose: Create a relationship between elements
Usecase: Manually create correlation link, add similarities
▶ Basically the same as a `Analyst Note` but includes the target element
▶ Example could be an `Event` → `Event` relationship where one is *Suspected to be part of the same campaign based on HUMINT sources*

Element Collection

Group element into collection

Purpose: Allow groupping multiple elements into a single collection
Usecase: Grouping `Events` together if they are part of the same campaing

Representation of an incident in MISP

Failed spear-phishing attempt

UUID	28b1cd2e-46a7-4ee2-a364-c3d26451b089
Date	2021-12-09
Creator Org.	CIRCL.lu
Distribution	Connected Communities
Published	✓

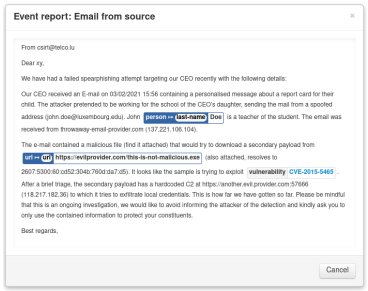
Galaxies

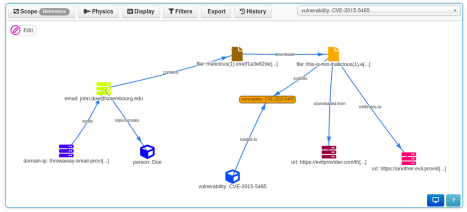
- **Sector**
  - Telecoms
- **Country**
  - luxembourg
- **Attack Pattern**
  - Spearphishing Attachment - T1566.001
  - Phishing - T1566

Taxonomies

- workflow:state="draft"
- tlp:amber
- PAP:RED
- phishing:techniques="email-spoofing"
- phishing:distribution="spear-phishing"

> Intelligence Visualization Widgets





> Attributes

☐	2021-11-25	Payload delivery	ip-src	118.217.182.3
☐	2021-11-25	Payload delivery	url	https://evilprovider.com/this-is-not-malicious.exe

...

> Objects

	2021-12-09	**Object name:** file **References:** 1 **Referenced by:** 1		
☐	2021-12-09	Payload delivery	**malware-sample:** malware-sample	malicious.exe f1a3e62de12faecee82bf4599cc1fdcd
☐	2021-12-09	Payload delivery	**filename:** filename	malicious.exe
☐	2021-12-09	Payload delivery	**md5:** md5	f1a3e62de12faecee82bf4599cc1fdcd
☐	2021-12-09	Payload delivery	**sha1:** sha1	d836f2ee449b74913d1efc615eeb459b65e4f791
☐	2021-12-09	Payload delivery	**sha256:** sha256	d90401420908dbb4b3488a306467e8fffc57577ce9d5eee016578ff6a3ada12e
☐	2021-12-09	Other	**size-in-bytes:** size-in-bytes	751328

...

Event: Encapsulates contextually linked information.
Events also have basic information including ownership and access-control
Here: Contains all the information related to the spear-phishing incident.

Taxonomies: Simple label standardised on common set of vocabularies.
Here: Usage of labels to classify the current completeness of the Event, what recipient can do with the information and the category of the incident.

Galaxies & Galaxy-Clusters: Advanced label containing meta-data
Here: The sector affected by the incident as well as the country. The kill-chain of the attack can be described using the MITRE ATT&CK framework

Event Graph: Visualization of the relationships between entities contained in the Event.
Here: The whole story of the attack can be described with relationships defined between Attributes and Objects

Event Timeline: Visualization of the temporality of the data contained in the event.
Here: A timeline of the steps performed during the attack. The time data is taken directly from the Attributes and Objects belonging to the Event.

Event Report: Markdown-aware supporting text document to describe events or incidents
Here: The report describe the steps taken by the attacker and provide additional contextual information. It also contains references to Attributes and Object encoded in the Event

Attributes: Basic building block to represent information.
They can have context such as taxonomy and express if they are supportive data or meant for automation. An Event can have multiple Attributes
Here: Two Attributes representing payload delivery. One is an IP address, the other is an URL.

Objects: Advanced building block allowing Attribute composition via predefined templates.
As an Object is an instantiation of its template, it is composed of Attributes that make sense Together. They can also have relationship to other entity contained in the Event
Here: A file object composed of Attributes such as the filename, size and hashes. It also have a relationship

MISP User & Admin Cheat Sheet

- User -

API

Wildcard searches:

```
POST /attributes/restSearch
{"value": "1.2.3.%"}
```

Or and Negation searches:

```
POST /attributes/restSearch
{"tags": ["tlp:white", "!tlp:green"]}
```

And and Negation searches:

```
POST /attributes/restSearch
{"tags": {"AND": ["tlp:green", "Malware"], "NOT": ["%ransomware%"]}}
```

Galaxy Cluster metadata searches:

```
POST /attributes/restSearch
{
    "galaxy.synonyms": "APT29",
    "galaxy.cfr-target-category": "Financial sector"
}
```

Attach tags:

```
POST /tags/attachTagToObject
{
    "uuid": "[Could be UUID from Event, Attribute, ...]",
    "tag": "tlp:amber"
}
```

Timestamps:

`timestamp`: Time of the last modification on the data

- Usecase: Get data was modified in the last t
- E.g.: Last updated data from a feed

`publish_timestamp`: Time at which the event was published

- Usecase: Get data that arrived in my system since t
- E.g.: New data from a feed

`event_timestamp`: Used in the Attribute scope

- Usecase: Get events modified in the last t

Usage:

```
{"timestamp": 1521846000}
{"timestamp": "7d"}
{"timestamp": ["2d", "1h"]}
```

Tips & Tricks

Get JSON Representation: Append `.json` to any URLs to get their content in JSON format. Example: `/events/view/42.json`

- Admin -

Reset Password

API: `POST /users/initiatePasswordReset/[id]` `{"password": "***"}`

CLI: `MISP/app/Console/cake Password [email] [password]`

Reset Bruteforce login protection

CLI: `MISP/app/Console/cake Admin clearBruteforce [email]`

Upgrade to the latest version

All in 1-shot: `MISP/app/Console/cake Admin updateMISP`

Manually:

1. `cd /var/www/MISP`
2. `git pull origin 2.4`
3. `git submodule update --init --recursive`
4. `MISP/app/Console/cake Admin updateJSON`
5. Check live update progress `GET /servers/updateProgress`

Workers

Restart All: `MISP/app/Console/cake Admin restartWorkers`

Add: `MISP/app/Console/cake Admin startWorker [queue]`

Stop: `MISP/app/Console/cake Admin stopWorker [pid]`

Settings

Get: `MISP/app/Console/cake Admin getSetting [setting]`

Set: `MISP/app/Console/cake Admin setSetting [setting] [value]`

Base URL: `MISP/app/Console/cake Baseurl [baseurl]`

Miscellaneous

Clean Caches: `MISP/app/Console/cake Admin cleanCaches`

Get IPs For User ID: `MISP/app/Console/cake Admin UserIP [user_id]`

Get User ID For User IP: `MISP/app/Console/cake Admin IPUser [ip]`

Documentation: `/events/automation`

Logs files location: `MISP/app/tmp/logs`

User

Check	Description	Length
☐	**Add events**	
☐	- via Standard UI	
☐	- Distribution levels and publication	
☐	- Different timestamps & publish_timestamp	
☐	**Add attributes**	
☐	- Freetext	
☐	- Standard UI	
☐	- Template	
☐	- ReST API	
☐	- via EventGraph	
☐	**Object**	
☐	- add Object	
☐	- add References	
☐	- show via EventGraph	
☐	***-lists**	
☐	- Warninglists: show warnings raised in steps above	
☐	- Noticelists: show warnings when adding data	
☐	- Import Regexp: avoid leaking private/personal data	
☐	**Correlations**	
☐	- show correlations that were added	
☐	- pivot to events via correlations	
☐	- show correlations graph	
☐	- feeds & servers correlation	
☐	**Tags and Galaxies**	
☐	- Tag from Taxonomy	
☐	- GalaxyCluster	
☐	- ATT&CK pattern & Galaxy matrix	
☐	- Tag Collection	
☐	**Sighting**	
☐	- via UI & API	
☐	**Delegation**	
☐	**Proposal**	
☐	**Delete (including soft versus hard delete)**	
☐	- Event blocklist when deleting	
☐	**Extending event** (how and when to use it)	
☐	**Exporting data**	
☐	- download from	
☐	- download from via modules	
☐	- .json routing	
☐	- RestSearch	
☐	**Searching for data**	
☐	- Attribute search	
☐	- Event index filter search	
☐	**Advanced features**	
☐	- Event graph, Event timeline, Event report	
☐	- Decaying of IoC	
☐	- Galaxy 2.0	
☐	**Enrichments**	
☐	- Hover & persistent	

Administrator (Community)

Check	Description	Length
☐	**Organisations**	10m
☐	- local and remote	
☐	- administration: Creation and merge	
☐	**User**	5m
☐	- administration and contact via standard UI	
☐	- Pasword/Auth key reset	
☐	- Disabling (never remove)	
☐	**Roles and permissions**	3m
☐	- Constraints & special sync-user	
☐	**Sharing group**	10m
☐	- administration via standard UI	
☐	**Block listing**	3m
☐	- Events & Organisations	
☐	**Synchronisation**	35m
☐	- MISP to MISP (sync_user, test & preview, flow control)	
☐	- Feeds to MISP (Options, overlap)	
☐	- Pub-Sub	
☐	**Collaboration settings**	
☐	- 'proposal_block_attributes', 'sanitise_attribute_on_delete', 'Sightings_anonymise'	
☐	**Templates**	
☐	- administration via standard UI	

Administrator (Instance)

Check	Description	Length
☐	**Advanced Auth keys**	3m
☐	- Migration from old system	
☐	- Usage	
☐	**Server settings**	5m
☐	**Maintenance**	15m
☐	- Updating & release process	
☐	- Submodules and populate DB	
☐	- Diagnostic	
☐	**Jobs and Workers**	10m
☐	- Administration via standard UI	
☐	- Scheduled Tasks and CRON jobs	
☐	**User settings & User management**	5m
☐	- User settings	
☐	- User monitoring, self-management, auto-registration	
☐	**Logging & auditing**	10m
☐	- Logs (and purge: event history)	
☐	- Paranoid, IP & Auth log, Sync audit	
☐	**Troubleshooting**	5m
☐	- Clean cache & DB Schema diagnostic	
☐	- Stuck workers	
☐	- Update in progress	
☐	- Apache logs & workers logs	

MISP Training Slide Decks

MISP[1] is a threat intelligence platform for gathering, sharing, storing and correlating Indicators of Compromise of targeted attacks, threat intelligence, financial fraud information, vulnerability information or even counter-terrorism information.

This document includes the slides which are the support materials[2] used for MISP trainings. The content is dual-licensed under CC-BY-SA version 4 license or GNU Affero General Public License version 3 which allows you to freely use, remixes and share-alike the slides while still mentioning the contributors under the same conditions.

Contributors

- Steve Clement `https://github.com/SteveClement`
- Alexandre Dulaunoy `https://github.com/adulau`
- Andras Iklody `https://github.com/iglocska`
- Sami Mokaddem `https://github.com/mokaddem`
- Sascha Rommelfangen `https://github.com/rommelfs`
- Christian Studer `https://github.com/chrisr3d`
- Raphaël Vinot `https://github.com/rafiot`
- Gerard Wagener `https://github.com/haegardev`

Acknowledgment

The MISP project is co-financed and resource supported by CIRCL Computer Incident Response Center Luxembourg[3] and co-financed by a CEF (Connecting Europe Facility) funding under CEF-TC-2016-3 - Cyber Security as *Improving MISP as building blocks for next-generation information sharing.*

Co-financed by the Connecting Europe Facility of the European Union

[1] `https://www.misp-project.org/`
[2] `https://github.com/MISP/misp-training`
[3] `https://www.circl.lu/`

CIRCL
Computer Incident
Response Center
Luxembourg